

Custom Compiler™ SDL and Visually Assisted Automation Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	13
Related Products, Publications, and Trademarks	13
Conventions	13
Customer Support	14
Statement on Inclusivity and Diversity	15

1. Introduction to SED, SDL and Visually Assisted Automation	16
About Visually Assisted Layouts	16
Tools for Visually Assisted Layout Processes	16
Realizing and Implementing a Layout with SDL	18

2. Initializing Schematic Driven Layout	22
Overview of Schematic Driven Layout Initialization	22
Initializing Schematic Driven Layout From Schematic	23
Advanced Steps in Initializing From Schematic	25
Initializing Schematic Driven Layout From Layout	26
Logical Source Errors	28
Reopening the Logical Source	29
Reinitializing Schematic Driven Layout	29
Verifying Schematic-Layout Correspondence	30
Checking the Physical Correspondence of a Schematic Instance	30
Checking the Elaboration Rules of an Instance	30
Establish Correspondence Feature	31
Setting Schematic Driven Layout Options	31
Main Tab	32
Label Tab	36
Feedthrough Net Name Selection Rules	38
Expanding M-Factor and S-Factor	40
M-Factor Expanding	40
S-Factor Expanding	40
M-Factor and S-Factor Expanding	42

3. Browsing the Schematic Using the Design Navigator	43
Starting the Design Navigator with Physical and Logic Views	43
Using Toolbar Icons for the Design Navigator in Schematic Mode	44
Symbolic Editor With Chaining Mode	45
Symbolic Editor With Matching Mode	45
Pick and Place	45
Cross Shadow	45
Create Template	46
Create Placement Constraint	47
Set Flight Line	48
More Options	48
Using the Design Navigator as an SDL Browser	49
Filter	49
Realized and Unrealized Background Color	50
Display and View	51
Instance, Net, and Terminal Icons	54
Insts, Nets, Terms, and Groups Tabs	56
Insts Tab	56
Nets Tab	58
Terms Tab	61
Groups Tab	62
Finding and Displaying Mismatches	62
Design Navigator Shortcut Menus	64
Instances Shortcut Menu Commands	65
Layout Ignored Instances	73
Hierarchy Instances	76
Extra Instances	77
Nets Shortcut Menu Commands	78
Terms Shortcut Menu Commands	79
Groups Shortcut Menu Commands	80
Using the Schematic Assistant	80
Starting the Schematic Assistant	80
Schematic Assistant Shortcut Menus	83
Shortcut Menu on Canvas	83
Shortcut Menu on Instances	85
Shortcut Menu on Nets	86
Shortcut Menu on Pins	87

4. Using a Top-Down Flow	89
---------------------------------	----

Contents

Soft Macro Creation	89
Effective Parameters	91
Soft Macro Cell Creation	91
Soft Macro Layout	92
Soft Macro Config	92
Soft Macro Options	92
Constraint Update for Soft Macro Creation	93
Soft Macro Reuse	93
Expand Schematic	94
Setting Expand Schematic Options	95
Library Tab	96
Schematic Tab	97
Options Tab	98
Config Setup Tab	99
Log File Tab	100
Copy Schematic Functionality in Expand Schematic	101
ECO Flow in Expand Schematic	102
Expanding Schematic With Different Version Control	103
Flatten	104
Constraint Update for Flatten	105
Recover	105
Constraint Update for Recover	106
Soft Macro Verification	107
Fix Soft Macro Mismatch	107
5. Placing Schematic Instances Into Layout	108
Generating Layout	109
Using Generate Layout	109
Generate Layout Options — Main Tab	110
Generate Layout Options — Pins Tab	114
Generate Layout Options — Label Tab	116
Resolving Terminal Connection Conflicts	116
Placing by Floorplan Template	116
Picking and Placing	117
Setting Up Pick and Place	118
Picking and Placing a Single Object	118
Snapping and Flipping	119
Creating Photonic Pins	119

Contents

Picking and Placing Multiple Objects	120
Placing Multiple Objects by Selection Order	121
Aligning Gates While Placing Objects	121
Pick and Place Options	122
Pick and Place Dialog Box Options	122
Pick & Place Toolbar Options	130
Realizing a Layout With Preset Parameters	131
Realizing Layouts	131
Customizing IxPresetConfig Setting	133
Customizing Preset Designs	135
Add Valid CDF Parameters in IxPresetConfig Setting	135
Pick Valid Preset Instances	136
Chaining Instances	140
Folding Instances	140
Establishing Correspondence	141
Viewing Correspondence Relationships	142
Setting and Unsetting Correspondence Relationships	142
Setting Correspondence Relationships	142
Unsetting Correspondence Relationships	143
Establish Correspondence Toolbar Options	143
Establish Correspondence Dialog Box Options	144
Establish Correspondence Shortcut Menu	146
Establish Correspondence From LVS Results	146
Cloning Devices	148
Cloning From Toolbar	148
Cloning From Dialog Box	149
Clone Toolbar and Dialog Box Options	150
Array Placement Using the Clone Command	154
Re-editing the Clone Group Using Symbolic Editor	156
Splitting Devices	158
Splitting Placed Devices	158
Splitting Preselected Devices	159
Splitting Post-selected Devices	159
Split Device Toolbar Options	160
Adding and Editing SDL Mapping Rules	160
SDL Mapping Options	161
Applying Preset Parameters	163

Realizing Layouts Using Drag-And-Drop Operations	163
Triggering Callback Functions When Realizing Layout	164
Establish Correspondence Feature	165
Cross-Object Referencing	165
Cross-Selecting	165
Enabling Cross-Selection	166
Cross-Probing	166
Adding Probes	167
Removing All Probes	168
Finding Probes	168
6. Using Templates and the Template Manager	169
Creating Templates	170
Introduction to Create Template	170
Template Modes	170
Prerequisites for Create Template	171
Invoking Create Template	172
Using the Create Template Dialog Box	172
Creating a New Template Cell	173
Adding Views to an Existing Template Cell	174
Using the Template Manager	176
Introduction to Template Manager	176
Prerequisites for Template Manager	177
Invoking Template Manager	178
Using the Template Manager Interface	179
Searching for Matches in the Active Design	183
Searching the Scope of the Active Design	183
Searching Scope of the Template Candidates	184
Browsing Matches in the Find and Bind Tab	186
Results Viewer	186
Template Description	188
Template Viewer	188
Mapping Table	189
Working with Bound Templates in the Management Tab	192
Toolbar Actions in the Management Tab	193
Display Styles in the Management Viewer	194
Menu Actions in the Management Viewer	195
Hierarchical Templates in the Management Viewer	196
Previewing the Layout Realization	196

Contents

Copying Template Results	198
Saving Template Constraints	198
Post-Process Procedure	201
Realizing a Template to the Editing Layout	201
Realize Toolbar	202
Realize Menu	203
Post-Process Procedure	203
Batch Creating Templates	204
Working With Static Flow	206
Enabling Static Template	206
Using Matching Parameters and Criteria	208
Browsing Matching Results	211
Saving and Realizing Template Design	212
Reconfiguring Templates	212
Setting Options for Template Manager	216
Showing and Hiding Sections of Template Manager	220
Using the Template Browser	221
Introduction to Template Browser	221
Prerequisites for Template Browser	221
Invoking Template Browser	222
Using the Template Browser Interface	224
Working With Template Browser	225
Setting Template Attributes	229
Invoking the Template Attributes Assistant	229
Using the Template Attributes Assistant	231
7. Using the Placement Assistant	233
Introduction to Placement Assistant	234
Prerequisites for Placement Assistant	234
PDK Requirements for Placement Assistant	234
Preparing the Layout Hierarchy	237
Enabling the Placement Menu	237
Using the Placement Menu	239
Create Placement Area Command	240
Placement Area Toolbar	240
Using the Create Placement Area Command	241
Auto Place Command	243
Place Toolbar	243

Contents

Using the Auto Place Command	244
Auto Place Command Usage Notes	245
Placement Jobs Submission	247
Placement Library	247
Place Pins Command	249
Place Pins Toolbar	249
Using the Place Pins Command	249
Optimize Pins Command	251
Optimize Pins	251
Using the Optimize Pins Command	252
Placement Options Dialog Box	253
Main Tab	253
Create New Placement Library Dialog Box	258
Pins Tab	259
Modifying Pin Settings	262
Pins Tab Menu	265
Placement Results	266
Viewing Placement Results	267
Selection and Highlighting of Placement Results	269
Placement Results Browser	271
Filters	272
List Tab	275
Chart Tab	276
Copy Result Command	277
Copy Result Toolbar	278
Copy Result Menu	279
Using the Copy Result Command	279
Generating Constraints	281
Generate Constraints Dialog Box	281
Apply Templates Option	282
Group Devices Option	282
Create Constraint Area Command	283
Extract Topology Constraints Command	285
Extract Topology Toolbar	288
Using the Extract Topology Constraints Command	289
Relative Constraint	290

8. Updating Schematic From Layout	292
Updating Schematic From Dummy and Extra Devices	292
Updating Schematic From Parameters Difference	296
Update Schematic From Layout Options	297
Update Schematic From Layout Options — Main Tab	298
Update Schematic From Layout Options — Dummy Devices Tab	299
Update Schematic From Layout Options — Extra Devices Tab	303
Update Schematic From Layout Options — Parameters Tab	303

9. Using the Via Track Planner	305
About Via Track Planner	305
Enabling the Via Track Planner	305
Tcl Procedures	307
Starting Via Track Planner	309
Creating and Deleting Vias in MOS Transistors	312
Creating and Deleting Interconnects in MOS Transistors	312
Creating and Deleting Cut-poly Segments in MOS Transistors	314

10. Setting Up and Performing Verification	315
Mismatches Between Physical Layout and Logic Source	315
Schematic Driven Layout Engineering Change Orders	315
Setting Up Schematic Driven Layout ECO	316
Specifying Error Types and Auto Update Actions	317
Ignoring Cells During Verification	320
Specifying the Update Options	322
Specifying the Verify Options	323
Honor M-Factor/Finger	324
Extend Gate	325
Finding Mismatches between Source and Target Designs	326
Verify Current Design	326
Hierarchy Verify	327
Resolving Mismatches between Source and Target Designs	328
Using the Auto Update Command to Resolve Mismatches	328
Using the Error Viewer to Resolve Mismatches	328
Using the Incremental Update Command to Resolve Mismatches	329

A. Placement and Resistance Constraints	332
Placement Constraints	332
Affinity	333
Example — Affinity Constraint	334
Alignment	335
Example — Alignment Constraint	336
Checking Intent	336
Block Size	337
Example — Block Size Constraint	338
Cluster	338
Example — Cluster Constraint	340
CMOS	340
Example — CMOS Constraint	345
Column	345
Example — Column Constraint	347
Edge Assign	347
Example — Edge Assign Constraint	348
Enclosing Shape	349
Example — Enclosing Shape Constraint	351
Guard Ring	351
Example — Guard Ring Constraint	353
Halo	353
Example — Halo Constraint	354
Inst Matching	355
Example — Inst Matching Constraint	362
Orientations	362
Example — Orientations Constraint	364
Pattern Route	364
Placement Grid	364
Example — Placement Grid Constraint	366
Resistor Bank	366
Example — Resistor Bank Constraint	370
Routing Stripes	370
Example — Routing Stripes Constraint	371
Rows	372
Example — Rows Constraint	374
Standard Cells	374
Example — Standard Cells Constraint	378
Symmetric	378

Example — Symmetric Constraint	379
Resistance Constraints	379

B. Setting up a Foundry Tool Kit for SDL	380
Overview of Logical to Physical Mapping	380
Basic View Switch / Stop Lists	380
Library Search List	381
Ignoring Objects	381
Ignoring Terminals	382
Logical to Physical Mapping Using Configurations	382
About IxComponentType	382

About This User Guide

This guide explains how to use the Custom Compiler Visually Assisted Automation tools to create integrated circuit designs.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® Custom Compiler™

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .

Convention	Description
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN.</code>
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to SED, SDL and Visually Assisted Automation

This chapter introduces the following Custom Compiler Layout Assistant tools: Schematic Driven Layout (SDL), Symbolic Editor (SED), Placement Assistant (PA), Template Assistant (TA), and Via Track Planner (VTP).

This chapter describes the following topics:

- [About Visually Assisted Layouts](#)
- [Tools for Visually Assisted Layout Processes](#)
- [Realizing and Implementing a Layout with SDL](#)

About Visually Assisted Layouts

Custom Compiler layout assistants are user-guided layout and placement tools that automate and accelerate layout tasks. Instead of using text-based constraints to produce circuit designs in a rigid process, you can use Custom Compiler layout assistants to interactively create layouts and refine placements.

The layout creation tool you choose depends on your needs. Some tools allow you to have precise control over pick and place tasks. Others automate the layout process by intelligently detecting similar layout patterns so that you can reuse them in another area of the design or in a different design.

Tools for Visually Assisted Layout Processes

 [Show me](#) how to use SDL and visually assisted automation features for device placement.

SDL and other Visually Assisted Automation tools can be used in the following ways:

Tool or Feature	Description
Schematic Driven Layout (SDL)	Generates a layout design from a logical source. SDL creates an equivalent layout design in which the layout is placed like the schematic. The resultant layout can serve as a useful starting point, but it is not readily usable. For more information, see Generating Layout. Creates a layout by picking instances and pin semantic objects from the schematic and placing them into the layout. If a net or term does not exist in the layout, it is created. Each placement creates a pin and a shape on the pin. For more information, see Picking and Placing.
Symbolic Editor	Quickly places multiple devices into a layout for device-level floorplanning that meets DRC requirements. You can use the chaining mode to automatically abut devices, or use the matching mode to place multi-transistor devices using preset matching patterns to support interdigitation. You can modify the placement of the symbolic layouts in the Symbolic Editor, and save customized matching patterns for future use. The Symbolic Editor in chaining mode provides a visual estimate of required interconnect as a reference. For more information, see About the Symbolic Editor in the <i>Custom Compiler Symbolic Editor User Guide</i>.
Template Manager	Reuses existing layout patterns in new designs, or in another region of the same design. The Template Manager learns from the work done with the Placement Assistant and Custom Router. It recognizes circuits that are similar to ones that were already completed and lets you apply the same placement and routing pattern as a template to the new circuits. The Custom Compiler environment has pre-loaded templates for commonly used circuits, such as current mirrors, level shifters, and differential pairs. For more information, see Using Templates and the Template Manager.
Placement Assistant	Lets you make successive refinements to device placements. You can use the auto mode or the interactive mode. The auto mode does not require any intervention. The interactive mode allows you to provide some guidance during the operation. For more information, see Using the Placement Assistant.
Via Track Planner	Creates and deletes vias, interconnects, and cut-poly segments in MOS transistor devices. For more information, see Using the Via Track Planner.
SDL ECO	Finds and fixes mismatches between a logic design and the corresponding layout design. For more information, see Schematic Driven Layout Engineering Change Orders.

Realizing and Implementing a Layout with SDL

Schematic Driven Layout (SDL) is a visually assisted automation tool to help you realize and implement a layout from a schematic.

Physical realization is the process by which the layout equivalent of a schematic design is created. You can use Schematic Driven Layout to automate physical realization, significantly improving design productivity.

SDL uses the schematic design to produce a layout design with instances and pins that have the correct set of parameter values. The instances and pins placed in the layout design match their logical correspondences' connectivity. SDL creates a net, terminal, pin, or shape in a layout from what it finds in the schematic. Nets, terminals, pins, and shapes are also created for any global net and any unresolved net connection definition.

SDL flattens the hierarchy present in the logical source to provide a flattened physical design that corresponds to the logical source in terms of instances and pins.

SDL is optimized for bottom-up physical designs. All leaf instances must exist in the layout.

 [Show me](#) how to use SDL and visually assisted automation features to create a block-level layout.

Layout requires symmetry for placement and routing. Due to the discrete nature of advanced nodes, transistors are often arranged in series-parallel structures to achieve the desired effective length and width. You can create this type of layout placement quickly and efficiently by using SDL and related automation tools, as described in the following steps.

1. Open a schematic and choose **Tools > SDL** to enable SDL features. The Design Navigator, Schematic Assistant and Layout Editor are displayed after you define the physical and logical design information in the **Design Physical Target** dialog box.

See [Initializing Schematic Driven Layout](#).

2. Select all instances in the Design Navigator and flatten the hierarchy to all levels by right-clicking the Design Navigator. This will flatten the layout hierarchy without modifying the schematic design.

See [Browsing the Schematic Using the Design Navigator](#).

3. Define parameters for a single instance and use those parameters for all instances.

See [Realizing a Layout With Preset Parameters](#).

4. Select the corresponding instances in the Design Navigator and then invoke the Symbolic Editor Chaining Mode to design the placement. You can preview the placement; the placement is updated automatically in the preview assistant. You also

can specify the SDL presets in chaining mode. Click **Realize** to finish the design and place to layout. A group is created with a [CMOS](#) constraint.

The placement also can be designed by using Symbolic Editor Matching Mode. You can choose a variety of built-in patterns under the Pattern Panel. Even if you do not find a perfect match, you can still pick a viable starting pattern and edit it in a graphical manner. Invoke the preview assistant to visualize the layout while you arrange the pattern. A group is created with an [Inst Matching](#) constraint when realizing the pattern. Resistor matching is also supported in matching mode.

See About the Symbolic Editor in the *Custom Compiler Symbolic Editor User Guide*.

5. Select the fig group created from Symbolic Editor, and descend the group using **Edit in Group**. Create NWell and PSubstrate guard rings around PMOS devices and create PWell and Deep NWell guard rings around NMOS devices.

See Creating and Editing Guard Rings and Editable PCells in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

6. After the group design is ready, use **SDL > Clone** to reuse all instances in the cell or create all instances in the design.

See [Cloning Devices](#).

7. The Routing Assistant provides advance welding and cloning capabilities. You can enable these features in the Interactive Route toolbar. As you start from a pin and slide the cursor across, Routing Assistant will automatically connect to the pins as your cursor moves along. Besides the routes created under the cursor, others branches are created simultaneously across all channels.

See Interactive Routing in the *Custom Compiler Custom Router User Guide*.

Pattern Route is also supported under the Routing Assistant. You can specify the net that you want it to learn the pattern from by first selecting that net.

See Pattern Routing in the *Custom Compiler Custom Router User Guide*.

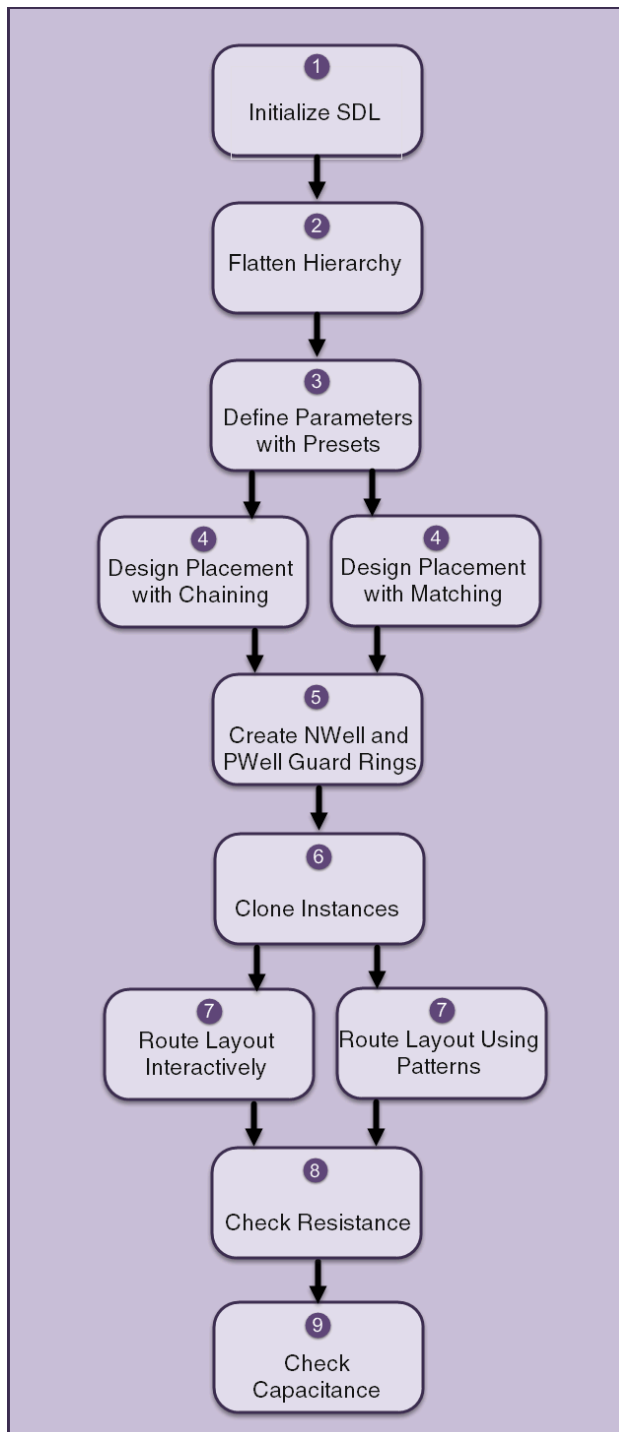
8. At advanced nodes, besides design rules, parasitics and EM also drive critical layout choices. From the Design Navigator, in-design assistants can be invoked to make sure that two nets not only meet the resistance budget, but also have balanced capacitance. Bring up the Resistance Reporter by clicking on any two points on a net. The top table shows the total of each path. The resistance breakdown is displayed in the bottom table in a sorted order. That helps you to easily identify where the bottleneck is, which could be a via or a narrow metal.

See Reporting and Checking Resistance in the *Custom Compiler Electrical Checking and Reporting User Guide*.

Capacitance Reporter can be invoked the same way.

See Reporting and Checking Capacitance in the *Custom Compiler Electrical Checking and Reporting User Guide*.

Figure 1 *SDL and Related Automation Tools Workflow*



2

Initializing Schematic Driven Layout

This chapter describes the different ways of initializing SDL by verifying the correspondence between schematic and layout designs.

This chapter describes the following topics:

- [Overview of Schematic Driven Layout Initialization](#)
- [Initializing Schematic Driven Layout From Schematic](#)
- [Initializing Schematic Driven Layout From Layout](#)
- [Logical Source Errors](#)
- [Reopening the Logical Source](#)
- [Reinitializing Schematic Driven Layout](#)
- [Verifying Schematic-Layout Correspondence](#)
- [Establish Correspondence Feature](#)
- [Setting Schematic Driven Layout Options](#)

Note:

To ensure correspondence between your schematic and layout, you must provide the `sdl simInfo` section in the Component Description Format (CDF) for cells.

See Parameter Definition Editor Tool in the *Custom Compiler Environment User Guide* for more information.

See Adding `simInfo` in the *Custom Compiler iPDK Developer Guide* for more information.

Overview of Schematic Driven Layout Initialization

Initialization is the process of creating a correspondence between the layout design and the configured schematic design.

When you initialize Schematic Driven Layout (SDL), SDL creates a new session and adds the SDL menu to the corresponding Schematic Editor and Layout Editor windows. During the process of initialization, the elaboration rules for the schematic are also defined.

When SDL is initialized, the **Connectivity Engine** option is enabled. For more information regarding the Connectivity Engine, see the Connectivity Tab.

Note:

Although the **Connectivity Engine** option is enabled when SDL is initialized, the **Connectivity Engine** option is not disabled by turning off SDL. You can disable the **Connectivity Engine** option in the **Connectivity** tab from the **Layout Design Options** dialog box by choosing **Layout > Options > Design**.

You can initialize SDL either from a schematic design or from a layout design. See [Initializing Schematic Driven Layout From Schematic](#) or [Initializing Schematic Driven Layout From Layout](#) for more information.

You also can use `lx::setupSDL` to set up SDL environment by generating a default `layout.config` cellview in the same cell if the config is not specified. This command can also be used to generate a new layout cellview. See Commands.

Initializing Schematic Driven Layout From Schematic

The most typical use model is to initialize SDL from a schematic design that is open in Schematic Editor.

After SDL is initialized from schematic successfully, both the Layout Editor and Schematic Editor windows have the SDL menu.

 [Show me](#) how to initialize schematic driven layout from schematic.

To initialize SDL from a schematic:

1. Choose **Tools > SDL**.

The **Define Physical Target** dialog box opens.

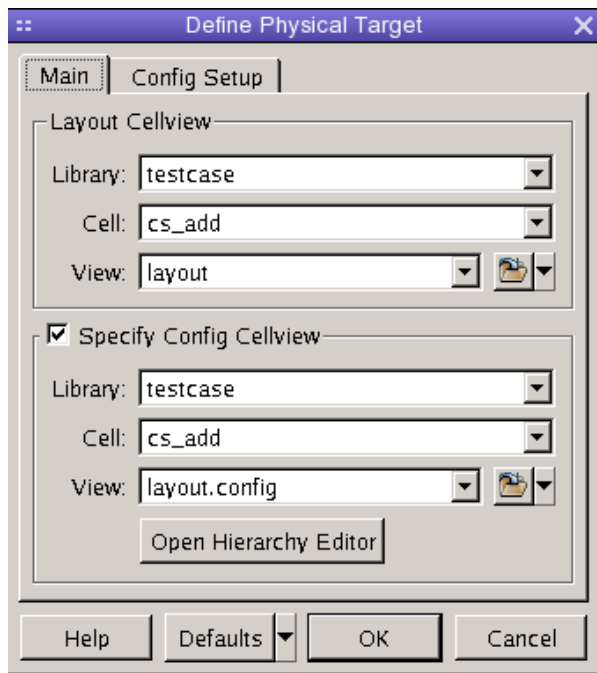
Note:

The preference `lxOpenSDLWithSameCell` controls the **Define Physical Target** dialog box.

When `lxOpenSDLWithSameCell` is false, SDL prompts the **Define Physical Target** dialog box with the target layout design and configuration cellview.

When `lxOpenSDLWithSameCell` is true, the target layout design and configuration are automatically set to the same cell views. A SDL session is created and the layout is changed to the SDL workspace.

Figure 2 Define Physical Target Dialog Box



2. In the **Layout Cellview** region, specify the library, cell, and view of the target layout design that you want to create.
3. (Optional) If you want to make advanced changes, such as changing the default config options or the library search list, see [Advanced Steps in Initializing From Schematic](#).
4. Click **OK**. The result depends on whether the logical source contains errors:
 - If the logical source contains errors, a dialog box opens (see [Logical Source Errors](#)).

The `lxShowLogicalSourceError` specifies whether error messages are displayed in the **Logical Source Errors** dialog box or in the Console assistant. Valid values: dialog (default) or console.

- If there are no errors, the new layout cellview is created and opened in edit mode.

If the layout and the config cellviews do not exist, SDL creates these views and initializes the layout. If the layout view exists, SDL opens this cellview and uses the config pointer to open the layout and the schematic using that config. You must use the Hierarchy Editor to modify an existing config cellview.

If the config cellview cannot be created, SDL creates the configuration view in a system temporary library.

When the layout opens, the correspondence between the new layout cellview and the logical source is established and the **Connectivity Engine** option is enabled.

Advanced Steps in Initializing From Schematic

The following procedure describes advanced steps available when initializing Schematic Driven Layout to create a new layout cellview. The values for fields that are updated in these steps are already populated from defaults.

This procedure continues from Step 3 in [Initializing Schematic Driven Layout From Schematic](#).

1. In the **Main** tab, select the **Specify Config Cellview** check box.

Note:

If the config cellview already exists, the config options on the **Config Setup** tab are disabled.

2. (Optional) In the **Main** tab, select the library, cell, and view of the configuration cellview. The default view name of the config view is *<target layout name>.config*, where *<target layout name>* is the name of the layout design.
3. (Optional) If you want to edit the cellview configuration (for example, to change the **Library Search List**, **View Search List** or **View Stop List**), click the **Open Hierarchy Editor** button.

For more information about the library search list, see [Library Search List](#).

For more information about view lists, see [Basic View Switch / Stop Lists](#).

For information about using the Hierarchy Editor, see the Hierarchy Editor Tool in the *Custom Compiler™ Environment User Guide*.

4. Use an XML editor to edit the **Ignore Name List** to specify property names that must be considered to designate a specific instance as ignored. For example:
 - *!xIgnore*
 - *nlAction* (Included automatically; need not be specified.)

Note:

The ignore property can be specified on an instance, master, or parameter defined in cell dmData (Component Description Format). In most instances, SDL looks for a property whose value is "true." However, some properties that SDL considers can also be string values.

If the property name is found in a master, all instances of the master are ignored.

5. Use an XML editor to edit the **Short Name List** to specify the property names that must be considered to designate a specific instance as having a terminal short. n-terminals can also be shorted.

Note:

The property can be specified on an instance, master or parameter defined in cell dmData (Component Description Format). In most instances, SDL looks for a property whose value is *true*. However, some properties that SDL considers can also be string values:

- To ignore the instance: "ignore", "true", "t", and "1".
- To explicitly include the instance: "false", "nil", and "0".

If the property name is found in a master, all instances of the master are shorted.

6. Click **OK**.

Initializing Schematic Driven Layout From Layout

You can initialize SDL from the Layout Editor. When SDL is initialized from a layout, the corresponding schematic design is opened in the schematic assistant.

Note:

The preference `lxOpenInNewFrames` controls whether new Schematic Editor and Layout Editor frames are created during SDL initialization. The geometry of the new frames is set by the preferences `lxOpenedSchematicFrameSize` and `lxOpenedLayoutFrameSize`.

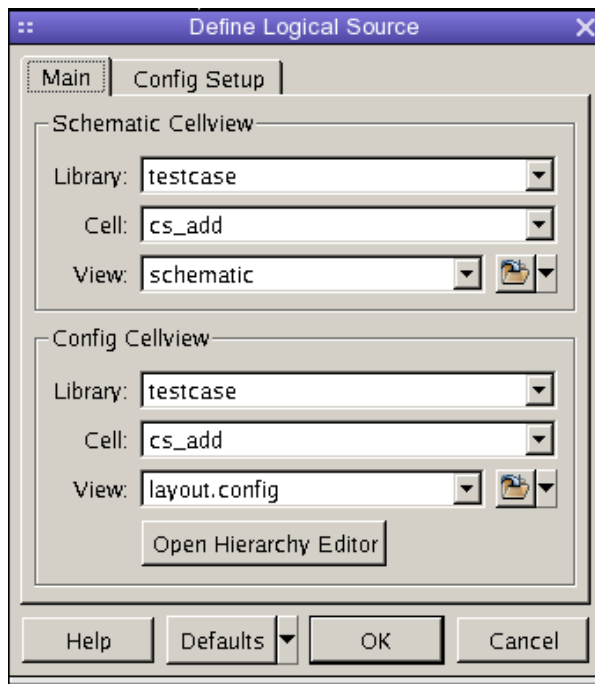
After SDL is successfully initialized from a layout, the layout window displays the SDL menu.

To initialize SDL from the layout:

1. From the layout design toolbar, choose **Tools > SDL**.

The **Define Logical Source** dialog box opens.

Figure 3 Define Logical Source Dialog Box



2. In the **Schematic Cellview** region, specify the library, cell, and view of the schematic view that you want to use as the source.
3. Select the library, cell, and view of the configuration cellview you want to create.

If a layout has been initialized with SDL before, the config cellview might already exist. In that case the values in the config cellview are automatically displayed. Preference `lxSDLPhysicalConfig` is used to find which config view should be used to open SDL.

Note:

The `lxUpdateSDLDataOnCopyMove` preference is provided to update the SDL binding when moving or copying a library, cell or cellview. See *Schematic Driven Layout Tcl* in the *Custom Compiler Tcl Reference Manual*.

4. Click **OK**.

The result depends on whether the logical source contains errors:

- If the logical source contains errors, the error messages are displayed in the Console assistant.
- If there are no errors:

If you specify a nonexistent config cellview, a new config cellview is created. SDL opens the specified config cellview in the schematic assistant.

SDL opens the layout design in the context of the newly created config cellview. If a config cellview with the same name already exists, SDL asks you to confirm whether you want to overwrite the existing config cellview.

The **Connectivity Engine** option is enabled.

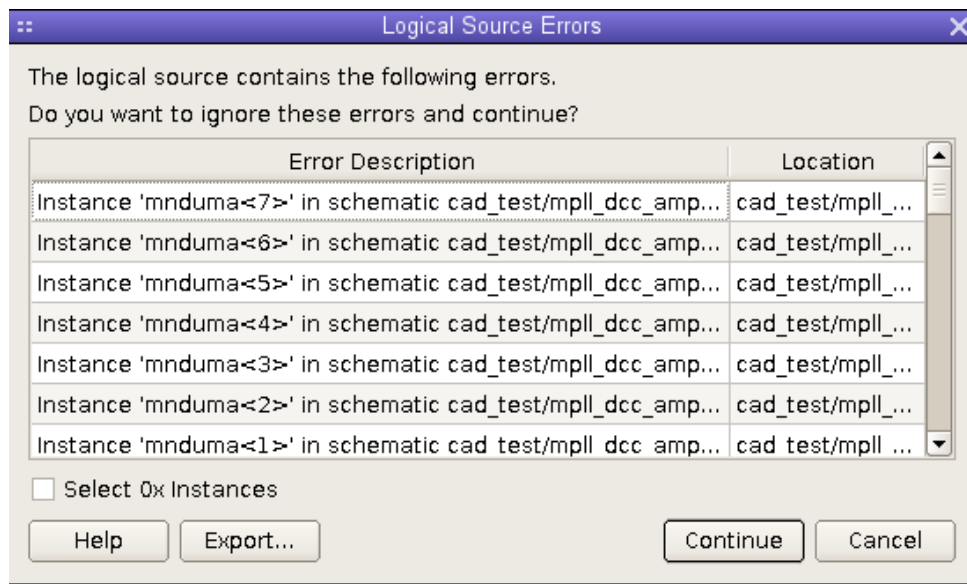
Note:

By default, SDL rearranges the layout and schematic windows. To disable this behavior, set the `lxAutoArrangeWindows` preference to *false*.

Logical Source Errors

The **Logical Source Errors** dialog box opens when you try to initialize SDL from a schematic that contains logical errors ([Figure 4](#)).

Figure 4 Logical Source Errors Dialog Box



You can:

- Cancel initialization
- Continue initialization, but show only the 0X instance errors (**Select 0X Instances** enabled)
- Continue initialization, but ignore all errors (**Select 0X Instances** disabled)
- Click **Export** to export all errors to a file

Reopening the Logical Source

To reopen the logical source:

- ▶ From Layout Editor, choose **SDL > Logical Source > Reopen**.

The logical source is reopened in the schematic assistant.

Reinitializing Schematic Driven Layout

After performing SDL initialization, if you want to reset the logical-physical correspondence, you can reinitialize SDL from either a logical source or from the physical target.

When you reinitialize, the existing correspondence between the logical source and physical target is broken and you can specify a new logical-physical correspondence.

To reinitialize SDL from the logical source:

1. From the Schematic Editor, choose **SDL > Reinitialize**.
2. The **Define Physical Target** dialog box opens.
3. Continue from step 2 in the [Initializing Schematic Driven Layout From Schematic](#) procedure.

To reinitialize SDL from the physical target:

1. From the Layout Editor, choose **SDL > Logical Source > Reinitialize**.
2. If you are reinitializing from a layout, the **Define Logical Source** dialog box opens.
3. Continue from step 2 in [Initializing Schematic Driven Layout From Layout](#).

Verifying Schematic-Layout Correspondence

After SDL initialization has completed, you can verify the schematic-to-layout correspondence of each schematic instance and also check the elaboration rules of any schematic instance.

- [Checking the Physical Correspondence of a Schematic Instance](#)
- [Checking the Elaboration Rules of an Instance](#)

Checking the Physical Correspondence of a Schematic Instance

[Table 1](#) shows three methods for checking the physical correspondence of a schematic instance after the logical to physical correspondence is established.

Table 1 Methods for Checking the Physical Correspondence of a Schematic Instance

Editor	Description
Layout	Use the Establish Correspondence command to verify the correspondence between each instance or terminal in the layout with its equivalent in the schematic design. See Establishing Correspondence .
Schematic	Open the physical config view of the specific SDL session in the Hierarchy Editor and view the physical correspondence by clicking the Open Hierarchy Editor button in the Define Logical Source dialog box. For information about using the Hierarchy Editor Tool, see <i>Custom Compiler Environment User Guide</i> .
Both	Enable cross view highlighting and then select objects in either editor to see the correspondence between physical and logic designs. For information about cross view highlighting, see Setting Schematic Driven Layout Options .

Checking the Elaboration Rules of an Instance

You can check the elaboration rules that apply to a schematic instance using the Hierarchy Editor.

To check the elaboration rules:

1. In the Schematic Editor, click the **Open Hierarchy Editor** button in the **Define Logical Source** dialog box.

The Hierarchy Editor dialog box opens.

2. Right-click on the instance whose elaboration rules you want to see.

The menu appears.

3. Click **Explain**.

All available details about the instance including elaboration rules are displayed.

4. Click **OK**.

Establish Correspondence Feature

You can view the correspondence between instances in a schematic netlist and a layout netlist using the Establish Correspondence feature. You can also use this feature to establish correspondence relationships between two elements and modify existing correspondence relationships.

For more information on the Establish Correspondence feature, see [Establishing Correspondence](#).

Setting Schematic Driven Layout Options

The **SDL Options** let you control the display of layout designs.

To open the **SDL Options** dialog:

1. Select the **Tools > SDL** command.

The **Define Logical Source** dialog opens (see [Figure 3](#)).

2. Complete or accept the Define Logical Source options and click **OK**.

The SDL menu is added to the Layout Editor toolbar.

For information about completing the **Define Logical Source** dialog box, see [Initializing Schematic Driven Layout From Layout](#).

3. Select the **SDL > Options** command in the Layout Editor.

This SDL options are described in the following sections:

- [Main Tab](#)
- [Label Tab](#)
- [Feedthrough Net Name Selection Rules](#)
- [Expanding M-Factor and S-Factor](#)

Main Tab

Figure 5 shows the options available from the **SDL Options, Main** tab.

Figure 5 *SDL Options, Main Tab*

The screenshot shows the 'SDL Options' dialog box with the 'Main' tab selected. The dialog is organized into several sections with expandable/collapsible headers. The 'Schematic Region Filter' section is expanded, showing checkboxes for 'Instance' and 'Pin'. The 'Schematic Annotations' section is also expanded, showing radio button options for 'Style' (Marker selected, Shadow unselected), and radio button options for 'Instance', 'Pin', and 'Wire' (all set to 'None'). Below this, there are text input fields for 'Comparison Precision' (0.001), 'M-Factor Names' (m M), and 'Tool Search List' (sdl), followed by a checkbox for 'Verify Total Width Only'. The 'Cloning' section is expanded, showing checked checkboxes for 'Match Parameters' and 'Replace Existing Layout Instances'. The 'Placement' section is expanded, showing checkboxes for 'Split Devices', 'Align Gates' (checked), 'Add Dummy', 'Group Device with Bus Net', 'SED Preview In-Design', and 'Post-process Procedure'. Below these is a 'Resistor Direction' section with radio buttons for 'Auto' (selected), 'Vertical', and 'Horizontal'. The 'Parameter Preset' section has a 'Library' dropdown, a 'Preset' dropdown, and a gear icon. The 'Update Schematic' section has a checkbox for 'Dummy Pre-process Procedure'. At the bottom, there are buttons for 'Help', 'Defaults' (with a dropdown arrow), 'OK', 'Apply', and 'Cancel'.

SDL Options

Main Label

Schematic Region Filter

- ☐ Instance
- ☐ Pin

Schematic Annotations

Style: ☒ Marker ☐ Shadow

Instance: ☒ None ☐ Partial ☐ Detailed

Pin: ☒ None ☐ Partial ☐ Detailed

Wire: ☒ None ☐ Partial ☐ Detailed

Comparison Precision: 0.001 %

M-Factor Names: m M

Tool Search List: sdl

☐ Verify Total Width Only

Cloning

- ☒ Match Parameters
- ☒ Replace Existing Layout Instances

Placement

- ☐ Split Devices
- ☒ Align Gates
- ☐ Add Dummy
- ☐ Group Device with Bus Net
- ☐ SED Preview In-Design
- ☐ Post-process Procedure

Resistor Direction: ☒ Auto ☐ Vertical ☐ Horizontal

Parameter Preset

Library: Preset:

Update Schematic

- ☐ Dummy Pre-process Procedure

Help Defaults OK Apply Cancel

Table 2 describes the options available from the **SDL Options - Main** tab.

Table 2 Main Tab Options

Option	Description
Schematic Region Filter	- Allows you to area-select the specified object types using a click-drag operation in the Schematic Assistant and Schematic Editor. Instance: Click-drag to area-select instances. Tcl preference: <code>lxSchematicInstanceRegionSelection</code> Pin: Click-drag to area-select pins. Tcl preference: <code>lxSchematicPinRegionSelection</code> The Schematic Region Filter is <i>on</i> when either (or both) the Instance or Pin option is <i>on</i>. Region selection applies only to instances and pins. Other objects are not selected. The Schematic Region Filter is <i>off</i> when both Instance and Pin options are off.
Schematic Annotations	Specifies the Style of the annotations: Maker: Displays pink makers over unplaced objects. Shadow: Dims the placed objects. Tcl preference: <code>lxSchematicAnnotationStyle</code> Annotates the specified objects (Instance, Pin, Wire) in the schematic design that have yet to be placed in the layout. None: No markers are displayed. Partial: Displays a marker over unplaced objects. Detailed: Displays the number of placed objects out of the total number of objects. Tcl preferences: <code>lxShowInstanceAnnotation</code>, <code>lxShowPinAnnotation</code>, <code>lxShowWireAnnotation</code>
Comparison Precision	Threshold precision considered for matching the floating-point parameters, length and width. Tcl preference: <code>lxParamPrecision</code>
M-Factor Names	Based on the value of m-factor parameter for an instance in the schematic may create m-copies in the layout, where <i>m</i> is the value of m-factor. Naming convention that is followed while m-factored instances are created. <i>m M</i> signifies a total of <i>m</i> instances of the type <i>M</i>. Tcl preference: <code>lxMFactorNames</code>
Tool Search List	Tools from which details of parameter mapping are taken for comparing the layout and schematic designs. Parameter details are defined under the <code>simInfo</code> section in the Parameter Definition Editor. Tcl preference: <code>lxToolSearchList</code>
Verify Total Width Only	Controls whether the total width of device should be checked when the device with m-factor is expanded or the device with the number of fingers is split. Tcl preference: <code>lxVerifyTotalWidthOnly</code>

Table 2 Main Tab Options (Continued)

Option	Description
Cloning	• Match Parameters: Controls whether a potential clone target should match parameters of the clone source to be qualified as a valid clone target. Tcl preference: <code>lxCloneMatchParams</code> • Replace Existing Layout Instances: If any instance of the clone target has already been placed (but not in the clone source), Auto Clone replaces the existing layout instance when committing the clone target. Tcl preference: <code>lxCloneReplaceExisting</code>
Placement	• Split Devices: Controls whether layout generation automatically splits devices having a number of fingers greater than one when placed in the design. For information about splitting devices interactively, see Splitting Devices. Tcl preference: <code>lxSplit</code> • Align Gates: Controls whether to automatically align PMOS and NMOS gates when picking and placing objects to layout. The default is <i>on</i>. For more information about gates alignment, see Aligning Gates While Placing Objects. Tcl preference: <code>lxAlignGates</code> • Group Device with Bus Net: Controls whether to group vector device as one item on SED device panel if the device contains bus net. For example: <code>I0<0:1></code> has bus net <code>S<0:1></code>. If the option is off, two items are created on SED device panel; if the option is on, only one item <code>I0<0:1></code> is created on SED device panel. The default is <i>off</i>. Tcl preference: <code>lxGroupVectorDevice</code> • Add Dummy: Controls whether to add dummy devices to align gates so they share their OD (oxide definition) areas and achieve an equal number of PMOS or NMOS devices for perfect gate alignment. The default is <i>off</i>. Tcl preference: <code>lxAddAlignDummy</code> • SED Preview In-Design: Symbolic Editor allows you to view the placement in the whole layout design using the Preview assistant. See Using Preview Assistant to View Placement in Layout. The default is <i>off</i>. Tcl preference: <code>lxPreviewInDesign</code> • Post-process Procedure: Controls whether to call a user-specified Tcl procedure after completing the Pick and Place or Generate Layout operation. Tcl preferences: <code>lxUseSDLPostProc</code>, <code>lxSDLPostProcName</code>. • Resistor Direction: Controls the direction for resistors in SED. When this option is <i>Auto</i>, SED determines the direction from the direction parameter <code>lxResDirection</code> in the <code>lxComponentType</code>, and uses vertical direction if the direction parameter is not defined. Otherwise, the resistor direction is decided based on the option setting. The default is <i>Auto</i>. Tcl preference: <code>lxResDirection</code>

Table 2 Main Tab Options (Continued)

Option	Description
Parameter Preset	You can preset the instance parameters in a layout cellview by specifying the following selection fields. If the Library selection field is empty, the Parameter Preset feature is disabled. Click the icon  to customize the <code>lxPresetConfig</code> setting. For more information about Parameter Preset, see Realizing a Layout With Preset Parameters. • Library: Specify the library to preset the layout cellview with the preset instance parameters. Tcl preference: <code>lxPresetLCV</code> • Preset: Specify the cellview with the preset instance parameters.
Update Schematic	Dummy Pre-process Procedure: Enables this option and specifies the procedure name to decide which dummy devices should be back-annotated and custom some parameters for the schematic dummy devices. The following two preferences are used to set this procedure: <code>lxUseUpdateDummyPreProc</code> and <code>lxUpdateDummyPreProcName</code>. If <code>lxUseUpdateDummyPreProc</code> is true, this procedure updates dummy devices which are selected in the Device Mapping table in the Update Schematic from Layout dialog box. This procedure follows the following rules: • The input is an oalnst (layout dummy devices). • The dummy devices are not back-annotated to the schematic when specifying <code>return 0</code> in the procedure. • The dummy devices are back-annotated to the schematic when specifying <code>return 1</code> in the procedure. • You also can change the specific parameters in the schematic when specifying <code>return {1 {{p_1 v_1} {p_2 v_2} ... {p_n v_n}}} </code> in the procedure; <code>p</code> is parameter name and <code>v</code> is parameter value. Note: If parameter <code>p</code> is defined in <code>instParameters</code> of <code>simInfo</code>, this parameter is skipped.

Label Tab

Figure 6 shows the options available from the **SDL Options, Label** tab. See pinTextLayers.

Figure 6 SDL Options, Label Tab

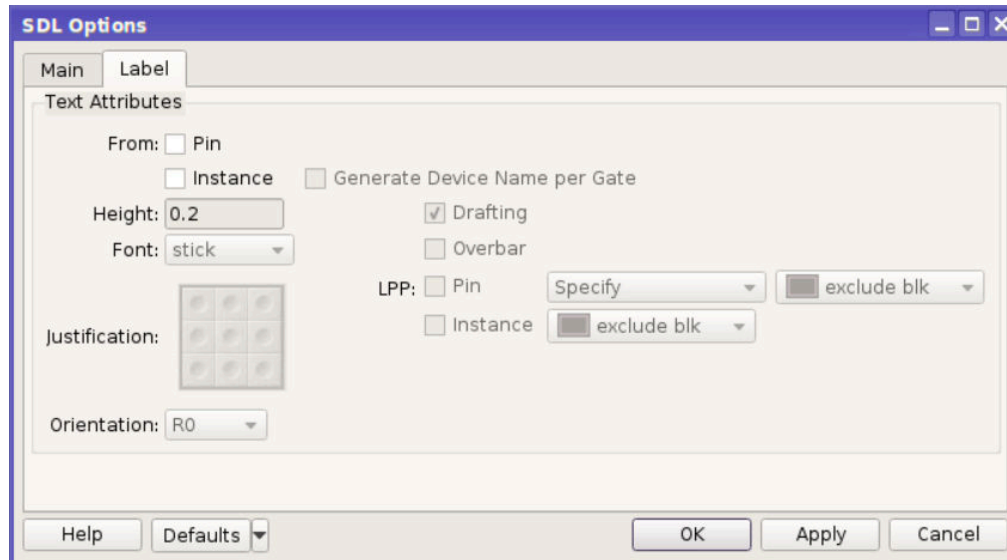


Table 3 lists the options available from the **Label** tab.

Table 3 Label Tab Options

Option	Description
From	Enables Pin or Instance option to create the labels for pins or instances. Tcl preferences: <code>lxCreatePinLabel</code>, <code>lxCreateInstanceNameLabel</code> • Generate Device Name per Gate: Controls whether to generate duplicate labels on each gate or just one label for realized instances. This option is available when the Instance option is enabled. Tcl preference: <code>lxGenerateLabelPerGate</code>.
Height	Specifies the height of the pin labels. Tcl preference: <code>lxTextHeight</code>
Drafting	Specifies how the pin label should be drawn. If you check this option, the pin label is always drawn either left-to-right or bottom. • Left-to-right: If the object orientation is R0, R180, MX, or MY. • Top-to-bottom: If the object orientation is R90, R270, MXR90, or MYR90. If you leave the Drafting option deselected, the pin label is drawn using the orientation set on the object. Tcl preference: <code>lxTextDrafting</code>

Table 3 Label Tab Options (Continued)

Option	Description
Font	Specifies the font to be used for the pin labels. Tcl preference: <code>lxTextFont</code>
Overbar	Specifies overbar characteristics for text, if available. Tcl preference: <code>lxTextOverbar</code>
Justification	Specifies the text justification for the pin labels. The first part of the option refers to the vertical alignment and the second part refers to the horizontal alignment. For example, lowerCenter implies that the pin label is created with a lower vertical alignment and center horizontal alignment. Tcl preference: <code>lxTextJust</code>
LPP: Pin/Instance	Indicates that the pin labels or instance labels should be created on the layer/purpose pair (LPP) of your choice. - When Pin is enabled, creates the label by defining LPP with the following methods: Specify: Specifies layer purpose pair. Technology Library: The associated technology library. Map: Clicks the Browser button to select a map file defined shape and text LPP mapping pair. For example: <pre>##### # Shape LPP Text LPP # ##### m1 drawing m1text drawing m2 drawing m2text drawing m3 drawing m3text drawing</pre> Purpose: Specifies a list of LPP purpose names for pin label. When generating pin labels along with the pin shape, the layer follows the pin shape, and the purpose follows the order of the list of Purpose. Tcl preferences: <code>lxTextInstLPP</code>, <code>lxTextPinLPP</code>, <code>lxTextSpecifyPinLPP</code>, <code>lxTextSpecifyPinLPP</code>, <code>lxTextSpecifyInstLPP</code>, <code>lxTextPinLPPMethod</code>, <code>lxTextPurposeList</code>.
Orientation	Specifies the orientation of the label text. The orientation options are: R0, R90, R180, R270, MX, MY, MXR90, MYR90. This orientation is used only if you leave the Drafting check box deselected. Tcl preference: <code>lxTextOrientation</code>

Feedthrough Net Name Selection Rules

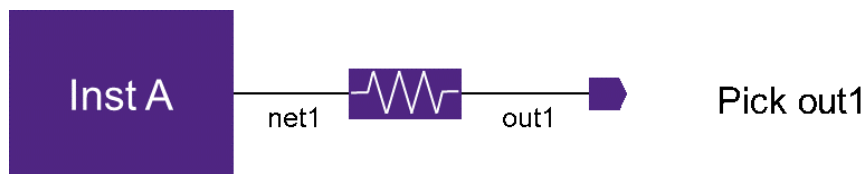
The `lxEquivalentNetMode` preference is used to control what rule to select net names. You can specify one of the following values to choose the net names selecting rule:

After the value is specified, the net names are used according to the individual priority rules.

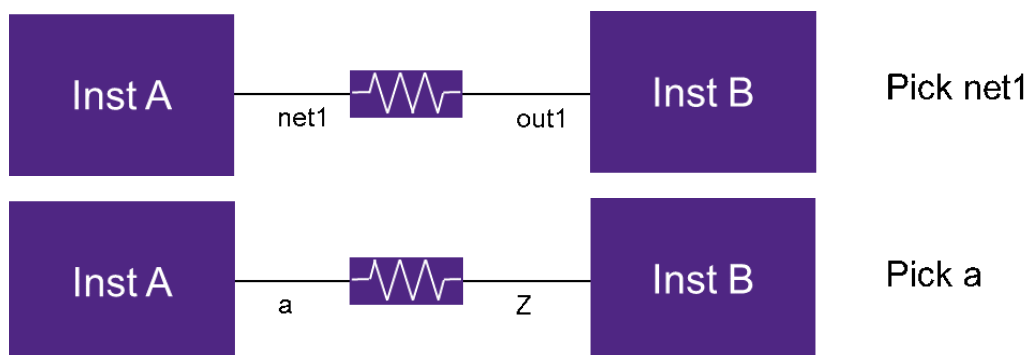
- SDL (This is default):
 1. Least depth (highest in the hierarchy).
 2. Pick the global net.
 3. If both nets are global or are not global, pick the net has connection definition.
 4. If both nets have or don't have connection definition, pick the net connected to a port. (term in Custom Compiler)
 5. If both nets are or are not connected to a port, pick the net with label.
 6. If both nets have or don't have label, pick the preferred equivalent net.
 7. If both nets have or don't have the preferred equivalent flag, pick the net connected to higher priority port. (Input > InputOutput > Output > Other types)
 8. If both nets are or are not connected to same priority port, pick the net in ascending order.
- ICC:
 1. Least depth (highest in the hierarchy).
 2. If both nets are global or are not global, pick the net connected to a port. (term in Custom Compiler)
 3. If both nets are or are not connected to a port, pick the nets with net type in the following order:
 - a. Power/Ground
 - b. Tie
 - c. Signal
 4. If both nets have same type, pick the net with more port or instance pin connections.

5. If both nets have or don't have the preferred equivalent flag, pick the net connected to higher priority port. (Input > InputOutput > Output > Other types)
 6. If both nets are or are not connected to same priority port, pick the net in ascending order.
- CDL:
 1. Least depth (highest in the hierarchy).
 2. If both nets are global or are not global, pick the net has connection definition.
 3. If both nets have or don't have connection definition, pick the net connected to a port. (term in Custom Compiler)
 4. If both nets are or are not connected to a port, pick the nets pick the net with label.
 5. If both nets have or don't have label, pick the preferred equivalent net.
 6. If both nets have or don't have the preferred equivalent flag, pick the net connected to higher priority port. (Input > InputOutput > Output > Other types)
 7. If both nets are or are not connected to same priority port, pick the net with preference `nlCDLShortPreferredOrder` (ascending or descending order).

The example of net connected to a port:



Examples of ascending order:



Expanding M-Factor and S-Factor

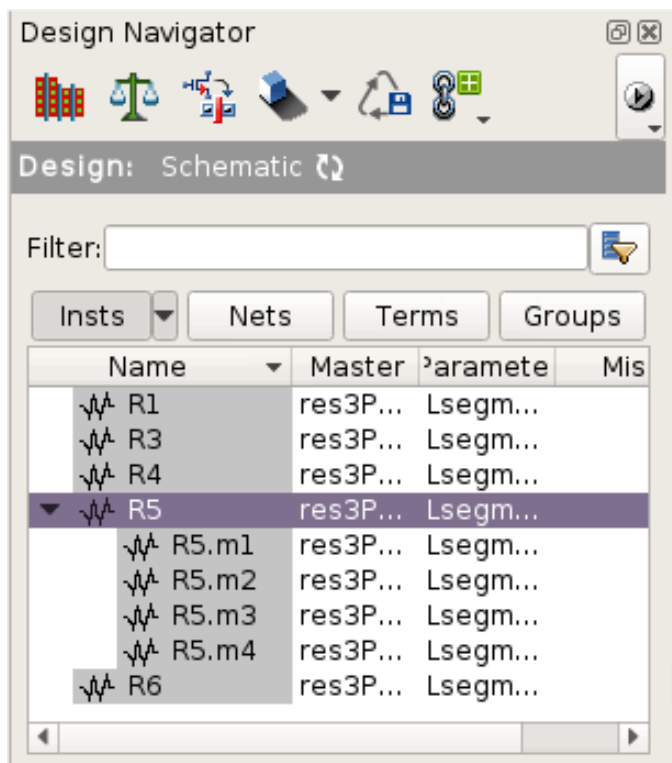
This chapter describes the following sections:

- [M-Factor Expanding](#)
- [S-Factor Expanding](#)
- [M-Factor and S-Factor Expanding](#)

M-Factor Expanding

The `lxExpandMFactor` preference is used to control whether the device with m-factor needs to be expanded or not during SDL elaboration. The default is true.

When the `lxExpandMFactor` is set to true, the device is expanding as shown in the following figure:

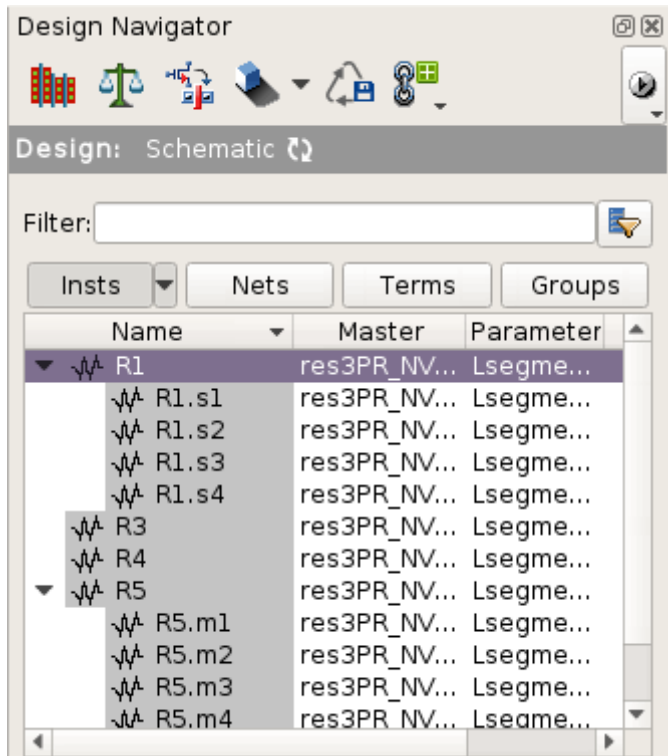


See [M-Factor Names](#) for naming convention.

S-Factor Expanding

The `lxExpandSFactor` preference is used to control whether the resistor device with s-factor needs to be expanded or not during SDL elaboration. The default is false. The expanding result is shown until next elaboration.

When the `lxExpandSFactor` preference is set to true, the device is expanding as shown in the following figure:

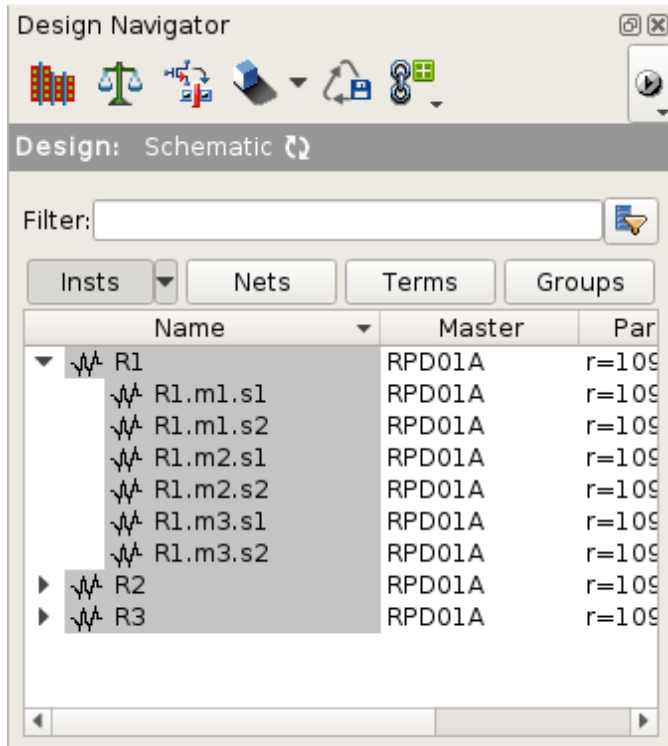


S-factor is only used for resistor devices in series. Based on the value of s-factor parameter for an instance in the schematic may create s-copies in the layout, where s is the value of s-factor.

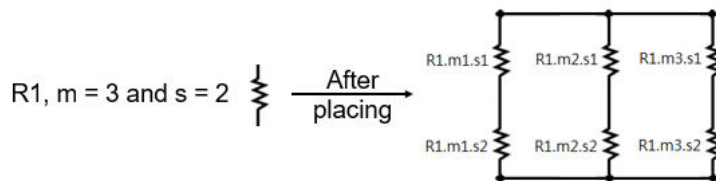
Tcl preference: `lxSFactorNames`

M-Factor and S-Factor Expanding

SDL can expand m-factor and s-factor at the same time. When the `1xExpandMFactor` and `1xExpandSFactor` preferences are set to true, the device is expanding as shown in the following figure:



When a schematic resistor has m-factor and s-factor and both values are bigger than one, this is a series-parallel circuit. To place the resistors, the tool places series first and then parallel them.



3

Browsing the Schematic Using the Design Navigator

This chapter describes how to use the Design Navigator assistant to browse the logic information and the Schematic Assistant as a viewer of logical design.

This chapter describes the following topics:

- [Starting the Design Navigator with Physical and Logic Views](#)
- [Using Toolbar Icons for the Design Navigator in Schematic Mode](#)
- [Using the Design Navigator as an SDL Browser](#)
- [Using the Schematic Assistant](#)

Starting the Design Navigator with Physical and Logic Views

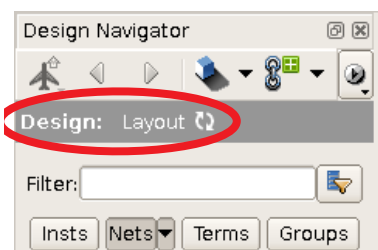
When SDL is initialized from a layout (choose **Tools > SDL** to define the logic data in the **Define Logical Source** dialog box), you can enable the Design Navigator assistant to show the physical and logic information. (See [Initializing Schematic Driven Layout From Schematic](#) for details on how to start the SDL flow.)

To open the Design Navigator assistant, choose **Window > Assistants > Design Navigator**. The physical (Layout mode) and logic (Schematic mode) information can be switched by clicking the **Design: <mode>** button (this button shows you the mode you are currently in). The default is Layout mode.

The assistant docks along the window edge ([Figure 7](#)). Undock it by clicking and dragging it.

The Design Navigator opens by default in Layout mode ([Figure 7](#)). (See the Design Navigator Assistant in the *Custom Compiler™ Environment User Guide* for information about using the Design Navigator Assistant in Layout mode.)

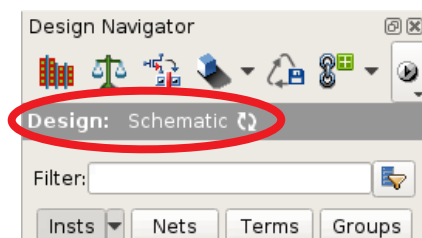
Figure 7 Design Navigator upon Opening, in Layout Mode



Click the **Design: Layout** button (circled in red in [Figure 7](#)) to enable Schematic mode in the Design Navigator assistant.

The Design Navigator switches to Schematic mode. To switch back to Layout mode, click the **Design: Schematic** button (circled in red in [Figure 8](#)).

Figure 8 Design Navigator in Schematic Mode



Using Toolbar Icons for the Design Navigator in Schematic Mode

This section explains how to use the toolbar icons for the Design Navigator in Schematic mode. The assistant consists of the following toolbar icons:

- [Symbolic Editor With Chaining Mode](#)
- [Symbolic Editor With Matching Mode](#)
- [Pick and Place](#)
- [Cross Shadow](#)
- [Create Template](#)
- [Create Placement Constraint](#)

- [Set Flight Line](#)
- [More Options](#)

Symbolic Editor With Chaining Mode

Clicking the **Symbolic Editor with Chaining Mode** icon  opens the Symbolic Editor in chaining mode. You can realize the selected devices in your layout using chaining mode.

For more information about **Symbolic Editor with Chaining Mode**, see About the Symbolic Editor in the *Custom Compiler Symbolic Editor User Guide*.

Symbolic Editor With Matching Mode

Clicking the **Symbolic Editor with Matching Mode** icon  opens the Symbolic Editor in matching mode. You can realize the selected devices in your layout using matching mode.

For more information about **Symbolic Editor with Matching Mode**, see About the Symbolic Editor in the *Custom Compiler Symbolic Editor User Guide*.

Pick and Place

Clicking the **Pick and Place** icon  lets you select individual devices from the Design Navigator and realize them in the layout, according to the mapping information between the schematic and layout.

For more information about **Pick and Place** see [Picking and Placing](#).

Cross Shadow

This command highlights the selected devices and dims the color of unselected objects from the Design Navigator in Schematic mode. A cross-probing capability is provided between the Design Navigator in Schematic mode, the Schematic Editor, and the Layout Editor. Devices, instances, or nets can be highlighted from each window.

For example, when clicking an instance or a net in the Design Navigator in Schematic mode, the corresponding object is highlighted in the Schematic Editor and Layout Editor.

To highlight an object in the Design Navigator in Schematic mode, follow the steps below:

1. Click **Enable Cross Select**  on the toolbar.
2. Click the arrowhead in the **Cross Shadow** toolbar icon  to set the level of cross shadow brightness using the slider. The default is 50% intensity.

Figure 9 Cross Shadow Brightness



3. Select objects in the Design Navigator in Schematic mode. The objects are highlighted in the Design Navigator in Schematic mode, the Schematic Editor, and the Layout Editor.
4. When you click the **Cross Shadow** icon , any unselected objects are dimmed to the level of cross shadow brightness you set in Step 2.

Create Template

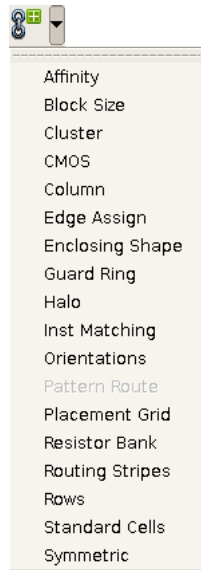
Clicking the **Create Template** icon  displays the **Create Template** dialog box. This feature provides the capability for you to construct your own template library. You can create a new template cell or add views to an existing template cell by selecting a partial layout and then invoking **Create Template**.

For more information, see [Creating Templates](#).

Create Placement Constraint

Clicking the **Create Placement Constraint** icon  displays a menu listing the placement constraints which control the placement of objects.

Figure 10 Create Placement Constraint Menu



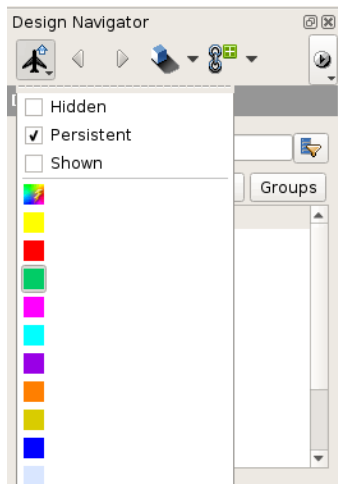
For more information about placement constraints, see [Placement Constraints](#).

For more information about the Placement Assistant, see [Using the Placement Assistant](#).

Set Flight Line

Clicking the **Set Flight Line** icon  when choosing the **Nets** tab displays a menu listing the flight line settings. This option is disabled when you deselect the **Query > Flight Lines > Enable** check box.

Figure 11 Set Flight Line Menu



For more information about setting flight line, see Setting Flight Line Display Options in the *Custom Compiler Environment User Guide*.

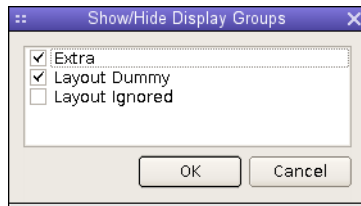
More Options

Clicking the icon  customizes the display logic information shown in the Design Navigator assistant.

The menu includes the following list options:

- **Select All:** Select all logic information.
- **Deselect All:** Deselect all logic information.
- **Expand All:** Expand all logic information.
- **Collapse All:** Collapse logic information.
- **Show/ Hide Display Groups:** Clicking the **Insts** or **Nets** tab enables this option to customize the display groups in the **Show/Hide Display Groups** dialog box. Turning on the options shows the groups in the Design Navigator assistant.

Figure 12 Show/Hide Display Groups



- **Extra:** Displays the objects (instances or nets) created from layout and not matched with the schematic. The default is *on*.
- **Layout Dummy:** Displays the dummy devices which are back annotated from layout to schematic. The default is *on*.
- **Layout Ignored:** Displays the objects created from layout with the ignore property set on. These objects do not match with the schematic. The default is *off*.

Using the Design Navigator as an SDL Browser

This section describes the components of the Design Navigator in Schematic mode and also explains how to use the tabs and menus. The assistant consists of the following components:

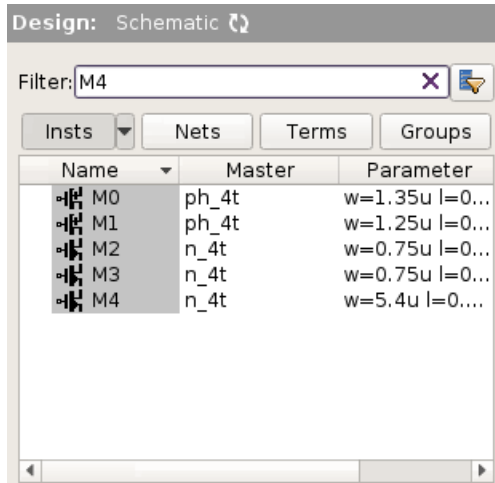
- [Filter](#)
- [Realized and Unrealized Background Color](#)
- [Display and View](#)
- [Instance, Net, and Terminal Icons](#)
- [Insts, Nets, Terms, and Groups Tabs](#)
- [Finding and Displaying Mismatches](#)
- [Design Navigator Shortcut Menus](#)

Filter

This text field is used to search data under the specific tab. The search function is activated whenever you enter any text string in the **Filter** text field. (See the Design Navigator Assistant section in the *Custom Compiler Environment User Guide* for more details.)

For example, enter M4 in the **Filter** text field to find all the matching instances under the **Insts** tab.

Figure 13 Filter Text Field

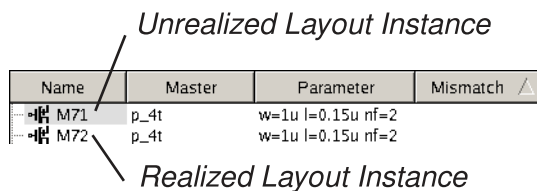


Realized and Unrealized Background Color

When you create a layout from a schematic in SDL, you choose components from the schematic to place in the layout. The schematic components are considered “unrealized” in the layout until you place them.

The Design Navigator assistant shows unrealized layout instances with a dark gray background color in the **Insts** tab, unrealized terminals with a dark gray background color in the **Terms** tab and unrouted nets with a dark gray background color in the **Nets** tab.

Figure 14 Background Color for Realized/Unrealized Layout Instance in the Insts Tab

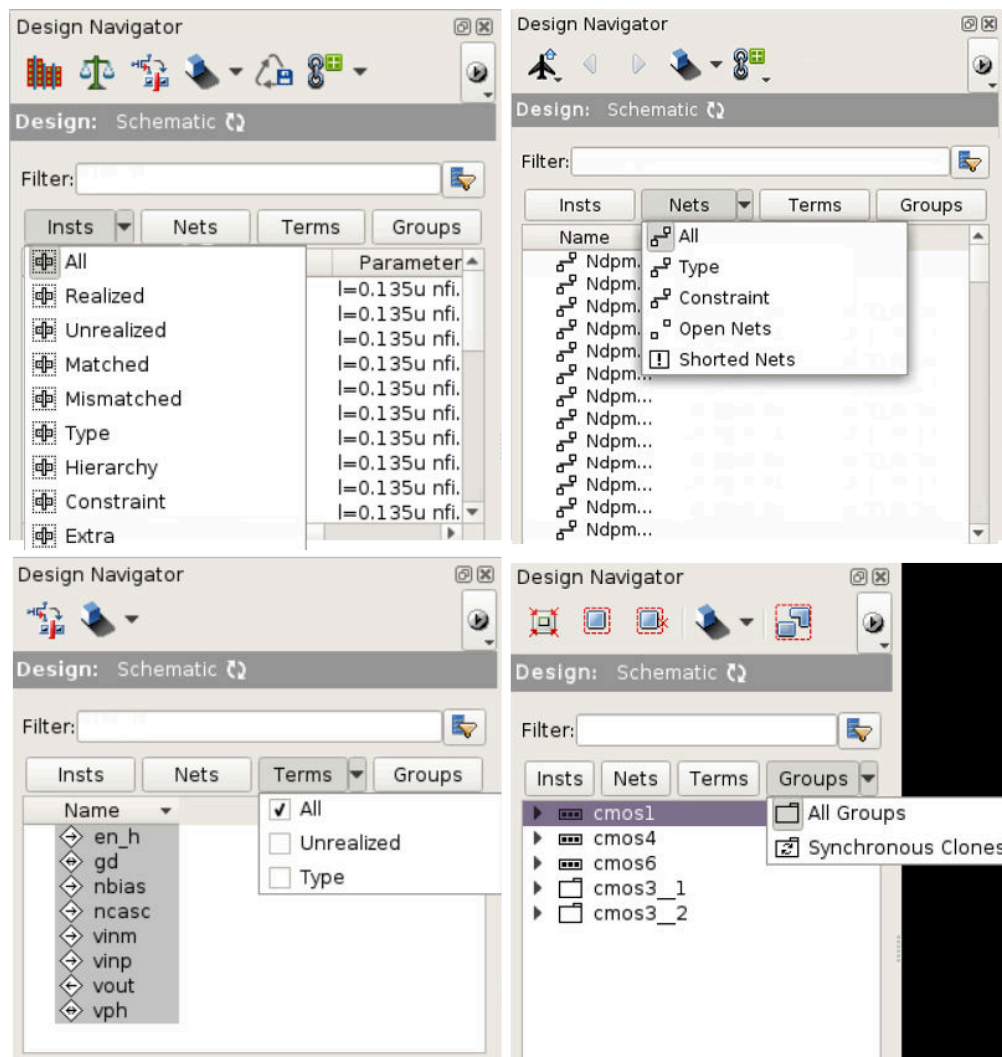


You also can set the preference `lxSdlBrowserColorStatus` to `Realized` to show the realized layout instances, realized terminals, and routed nets with a dark gray background color.

Display and View

The logic information is displayed in the Design Navigator assistant. You can determine what type of objects you are viewing directly by selecting the list option. Click the arrowhead in the **Insts/Nets/Terms/Groups** tab to select the list option from the pull-down menu.

Figure 15 Instance, Net, Term, and Group List Options



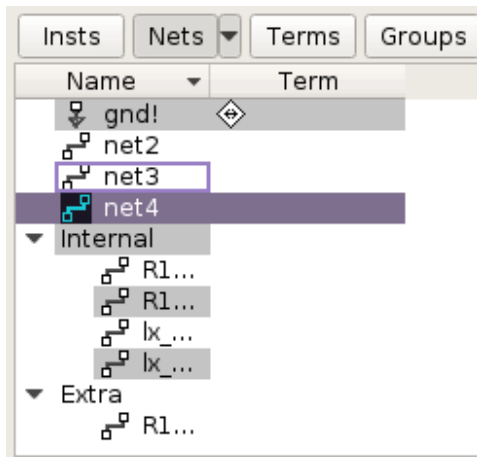
The **Insts** tab menu includes the following list options:

- **All:** Shows all instances.
- **Realized:** Shows the realized instances.

- **Unrealized:** Shows the unrealized instances.
- **Matched:** Shows all instances that appear in both the logic and layout views with identical parameters and other information.
- **Mismatched:** Shows the instances whose parameters or other information is not identical between the schematic and the layout view.
- **Type:** Shows the device types by **Instance**, **PMOS**, **NMOS**, **Capacitor**, **Resistor** and **Stacking MOS**.
- **Hierarchy:** Shows the instances with hierarchy. See [Hierarchy Instances](#).
- **Constraint:** Shows the instances with constraint hierarchy. The constraints transferred from the schematic are in the italic font style.
- **Extra:** Shows the instances that appear in the layout view but not in the logic view. The extra instances cannot be grouped and ignored type. See [Extra Instances](#).

The **Nets** tab menu includes the following list options:

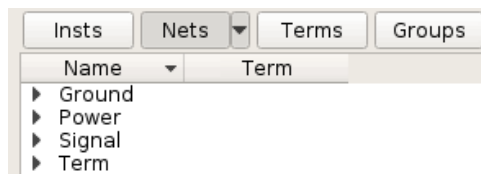
- **All:** Shows all nets in the **Name** column. The **Extra** group shows the nets that appear in the layout view but not in the logic view. The **Internal** group shows the physical nets that connect to the same logic resistor but not in the logic view. The **Term** column shows the direction of connected term.



- **Type:** Shows the net types by **Ground**, **Power**, **Signal**, and **Term**.

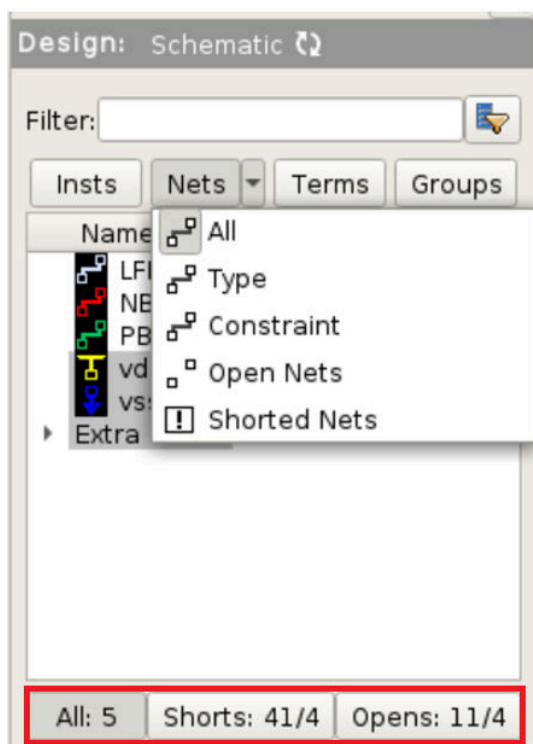
The **Term** type includes **Ground with Terminal**, **Power with Terminal**, and **Signal with Terminal**.

Chapter 3: Browsing the Schematic Using the Design Navigator Using the Design Navigator as an SDL Browser



- **Constraint:** Shows all routing constraints in the layout and displays the constraint hierarchy with leaf instances. The leaf level objects are grouped by **Constraint Type**, **Unconstrained**, **Internal**, and **Extra**.
- **Open Nets:** Shows the open nets. See Viewing Open Nets.
- **Shorted Nets:** Shows the shorted nets. See Viewing Shorted Nets.

You can also switch to **All**, **Open Nets**, or **Shorted Nets** views by clicking the buttons at the bottom of the **Nets** page.



These buttons display the following information:

- The **All** button shows the total count of the nets in the logic view.
- The **Shorts** button shows the shorts count and the number of shorted nets.
- The **Opens** button shows the open count and the number of open nets.

The **Terms** tab menu includes the following list options:

- **All:** Shows all terminals.
- **Unrealized:** Shows the unrealized terminals.
- **Type:** Shows the terminal types by **IO**, **Input**, and **Output**.

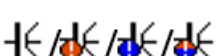
Instance, Net, and Terminal Icons

The Design Navigator assistant shows instances, nets, and terminals with different icons, as listed in [Table 4](#), [Table 5](#), and [Table 6](#), respectively.

Note:

Red exclamation point indicates mismatch.

Table 4 Instance Type Icons

Instance Type ¹²³	Instance Icon	Unplaced Instance Icon
Instance/Array		
PMOS		
NMOS		
Resistor		
Capacitor		
Superconductor		

1. Red exclamation point indicates layout and schematic mismatch.
2. Blue exclamation point indicates schematic mismatch.
3. Half red and half blue exclamation point indicates layout and schematic mismatch.

Table 5 Net Type Icons

Net Type	Net Icon
Net	
Power	
Ground	
Signal with Input Terminal	
Signal with Output Terminal	
Signal with Input/Output Terminal	
Power with Terminal	
Ground with Terminal	

Under the **Nets** type, the **Term** column shows the direction of connected term. See [Table 6](#) for the icons.

Table 6 Terminal Type Icons

Terminal Type	Terminal Icon
Input	
Output	
I/O	

Insts, Nets, Terms, and Groups Tabs

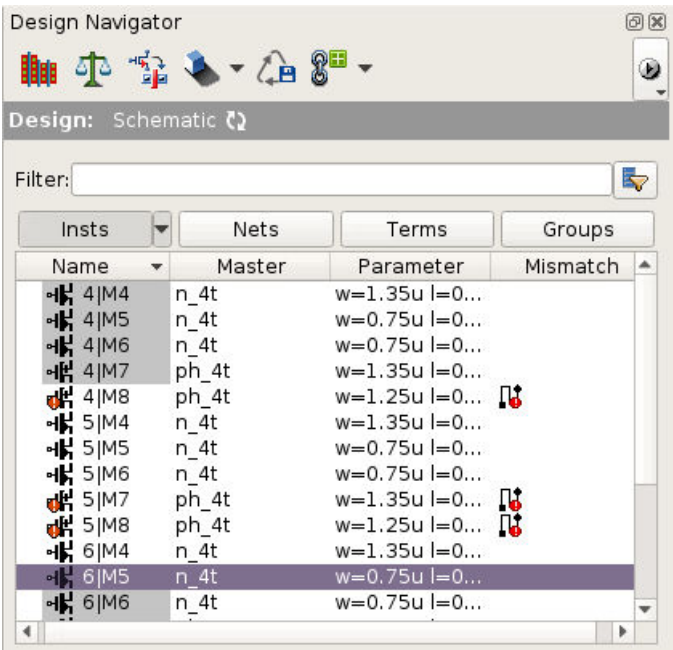
The schematic data corresponding to the layout view from the layout design is shown under the following tabs:

- [Insts Tab](#)
- [Nets Tab](#)
- [Terms Tab](#)
- [Groups Tab](#)

Insts Tab

Click the **Insts** tab on the Design Navigator in Schematic mode to show all instances in the current scope with their corresponding logic information, such as the instance name or instance type, length and width of transistor gates, matched status, and realized status in the layout design.

Figure 16 *Insts Tab*



The **Insts** tab displays the following information:

- **Name:** Shows the instance type icon (see [Table 4](#)) and instance name. The instance type icon warns if the instance has a mismatch. You can sort the instance names in either alphabetical or numerical order by clicking the column heading.
- **Master:** Shows the master cell name of schematic instances. You can set the `lxSdlBrowserMasterName` preference to display the master name with one of the following formats: Lib/Cell/View, Cell/View, or Cell (default). You can sort the master names in either alphabetical or numerical order by clicking the column heading.

Tcl preference: `lxSdlBrowserShowInstMasterColumn`

- **Layout Master:** Shows the switched master of instance. This option is hidden by default. You can set the `lxSdlBrowserMasterName` preference to display the switched master name with one of the following formats: Lib/Cell/View, Cell/View, or Cell (default). You can sort the master names in either alphabetical or numerical order by clicking the column heading.

Tcl preference: `lxSdlBrowserShowInstLayoutMasterColumn`

- **Parameter:** Shows the reserved parameters of instances. You can sort the parameters in either alphabetical or numerical order by clicking the column heading. The reserved parameters of the master instances are specified in the `instParameters` section of `sdl simInfo`.

Tcl preference: `lxSdlBrowserShowInstParameterColumn`

- **Mismatch:** Uses icons to show the discrepancies between the logic and layout views when mismatches of instances are found, including mismatches of leaf instances

in grouped instance nodes. The icons show the Master Mismatch , Parameter

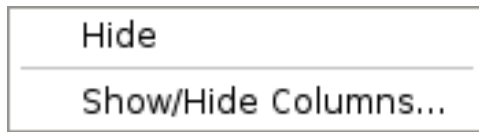
Mismatch , Connection Mismatch , Soft macro Mismatch , and Constraint Mismatch . You can place the cursor over a mismatched instance to display a tooltip containing more information about the mismatch (see [Soft Macro Verification](#)). You can also sort the mismatched instances by clicking the column heading.

If the `instParameters` section of `sdl simInfo` is not defined, all instance parameters are verified and compared. You can set the preference `lxVerifyAllParamsWithoutInstParameters` to control whether the instance parameters are verified.

Tcl preference: `lxSdlBrowserShowInstMismatchColumn`

When you right-click the column headings (**Name**, **Master**, **Layout Master**, **Parameter**, and **Mismatch**), the menu displays.

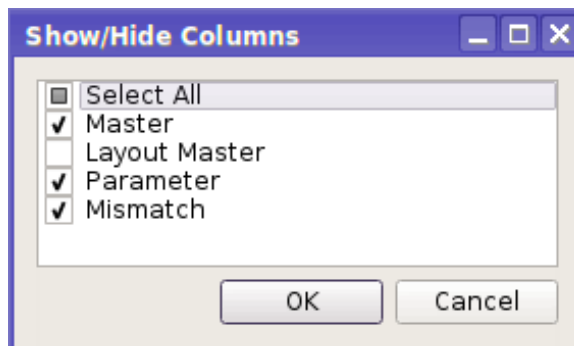
Figure 17 Column Heading Shortcut Menu for Insts



The menu is customized with the following options:

- **Hide:** Right-click a column and then click this option to hide the column.
- **Show/Hide Columns:** Click this option to customize the columns in the **Show/Hide Columns** dialog box. Turn on the **Select All**, **Master**, **Layout Master**, **Parameter**, **Mismatch** options to show the columns.

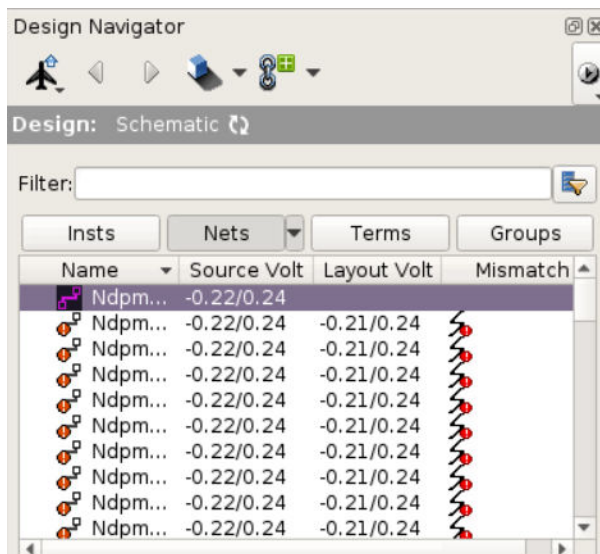
Figure 18 Show/Hide Columns for Insts



Nets Tab

Click the **Nets** tab on the Design Navigator in Schematic mode to show all nets in the current scope with their net types.

Figure 19 Nets Tab



The **Nets** tab displays the following information:

- **Name:** Shows the net type icon (see [Table 5](#)) and net name. You can sort the net names in alphabetical order by clicking the column heading.

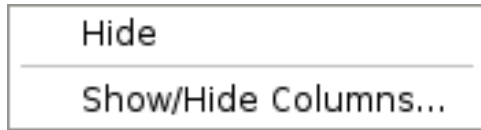
You can annotate the voltage data in the **Net** tab when choosing **Electrical > Voltage> Import Voltages** in the Layout Editor to display the following columns. See *Annotating Schematics With Voltage Data in the Custom Compiler Electrical Checking and Reporting User Guide* for more details.

- **Source Volt:** Shows the minimum and maximum voltage values of the logical net in the column. This option is available when the voltage value is annotated.
- **Layout Volt:** Shows the minimum and maximum voltage values of the layout net in the column when there is a voltage mismatch. This option is available when the voltage value is annotated.
- **Mismatch:** The voltage mismatch icon  is shown when the source type or voltage value is different between the annotated value and the value in layout. To find out more about the mismatch, rest the pointer on the net name. A tooltip displays the original value, the source of the original value, and the override value. See *Verifying Edited Voltage Data in the Custom Compiler Electrical Checking and Reporting User Guide* for more details.

See [Nets Shortcut Menu Commands](#) for shortcut options in the **Nets** tab.

When you right-click the column headings (**Name**, **Source Volt**, **Layout Volt**, and **Mismatch**), the menu displays.

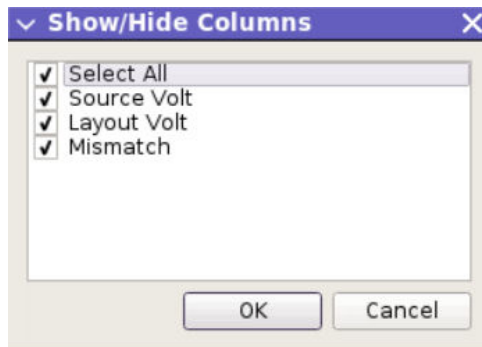
Figure 20 Column Heading Shortcut Menu for Nets



The menu is customized with the following options:

- **Hide:** Right-click a column and then click this option to hide the column.
- **Show/Hide Columns:** Click this option to customize the columns in the **Show/Hide Columns** dialog box. Turn on the options to show the columns.

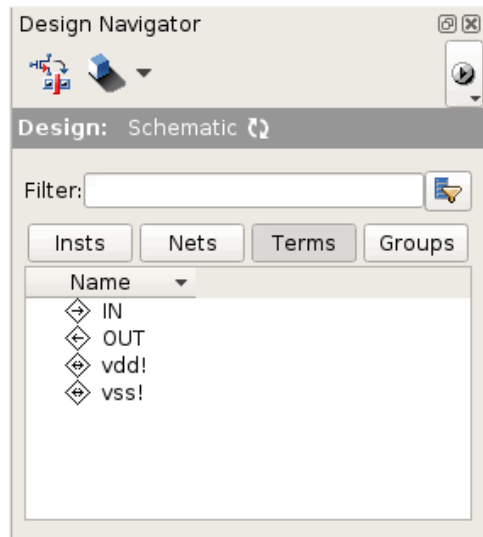
Figure 21 Show/Hide Columns for Nets



Terms Tab

Click the **Terms** tab on the Design Navigator in Schematic mode to show all pins in the current scope with their pin types.

Figure 22 Terms Tab



The **Terms** tab displays the following information:

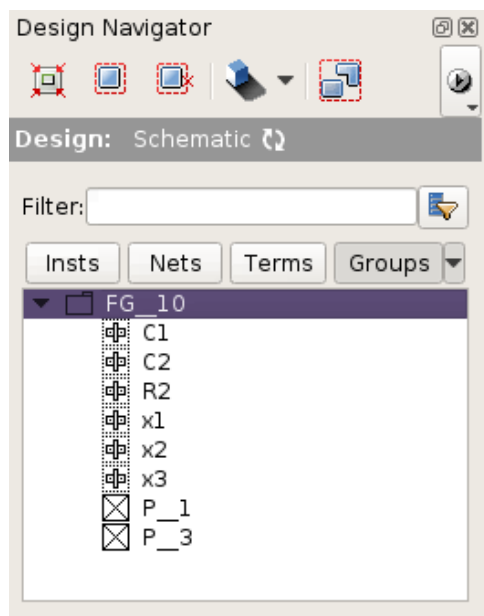
- **Name:** Shows the pin type icon (see [Table 6](#)) and pin name. You can sort the pin names in alphabetical order by clicking the column heading.

You can use the **Generate Pin** command to generate pins to layout for nets that are terminals in the schematic. See *Generating Multiple Pins Simultaneously* in the *Custom Compiler Layout Editor User Guide for Connecting Objects* for more details.

Groups Tab

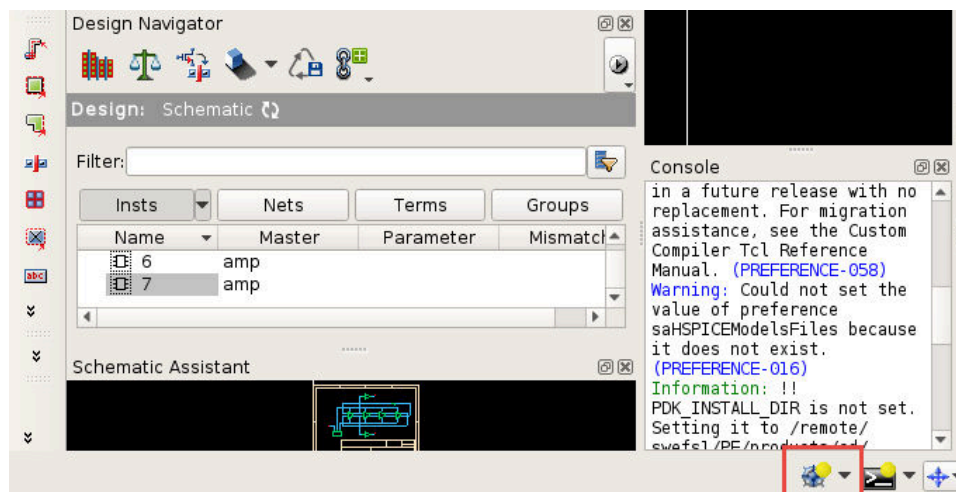
Click the **Groups** tab on the Design Navigator in Schematic mode to show all groups. See [Groups Shortcut Menu Commands](#) for shortcut options in the **Groups** tab.

Figure 23 Groups Tab



Finding and Displaying Mismatches

A notification button **SDL Warnings** in the layout window shows the mismatch status.



The mismatch status includes Warning and Error.

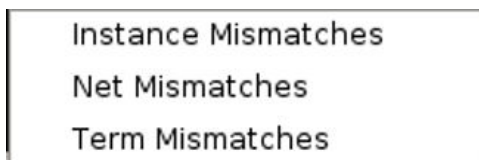
- Warning includes the following rules:
 - Unrealized instances, nets, or terms
 - Extra instances, nets, or terms
 - Uncompleted Nets
- Error includes the following rules:
 - Master mismatch
 - Connection mismatch
 - Parameter mismatch

The **SDL Warnings** button uses the following colors to display the mismatch status:

Icon	Status
	No mismatch
	Warning
	Error

To find the mismatches:

1. Click the button  to navigate to the SDL Browser to find the mismatches.
2. Or click the arrowhead next to the button to specify one of the following mismatches types.

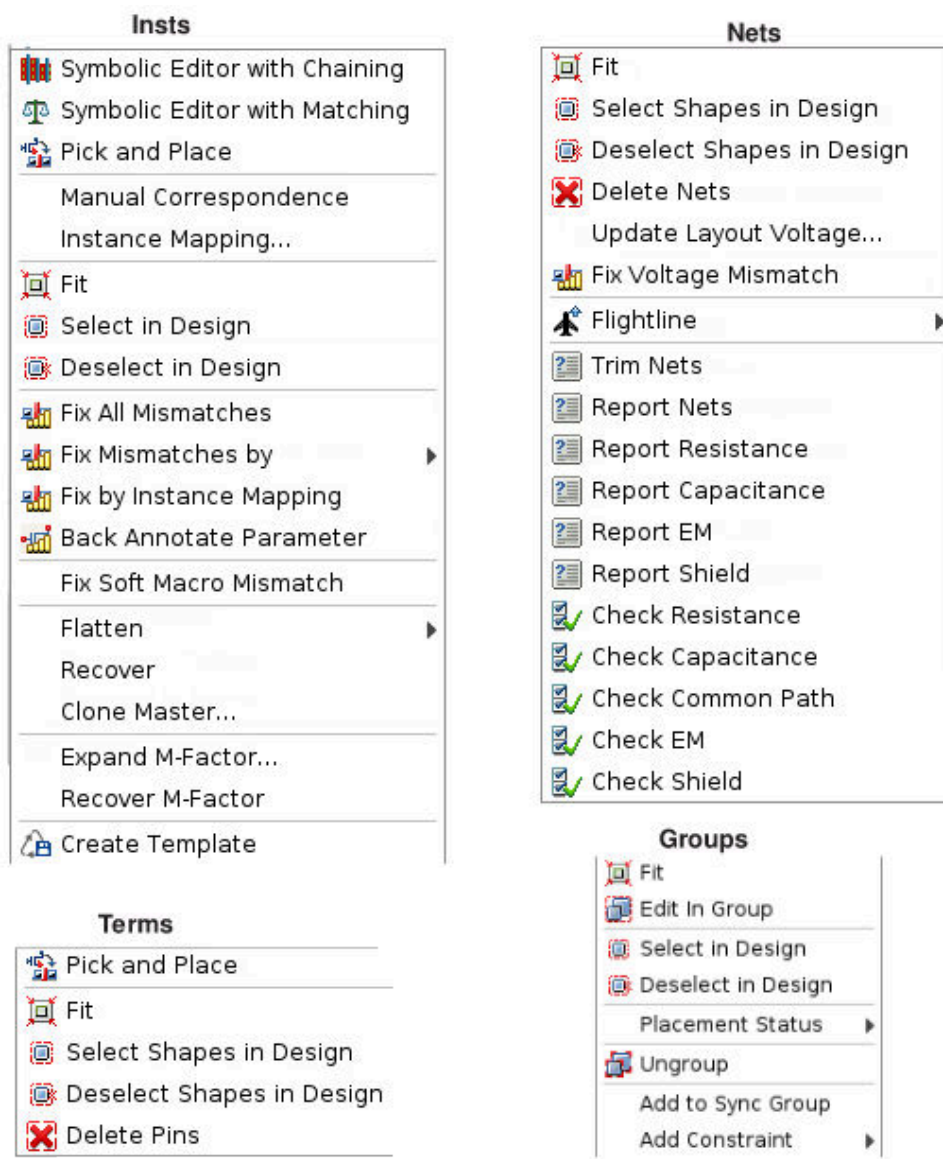


The selected type of mismatches are displayed in the SDL Browser.

Design Navigator Shortcut Menus

Right-clicking objects displayed under the **Insts**, **Nets**, **Terms**, or **Groups** tabs in the Design Navigator in Schematic mode shows a menu (Figure 24), which includes the following commands.

Figure 24 Shortcut Menus for the Design Navigator in Schematic Mode



This section describes the following topics:

- [Instances Shortcut Menu Commands](#)
- [Layout Ignored Instances](#)
- [Hierarchy Instances](#)
- [Extra Instances](#)
- [Nets Shortcut Menu Commands](#)
- [Terms Shortcut Menu Commands](#)
- [Groups Shortcut Menu Commands](#)

Instances Shortcut Menu Commands

The following topics describe menu commands for instances. Some commands appear only under appropriate design circumstances.

- [Symbolic Editor with Chaining](#)
- [Symbolic Editor with Matching](#)
- [Pick and Place](#)
- [Manual Correspondence](#)
- [Instance Mapping](#)
- [Fit](#)
- [Select in Design](#)
- [Deselect in Design](#)
- [Select Shapes in Design](#)
- [Deselect Shapes in Design](#)
- [Fix All Mismatches](#)
- [Fix Mismatches by](#)
- [Fix by Instance Mapping](#)
- [Back Annotate Parameter](#)
- [Fix Soft Macro Mismatch](#)
- [Flatten](#)
- [Recover](#)

- [Clone Master](#)
- [Expand M-factor](#)
- [Recover M-factor](#)
- [Create Template](#)

Symbolic Editor with Chaining

Right-click unrealized instances under the **Insts** tab to invoke the Symbolic Editor in chaining mode. (See About the Symbolic Editor in the *Custom Compiler Symbolic Editor User Guide*.)

Symbolic Editor with Matching

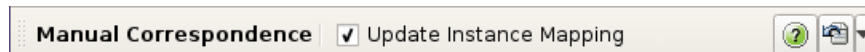
Right-click unrealized instances under the **Insts** tab to invoke the Symbolic Editor in matching mode. (See About the Symbolic Editor in the *Custom Compiler Symbolic Editor User Guide*.)

Pick and Place

Right-click an object under the **Insts** or **Terms** tabs to realize the device in the layout design according to the schematic and layout mapping information. (See [Picking and Placing](#) for more details.)

Manual Correspondence

Right-click objects under the **Insts** tabs to set the corresponding logical instance and physical instances in the **Manual Correspondence** dialog box.

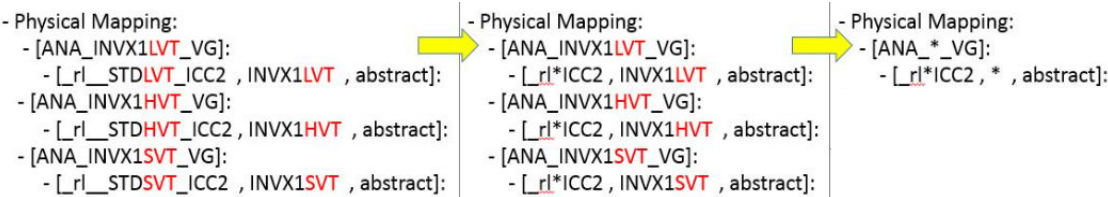


Option	Description
Update Instance Mapping	Selects this option to update the instance mapping when the correspondence settings are different between the physical instance master and logical instance master.

Instance Mapping

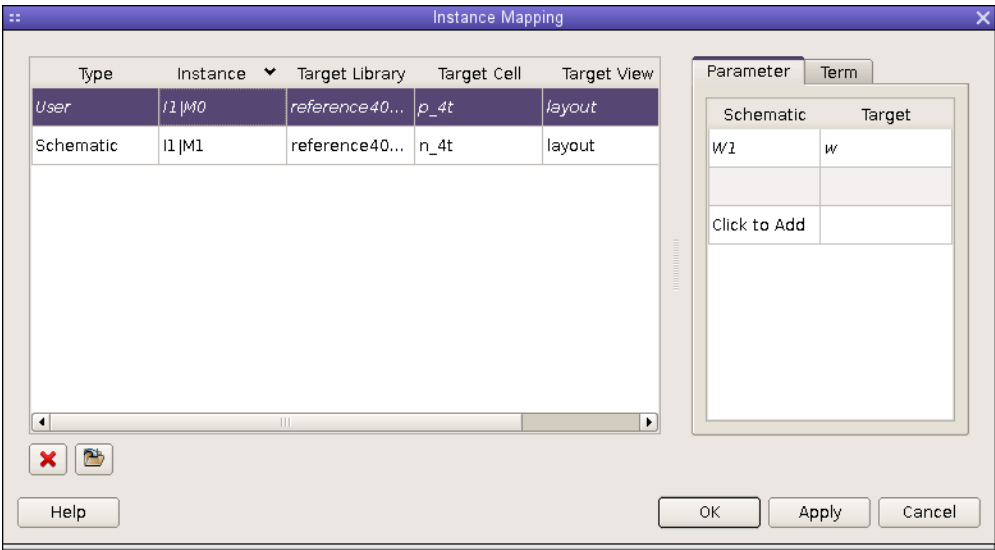
Right-click objects under the **Insts** tabs to show the current instance mapping rules for the selected instances in the **Instance Mapping** dialog box. You can change the mapping rules in the dialog box by double-clicking the text field in the table or clicking the icon . The changed rules are in *italics* before applying the changed rules.

Library name and cell name support wildcards. The following example creates model map file using wildcards.



You can also change the mapping rules from **SDL > SDL Mapping**. For more information about SDL Mapping, see [Adding and Editing SDL Mapping Rules](#). The following figure and table describe the **Instance Mapping** dialog box.

Figure 25 Instance Mapping



Option	Description
Type	Display the source of the mapping rules: • User: Use the instance mapping rules. • SDL Map: Use on the SDL mapping rules. • Schematic: Use the config mapping rules.
Instance	Display the selected instance names to be mapped.
Target Library	Lets you select the target library to which you want to map this instance.
Target Cell	Lets you select the target cell to which you want to map this instance.
Target View	Lets you select the target view to which you want to map this instance.

Option	Description
Parameter	In the Schematic column, display or specify the selected instance parameters to be mapped. In the Target column, display or specify the target parameters to which you want to map this instance.
Term	In the Schematic column, display or specify the selected instance terminals to be mapped. In the Target column, display or specify the target terminals to which you want to map this instance. Specifies the schematic or target terminal in the Ignored Terms tab to ignore the mismatch checking.
Reset Selected Instance Mapping (s)	Click the icon  to reset the mapping rules to the original mapping rules. Any rules you changed are now crossed out after clicking the icon.
Changed Selected Instance Mapping (s)	Click the icon  to change the instance mapping rules in the Select lib/cell/view dialog box.

If the source instance is a vector instance, each vector bit can be mapped to different layout view. All vector instance bits can be expanded or collapsed in the **Instance Mapping** table by clicking + or -.

Type	Instance	Target Library	Target Cell	Target View
+ Schematic	N0<0:9>	devices	nfin	layout
- Schematic	N0(0)	devices	nfin	layout
Schematic	N0(1)	devices	nfin	layout
Schematic	N0(2)	devices	nfin	layout
Schematic	N0(3)	devices	nfin	layout
Schematic	N0(4)	devices	nfin	layout
Schematic	N0(5)	devices	nfin	layout
Schematic	N0(6)	devices	nfin	layout
Schematic	N0(7)	devices	nfin	layout

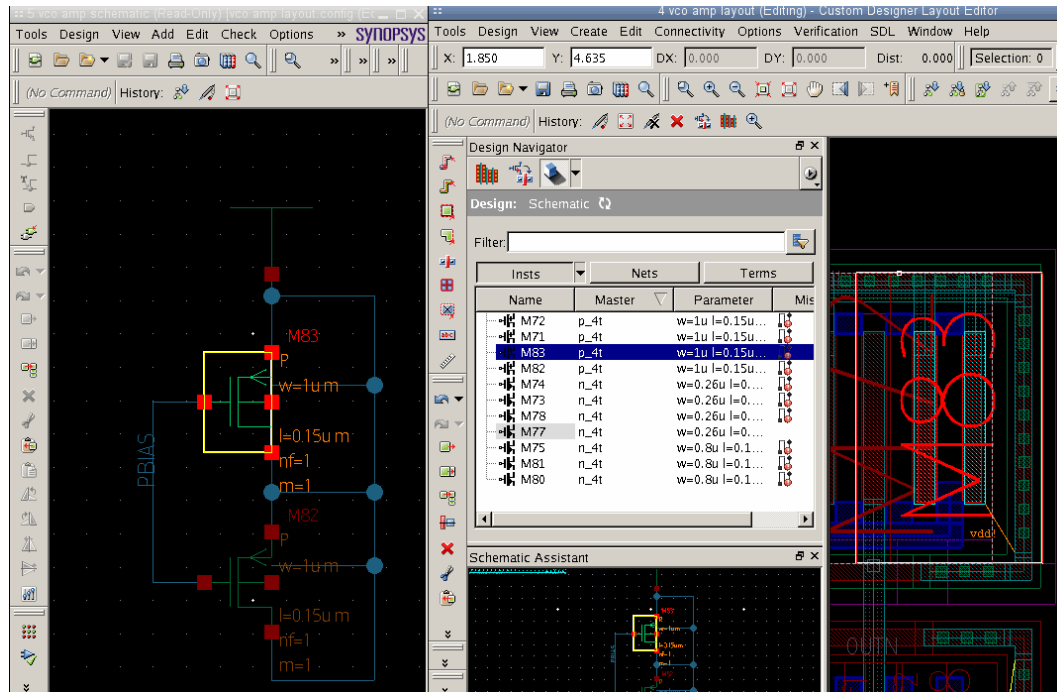
If there are different values for its vector bits, the columns in the **Instance Mapping** table display **As is**.

Type	Instance	Target Library	Target Cell	Target View
- As is	I1<2:0>	TEST	As is	layout
User	I1(0)	TEST	inv3	layout
User	I1(1)	TEST	inv3	layout
Schematic	I1(2)	TEST	inv	layout

Fit

Right-click an object under the **Insts**, **Nets**, **Terms**, or **Groups** tabs to zoom the corresponding object to fit each window. Devices, instances, or nets can be located from each window.

Figure 26 Zoom Instance M83 to Fit Schematic Editor, Layout Editor, and Schematic Assistant



Select in Design

Right-click an object under the **Insts** or **Groups** tabs to select this instance in the design.

Deselect in Design

Right-click an object under the **Insts** or **Groups** tabs to deselect this instance in the design.

Select Shapes in Design

Right-click an object under the **Nets** or **Terms** tabs to select these net or terminal shapes in the design.

Deselect Shapes in Design

Right-click an object under the **Nets** or **Terms** tabs to deselect these net or terminal shapes in the design.

Fix All Mismatches

Right-click mismatched instances under the **Insts** tab to fix all type of mismatches using SDL ECO. (See [Schematic Driven Layout Engineering Change Orders](#) for more details.)

Fix Mismatches by

Right-click mismatched instances under the **Insts** tab to fix the selected type of mismatches individually using SDL ECO.

Tcl preference: `lxBrowserFixConnectivityMismatch`

- **Connectivity:** Fixes connectivity mismatches from logic selection.
- **Master:** Fixes master mismatches from logic selection.
- **Parameter:** Fixes parameter mismatches from logic selection.

Fix by Instance Mapping

Right-click instances with master mismatch under the **Insts** tab to automatically change the masters matched to layout masters.

Instances with multiple bits such as hierarchy vector, stacking MOS, and m-factor are skipped.

Back Annotate Parameter

The parameter back annotation copies all instance parameters from layout to schematic.

There are some conditions show the warning message without updating:

1. The correspondence between schematic to layout is one-to-many.
2. The schematic instance is not the top schematic design.

Fix Soft Macro Mismatch

Under the **Insts** tab, right-click a soft macro instance that has an effective parameter mismatch. To fix the mismatch, the tool looks for a reuse candidate in the reuse table for the same schematic master, but with a new effective parameter set. If a reuse cell exists, then it is bound to the instance. (See [Fix Soft Macro Mismatch](#) for more details.)

Flatten

Under the **Insts** tab, right-click a leaf cell and select one of the following options from the pull-down menu. (See [Flatten](#) for more details.)

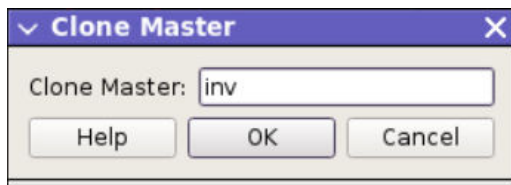
- **One Level:** Replace a binding to an existing layout with a binding to a layout cellview which is a layout.config pointed to by the layout.
- **All Levels:** Recursively flatten the selected leaf cell until there are no more layout cellviews to flatten in the hierarchy.
- **To Level:** Recursively flatten the selected leaf cell to the specified hierarchy level.

Recover

Under the **Insts** tab, right-click leaf cells to recover (that is, un-Flatten) the parent or right-click intermediate nodes to recover that node itself. (See [Recover](#) for more details.)

Clone Master

Under the **Insts** tab, right-click leaf instances to identical copy the instance master with another name using the **Clone Master** dialog box.



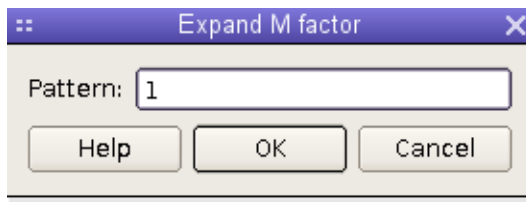
The new copy includes the original attributes and existed layout or layout.config view. If the instance has already been realized in the layout, the master can be replaced by the new copy.

- **Clone Master:** Specifies new cell name of copied.

Expand M-factor

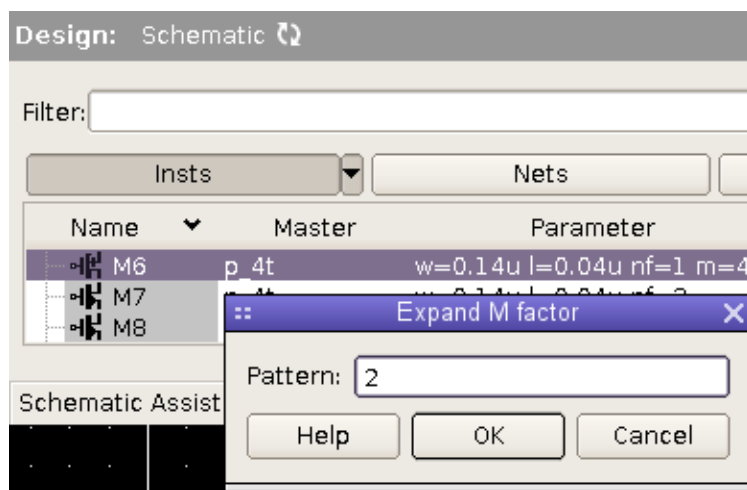
Under the **Insts** tab, right-click unrealized leaf instances to expand an m-factor parent node. This command is available when the preference `lxExpandMFactor` is false. You can specify the expanded pattern in the **Expand M-factor** dialog box. The default of **Pattern** is 1.

Figure 27 Expand M-factor

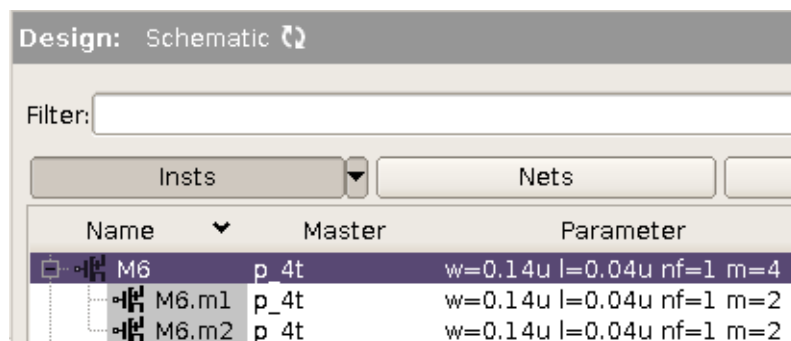


For example, if you want to expand the m-factor parent node M6 from m=4 to m=2, you can set **Pattern** to 2.

Chapter 3: Browsing the Schematic Using the Design Navigator Using the Design Navigator as an SDL Browser



You can see two leaf instances with m-factor m=2 (M6.m1 and M6.m2).



If **Pattern** is empty, the default of 1 is used, which results in four leaf instances with m-factor m=1 (M6.m1, M6.m2, M6.m3, and M6.m4).

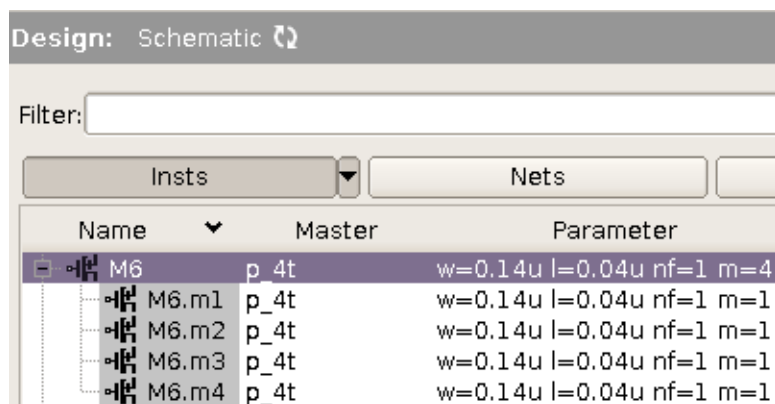


Figure 28 shows more examples.

Figure 28 Example of M1 With M-factor m=4

Original Status	M1m = 4				
Settings	Empty value	1	1 2 1	2	6
Results	M1.m1 M1.m2 M1.m3 M1.m4	M1.m1 M1.m2 M1.m3 M1.m4	M1.m1 M1.m2 M1.m3	M1.m1 M1.m2	M1
M-Factor Values	All m = 1	All m = 1	M1.m1 = 1 M1.m2 = 2 M1.m3 = 1	M1.m1 = 2 M1.m2 = 2	M1 = 4

Recover M-factor

Under the **Insts** tab, right-click leaf instances to collapse m-factor leaf instances that belong to the same parent node. This command is available when the preference `lxEExpandMFactor` is false.

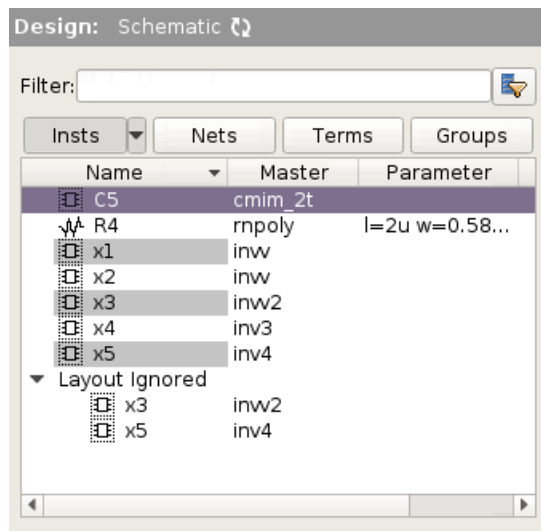
Create Template

Right-click unrealized instances under the **Insts** tab to invoke the **Create Template** dialog box. (See [Creating Templates](#) for more details.)

Layout Ignored Instances

Design Navigator can display instances in the schematic that are ignored for layout placement. You must enable this display by setting a Design Navigator option, as shown in [Figure 12](#).

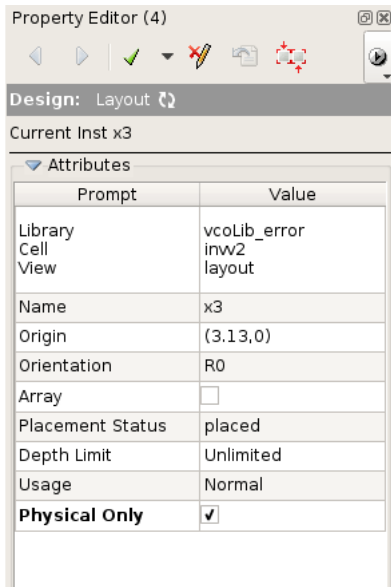
An example of ignored instances in Design Navigator is as follows:



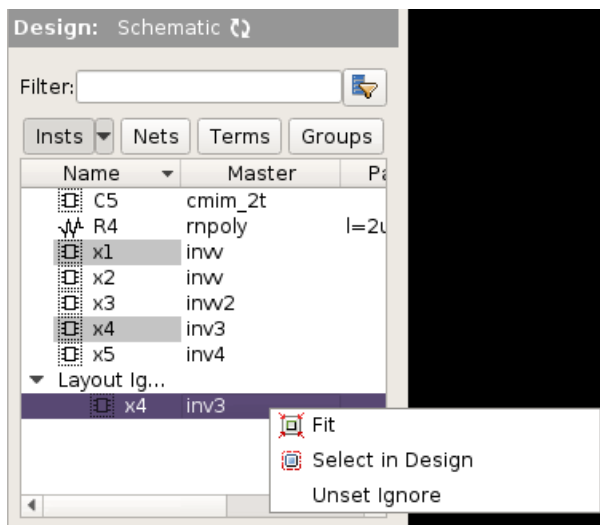
An instance might be ignored for any of the following reasons:

- An oaBooleanProp is specified on the instance or the master
- The instance name is found in the lxlgnoreNames preference: nlAction, lvsIgnore, lxlgnore
- The instance cell type is selected as an ignored cell type in the **ECO Setup** dialog box or by ECO preferences. For more information, see [Ignoring Cells During Verification](#).
- The Physical Only attribute is set for the instance. Physical-only instances are ignored by default.

An example of the **Property Editor** dialog box with the Physical Only attribute is as follows:



The menu for a Layout Ignored instance is limited to a few commands:



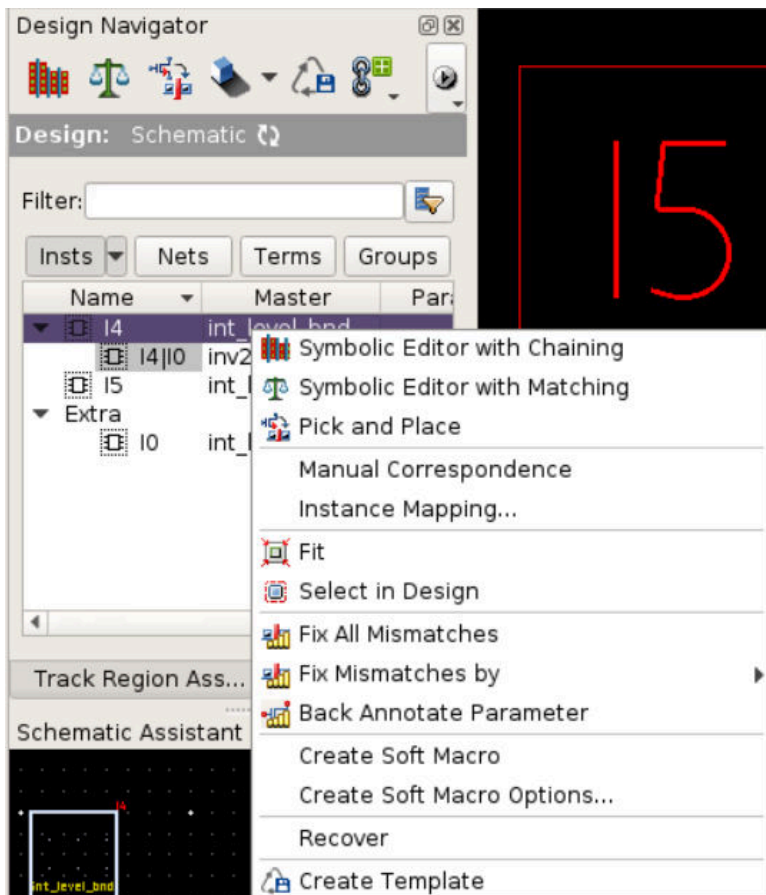
You can use the **Unset Ignore** command to change the status of a physical-only instance. However, if you specify cell types to ignore in SDL ECO, you cannot use the Unset Ignore right-click option in Design Navigator to stop ignoring instances of the specified cell types. You must modify the selection in SDL ECO.

Tcl preference: `lxUnsetIgnore`.

Hierarchy Instances

The Hierarchy model in the **Insts** tab (**Insts** > **Hierarchy**) shows the instances with hierarchy. The node name only displays the name without full path of hierarchy because this model includes hierarchy in the tree. All leaf instances show the full name with hierarchy path. The is an example of Hierarchy model, and the intermediate node also displays the master name for users.

An example of hierarchy instances in Design Navigator is as follows:



The hierarchy mode supports the following specific groups:

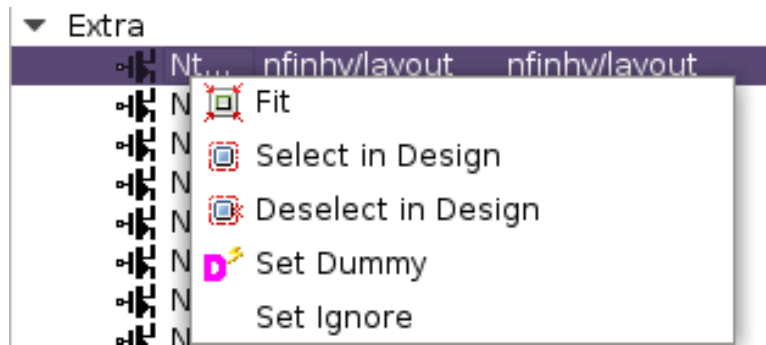
- Extra
- Unbound
- Layout Ignored
- Layout Dummy

See [Instances Shortcut Menu Commands](#) for the menu commands descriptions. For **Create Soft Marco** and **Create Soft Marco Options**, see [Soft Macro Creation](#).

Extra Instances

Design Navigator can display the instances that appear in the layout view but not in the logic view. You must enable this display by setting a Design Navigator option, as shown in [Figure 12](#).

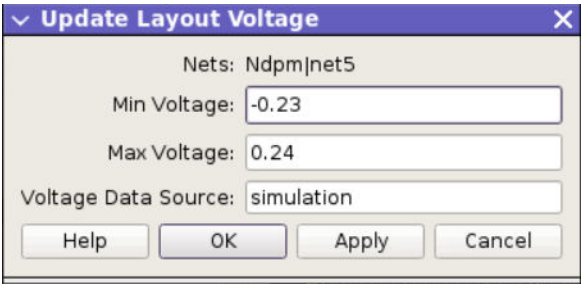
An example of extra instances in Design Navigator and the menu for an extra instance is limited to a few commands:



Button	Tooltip	Action
	Fit	See Fit section.
	Select in Design	See Select in Design section.
	Deselect in Design	See Deselect in Design section.
	Set Ignore	Sets the extra instances to ignore and then the instances are Layout Ignored instances. See Layout Ignored Instances . Tcl preference: <code>lxSetIgnore</code> .
	Set Dummy	Sets the extra instances to dummy devices. Tcl preference: <code>lxSetDummy</code> .

Nets Shortcut Menu Commands

Under the **Nets** tab, right-click on objects to display the following commands:

Button	Tooltip	Action
	Fit	See Fit section.
	Select Shapes in Design	See Select Shapes in Design section.
	Deselect Shapes in Design	See Deselect Shapes in Design section.
	Delete Nets	Deletes nets on the selected nets. Tcl Preference: <code>lxBrowserDeleteNets</code>
	Update Layout Voltage	This option is available when choosing Electrical > Voltage > Import Voltages in the Layout Editor. You can click this option to modify the minimum and maximum voltages of layout nets in the Update Layout Voltage dialog box. 
	Fix Voltage Mismatch	This option is available when choosing Electrical > Voltage > Import Voltages in the Layout Editor. You can click this option to fix the voltage mismatch when the net has mismatches.
	Flight Line	See Set Flight Line section.
	Trim Nets, Report Nets, Report Resistance, Report Capacitance, Report EM, Report Shield	See Design Navigator Nets Shortcut Menu in the Manipulating Components in the <i>Custom Compiler™ Environment User Guide</i> .
	Check Resistance, Check EM, Check Shield	See Design Navigator Nets Shortcut Menu in the Manipulating Components in the <i>Custom Compiler™ Environment User Guide</i> .

Terms Shortcut Menu Commands

Under the **Terms** tab, right-click on objects to display the following commands:

Button	Tooltip	Action
	Pick and Place	See Picking and Placing for more details.
	Fit	See Fit section.
	Select Shapes in Design	See Select Shapes in Design section.
	Deselect Shapes in Design	See Deselect Shapes in Design section.
	Delete Pins	Deletes pins on the selected terminals. Tcl preference: <code>lxBrowserDeletePins</code> .
	Set Ignore	Sets the extra terminals to ignore and then the terminals are added the ignored property with value true. ▼ Extra ◊ ▼ Layout ◊  Pick and Place  Fit  Select Shapes in Design  Deselect Shapes in Design  Delete Pins
	Unset Ignore	Unsets the layout ignored terminals and then the terminals are removed ignored property. ▼ Extra ◊ bb ▼ Layout Ig... ◊  Fit  Select Shapes in Design  Deselect Shapes in Design  Delete Pins Unset Ignore

Groups Shortcut Menu Commands

Under the **Groups** tab, right-click on objects to display the following commands:

Button	Tooltip	Action
	Fit	See Fit section.
	Edit In Group	Activates, and deactivates, the edit-in-group feature, which descends into the selected group for editing in place.
	Select In Design	See Select in Design section.
	Deselect In Design	Deselects currently selected groups in the open design.
	Placement Status	Shows, and allows assignment of, placement status for this instance: Normal , Fixed , or Locked . Where placement status varies among a set of selected instances, the tool does not indicate a status.
	Ungroup	Deletes the selected group or groups. Objects in the group remain in the design.
	Add to Sync Group	Adds selected groups to an existing or new synchronous group. If a figure group already belongs in a synchronous group, the command moves the figure group from the current synchronous group to the newly designated synchronous group.
	Add Constraint	Adds the constraint to the selected groups.

Using the Schematic Assistant

You can use the Schematic Assistant to view the logic information for a layout.

This section describes the following topics:

- [Starting the Schematic Assistant](#)
- [Schematic Assistant Shortcut Menus](#)

Starting the Schematic Assistant

When SDL is initialized from a layout (choose **Tools > SDL** to define the logic data in the **Define Logical Source** dialog box), you can enable the Schematic Assistant to show the schematic. (See [Initializing Schematic Driven Layout From Schematic](#) for details on how to start the SDL flow.)

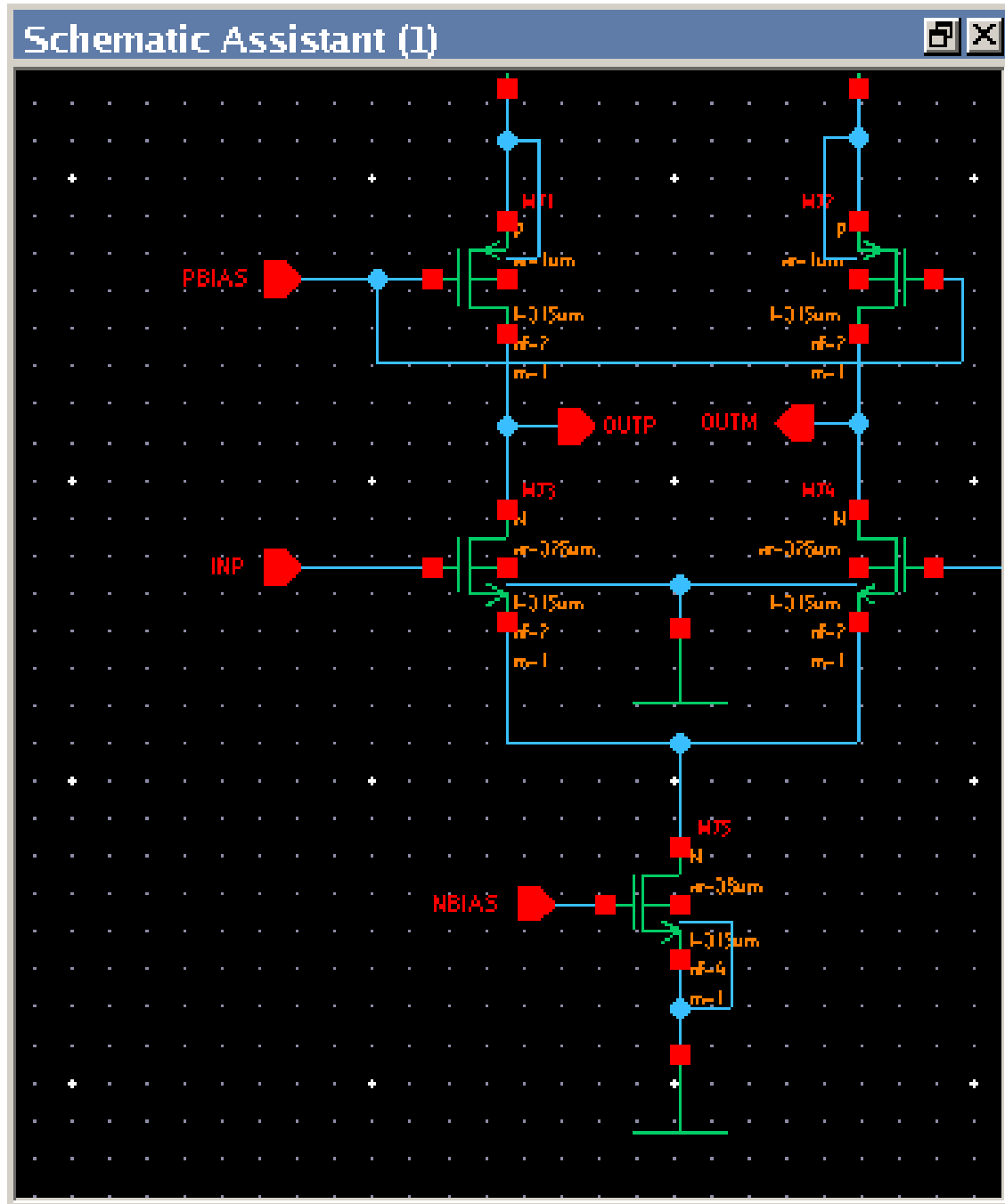
Note:

The Schematic Assistant is a schematic viewer. To edit the schematic, use the Schematic Editor. (See About the Schematic Editor in the Custom Compiler Schematic Editor User Guide for details on how to use the Schematic Editor.)

The Trace Net result in Schematic Editor is displayed in Schematic Assistant in Layout canvas in SDL mode.

To open the Schematic Assistant, choose **Window > Assistants > Schematic Assistant**.

Figure 29 Schematic Assistant



Schematic Assistant Shortcut Menus

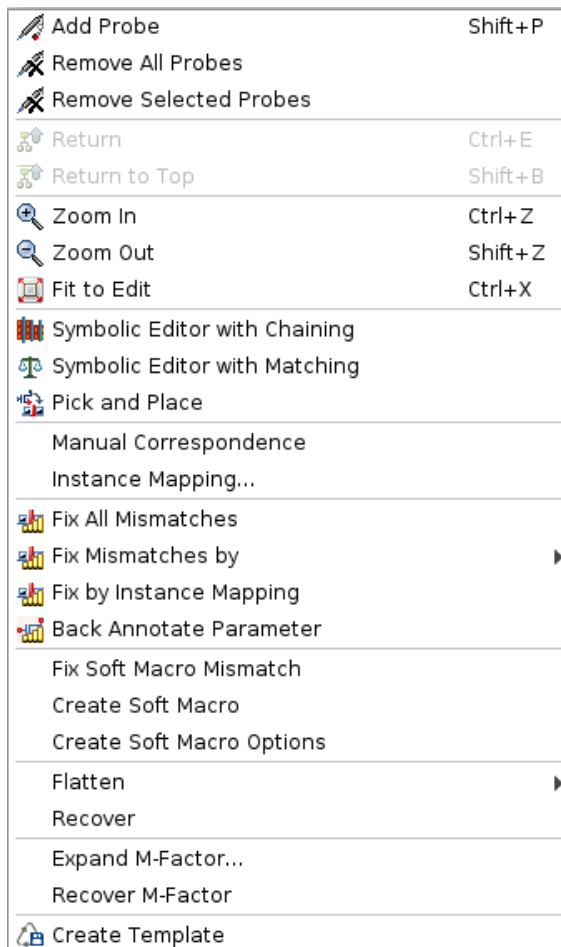
You can right-click items in the Schematic Assistant to display any of the following menus:

- [Shortcut Menu on Canvas](#)
- [Shortcut Menu on Instances](#)
- [Shortcut Menu on Nets](#)
- [Shortcut Menu on Pins](#)

Shortcut Menu on Canvas

Right-clicking the canvas in the Schematic Assistant shows the following menu.

Figure 30 Schematic Assistant Canvas Shortcut Menu



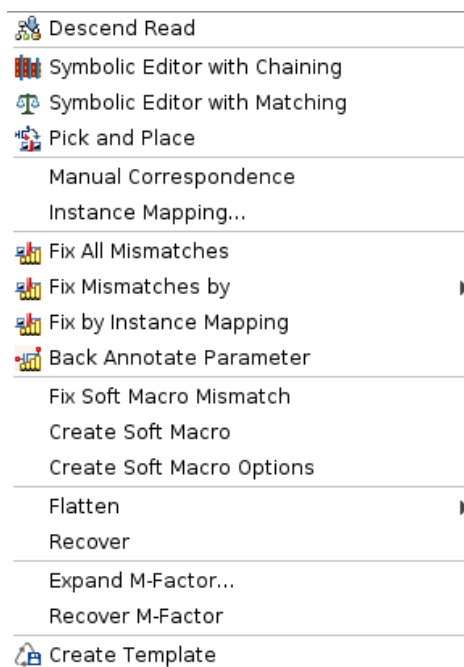
Option	Description
Add Probe	Adds probes to the selected pin or segment and its physically connected net. Click the Add Probe option to highlight the net with the active selection color. The highlight colors cycle through the color set.
Remove All Probes	Removes all added probes on nets. To add probes, see Shortcut Menu on Nets .
Remove Selected Probes	Removes selected probes on nets by dragging the mouse over the nets. To add probes, see Shortcut Menu on Nets .
Return	Returns you to the parent for this design. (Active only if this is not the top design.)
Return to Top	If this is not the top design, returns you to the top design.
Zoom In	Magnifies the schematic view area.
Zoom Out	Manipulates the schematic view area at a reduced size.
Fit to Edit	Fits all items into the schematic view area.
Symbolic Editor with Chaining	See About the Symbolic Editor in the <i>Custom Compiler Symbolic Editor User Guide</i> .
Symbolic Editor with Matching	See About the Symbolic Editor in the <i>Custom Compiler Symbolic Editor User Guide</i> .
Pick and Place	See Picking and Placing .
Manual Correspondence	See Manual Correspondence .
Instance Mapping	See Instance Mapping .
Fix All Mismatches	See Fix All Mismatches .
Fix Mismatches by	See Fix Mismatches by .
Fix by Instance Mapping	See Fix by Instance Mapping .
Back Annotate Parameter	See Back Annotate Parameter .
Fix Soft Macro Mismatch	See Fix Soft Macro Mismatch .
Create Soft Macro	See Soft Macro Creation .
Create Soft Macro Options	See Soft Macro Options .
Flatten	See Flatten .
Recover	See Recover .

Option	Description
Create Template	See Creating Templates .

Shortcut Menu on Instances

Right-click a selected instance in the Schematic Assistant to show the following menu:

Figure 31 Schematic Assistant Instance Shortcut Menu



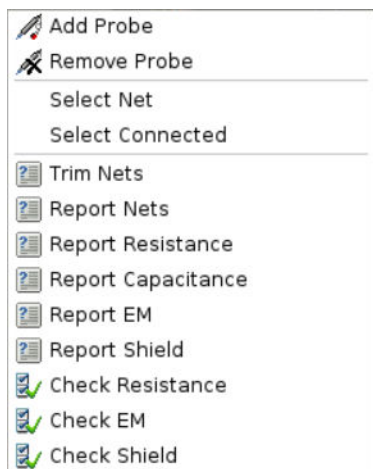
Option	Description
Descend Read	Changes the current schematic view to the next view down in the hierarchy. The Schematic Assistant and the Schematic Editor are synchronized as you move through the hierarchy.
Symbolic Editor with Chaining	See About the Symbolic Editor .
Symbolic Editor with Matching	See About the Symbolic Editor .
Pick and Place	See Picking and Placing .
Manual Correspondence	See Manual Correspondence .
Instance Mapping	See Instance Mapping .
Fix All Mismatches	See Fix All Mismatches .

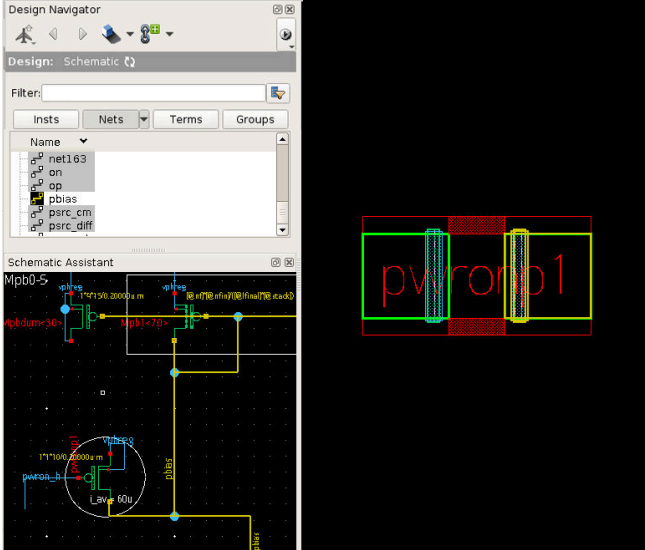
Option	Description
Fix Mismatches by	See Fix Mismatches by .
Fix by Instance Mapping	See Fix by Instance Mapping .
Back Annotate Parameter	See Back Annotate Parameter .
Fix Soft Macro Mismatch	See Fix Soft Macro Mismatch .
Create Soft Macro	See Soft Macro Creation .
Create Soft Macro Options	See Soft Macro Options .
Flatten	See Flatten .
Recover	See Recover .
Create Template	See Creating Templates .

Shortcut Menu on Nets

Right-click a selected net in the Schematic Assistant to show the following menu:

Figure 32 Schematic Assistant Net Shortcut Menu



Option	Description
Add Probe	Adds probes to the selected pin or segment and its physically connected net. Click the Add Probe option to highlight the net with the active selection color. The highlight colors cycle through the color set. 
Remove Probe	Removes the probe to the selected pin or segment and its physically connected net.
Select Net	Selects a pin or segment and its physically connected net and deselects any other selected objects.
Select Connected	Selects a pin or segment and its entire net and deselects any other selected objects.
Trim Nets, Report Nets, Report Resistance, Report Capacitance, Report EM, Report Shield	See Design Navigator Nets Shortcut Menu in the Manipulating Components in the <i>Custom Compiler™ Environment User Guide</i> .
Check Resistance, Check EM, Check Shield	See Design Navigator Nets Shortcut Menu in the Manipulating Components in the <i>Custom Compiler™ Environment User Guide</i> .

Shortcut Menu on Pins

Right-click a selected pin in the Schematic Assistant to show the following menu:

Figure 33 Schematic Assistant Pin Shortcut Menu



Option	Description
Pick and Place	See Picking and Placing .

4

Using a Top-Down Flow

This chapter describes how to use a top-down methodology to generate layouts from schematics in SDL.

The top-down approach makes use of *soft macro* blocks to determine the physical hierarchy of the layout. A soft macro for a schematic cellview comprises a PR boundary whose size is estimated based on the sizes of the leaf cells of the schematic and all the corresponding terminals so you can see the connectivity between blocks.

This chapter covers the following topics:

- [Soft Macro Creation](#)
- [Soft Macro Reuse](#)
- [Expand Schematic](#)
- [Flatten](#)
- [Recover](#)
- [Soft Macro Verification](#)
- [Fix Soft Macro Mismatch](#)

Soft Macro Creation

Soft macros can be created in the following methods:

1. Via the shortcut command menu in the Design Navigator in an SDL session.
2. During Expand Schematic outside an SDL session.
3. Using **Create Soft Macro** in [Pick and Place Options — Main Tab](#).

In both cases, soft macro creation involves selecting an instance in the hierarchy of a Top schematic whose switch master is a schematic and then creating two new soft macro cellviews: a *layout* and a *layout.config*.

The *layout* properties are described in [Soft Macro Layout](#) and the *layout.config* properties are described in [Soft Macro Config](#). The instance boundary of soft macros is described in [Soft Macro Options](#).

After the soft macro is created, the layout is bound to the selected instance by adding an occurrence rule to the Top config.

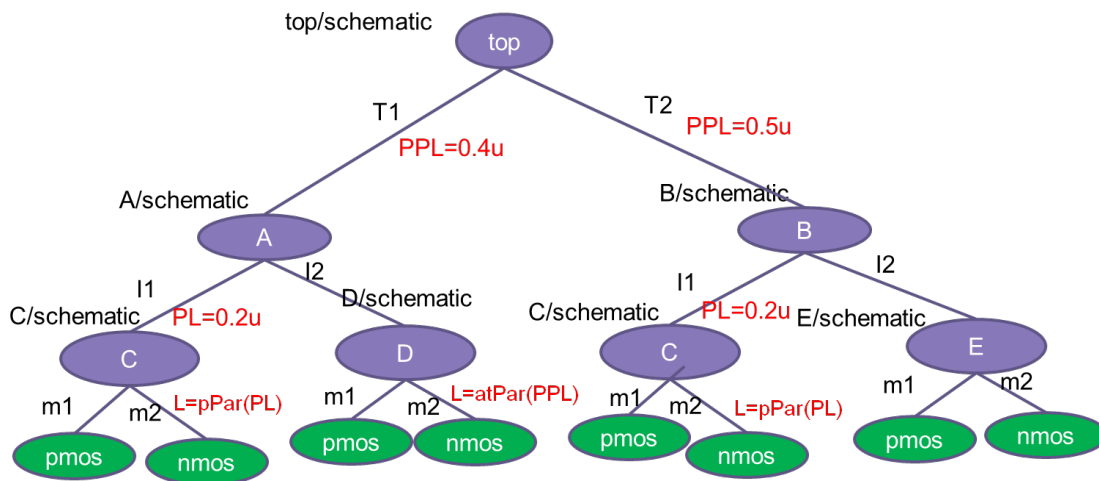
Note:

In an SDL session, if any one of the leaf cells of the master of the selected instance is already realized in the layout, then a soft macro cannot be created for it.

The creation of a new cell is subject to whether it can be reused as described in [Soft Macro Reuse](#) and the new cell name is decided by the `lxSoftMacroCellName` preference.

[Figure 34](#) illustrates a generic SDL flow and provides an example that is referred to throughout this chapter. In the figure, schematics are shown in purple, layout cellviews (instances) are shown in green, and parameters applied to each schematic and layout instance are shown in red.

Figure 34 Generic SDL Flow



This section describes the following topics:

- [Effective Parameters](#)
- [Soft Macro Cell Creation](#)
- [Constraint Update for Soft Macro Creation](#)

Effective Parameters

The effective parameters for a soft macro are those parameters that are defined in the lineage from the Top to the instance that instantiates the soft macro, and upon whose values the leaf cells in the soft macro lineage depend. For the example in [Figure 34](#), the effective parameter and value for T1/I1(C) is PL(0.2), and for T1(A) is PPL(0.4).

While the effective parameters of a schematic are dependent on the set of schematics under the soft macro instance, the *values* of those effective parameters depend on the lineage from the Top to the instantiation of the schematic and can vary from one instantiation to another.

Soft macros with effective parameters are referred to as *parameterized*. Similarly, soft macros that do not have any effective parameters are referred to as *non-parameterized*. For the example in [Figure 34](#), the soft macro for T1(A) is a parameterized soft macro, while the soft macro for T2(B) is a non-parameterized soft macro. Although parameters have been defined above B's instantiation, they are not used in all of the leaf cells of B.

The primary distinction between handling parameterized and non-parameterized soft macros is in the naming of the soft macro cell as described in [Soft Macro Cell Creation](#).

The effective parameters of a soft macro are stored in the data.dm of the soft macro config cellview. Opening the schematic cellview via a soft macro config also reads in the effective parameters, and so all parameters in the schematic are fully evaluated. For the example in [Figure 34](#), if the schematic for A is opened via its soft macro config, then all of its leaf cell parameters are properly evaluated, including L=0.4u in T1/I2/m2.

Soft Macro Cell Creation

The default output library for soft macros while in an SDL session is the Top layout library. If the target and source libraries are the same, then non-parameterized soft macro cellviews use the same cell name as the schematic. If they are different, then a new cell may be created in the target library.

For parameterized soft macros, the target cell naming convention follows the `lxSoftMacroCellName` preference:

- `%L` — Include library name in the new cell name.
- `%C` — Include source cell name in the new cell name.
- `%N` — Include a running index in the cell name starting from 1.
- `%F` — Include the values of effective parameters in the sorted order of the effective parameter names.

For the example in [Figure 34](#), if the preference is `%C_%F`, then the soft macro cell name for T1(A) is A_0.4.

Note:

The default naming convention is %C_%N.

In order to avoid name collisions for parameterized soft macros with the same master, %C and %N must be specified.

This section describes the following topics:

- [Soft Macro Layout](#)
- [Soft Macro Config](#)
- [Soft Macro Options](#)

Soft Macro Layout

SDL creates the soft macro *layout* by elaborating the Top schematic along the path of the selected instance and identifying all the leaf level layouts for which the selected instance is an ancestor. The layout consists of oaPRBoundary, terminals, and inherited connections from the leaf cells. The oaCellType of the created layout is set to softMacro.

Similar to the SDL elaboration on the Top schematic, the terminals and inherited connections to be added to the soft macro are identified by elaborating the selected schematic.

The PRBoundary for the soft macro is computed similar to SDL layout generation functionality, using the sum of area of all evaluated leaf cells within soft-macro hierarchy and the preferences lxBoundaryUtilization and lxBoundaryWidthHeightRatio.

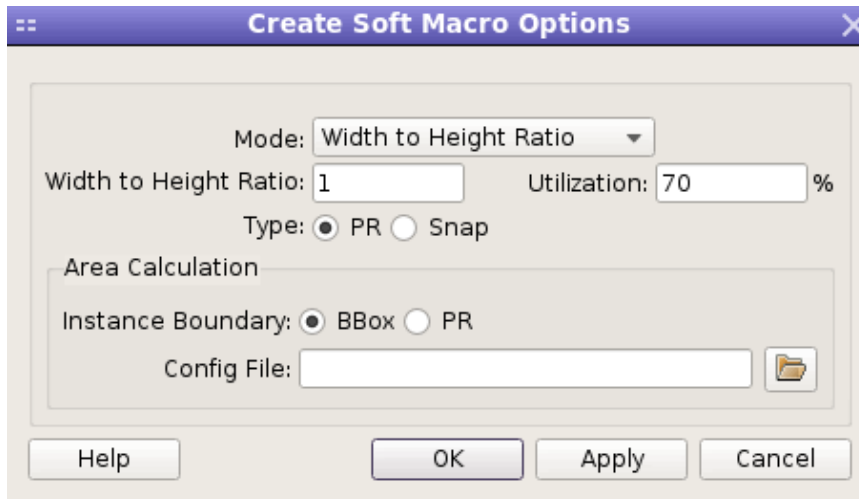
Soft Macro Config

The SDL also creates a soft macro *layout.config* for each soft macro *layout*. The SDL layout attribute is set on the soft macro layout to point to the layout.config to aid in starting an SDL session to complete the soft macro layout.

The new config is derived from the Top config by copying all search/stop lists and the rules that are applicable to the selected instance and below its hierarchy. The copied rules also remain in the Top config. For the example in [Figure 34](#), while creating a soft macro for A, only occurrence rules below T1 are copied to the soft macro config of A.

Soft Macro Options

The command is opened from [Schematic Assistant Shortcut Menus](#). The command opens the **Create Soft Macro Options** dialog box and uses the lxInstanceBoundaryCalculation to calculate the instance boundary.



For options description, see the **Create Boundary** section in the [Generate Layout Options — Main Tab](#).

Constraint Update for Soft Macro Creation

After a soft macro is created and the hierarchical instance that binds to it becomes a leaf instance, some previously transferred constraints in the Top level layout are rendered invalid and are deleted. For example, if a soft macro is created for instance i1, any previously transferred constraints with hierarchical path i1|...|... are deleted from the Top level layout.

Constraints containing the soft macro instance may also have to be transferred. For example, if there is a matching constraint for instances i1 and i2 (where i2 is a leaf instance and i1 was not previously a leaf instance), when a soft macro is created for i1, this constraint is transferred to the Top level layout.

Constraints for the soft macro layout are not transferred immediately; rather, the transfer happens when its own SDL session is started.

Soft Macro Reuse

While creating a soft macro for a schematic master, if there is an already existing soft macro for another instance of the same master that matches the same requirements, then the existing soft macro is reused. In order to reuse a soft macro for a schematic master, the effective parameters and their values must be the same.

The reuse table for soft macros is maintained in the `data.dm` of the target layout library. The key for the table is the concatenation of layout cellviews of the schematic master

and the value is a list of soft macro layout cells for that schematic with different effective parameters.

During expand schematic, if there is a reuse match available for a given layout cellview and the effective parameters in the reuse table, but the reuse cell has only a layout, then also the existing layout is reused. A new `layout.config` is created in the cell with effective parameters stored in its `data.dm` and the layout's config property is updated to point to the new `layout.config`.

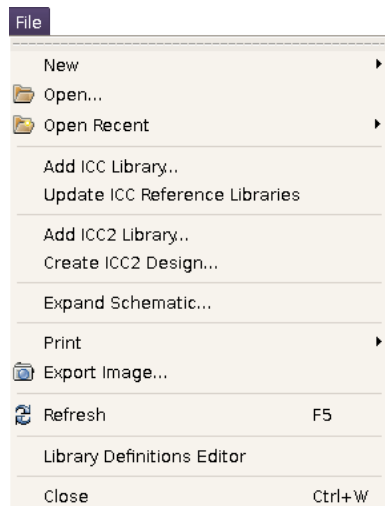
Expand Schematic

Expand Schematic is an extension of soft macro creation to the library and cellview level. It takes either a schematic cellview or a config cellview pointing to a schematic as input, and then recursively generates soft macro cells for all instances whose switch masters are schematics at each hierarchy level in the schematic. For the example in [Figure 34](#), Expand Schematic tries to create soft macros for all six instances (top, A, B, C, D, E) of schematic cellviews in the entire hierarchy and not for any of the leaf-level transistors. If Expand Schematic is unable to process an instance, it ignores it and moves to the next one.

Soft macro creation for each schematic instance is described in [Soft Macro Creation](#). It respects the `lxSoftMacroCellName` and `lxSoftMacroSchematicSource` preferences and always tries to reuse soft macros as described in [Soft Macro Reuse](#). The generated soft macro cells can then be used to define the physical hierarchy in an SDL session on the Top schematic.

From the Library Manager, select **File > Expand Schematic** to launch the **Expand Schematic** dialog box.

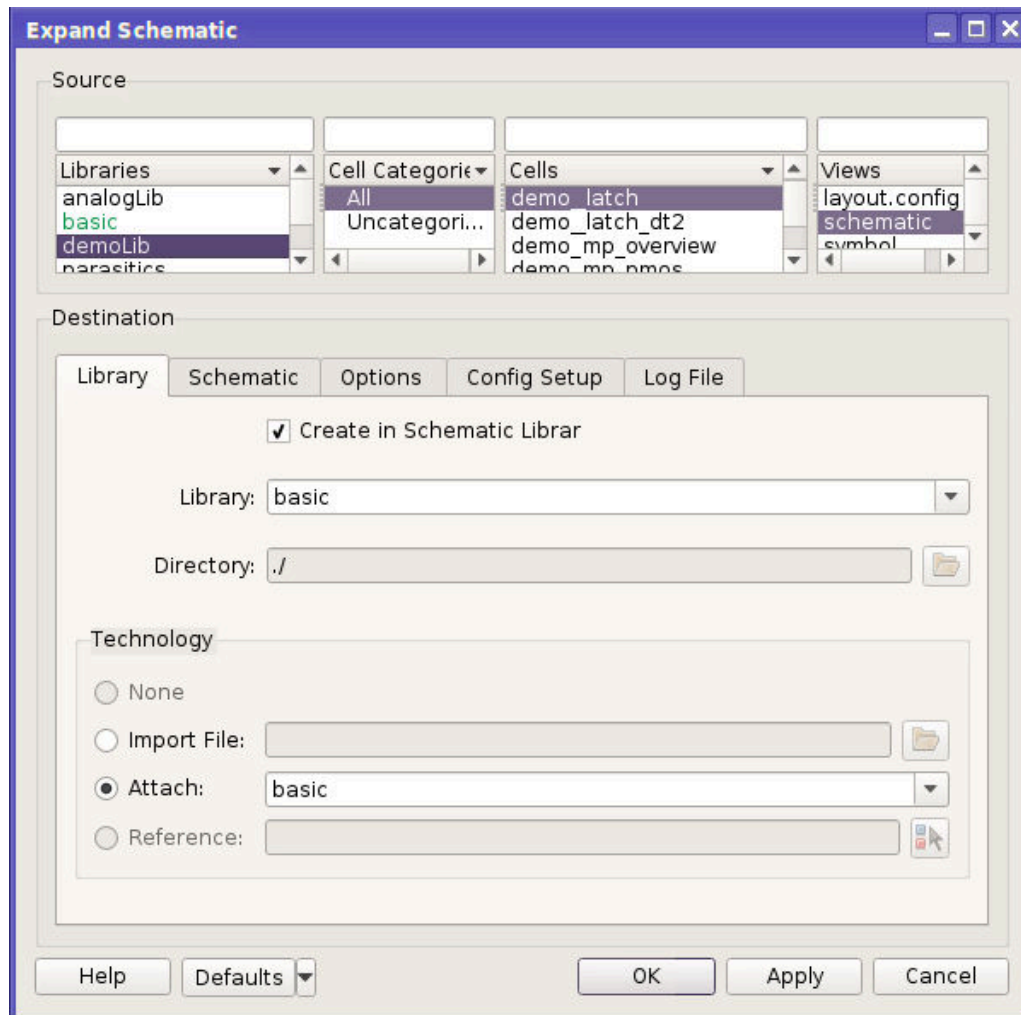
Figure 35 *Launching Expand Schematic From Library Manager*



Setting Expand Schematic Options

As shown in [Figure 36](#), the **Expand Schematic** dialog box consists of two areas at the top level, one for specifying the **Source** and the other for specifying the **Destination**.

Figure 36 Expand Schematic Dialog Box - Library Tab



The **Source** area is a layout cellview selector with columns for **Libraries**, **Cell Categories**, **Cells**, and **Views**. Pre-selections from the Library Manager are preloaded.

The valid selections for Expand Schematic are as follows.

- **Schematic Cellview:** Expand Schematic creates a new config cellview based on the SDL default layout config and uses it to elaborate the schematic. During elaboration, for each instance whose switch master is a schematic, Expand Schematic creates a soft macro layout and config.

- **Config Cellview:** For Expand Schematic to be invoked on a config cellview, its Top cellview must point to a schematic. Expand Schematic uses the selected config to traverse the schematic and recursively generates all soft macros.
- **Library Only:** If only one of the **Libraries** is selected with no selections in the **Cells** and **Views** columns, then all of the Top schematic cellviews in the selected library are identified first, based on whether a cell is referenced in another schematic in the library. Expand Schematic is then invoked on each one of the Top schematic cellviews.

During Expand Schematic, if the soft macro layout cellview already exists, then that layout is not overwritten. If the soft macro config also exists, then soft macro creation is ignored for the corresponding schematic master.

If the layout exists, but the config does not exist, then a new config (along with the effective parameters) is generated and the existing layout's attribute is set to point to the new config. However, if the config alone exists without the layout, then a new layout and config are generated and the existing config is overwritten.

When any design is opened in the target library of Expand Schematic, a dialog box displays with designs that are opened for users to choose to close the designs. Expand Schematic cannot continue unless all designs in the target library are closed.

The **Destination** area specifies other settings in the **Library**, **Schematic**, **Options**, and **Config Setup** tabs.

- [Library Tab](#)
- [Schematic Tab](#)
- [Options Tab](#)
- [Config Setup Tab](#)
- [Log File Tab](#)

Library Tab

The **Library** tab contains the following elements:

- **Create in Schematic Libraries:** Specifies the location of schematic libraries to save the layout design and layout.config view.

Tcl preference: `lxSoftMacroSchematicLibrary`

- **Library:** Specifies the library where the soft macro blocks are created during Expand Schematic. All of the existing libraries in the Library Manager are populated in the list. When an existing library is selected only **Import File** and **Attach** in the **Technology** field are enabled. If a new library name is specified, then the **Directory** and **Technology** fields are enabled.

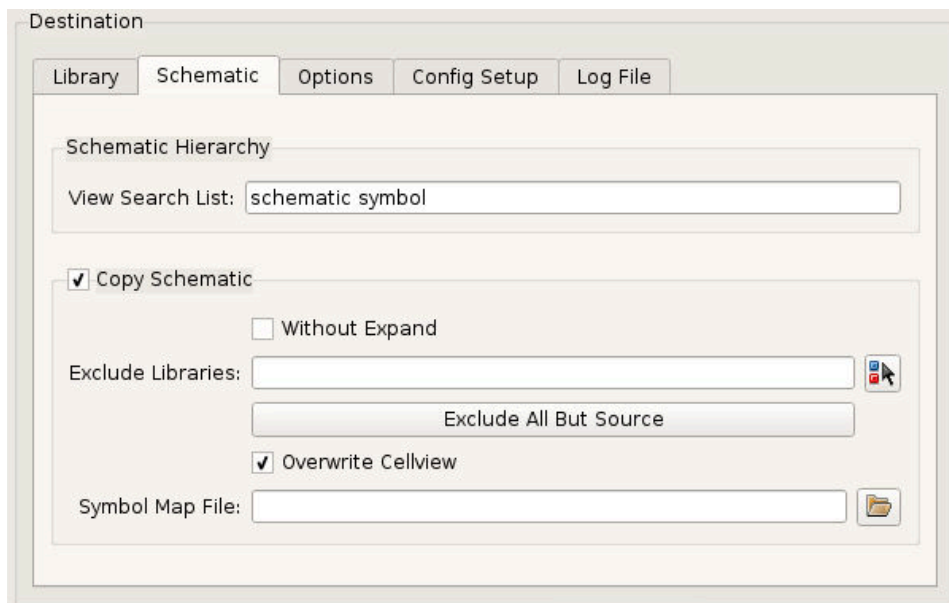
- **Directory:** Specifies the directory location for the new library. This is an optional field that is enabled only if a new library name is specified.
- **Technology:** Specifies whether and what type of technology to use for the new library:
 - **None:** Use no technology.
 - **Import File:** Import a technology file.
 - **Attach:** Attach a technology.
 - **Reference:** Attach reference libraries.

If a library name is specified, the specified technology is used to the library.

Schematic Tab

This tab is enabled only if the source and target libraries are different and **Create in Schematic Libraries** is not selected. The **Schematic** tab contains the following elements:

Figure 37 Expand Schematic Dialog Box - Schematic Tab



- **Schematic Hierarchy:** Specifies the schematic view names for the expand schematic in the **View Search List** text field.
- **Copy Schematic:** Specifies whether the symbol, schematic, and constraint views must be copied over to the target library. This feature supports only the *Schematic Cellview* and *Library Only* selections.

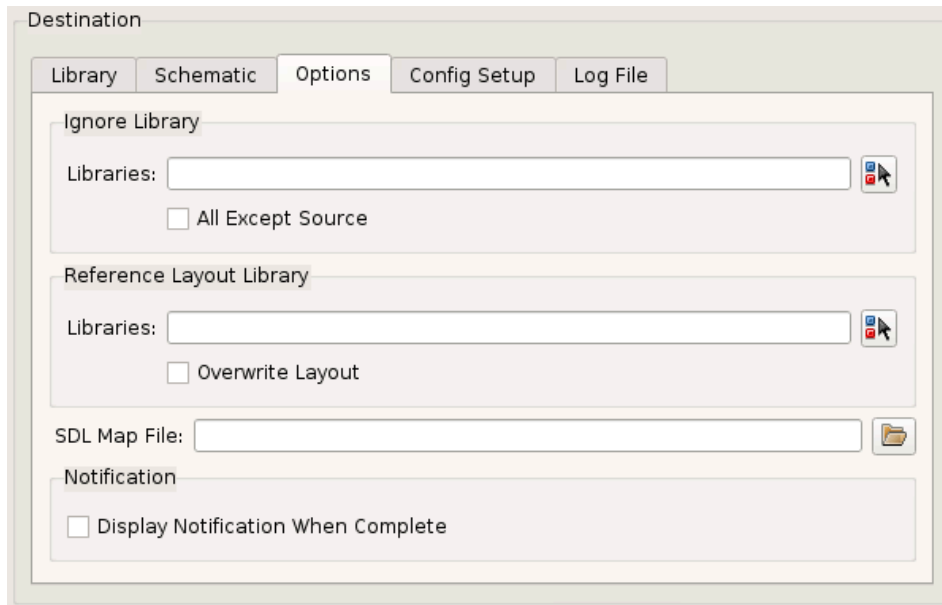
For more information, see [Copy Schematic Functionality in Expand Schematic](#).

- **Without Expand:** Only copies schematic, symbol and constraint views to the target library. Soft macros do not need to be generated in the target library.
- **Exclude Libraries:** Specifies a list of libraries from which *not* to copy the source schematic or symbol views, although Expand Schematic can still traverse these libraries for creating soft macros. This element is enabled only if **Copy Schematic** is enabled and selected.
- **Exclude All But Source:** Adds all libraries *except the source library* to the list of libraries to exclude. This element is enabled only if **Copy Schematic** is enabled and selected.
- **Overwrite Cellview:** Specifies whether to overwrite the schematic, symbol, and constraint views if the cellview to be copied is already present in the target library.
- **Symbol Map File:** Specifies a symbol mapping file for use in Expand Schematic. This element is enabled only if **Copy Schematic** is enabled and selected.

Options Tab

The **Options** tab contains the following elements:

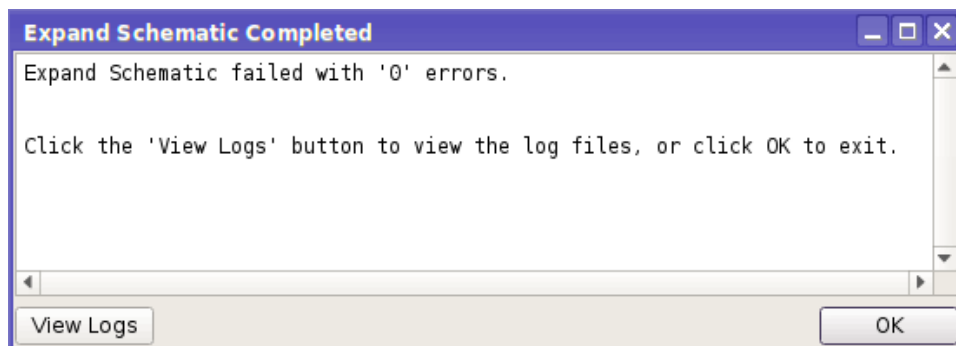
Figure 38 Expand Schematic Dialog Box - Options Tab



- **Ignore Library:** Specifies an existing list of libraries to be ignored during Expand Schematic. During Expand Schematic, if a schematic from an ignored library is found, expand schematic will ignore the schematic, not traverse further and return. Ignored libraries are also treated like exclude libraries during **Copy Schematic**.

Tcl preference: `lxExpandSchematicIgnoreLibs`

- **All Except Source:** When this option is enabled, all libraries except the specified source library are ignored.
- **Reference Layout Library:** Specifies existing libraries of layouts to be copied to the target library. All layouts are copied except those which already exist in the target library. If the layout being copied has a valid SDL config pointer, the corresponding layout.config is also copied to the same cell in the target library and the SDL config pointer for the target layout is updated. If a rule in the copied layout.config file refers to the reference layout library, that rule is updated to point to the new target library. Soft macros copied from the reference library to the target library are also considered for reuse when you generate the soft macro.
- **Overwrite layout:** Specifies whether to overwrite the layout if the layout to be copied is already present in the target library.
- **SDL Map File:** Specifies a layout mapping file for use in Expand Schematic and updates the physical mapping information in the target library's data.dm file.
- **Notification:** Enables the **Display Notification When Complete** option to specify if expand schematic should display the **Expand Schematic Completed** dialog box upon completion.



Click **View Logs** to display the details of log file in **Text Viewer**.

Tcl preference: `lxExpandSchematicNotify`

Config Setup Tab

This is used to specify the layout.config options to be used while generating all soft macros. Any existing soft macro config options can be overwritten by the specified options.

The **Config Setup** tab contains the following elements:

Figure 39 Expand Schematic Dialog Box - Config Setup Tab

The screenshot shows the 'Config Setup' tab of the 'Destination' dialog box. It features five tabs: 'Library', 'Schematic', 'Options', 'Config Setup' (selected), and 'Log File'. Below the tabs are five text input fields with labels on the left and icons on the right:

- Library Search List: %PLACED|
- View Search List: abstract layout schematic
- View Stop List: abstract layout
- Ignore Name List: nlAction lvsIgnore lxIgnore
- Short Name List: lxRemoveDevice

- **Library Search List:** Specifies the library search list to seed the configuration or browse the library search list in the **Library List Browser** dialog box.
- **View Search List:** Specifies the view search list to seed the configuration.
- **View Stop List:** Specifies the view stop list to seed the configuration.
- **Ignore Name List:** Specifies the list of names of devices that are ignored to seed the configuration.
- **Short Name List:** Specified the list of names of devices that are ignored to seed the configuration. Definition includes terminal names to short.

See [Advanced Steps in Initializing From Schematic](#).

Log File Tab

The **Log File** tab contains the following elements:

Figure 40 Expand Schematic Dialog Box - Log File Tab

The screenshot shows the 'Log File' tab of the 'Destination' dialog box. It features five tabs: 'Library', 'Schematic', 'Options', 'Config Setup', and 'Log File' (selected). Below the tabs is a 'General' section with two text input fields:

- Log File Name: (empty)
- Limit Log Entries: 5

- **Log File Name:** Specifies the file name to override the default log file name. By default, the tool creates a log file name with the following syntax: `expandSchematic.<user name>.DD_MM_HH_MM_SS.log`.

If this field is specified, then the **Limit Log Entries** option is disabled.

- **Limit Log Entries:** Specifies the number of expand schematic log files to be retained. It only applies to the default log files. The valid range is from 1 to 999. The default is 5. This field is available when the **Log File Name** field is empty.

Tcl preference: `1xExpandSchematicLimitLogFiles`

Copy Schematic Functionality in Expand Schematic

If the **Copy Schematic** option is enabled in [Expand Schematic](#), then the source schematic, symbol, and constraint cellviews are copied hierarchically from the source library to the target library. Copy Schematic can only take an expanded library level or a schematic cellview as the source selection. The hierarchical traversal is performed using a schematic config. You can specify a list of libraries to omit from copying. In addition to copying cellviews, the cell level DM data of the corresponding schematic cellviews is also copied to the target library. In the special case where all libraries are selected in the **Exclude Libraries** field, then no schematic source is copied and the Copy Schematic option is ignored.

If there is a source conflict during copy, that is, if there are two cells from different source libraries with same name, then the corresponding cells in the target are automatically renamed to "\$L_\$C" format where "\$L" is the library name and "\$C" is the cell name. For example, if there is a cell named "inv" in library "A" as well as in library "B" and both are instantiated in the cellview "Demo/Top/schematic", then the corresponding target cellviews in a target library "CS" would be "CS/A_inv/schematic" and "CS/B_inv/schematic".

In case of a destination conflict, that is, if the cellview to be copied already exists in the target library, then **Overwrite Cellview** determines whether the cellview in the target library is deleted and copied from source or left as-is.

Copy Schematic is used in Expand Schematic for the following main reasons:

- To isolate the generated layout from schematic changes by copying schematics and taking a snapshot of their current state.
- To handle cases where the symbols cannot be read and must be remapped to equivalent cellviews.

In order to support the latter use model, Copy Schematic also supports specification of a symbol mapping file. The symbol mapping file maps a source symbol cellview (LCV) to a target symbol cellview (LCV). The format of the symbol map text file is as follows:

```
<{old_lib old_cell old_symbol_view}> <{new_lib new_cell new_symbol_view}>
```

During Copy Schematic, if a design contains an instance of a source symbol cellview (placed master) specified in the symbol mapping file, then the corresponding instance in the target design points to the target symbol cellview specified in the symbol mapping file. Copy Schematic does not expand further after the mapping is done.

For example, if a design "src/A/schematic" contains an instance "I" of "generic90RF/n_4t/symbol" and the symbol mapping file contains "{generic90RF n_4t symbol}{reference90RF n_4t symbol}", then the target cellview of "A", "tgt/A/schematic", has an instance "I" pointing to "reference90RF/n_4t/symbol".

Regardless of whether the reference libraries are specified in the **Exclude Libraries** field, the result of symbol mapping is identical. That is, the cellviews specified in the source section in the symbol mapping file are not copied to the target library. Instead their instantiations are remapped to the corresponding new symbol cellviews specified in the symbol mapping file.

ECO Flow in Expand Schematic

[Expand Schematic](#) supports an ECO (Engineering Change Order) flow, as follows:

1. Run Expand Schematic one time.
2. Perform layout realization operations on Top and/or other soft macros.
3. Update/modify the schematic. The modifications to the schematic may include parameter changes, addition/deletion of instances, change in the master, or a combination of all anywhere in the schematic hierarchy.
4. Run Expand Schematic again to update the soft macros.

If a schematic is selected as the source to expand, a new config that uses a view search list comprising {schematic abstract layout} and a stop list comprising {abstract layout} is used to traverse the schematic and determine the soft macros that must be generated. If a schematic and layout are both present in the same cell, Expand Schematic continues to traverse the hierarchy below the schematic. The config generated for all soft macros is derived from SDL layout.config. If a config itself is selected as the source, then that config is used as-is for creation of soft macros.

While creating soft macros for each instance in the schematic hierarchy, Expand Schematic reuses existing soft macros based on the schematic and the effective parameters. If no existing soft macro is found, then a new soft macro is created and the

Top config is updated. If a soft macro can be reused, then the existing soft macro config is updated to reflect any changes to the schematic as follows:

- If a new occurrence was created (for example, addition of an instance), then the new rule is added to the config.
- If an occurrence path is no longer valid (for example, deletion of an instance), then the occurrence rules corresponding to the occurrence are removed from the config.
- If the occurrence exists but has been modified (for example, a new master binding for the instance), then the library and cell name rules for the occurrence are updated while retaining the selection rules.
- If an occurrence rule exists and there is no change to the binding, then the existing rule is retained.

Soft macro layouts are never modified.

Expanding Schematic With Different Version Control

Expand Schematic can work with registered version control systems (for example: Perforce) while updating pre-existing layout.configs. If the target library is managed by a version control system, then expand schematic performs some additional actions, as follows:

- If the soft macro cellviews already exist and need to be updated or overwritten, expand schematic need to check out the cellview first. If an existing cellview is checked in, then expand schematic tries to check out the cellview using the underlying version control system Checkout Feature.
- If checkout fails, then expand schematic ignores the hierarchy and moves forward. If checkout succeeds, then the cellview is updated or overwritten.

This ability to automatically checkout previously checked in cellviews is controlled via the `lxExpandSchematicCheckOut` preference.

If the preference is set to true, then the cellview be checked out, otherwise, expand schematic ignores and moves forward

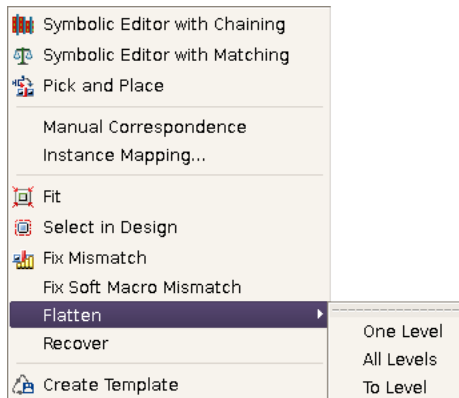
- Expand schematic never checks in any cellviews. Checking in must be done manually by the user.
- In some version control systems, it is possible that overwriting a managed cellview may remove it from the repository itself and such cellviews would have to be added to the repository again.

Flatten

When an schematic instance is bound to a layout, you can use the Flatten operation it to populate lower level schematic instances of that cell to the current level and to have a different layout.

As described in [Design Navigator Shortcut Menus](#), right-clicking objects displayed under the **Insts** tab in the Design Navigator in Schematic mode shows a command menu

Figure 41 Right-click to Flatten



To Flatten an instance, right-click a leaf cell and select one of the following options from the menu:

- **Flatten > One Level** is a config change operation in which a binding to an existing layout is replaced by a binding to a layout cellview which is a layout.config pointed to by the layout.

If the selected leaf cell is already realized in layout, or if it does not have a pointer to a config, or if the config does not exist, then Flatten triggers an error. In the example in [Figure 34](#), if the naming preference is set to %C_%F and Expand Schematic has been invoked on Top, then T1(A) is bound to A's soft macro, A_0.4. Invoking Flatten on T1(A_0.4) would expose the children of A_0.4, T1/I1(C_0.2) and T1/I2(D_0.4), by binding A_0.4/layout.config to Top/I1 in Top/layout.config.

- **Flatten > All Levels** recursively flattens the selected leaf cell until there are no more layout cellviews to flatten in the hierarchy. In the example in [Figure 34](#), flattening all levels of T1(A_0.4) would expose the leaf level four transistors by adding binding rules for A_0.4/layout.config, C_0.2/layout.config, and D_0.4/layout.config in Top/layout.config.
- **Flatten > To Level** calls a dialog box ([Figure 42](#)) and recursively flattens the selected leaf cell to the specified hierarchy level. In the example in [Figure 34](#), selecting

T1(A_0.4) to Flatten to level 2 would be the same as flattening one level on T1(A_0.4) and to Flatten to level 3 would flatten all levels on T1(A_0.4).

Figure 42 *Flatten->To Level Dialog Box*



Constraint Update for Flatten

The Flatten operation changes the binding of the flattened instance to a schematic switched master and brings any lower level leaf instances to the Top level layout. Incremental constraints transfer is performed for the flattened instance and for lower level hierarchical instances that have schematic switched masters with associated constraint cellviews.

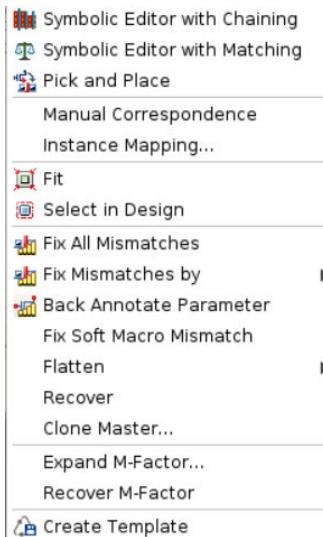
For example, instance i1 has its binding changed from a layout to a schematic through a Flatten operation. The bind-to schematic has two leaf instances, i2 and i3, with a matching constraint between them. After the Flatten operation, a matching constraint with leaf instances i1|i2 and i1|i3 is transferred to the Top level layout.

Recover

You can use the Recover operation to undo flattened instances.

As described in [Design Navigator Shortcut Menus](#), right-clicking objects displayed under the **Insts** tab in the Design Navigator in Schematic mode shows a command menu.

Figure 43 *Right-click to Recover*



Recover can be invoked on leaf cells and on intermediate nodes in the Design Navigator. While Recover on an intermediate node tries to recover that node itself, Recover on a leaf cell attempts to recover the parent.

Recovering a node restores the binding of the instance to its layout cellview from the original binding to a config cellview, provided the layout exists in the same cell as the layout.config and points to the config. In the example in [Figure 34](#), if after invoking **Expand Schematic** and **Flatten > All Levels** on T1(A_0.4), if T1/I1(C_0.2) is selected for Recover, then Top/T1/I1 is bound to C_0.2/layout in Top/layout.config. The result is the same if the leaf cell T1/I1/m1 is selected to recover.

If there are multiple valid layouts pointing to the config, then the first layout is selected to recover. If the selected instance does not have an existing config binding, or if a layout does not exist in the same cell, or if any leaf in the hierarchy of the instance is realized on layout, then Recover triggers a warning.

When there are multiple instances selected to recover, then the list of nodes to be recovered is sorted in decreasing order of hierarchy and then recovered in that order.

Constraint Update for Recover

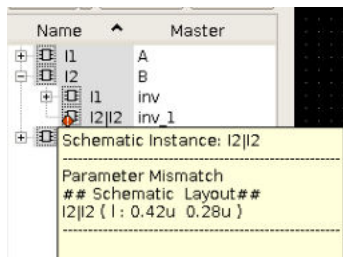
This is similar to constraint update for soft macro creation (see [Constraint Update for Soft Macro Creation](#)).

Soft Macro Verification

Verifying a soft macro entails generating the effective parameters of an instance bound to a soft macro and comparing them to the effective parameters in `data.dm` of the config cellview.

Soft Macro Verification is always on. For example, if a parameter change in the schematic affects the effective parameter of a bound soft macro, then a mismatch is shown in the Design Navigator immediately, along with the corresponding tooltip, as shown in [Figure 44](#).

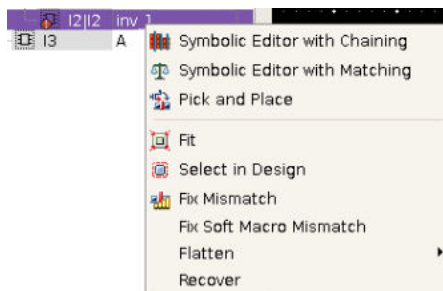
Figure 44 Soft Macro Parameter Mismatch



Fix Soft Macro Mismatch

The **Fix Soft Macro Mismatch** menu item is available for soft macros bound in the Design Navigator, as shown in [Figure 45](#).

Figure 45 Right-click to Fix Soft Macro Mismatch



If a soft macro instance has an effective parameter mismatch, then Fix Soft Macro Mismatch looks for a reuse candidate in the reuse table for the same schematic master, but with a new effective parameter set. If a reuse cell exists, then it is bound to the instance.

If a reuse candidate is not found, then the existing soft macro layout and config are copied to a new cell determined by the cell naming convention. The effective parameters in the `data.dm` of the copied config cellview are overwritten with the new set of effective parameters and their values, and the new cellview is bound to the instance.

5

Placing Schematic Instances Into Layout

This chapter describes the methods of placing a schematic design or part of a design into the layout design.

This chapter covers the following topics:

- [Generating Layout](#)
- [Picking and Placing](#)
- [Realizing a Layout With Preset Parameters](#)
- [Chaining Instances](#)
- [Folding Instances](#)
- [Establishing Correspondence](#)
- [Cloning Devices](#)
- [Splitting Devices](#)
- [Adding and Editing SDL Mapping Rules](#)
- [Applying Preset Parameters](#)
- [Realizing Layouts Using Drag-And-Drop Operations](#)
- [Triggering Callback Functions When Realizing Layout](#)
- [Establish Correspondence Feature](#)
- [Cross-Object Referencing](#)

Generating Layout

You can use the **SDL > Generate Layout** command to seed layout designs with instances, pin shapes, and a boundary.

SDL elaborates the referenced schematic and descends until it finds the physical stopping view defined in the SDL config file. SDL then places the stopping view (a flattened representation of all the instances in the reference schematic) in the layout design.

Consider the following case:

- vco schematic contains an instance of inv symbol
- layout is the stop view
- inv does not have a layout view
- inv contains an instance of pmos/symbol

In this case, when performing SDL between vco/schematic and vco/layout, the SDL engine checks if there is a layout view (the stop view) for inv. Because the layout view does not exist, SDL descends into inv/schematic and finds the instance of pmos/symbol. SDL then looks for a 'pmos/layout' view and if it exists, SDL places an instance of pmos/layout in vco/layout. It folds one level of hierarchy because inv/layout did not exist.

The following characteristics are applicable for layout designs created by the Generate Layout command.

- During the Generate Layout operation, pin shapes representing top level terminals, global nets, and unresolved net connection definitions are created and placed below the instances.

Using Generate Layout

You can generate a layout design using the Generate Layout feature after you initialize SDL. For more information on initialization, see [Overview of Schematic Driven Layout Initialization](#).

Generate Layout seeds the layout with instances, pins, and a boundary.

Use the following procedure to generate a layout:

1. When the target layout design specified during initialization is open in the Layout Editor, choose **SDL > Generate Layout**.

The SDL Generate Layout window appears.

2. Specify the options available on the **Main** area of the Generate Layout window. See [Generate Layout Options — Main Tab](#).

Note:

The three **Create** option regions control what is destroyed as well as what is created. When enabled (default), existing objects are destroyed and new objects created when you commit.

3. Specify the options available in the **Pins** area. See [Generate Layout Options — Pins Tab](#).

The **Override Default** options are available only when the **Create Pins** option on the **Main** tab has been enabled.

4. Click **OK**.

If you have disabled any of the "**Create...**" regions, the **Design Not Empty** prompt warns you about destroying existing physical objects and provides an option for preserving those objects.

- a. (Optional) Disable the **Preserve** option(s).
- b. Click **Continue**.

The layout is generated based on the specified schematic.

Generate Layout Options — Main Tab

[Figure 46](#) and [Table 7](#) describe the **Main** tab of the **Generate Layout** dialog box.

Figure 46 Main Tab of the Generate Layout Dialog Box

SDL Generate Layout

Template: Place Like Schematic ☒ Follow Schematic Orientation

☒ Create Instances

☒ Chain

☐ Fold

Device Class	Cell Name	Parameter	Threshold
nmos	nfin	nfin	
	nfinhv		
	nfin		

Spacing: 0.01

☒ Create Pins

Top/Bottom: M2 pin Width: 0.036 Length: 0.036 Pitch: 0.066

Left/Right: M1 pin Width: 0.024 Length: 0.024 Pitch: 0.048

Location: Boundary Side

☐ Create Boundary

Mode: Width to Height Ratio

Width to Height Ratio: 1 Utilization: 70 %

Type: ☒ PR ☐ Snap

Area Calculation

Instance Boundary: ☒ BBox ☐ PR

Config File:

Help Defaults OK Apply Cancel

Table 7 Options for the Main tab of the Generate Layout Dialog Box

Option	Description
Template	Lets you select the template that contains the algorithms used as the basis for generating the layout. The default template is <i>Place Like Schematic</i> . You can create your own template using Tcl procedures and add it to the list.
Follow Schematic Orientation	Let you create instances with the orientation in the schematic.
Create Instances	Controls whether instances are created automatically.

Table 7 Options for the Main tab of the Generate Layout Dialog Box (Continued)

Option	Description
Chain	Lets you specify whether the auto chaining feature must be enabled for selected objects; see Chaining Instances .
Fold	Lets you specify whether the auto folding feature must be enabled for selected objects; see Folding Instances. • <i>Device Class</i>: Specifies the device class for which folding is made available. • <i>Cell Name</i>: Specifies the cell name that is considered for folding. • <i>Parameter</i>: Specifies the parameter that is considered for folding. • <i>Threshold</i>: Lets you specify the folding threshold. If the instance parameter value is higher than the fold threshold, folding is initiated.
Spacing	Lets you specify the distance between each instance. The default is 0.01. Tcl preference: <code>lxDeviceSpacing</code>
Create Pins	Controls whether pins are created automatically. Tcl preference: <code>lxCreatePins</code> • <i>Top/Bottom</i>: Lets you specify the layer purpose pair on which the pin shapes must be created. • <i>Left/Right</i>: Lets you specify the layer purpose pair on which the pin shapes must be created. • <i>Width</i>: Lets you specify the width of the pin shapes, in user units • <i>Length</i>: Lets you specify the length of the pin shapes, in user units • <i>Pitch</i>: Lets you determine the distance between pin centers. Tcl preference: <code>lxTopBottomPinLPP</code>, <code>lxTopBottomPinWidth</code>, <code>lxTopBottomPinHeight</code>, <code>lxTopBottomPinPitch</code>, <code>lxLeftRightPinLPP</code>, <code>lxLeftRightPinWidth</code>, <code>lxLeftRightPinHeight</code>, <code>lxLeftRightPinPitch</code>, Note: The values of the width, height, and pitch in Generate Layout honor user-defined values if the values do not violate the constraint values in the technology library.
Location	Defines the pins location of pin creation by selecting one of the following options: • Boundary Side: Specifies pins are created on boundary side. • Lower Left: Specifies pins are created on the bottom left of whole top design. Tcl preference: <code>lxPinLocation</code>

Table 7 Options for the Main tab of the Generate Layout Dialog Box (Continued)

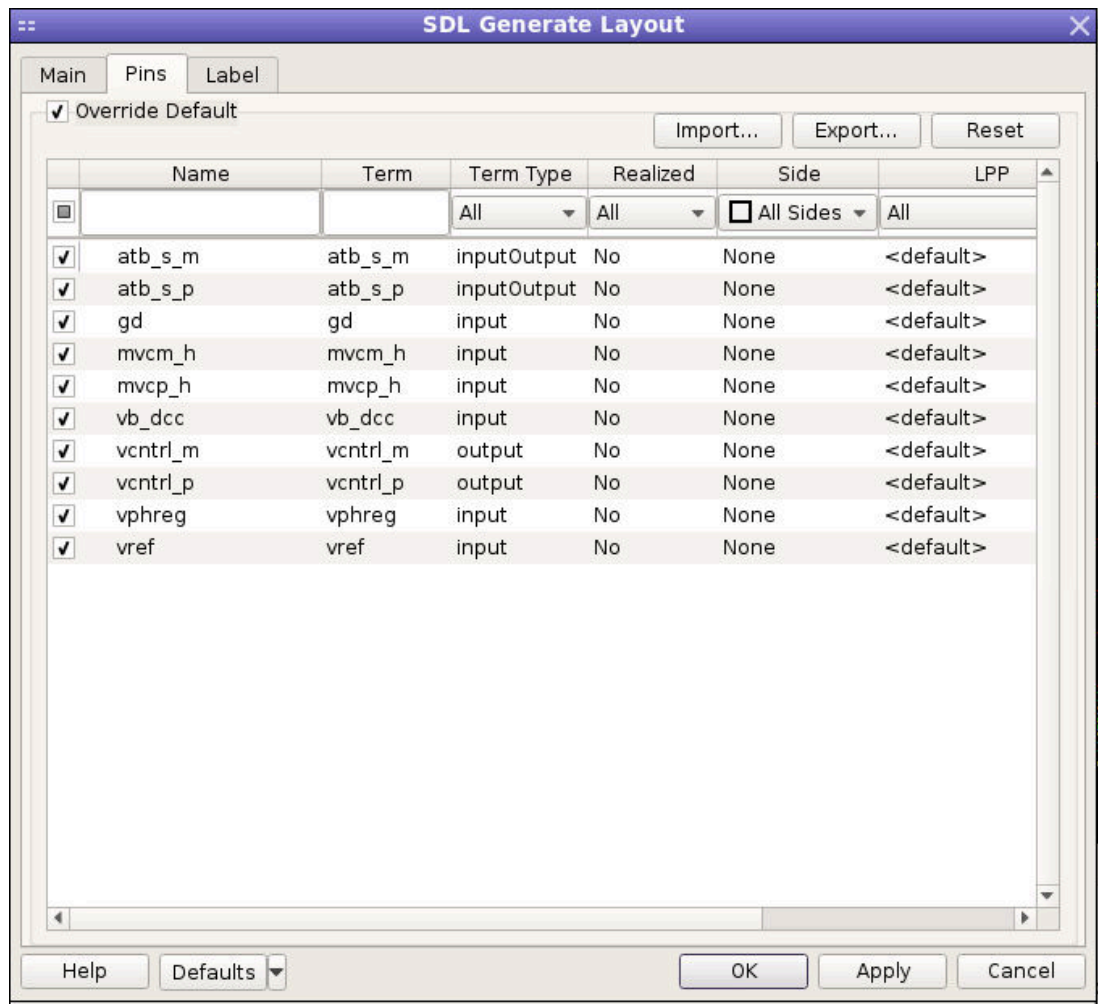
Option	Description
Create Boundary	Controls whether a boundary is created automatically. Tcl preference: <code>lxcreateBoundary</code>
Mode	Determines the aspect ratio (width to height ratio) of the boundary: • <i>Width to Height Ratio</i>: The width to height ratio is used as the basis for creating the boundary. • <i>Width</i>: The width of the boundary is used as the basis for creating the boundary. The height is calculated from the boundary area and the width you provide. • <i>Height</i>: The height of the boundary is used as the basis for creating the boundary. The width is calculated from the boundary area and the height you provide. • <i>Width and Height</i>: The width and height you provide is used as the basis for creating the boundary. Tcl preference: <code>lxBoundaryBasis</code>
Width to Height Ratio	Specifies a value for the width and height of the boundary.
Utilization	Lets you specify the percentage of the boundary area that is utilized by the design. The remaining area is set aside for placing and routing. The boundary area is calculated based on the utilization percentage provided. $\text{Utilization} = \text{Area required for instances} / \text{Boundary Area}$ Tcl preference: <code>lxBoundaryUtilization</code>
Type	Lets you specify the boundary type. • <i>PR</i>: Creates an <code>oaPRBoundary</code> object. In a top-down design flow, <code>oaPRBoundary</code> is commonly passed down to constrain the area in which the block's contents are placed. • <i>Snap</i>: Creates an <code>oaSnapBoundary</code> object. <code>oaSnapBoundary</code> is a rectangular boundary, typically used by Placement tools to place instances of the design in rows. Tcl preference: <code>lxBoundaryType</code>
Area Calculation	
Instance Boundary	Lets you choose BBox (bounding box) or PR (place and route boundary) as instance boundary when calculating area. Tcl preference: <code>lxInstanceBoundaryCalculation</code>

Table 7 Options for the Main tab of the Generate Layout Dialog Box (Continued)

Option	Description
Config File	Specifies a file using YAML syntax to specify scale factors and expansion for cells when estimating total boundary area. Note: Each instance area calculation method is expanding first and scaling after expanding. Area = (width + expansion) * (height + expansion) * scale Or Area = (width + expansion_x) * (height + expansion_y) * scale The YAML format of config file: <pre> --- - Boundary Scale: - scale: - [library name, cell name, view name] ... - Boundary Expansion: - expansion_x&y: - [library name, cell name, view name] - [expansion_x, expansion_y]: - [library name, cell name, view name] ... </pre> For example: <pre> --- - Boundary Scale: - 4: - [testA, C, layout] - [testB, D, layout] ... - Boundary Expansion: - [10]: - [testE, G, layout] - 3: - [testF, H, layout] - [0.5, 05]: - [test, simple_square, layout] ... </pre> Tcl preference: lxBoundaryConfigFile

Generate Layout Options — Pins Tab

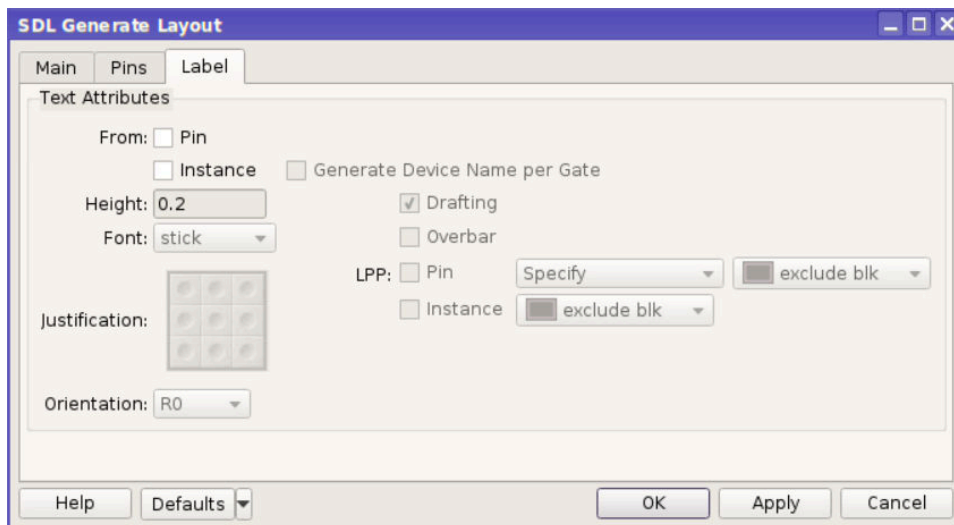
The following figure and table describe the **Pins** tab of the **Generate Layout** dialog box. See [Pins Tab](#) for more details.



Option	Description
Override Default	Specifies pin side and ordering, as well as overrides for LPP, Width, Length.

Generate Layout Options — Label Tab

You can create instance labels at the center of the instances or pin labels in the **Label** tab of the **Generate Layout** dialog box. The Label tab contains the following options:



See [Pick and Place Options — Label Tab](#) for the details of options.

Resolving Terminal Connection Conflicts

When you try to generate a layout and there exists more than one connection assignment in different parts of the source design, the tool prompts you to specify one assignment name for each default net.

To specify the assignment name:

1. Select a name from the drop-down menu in the **Terminal Connection Conflict** dialog box.
2. Click **OK**.

Placing by Floorplan Template

Floorplan is a placement template to generate layout according to the schematic design.

The floorplan template only can be used in command. To place by floorplan template:

1. (Optional) Specify properties `lxFloorPlanXScale` and `lxFloorPlanYScale` to rescale x-direction and y-direction from source schematic design in the Property Editor for top schematic or master symbol. The valid value must be a positive number.

Note:

The priority of the specified scale is: symbol property > command argument (`db::generateLayout`) > schematic property.

2. (Optional) The floorplan also supports two-dimensional spread when placing mfactor/vector instances or pins by defining either the `lxFloorPlanRow` or `lxFloorPlanColumn` property for top schematic instance. The valid value must be an integer greater than 0.

If `lxFloorPlanRow` or `lxFloorPlanColumn` is not specified, the floorplan spreads instances or pins into x-direction by default if the instance has mfactor > 1 or the instance or pin is a vector instance or pin.

3. Use `db::generateLayout` by giving the `-template` argument with `floorPlan` to create the master layout and the top layout design.
4. Floorplan uses the cell in the library of destination layout view as place master to do instance realization. If there is a valid master layout, then use the existed layout to do realization. Otherwise, the command creates layout based on schematic symbol.

If there is a selection shape in the symbol, the command uses it to create layout boundary. If there is no selection shape in the symbol, the command creates layout boundary by bounding. The top schematic properties `lxFloorPlanXScale` and `lxFloorPlanYScale` also affect to create the layout boundary. All pins attach into the closest side (top, bottom, left, or right) of boundary.

Picking and Placing

The **SDL > Pick and Place** command lets you select instances and pins in the schematic design and place them in the layout design.

The command also lets you place dummy instances on both sides of a selected device or an interdigitated device to maintain their electrical characteristics.

The following characteristics are applicable for the **Pick and Place** command and for the objects placed using the **Pick and Place** command:

- The command is context based. If you select the command and then traverse through the schematic design, the command remains active in the target layout design window.
- The object picked from the schematic design appears on the cursor. You can then specify the location in the layout design where you want to place the picked-up object.
- Vectored and m-factored objects are placed in a row to maximize area.

For information about adding devices or instances at the top (0x) level in digital blocks (IC Compiler II designs), see About Analog and Digital Co-designing in the *Custom Compiler and IC Compiler II Co-Design Assistant User Guide*.

Setting Up Pick and Place

You can pick and place objects from a schematic design into a layout design using the Pick and Place feature after you initialize SDL. For more information on initialization, see [Overview of Schematic Driven Layout Initialization](#).

Before using the Pick and Place feature, ensure that you have configured the Pick and Place options.

To configure the Pick and Place options:

1. When the target layout design specified during initialization is open in the Layout Editor, choose **SDL > Pick and Place**.

The **Pick & Place** toolbar opens.

2. Click the **Eject** button  at the right end of the **Pick & Place** toolbar.

The **Pick and Place** dialog box opens.

3. Specify the options (see [Pick and Place Options](#)).
4. Click **Hide**.

The Pick and Place command setup is complete.

Picking and Placing a Single Object

You can use the **Pick and Place** command to pick a single instance or pin from a schematic design and place it in the corresponding layout design.

Use the following procedure to pick and place a single object:

1. When the target layout design specified during initialization is open in the Layout Editor, choose **SDL > Pick and Place**.
2. Set the **Place** option as **Single Schematic Bit**.
3. In the **Pick & Place** toolbar, do one of the following:
 - Select **Instances** if you want to pick and place an instance.
 - Select **Pins** if you want to select a pin.
4. Make active the schematic design that was used during initialization.
5. Click the canvas to select the logical instance or pin that you want to pick and place.
6. Move the cursor to the Layout Editor window.

The picked object is visible on the cursor.

7. Click the location where you want to place the picked object.

The object is placed in the layout design and the command ends.

Note:

Pins are placed only on interconnect LPPs. If the active LPP is not an interconnect LPP, pins are placed on the nearest interconnect LPP.

Snapping and Flipping

When picking and placing the instance, the instance is snapped on track when specifying the following properties: `snapPointX`, `snapPointY`, and `snapTrackPatterns`. When `instFlipX` or `instFlipY` is specified in the `snapTrackPatterns` properties, the instance is flipped with specifying settings. For example, `((layer:M1 width:0.04 offset:0.082 instFlipX:{01}) (layer:POLY))`.

Properties		
Name	Value	Master
<code>snapPointX</code>	Add...	0.033
<code>snapPointY</code>	Add...	0.082
<code>snapTrackPatterns</code>	Add...	<code>((layer:M1 width:0.04 offset:0.082 instFlipX:{01}) (layer:POLY))</code>
Click to add		

Creating Photonic Pins

If a logical `oaTerm` has the `oaStringProp connectType` with the value `photonic`, the tool creates the `oaPinFig` from either **Pick and Place** or **Generate Layout**. The tool adds the `oaStringProp connectType` on layout `oaTerm`, and the value is `photonic`.

The `oaStringProps angel`, `width`, `location`, `radius`, and `crossSection` are added on layout `oaPinShape`. The values of `oaStringProps` are in `lePinPhotonicAngel`,

`lePinPhotonicWidth`, `lePinPhotonicLocation`, `lePinPhotonicRadius`, and `lePinPhotonicCrossSection`.

Picking and Placing Multiple Objects

You can use the **Pick and Place** command to place multiple objects at the same time. You can place multiple instances in two ways:

- Single Schematic Bit

When the **Place** option in the **Pick and Place** dialog box is set to **Single Schematic Bit**, SDL places multiple objects when the selected object is vectored, m-factored, or a hierarchical symbol.

- All Selected

When the **Place** option in the **Pick and Place** dialog box is set to **All Selected**, SDL places multiple objects based on what is selected in the schematic.

To pick and place multiple objects:

1. When the target layout specified during initialization is open in Layout Editor, choose **SDL > Pick and Place** to activate the **Pick and Place** command.

The **Pick & Place** toolbar opens.

2. Select **Single Schematic Bit** or **All Selected** from the **Place** drop-down menu.
3. Specify whether you want to pick and place instances, pins, or both.
4. Make active the schematic design that was used during initialization.
5. Click to select the objects that you want to pick and place.

Note:

To select all objects: Press Ctrl+A.

To select all unplaced objects: Press Ctrl+A; then deselect the **Replace Existing** check box in the **Pick & Place** toolbar.

6. Move the pointer to the Layout Editor window.

The picked objects are visible on the cursor.

7. Click the location in the layout where you want to place the picked objects.

The objects are placed in the layout.

Pins are placed only on interconnect LPPs. If the active LPP is not an interconnect LPP, pins are placed on the nearest interconnect LPP.

Note:

If you have set the **Place** option on the **Pick & Place** toolbar as **Single Schematic Bit**, the other selected objects are available in the cursor for successive placement.

8. If there are other objects remaining to be placed, click the locations where you want to place each of the other objects.

The picked objects are placed in the layout design.

Placing Multiple Objects by Selection Order

When the preference `lxSelectionOrderPlace` is true, the devices are placed by the selection order with the following conditions:

- **Place Like Schematic:** When this template is specified, the devices are placed one by one. If the preference is true, the devices are placed with the user-defined selection order. If the preference is false, the devices are placed alphabetically by name. **All Selected** is not supported in this situation.

For example, when you select I2 and then select I1 in browser. If the preference is false, the tool places I1, and then places I2. If the preference is true, the tool places I2, and then places I1.

- **Row Based:** When this template is specified, all devices are placed at a time. If the preference is true and no chaining or interdigitating in the selected objects, the devices are placed by the user-defined selection order from left to right in a row.

For example, when you select I2 and then select I1 in the browser, and no **chaining** and **interdigitating** in the selected objects. If the preference is false, the placed order is I1 I2 from left to right in the row. If the preference is true, the placed order is I2 I1 from left to right in the row.

Aligning Gates While Placing Objects

You can use the **Pick and Place** command to align objects by their gates as you place them.

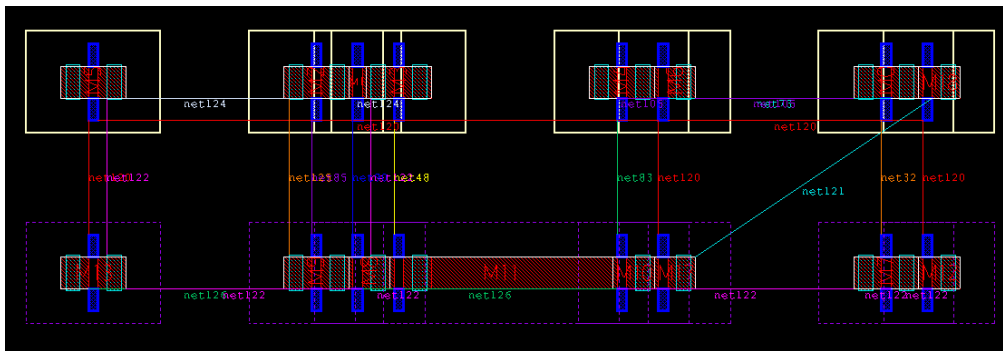
To align PMOS and NMOS gates during picking and placing:

1. Turn *on* the **Align Gates** option in the **SDL Options** dialog box before picking and placing.
2. Choose **SDL > Pick and Place** to activate the **Pick and Place** command. The **Pick & Place** dialog box opens.

3. Select the **Row Based** template and disable the **Interdigitate** section. (The gates cannot be aligned in the **Place Like Schematic** template or when enabling the **Interdigitate** section in the **Row Based** template.)
4. Follow the pick and place process to place the object in the layout.

Figure 47 shows the placed result with gates alignment. If the OD (oxide definition) extension parameter is defined, the abutting OD shapes may stretch to ensure gate alignment. If the OD extension parameter is not defined, a pair of abutted devices may be separated to ensure gate alignment.

Figure 47 Align Gates With OD Stretch



Pick and Place Options

The Pick and Place options are described in the following sections:

- [Pick and Place Dialog Box Options](#)
- [Pick & Place Toolbar Options](#)

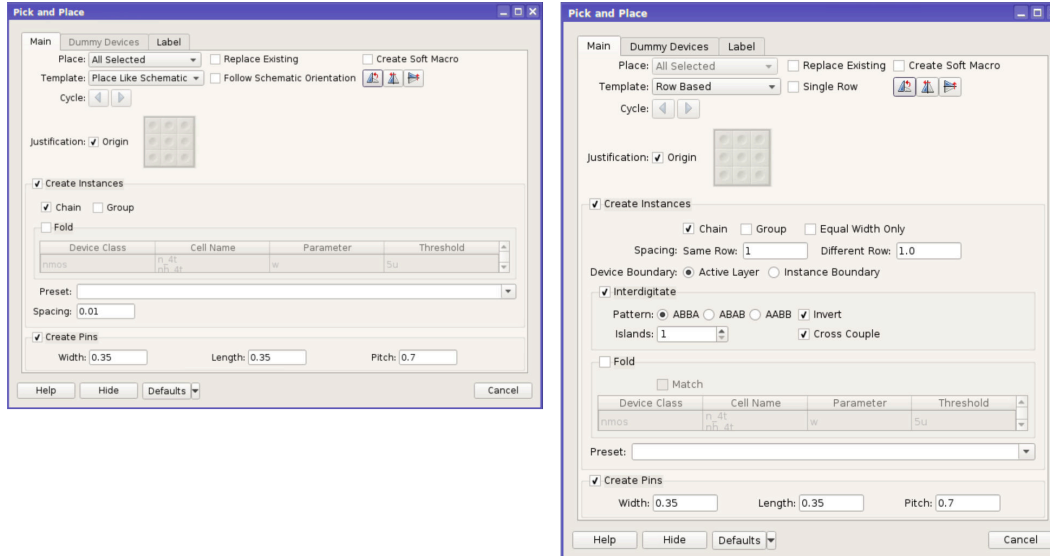
Pick and Place Dialog Box Options

The Pick and Place options available from the dialog box are described in the following sections:

- [Pick and Place Options — Main Tab](#)
- [Pick and Place Options — Dummy Devices Tab](#)
- [Pick and Place Options — Label Tab](#)

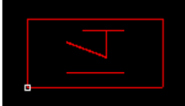
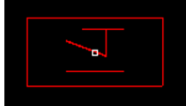
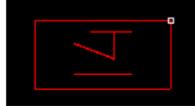
Pick and Place Options — Main Tab

The **Main** tab is always accessible, but the options available change depending upon which Template is selected: *Place Like Schematic* or *Row Based*.



Option	Description
Place	Lets you specify how objects must be placed when more than one objects are picked. <i>Single Layout</i>: Places an instance/pin, one layout device at a time. <i>Single Schematic Bit</i>: Places a single schematic bit instance/pin one at a time. <i>Single Schematic</i>: Places the selected schematic instance/pin one at a time. If the instance/pin is m-factored/vectored, then multiple objects are placed at one time. <i>All Selected</i>: Places all selected objects at the same time. Tcl preference: <code>lxPlaceSelectionMode</code>
Replace Existing	Lets you specify how to handle selected objects that already exist: <i>Enabled</i>: Specifies that already existing objects are replaced. <i>Disabled</i>: Specifies that already existing objects are ignored. Tcl preference: <code>lxReplaceExisting</code>
Create Soft Macro	Let you create a soft macro for the non-leaf node and then place the corresponding instances to layout. The default is <i>off</i>. See Soft Macro Creation for more details. Tcl preference: <code>lxCreateSoftMacro</code>

Option	Description
Template	Lets you select the template that contains the algorithms used as the basis while placing instances into the layout design. • <i>Place Like Schematic</i> (default): Hierarchically preserves the original schematic topology of the devices and instances. Removes instance and device overlap. • <i>Row Based</i>: If chaining has been enabled, performs chaining on different devices that belong to the same abutment class (abutClass), that might be present in different levels of the schematic hierarchy but connected on the same net. If chaining is disabled, then the devices are placed on different rows according to their abutment class. Also, when selected, adds the Dummy Devices tab to the form and adds the interdigitate options to the Main tab. To enable chaining, see Generate Layout Options — Main Tab. Tcl preference: <code>lxPlacementTemplate</code> Note: You can also create your own template using Tcl procedures and add that template to this list.
Single Row	Let you place all devices into a single row. If <code>lxSelectionOrderPlace</code> is true, the devices are placed by the selection order. If <code>lxSelectionOrderPlace</code> is false, the devices are sorted by name alphabet of device. Tcl preference: <code>lxSingleRowPlacement</code>
Follow Schematic Orientation	Let you create instances with the orientation in the schematic. Tcl preference: <code>lxFollowSchematicOrientation</code>
Transform buttons	Let you rotate or flip the objects during placement.
Cycle	Let you cycle through the set of selected objects. Note: The arrow buttons are available when the Pick and Place mode is other than <i>All Selected</i> and the template is not Row Based.

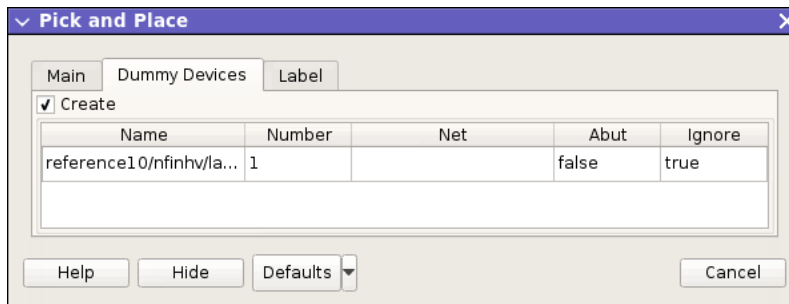
Option	Description
Justification	Specifies the anchor point of the instance. You can set the anchor point to Origin or specify the anchor point in the justification table, which provides the following options: Upper-Left, Center-Left, Lower-Left, Upper-Center, Center-Center, Lower-Center, Upper-Right, Center-Right, or Lower-Right. The default is Center-Center.    Lower-Left Center-Center Upper-Right Tcl preference: <code>leInstJustification</code>
Create Instances	Specifies to create instances. When enabled, various options become accessible depending on the specified template. Tcl preference: <code>lxCreateInstances</code>
Chain	Lets you specify whether the auto chaining feature must be enabled for selected objects; see Chaining Instances. Tcl preference: <code>lxAutoChain</code>
Group	Groups the multiple objects when realizing the selected objects. Tcl preference: <code>lxGroupRealizedInstances</code>
Equal Width Only	Accessible when template is <i>Row Based</i> and <i>Chain</i> is enabled. When enabled, any devices that have the same matching width parameters (as define in <code>lxComponentType</code>) are abutted or “chained” together. For FinFET devices, the width is represented by the number of fins. Note that <code>lxDeviceWidth</code> must be set to the appropriate parameter name for the number of fins (for example, <code>nfin</code>). If Fold is disabled, then the tool chains only equal dimension devices prior to folding. If Fold is enabled, the tool chains only equal dimension devices after folding. Tcl preference: <code>lxEqualWidth</code>

Option	Description
Spacing	Controls the spacing between devices and between rows. The unit can be specified by selecting Microns or Fins. • <i>Same Row</i>: The spacing between unchained and chained devices in the same row. Default is 1m. • <i>Different Row</i>: For non-FinFET devices, the spacing between each row. Default is 1m. • <i>Different Row Fins</i>: For FinFET devices, the number of row fins. Default is 1. Tcl preference: <code>lxIntraRowSpacing</code>, <code>lxInterRowSpacing</code>, <code>lxInterRowNumFins</code>
Device Boundary	Specifies the spacing of the instance by selecting one of the following options: • <i>Active Layer</i>: When selected, the row spacing is calculated according to the active layer setting. If the active layer is not defined, the setting of <code>leInstBoundary</code> is used. • <i>Instance Boundary</i>: When selected, the row spacing is calculated according to the setting of <code>leInstBoundary</code>. Tcl preference: <code>lxDeviceBoundary</code>
Interdigitate	Enables interweaving the device fingers to create interdigitated patterns. Tcl preference: <code>lxInterdigitatePattern</code>
Pattern	The interdigitation pattern to be applied (default: ABBA). Tcl preference: <code>lxInterdigitatePattern</code>
Invert	Specifies whether to invert the pattern within an island (default: true). Tcl preference: <code>lxInterdigitateInvertPattern</code>
Islands	The number of islands to create per chain (default: 1). Tcl preference: <code>lxInterdigitateNumIslands</code>
Cross Couple	Specifies whether to cross couple between islands (default: true). Tcl preference: <code>lxInterdigitateCrossCoupling</code>
Fold	Specifies to fold selected oalnst objects. Tcl preference: <code>lxAutoFold</code>
Match	Enabled when both <i>Fold</i> and <i>Chain</i> have been enabled and the template is <i>Row Based</i>. Specifies to find the highest common denominator folding parameter between the selected devices, using the folding threshold value. Tcl preference: <code>lxMatchedFolding</code>
Device Class	Specifies the device class for which folding is made available.

Option	Description
Parameter	Specifies the parameter that is considered for folding.
Threshold	Lets you specify the folding threshold. If the instance parameter value is higher than the fold threshold, folding is initiated.
Preset	Specify a preset parameters cellview to preset the instance parameters. To enable this option, specify the preset parameters library in the SDL Options dialog box. Tcl preference: lxPresetLCV
Spacing	See the Spacing option in the Main Tab of the Generate Layout Dialog Box table. Tcl preference: lxDeviceSpacing
Create Pins	Specifies to create pins with the specified width, length, and pitch values. For information about creating pin labels, see Setting Schematic Driven Layout Options . Tcl preference: lxPinLPP, lxPinWidth, lxPinHeight, lxPinPitch,

Pick and Place Options — Dummy Devices Tab

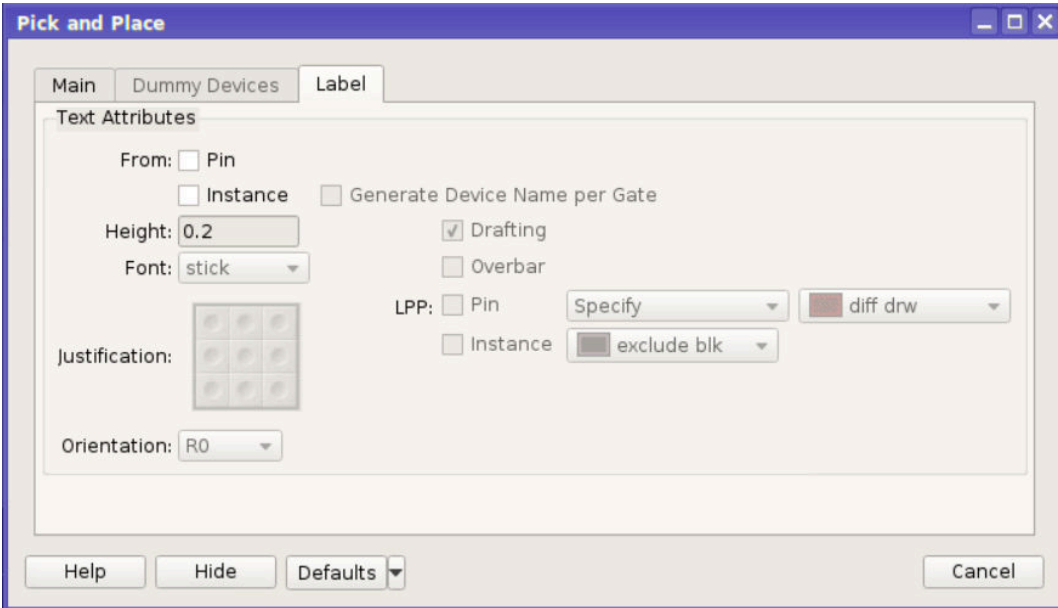
The **Dummy Devices** tab is accessible only when you select a *Row Based* template on the **Main** tab.



Option	Description
Create	Adds dummies to both ends of an island. • <i>Name</i>: The layout cellview of the selected schematic device. • <i>Number</i>: A nonnegative integer specifying the number of dummies to be added at both ends of the island. A value of 0 specifies no dummy device. Default is 1. • <i>Net</i>: Specifies the net on which to tie the instTerms of the dummy device. • <i>Abut</i>: Specifies whether to include the dummy devices in the chaining (abutment). Disabled if the <i>Chain</i> option on the Main tab is disabled. • <i>Ignore</i>: Specifies whether to add an <i>ignore</i> property on the dummy devices.

Pick and Place Options — Label Tab

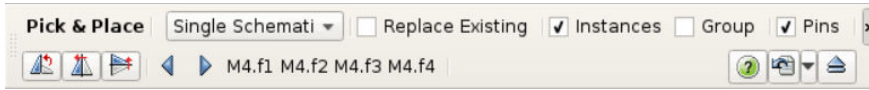
You can create instance labels at the center of the instances or pin labels in the **Label** tab. The **Label** tab contains the following options:



Option	Description
From	Enables Pin or Instance option to create the labels for pins or instances. Tcl preference: <code>lxCreatePinLabel</code> and <code>lxCreateInstanceNameLabel</code> • Generate Device Name per Gate: Controls whether to generate duplicate labels on each gate or just one label for realized instances. This option is available when the Instance option is enabled. Tcl preference: <code>lxGenerateLabelPerGate</code>
Height	Sets the text height in user units.
Drafting	Sets the text orientation to the conventional value based on the object orientation.
Font	Sets the font face. The available font types are euroStyle, gothic, math, roman, script, stick, fixed, swedish, and milSpec.
Overbar	When enabled, displays an overline above the text.
Justification	Positions the text relative to the origin by Upper-Left, Center-Left, Lower-Left, Upper-Center, Center-Center, Lower-Center, Upper-Right, Center-Right, or Lower-Right. The default is Center-Center. To center the text on the origin, choose Center-Center.
LPP	When the Pin or Instance option is enabled, creates the label on the specified layer purpose pair. When disabled, creates the label on the active layer. • When Pin is enabled, creates the label by defining LPP with the following methods: • Specify: Specifies layer purpose pair. • Map: Clicks the Browser button to select a map file defined shape and text LPP mapping pair. For example: <pre>##### # Shape LPP Text LPP # ##### m1 drawing m1text drawing m2 drawing m2text drawing m3 drawing m3text drawing</pre>
Orientation	Sets the rotation and mirroring of the label text by R0, R90, R180, R270, MX, MY, MXR90, or MYR90. The default is R0.

Pick & Place Toolbar Options

The **Pick & Place** toolbar options are a subset of the dialog box options.



To see more options, click the **Eject** button  to open the dialog box.

Option	Description
Pick & Place	Lets you specify how objects must be placed when more than one objects are picked. • <i>Single Layout</i>: Places an instance/pin, one layout device at a time. • <i>Single Schematic Bit</i>: Places a single schematic bit instance/pin one at a time. • <i>Single Schematic</i>: Places the selected schematic instance/pin one at a time. If the instance/pin is mfactored/vectored, then multiple objects are placed at one time. • <i>All Selected</i>: Places all selected objects at the same time.
Preset	Lets you specify preset parameters for the selected objects. To enable this option, specify the preset parameters library in the SDL Options dialog box.
Replace Existing	Lets you specify how to handle selected objects that already exist: • <i>Enabled</i>: Specifies that already existing objects are replaced. • <i>Disabled</i>: Specifies that already existing objects are ignored.
Instances	Specifies to create instances using the options accessible on the Pick and Place dialog box.
Group	Creates a group when realizing multiple objects on the Pick and Place dialog box.
Pins	Specifies to create pins using the options accessible on the Pick and Place dialog box.
Transform buttons	Let you rotate or flip the objects during placement.
Arrow buttons	Let you cycle through the set of selected objects. Note: The arrow buttons are available when the Pick and Place mode is other than <i>All Selected</i> and the template is not Row Based.

Realizing a Layout With Preset Parameters

This section describes the following topics:

- [Realizing Layouts](#)
- [Customizing IxPresetConfig Setting](#)
- [Customizing Preset Designs](#)

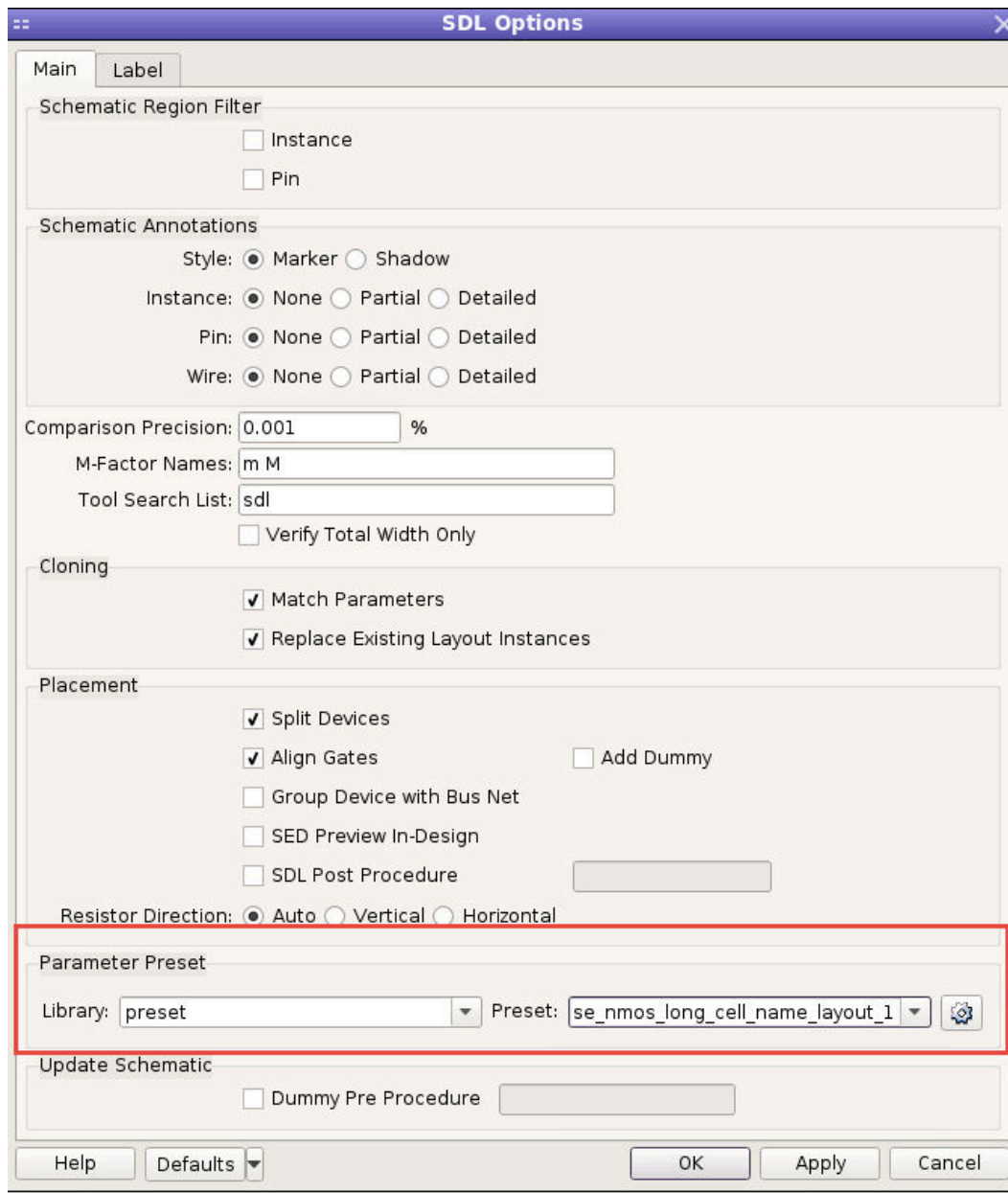
[Show me](#) how to create and use SDL preset.

Realizing Layouts

To realize a layout using preset parameters from the preset library:

1. Prepare preset designs in one preset library and create many instances with different parameters in your preset design.
2. Specify the preset parameters for picking the valid preset instances for the selected instances. See [Pick Valid Preset Instances](#).
3. Open the **SDL Options** dialog box to specify the preset library to enable the **Parameter Preset** feature.

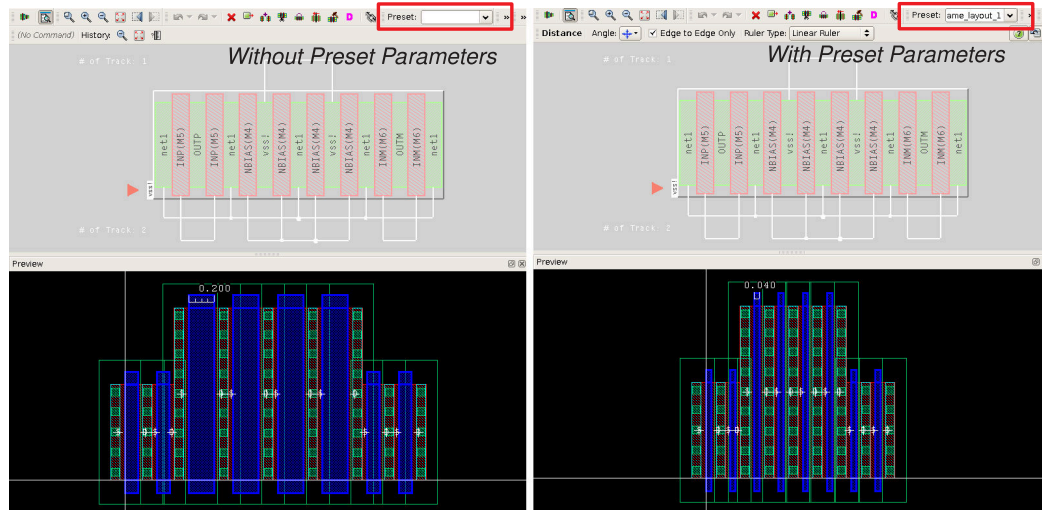
Figure 48 Enable Parameter Preset



4. In the Design Navigator, select the target objects that you want to place, and then do one of the following:
 - Choose **SDL > Pick and Place** to activate the **Pick and Place** command.
 - Choose **SDL > Symbolic Editor (Chaining or Matching)**.

5. From the Preset list, choose the layout view. [Figure 49](#) shows the objects with or without preset parameters in chaining mode.

Figure 49 Preset Parameters in Chaining Mode

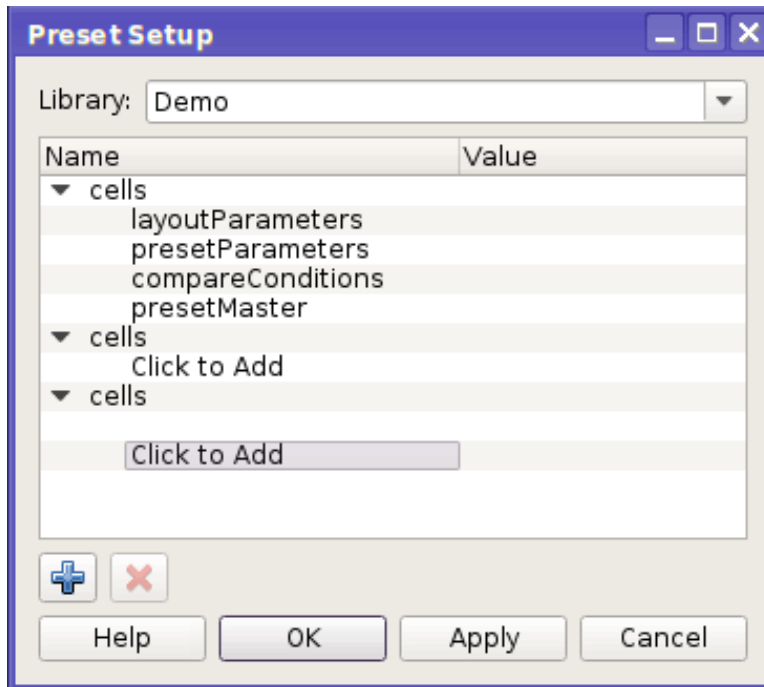


6. Realize the valid preset instances to layout.

Customizing lxPresetConfig Setting

You can customize the `lxPresetConfig` setting by using the `lx::showPresetSetup` command to invoke the **Preset Setup** dialog box.

Figure 50 Preset Setup Dialog Box



When specifying `lx::showPresetSetup` in the Console, the **Preset Setup** dialog box is displayed to set up the `lxPresetConfig`.

1. Specifies the library used to setup the `lxPresetConfig` in the **Library** selection field.
2. To add the cells with empty value, right-click on the **Name/Value** table and choose **Add Cells**. Or click the **Add** button  to add the cells.
3. To add the attribute with empty value, click the cells column and specify an attribute by selecting one of the following types:
 - [layoutParameters](#)
 - [presetParameters](#)
 - [compareConditions](#)
 - [presetMaster](#)
4. To delete the cells or attribute, right-click on the selected cells or attribute and choose **Delete**. Or click the **Delete** button  to delete the selected cells or attribute.

5. To import the `lxPresetConfig` setting, change the library in the **Library** selection field and the existed `lxPresetConfig` setting should be syntax correct and completed.
 6. To export the `lxPresetConfig` setting in the selected library, click **OK** or **Apply**.
-

Customizing Preset Designs

By default, you can place objects with preset parameters when specifying the preset library in the **SDL Options** dialog box. See **Parameter Preset** in [Main Tab](#).

However, you can create many instances with different parameters in your preset design when you need to have preset designs in one preset library. The view name of the preset design must be layout. The tool picks the valid preset instances for the selected instances according to the CDF parameters settings in `lxPresetConfig`.

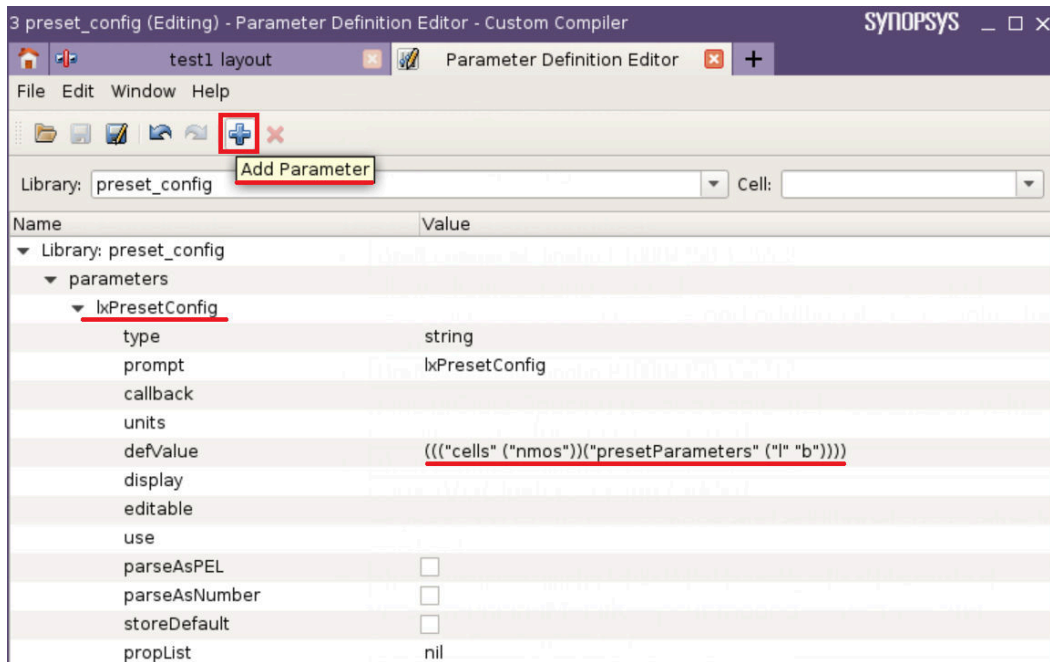
This section describes how to:

- [Add Valid CDF Parameters in lxPresetConfig Setting](#)
- [Pick Valid Preset Instances](#)

Add Valid CDF Parameters in lxPresetConfig Setting

To specify a valid `lxPresetConfig` setting in the preset library:

1. In the Library Manager, choose **Edit > Parameter Definitions**.
2. In the **Library** list, choose preset.
3. On the Parameter Definition Editor toolbar, click **Add Parameter**. `param_1` is added in the **parameters** list.
4. Rename `param_1` to `lxPresetConfig`.
5. Set `defValue` to `"cells" ("nmos") ("presetParameters" ("l" "b"))`.



Pick Valid Preset Instances

The rules for picking the valid preset instances for the selected instances are:

- Same layout master: Only the instances with the same layout master can be candidates or match the master defined in `lxPresetConfig`.
- Same parameter values: In the candidates, only the instances with the same parameter values defined in `instParameters` or `presetsParameters` as the preset design can be used.

The parameters defined in the `instParameters` section of the `sdl simInfo` are used to compare parameter values. However, if you used the Parameter Definition Editor to specify an `lxPresetConfig` parameter for some specified cells, the `lxPresetConfig` parameter values take precedence, and are used to find instances with matching parameter values. The `lxPresetConfig` setting must be specified in the preset library and the format of `lxPresetConfig` is

```
((("cells" ("cell_name1" "cell_name2" ...)) ("presetParameters" ("param1"
"param2" ...)) (("cells" ("cell_name3")) ("presetParameters" ("param1"
"param2" ...)))))
```

Figure 51 shows an example of a valid candidate when comparing `instParameters (l w)`:

Figure 51 Valid Values When Comparing `instParameters`

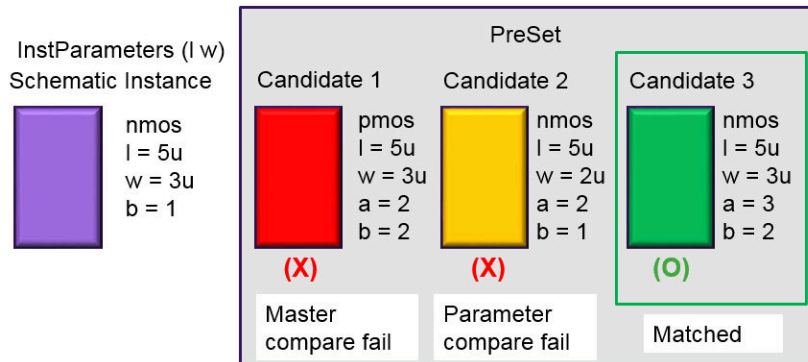
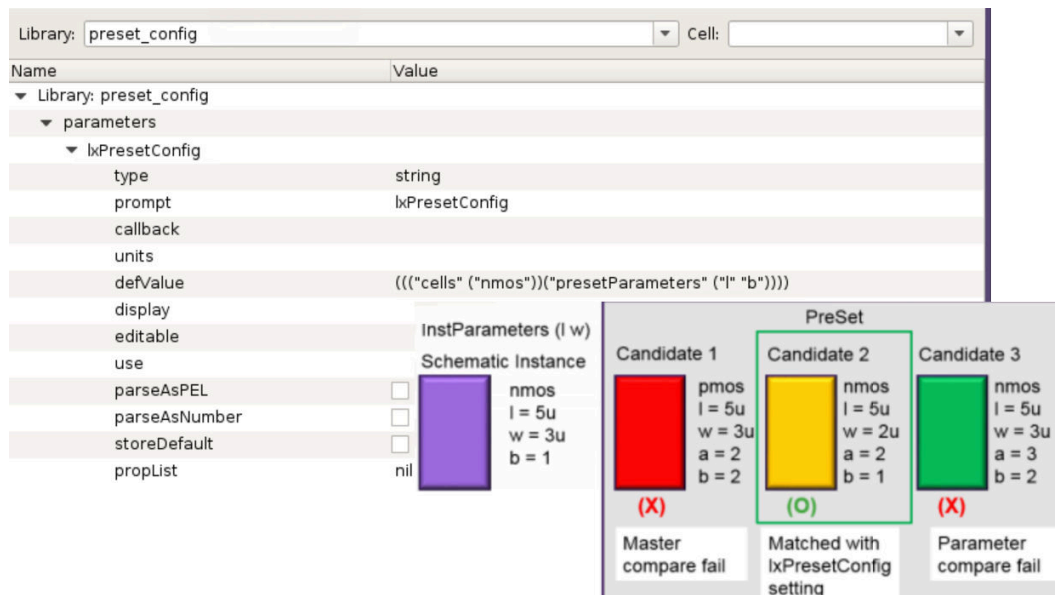


Figure 52 shows an example of a valid candidate when `lxPresetConfig` is specified.

Figure 52 Valid Values When Comparing `lxPresetConfig` Settings



Note:

For dummy devices, the preset candidates with dummy usage have higher priority.

- **Preset Master:** You can specify the `presetMaster` to add a new rule for picking the valid preset instances for the selected instances. The setting for the preset master is defined in the `lxPresetConfig` section. The format of `presetMaster` is

```
((("cells" ("cell_name1" "cell_name2" ...)) ("presetMaster" "cell_name2"))))
```

See [Add Valid CDF Parameters in lxPresetConfig Setting](#) or [Customizing lxPresetConfig Setting](#) to add the `presetMaster`. The settings of `presetMaster` and `presetParameters` can be specified together.

The first step is the same with the procedure of preset parameters. The instance master is checked. The instances with same layout master are candidates. If the instance with a different master is found, the instance specified in the `presetMaster` can also be the candidate for the selected instances.

After the candidate is found in the `presetMaster`, only the instances with the same parameter values defined in `instParameters` or `presetParameters` as the preset design can be used. See [Same parameter values](#).

[Figure 53](#) shows the example when specifying the `presetMaster` with `pfin_18` in the `lxPresetConfig` section. `pfin_18` is the candidate for schematic instances `pfin` and `pfin_27`, even though they have different masters. The parameters of `pfin_18` can be the preset parameters for `pfin` and `pfin_27`. If `pfin_18` has the same parameter values as `pfin` when comparing `instParameters(l w)`, the parameters of `pfin_18` are the preset parameters for `pfin`.

Figure 53 Preset Master Example

lxPresetConfig	
type	string
prompt	Custom Compiler Preset Config
callback	
units	
defValue	<u>((("cells" ("pfin_18" "pfin")) ("presetMaster" "pfin_18")) ((("cells" ("nfin_18" "nfin")) ("presetMaster" "nfin_18"))))</u>
display	nil
editable	t
use	t
parseAsPEL	☐
parseAsNumber	☐
storeDefault	☐

Figure 54 defValue Example

```
((("cells" ("pfin_18" "pfin")) ("presetMaster" "pfin_18")) ((("cells" ("nfin_18" "nfin")) ("presetMaster" "nfin_18"))))
```

- **Layout Parameters:** You can specify the `layoutParameters` to specify which layout parameters needed to be copied over from preset instance instead of copying

all of layout parameters. The setting for the preset parameters is defined in the `lxPresetConfig` section. The format of `layoutParameters` is

```
((("cells" ("cell_name1" "cell_name2" ...)) ("layoutParameters"
"parameter_name")))
```

See [Add Valid CDF Parameters in lxPresetConfig Setting](#) or [Customizing lxPresetConfig Setting](#) to add the `layoutParameters`.

Library: preset_config	
▼ parameters	
▼ lxPresetConfig	
type	string
prompt	lxPresetConfig
callback	
units	
defValue	((("cells" ("pch_lt_mac")) ("presetParameters" ("nFin") ("layoutParameters" ("dummyMdNumLeft" "dummyMdNumRight")))))
display	nil
editable	t
use	t

- **Compare Conditions:** You can specify the `compareConditions` to specify conditions to compare instance parameters of source instance and preset instance. The setting for the preset condition is defined in the `lxPresetConfig` section. The format of `compareConditions` is

```
((("cells" ("cell_name1" "cell_name2" ...))
("compareConditions" ("condition1" "condition2" ...))))
```

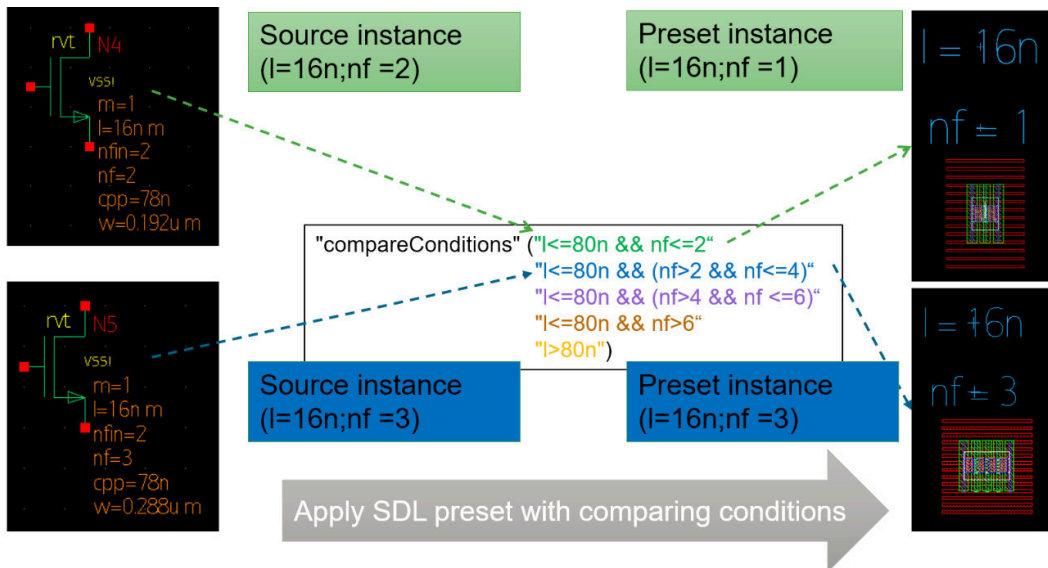
The settings of `compareConditions` and `presetParameters` can be specified at the same time. The priority of `compareConditions` is higher than `presetParameters`.

To compare conditions:

1. Use source instance parameters ($l=16n;nf=2$) to pick one condition.

The source instance is matched the condition ($l \leq 80n \ \&\& \ nf \leq 2$).

2. Use the same condition to find the corresponding preset instance. The preset instance ($l=16n;nf=1$) is found to use.



Chaining Instances

While placing instances into a layout design, the **Chain** feature lets you specify whether instances must be chained automatically.

When enabled, instances are aligned in a row and abutment is triggered wherever required, provided abutment is resident in the instance master.

M-factored and vectored instances are placed in a row.

Folding Instances

While placing instances into a layout design, you can specify whether instances must be folded automatically. Folding is the process of splitting devices automatically based on the folding threshold provided.

Folded layout instances have a **.f#** suffix added to their name to indicate that they are created as a result of folding.

Note:

1. If both **Fold** and **Chain** are enabled, the instances are first folded and then chained.

```
Example
Schematic w           = 4µ
Folding Threshold     = 1µ
Number of folded devices = Parameter value / Folding Threshold
                      = 4/1
                      = 4
Width of each instance = Parameter value / number of devices
                      = 1µ
```

2. You can define `lxValidWidths` in `lxComponentType` (see [Table 62](#)) to specify one or several folding widths. If the schematic width is divisible by the specified folding width, then the tool uses the width as parameter threshold to do folding. The tool finds the valid specified folding width from the first to last.

```
Example
Schematic w           = 2.4µ
lxValidWitdhs         = ("0.9µ" "0.6µ" "0.3µ")
Number of folded devices = Parameter value / Folding Threshold
                      = 2.4/0.9 (X)
                      = 2.4/0.6 (O)
Width of each instance = 0.6µ
```

Establishing Correspondence

You can view and modify the correspondence between terminals and instances in the schematic and layout views.

This section describes the following:

- [Viewing Correspondence Relationships](#)
- [Setting and Unsetting Correspondence Relationships](#)
- [Establish Correspondence Toolbar Options](#)
- [Establish Correspondence Dialog Box Options](#)
- [Establish Correspondence Shortcut Menu](#)
- [Establish Correspondence From LVS Results](#)

Viewing Correspondence Relationships

The **Establish Correspondence** dialog box displays a hierarchical netlist of the instances and terminals in both the logical design (schematic view) and its corresponding physical design (layout view).

To view correspondence relationships:

1. When the source logical design and target layout design specified during initialization are open, choose **SDL > Establish Correspondence**.

The **Establish Correspondence** toolbar appears.

2. Click the **Eject** button on the toolbar.

The **Eject** button  on the toolbar opens the dialog box. The **Hide** button at the bottom of the dialog box closes the dialog box and displays the toolbar. You can set the `lxEstablishCorrespondenceDialogMode` preference to control whether the dialog box (true) or the toolbar (false) opens by default.

3. Specify the **Display** options to filter the netlist.
4. (Optional) Click **Expand** if you want to expand the corresponding hierarchies in the netlist at all levels of the hierarchy.

Note:

You can right-click a physical instance and choose the **Set/Unset Ignore** and **Split Device** commands. See [Establish Correspondence Dialog Box Options](#) and [Splitting Devices](#).

Setting and Unsetting Correspondence Relationships

You can use the **Establish Correspondence** dialog box to set, unset, and modify correspondences.

See the procedures in the following sections:

- [Setting Correspondence Relationships](#)
- [Unsetting Correspondence Relationships](#)

Setting Correspondence Relationships

To set a new correspondence relationship:

1. View the existing correspondence relationships between the logical and physical designs. For more information, see [Viewing Correspondence Relationships](#).
2. In the display window, select the logical instance or terminal.

The selected objects are highlighted in the schematic window.

3. Select the corresponding physical instance or terminal.

The objects are highlighted in the layout window.

Note:

To select multiple objects, hold down the Ctrl key while clicking additional objects. Schematic Driven Layout supports one-to-many relationships (that is, one logical instance to multiple physical instances).

If a relationship does not already exist between the selected logical and physical instances, the **Set** button is enabled and the instances are highlighted with an orange packet.

4. Click **Set**.

The correspondence relationship is now established between the selected instances or terminal and the instances are highlighted with a green packet.

Unsetting Correspondence Relationships

To unset an existing correspondence relationship:

1. View the existing correspondence relationships between the logical and physical designs. For more information, see [Viewing Correspondence Relationships](#).
2. Select the logical instance or terminal by clicking it.
3. Select the physical instance to which the logical instance has a correspondence relationship.

If a relationship already exists between the selected logical and physical instances, the **Unset** button is enabled and the instances are highlighted with a green packet.

4. Click **Unset**.

The correspondence relationship between the selected instances (or terminals) is removed and the instances are highlighted with an orange packet.

Establish Correspondence Toolbar Options

The **Establish Correspondence** toolbar contains the following options:



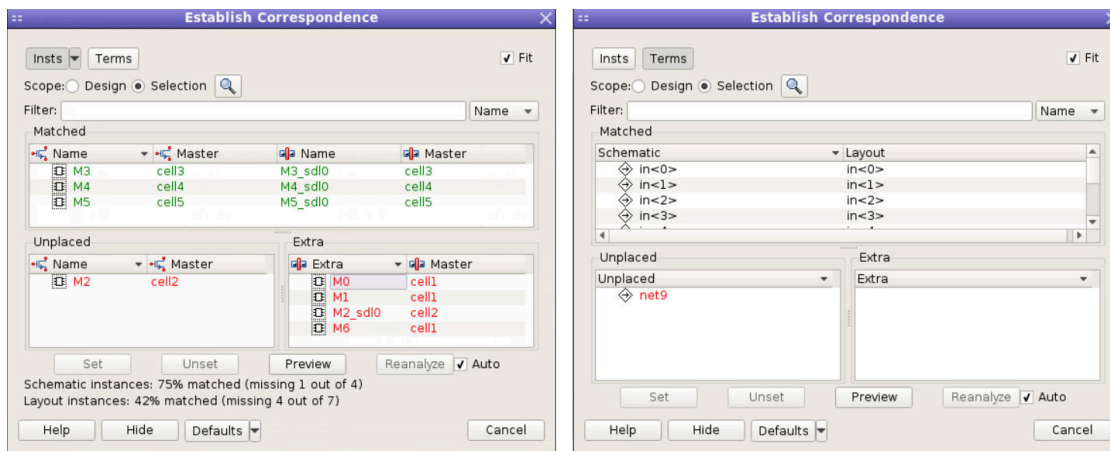
To see more options, click the **Eject** button  to open the dialog box.

Option	Description
Type	Specified one of the following type of objects to be applied to this command: Instance or Terminal .
Set	Enabled when a relationship does not already exist between selected logical and physical instances. Establishes a correspondence between the selected objects. Note: If you enable the Sync option, the logical name is applied to the physical instance. For more information, see Setting Schematic Driven Layout Options .
Unset	Enabled when a relationship already exists between the selected logical and physical instances. Removes the correspondence between the selected objects.

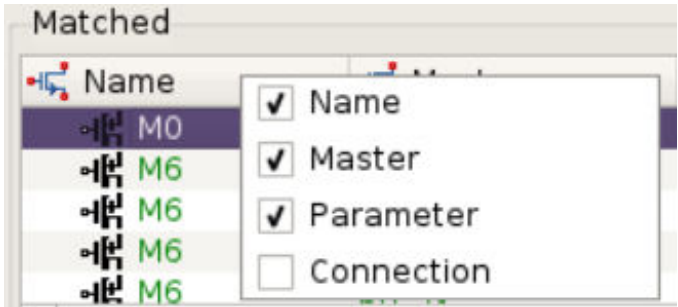
Establish Correspondence Dialog Box Options

The `lx::establishCorrespondence` command supports to run SDL Establish Correspondence command for establishing correspondence for the fully matched devices.

The **Establish Correspondence** dialog box contains the following options:



Option	Description
Insts	Potential Matched: Specifies whether to display instances having ambiguous topological correspondence (typically involving groups of similar devices or symmetrical subcircuits) Matched: Specifies whether to display instances where a relationship already exists between the logical (schematic) and physical (layout) designs. Ignore: Specifies whether to display instances marked as "ignored."

Option	Description
Terms	<i>Ignore</i>: Specifies whether to display terminals marked as "ignored."
Fit	When enabled (default), fits the current candidate in the logical (schematic or layout) view and candidates already existing in the layout view in their corresponding Schematic Editor and Layout Editor viewports.
Scope	Specifies whether to search the results from the schematic design or the selection by selecting Design or Selection .
Filter	When Name is selected, the matched objects are filtered by the contents of the Schematic or Layout column. When Master is selected, the matched objects are filtered by the contents of the Master column.
Matched	Displays elements from the schematic netlist and their corresponding matched elements from the layout netlist. In the Insts mode, you can enable the options to display the Name, Master, Parameter, and Connection information for the schematic netlist and layout netlist when right-clicking the Name/Master menu. 
Unplaced	Displays unplaced objects from the schematic netlist.
Extra	Displays extra objects from the layout netlist.
Set	Enabled when a relationship does not already exist between selected logical or physical instances or terminals. Establishes a correspondence between the selected objects. Note: If you enable the Sync option, the logical name is applied to the physical instance. For more information, see Setting Schematic Driven Layout Options.
Unset	Enabled when any selected logical or physical instance has existing correspondence. Removes the correspondence between the selected objects.
Preview	Clicks the button to display the Preview dialog box which lists the detailed of the mapping relationships for the selected matched objects. You can click Set or Unset to establish or remove the correspondence between the selected objects.

Option	Description
Reanalyze	Rechecks for potential matches and refreshes the display.
Auto	When enabled, the Reanalyze function is automatically invoked when the Set button is clicked.

Establish Correspondence Shortcut Menu

The **Establish Correspondence** menu contains the following options:

Set Correspondence	Return
Unset Correspondence	Shift+Return

Option	Description	Default Shortcut Key
Set Correspondence	Establishes a correspondence between the selected objects. Enabled when a relationship does not already exist between selected logical and physical instances. Note: If you enable the Sync option, the logical name is applied to the physical instance. For more information, see Setting Schematic Driven Layout Options .	Return
Unset Correspondence	Removes the correspondence between the selected objects. Enabled when a relationship already exists between the selected logical and physical instances.	Shift+Return

Establish Correspondence From LVS Results

The `lx::establishCorrespondenceFromLVS` command establishes the correspondence between schematic and layout when you have designs with LVS analysis succeeded, but layout views are not created with SDL.

The command generates a layout config file from the existing LVS results to rename layout instances to match schematic instances. The tool rebuilds the connection of layout instances to avoid connection mismatch after renaming layout instances.

Syntax

```
lx::establishCorrespondenceFromLVS -path <string> -layout <dmCellView |  
dmCellViewName> [-schematic <dmCellView | dmCellViewName>] [-hierarchy  
<bool>] [-skipCellFile <string> | -skipCells [<string>] | -skipLibraries
```

```
[<string>]] [-setIgnored <bool>] [-scalarizeArrayInst <bool>]  
[-scalarizeProc <string>] [-splitDevice <bool>] [-splitProc <string>]  
[-resetConnection <bool>]
```

- **path:** Specifies the existing LVS run directory.
- **schematic:** Specifies top schematic view of LVS run directory. The option is optional. If no specify schematic, the command will find top schematic from the existing layout config. If no existing layout config, the command will use the schematic from the same cell as layout.
- **layout:** Specifies top layout view of LVS run directory.
- **hierarchy:** If the value is false, the command works on top layout only. If the value is true, the tool establishes the correspondence for each layout view based on LVS hierarchy. The default is false.
- **targets:** Specifies the target cell views.
- **skipCellFile:** Specifies the skip cell list file which skips hierarchical verification.
- **skipCells:** The cells skip hierarchical verification.
- **skipLibraries:** The libraries skip hierarchical verification.
- **setIgnored:** Ignores the extra instances if all logical instances are matched
- **scalarizeArrayInst:** Scalarizes array instances. If the procedure to scalarize array instances is not defined, use **Edit > Hierarchy > Scalarize Array Instance**
- **scalarizeProc:** The procedure to scalarize instances.
- **splitDevice:** Splits devices. If the procedure to split devices is not defined, use **SDL > Split Device**.
- **splitProc:** The procedure to split devices.
- **resetConnection:** Resets connection using the **Query > Highlight > Connected** option.

The summary of the schematic instances and layout instances with different hierarchy is displayed in the console, or saved in the output file.

Note:

The correspondence cannot be established in the following situations even though the designs with LVS analysis succeeded.

- The schematic instance and layout instance are belonged to different cells. For example: A schematic instance of cell `INV` is mapped to a layout instance of cell `INV2`.
- The schematic instance and layout instance have different hierarchy. For example: A schematic instance binds to layout view in layout config, but it is mapped to flattened layout instances.
- The schematic instance and layout instance are unequal distribution. For example: there are one finger and two fingers at two schematic instances respectively, but the two schematic instances are mapped to a layout instance with three fingers.

Note:

The `lx::establishCorrespondenceFromLVS` command is available only with the Elite license.

Cloning Devices

Cloning identifies matching topologies in the elaborated design. Completed or partially completed sets of devices and interconnect can then be used as a template for other devices and interconnect.

Auto cloning is described in the following sections:

- [Cloning From Toolbar](#)
- [Cloning From Dialog Box](#)
- [Clone Toolbar and Dialog Box Options](#)
- [Array Placement Using the Clone Command](#)
- [Re-editing the Clone Group Using Symbolic Editor](#)

Cloning From Toolbar

The typical clone use model when using the **Clone** toolbar:

1. In the layout view, manually select a set of devices with completed connectivity.
2. Choose **SDL > Clone** to display the **Clone** toolbar.
3. Choose **From: Schematic** or **Layout** to identify the search area and then click **Search Clones in Target** to find the clone candidates.

4. Use the arrow buttons **Previous Candidate** and **Next Candidate** to select the target circuitries you want to copy into the layout. Those options are available when the **Cols** and **Rows** are equal to 1.

Note:

Enable **Highlight** to simultaneously highlight the selected candidate in the Design Navigator in Schematic mode, the schematic assistant, and the layout editor.

5. The runtime devices and their routings are automatically updated to match the schematic view by having the same device and connectivity. For example, when changing **Cols/Ros**, the runtime devices and their routings are automatically updated.

You can click to realize the selected clone candidates one by one in the layout view.

Note:

Choose **Reset Clone Source** to discard the current clone candidates and then use drag to select another device to be a new clone source.

Cloning From Dialog Box

The typical clone use model when using the **Clone** dialog box:

1. In the layout view, manually select a set of devices with completed connectivity.
2. Choose **SDL > Clone** to display the **Clone** dialog box.
3. Choose **From: Schematic** or **Layout** to identify the search area and then click **Search Clones in Target** to find the clone candidates.
4. (Optional) When enabling **Show Candidate Checkbox**, you can then enable the check boxes of those identified schematic circuitries and click to realize them in the layout view by the selected sequence in the **Candidates** table.

To select all candidates, you can:

- Right-click the candidate in the **Candidates** table to show the **Check All Selected** and **Uncheck All Selected** options.
- Click the check box next to the **Candidates** column heading to select or unselect all candidates.

When enabling, the selected candidates are highlighted in the Design Navigator in Schematic mode, the schematic assistant, and the layout editor when the **Highlight** option is selected.

Note:

In the Candidates table, unrealized candidates which do not exist in the layout are in bold and the candidates are in italic when realizing candidates in current procedure.

5. The runtime devices and their routings are automatically updated to match the schematic view by having the same device and connectivity. For example, when changing **Cols/Ros** or **DX/DY**, the runtime devices and their routings are automatically updated.

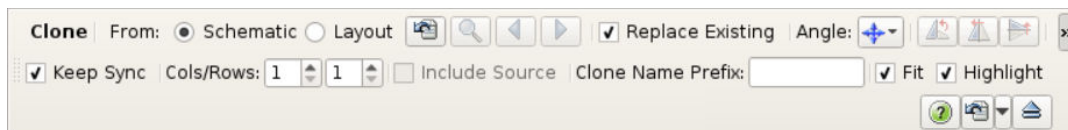
When the **Show Candidate Checkbox** option is disabled, you can realize the selected candidates by clicking on layout canvas one by one.

Note:

Choose **Reset Clone Source** to discard the current clone candidates and then use drag to select another device to be a new clone source.

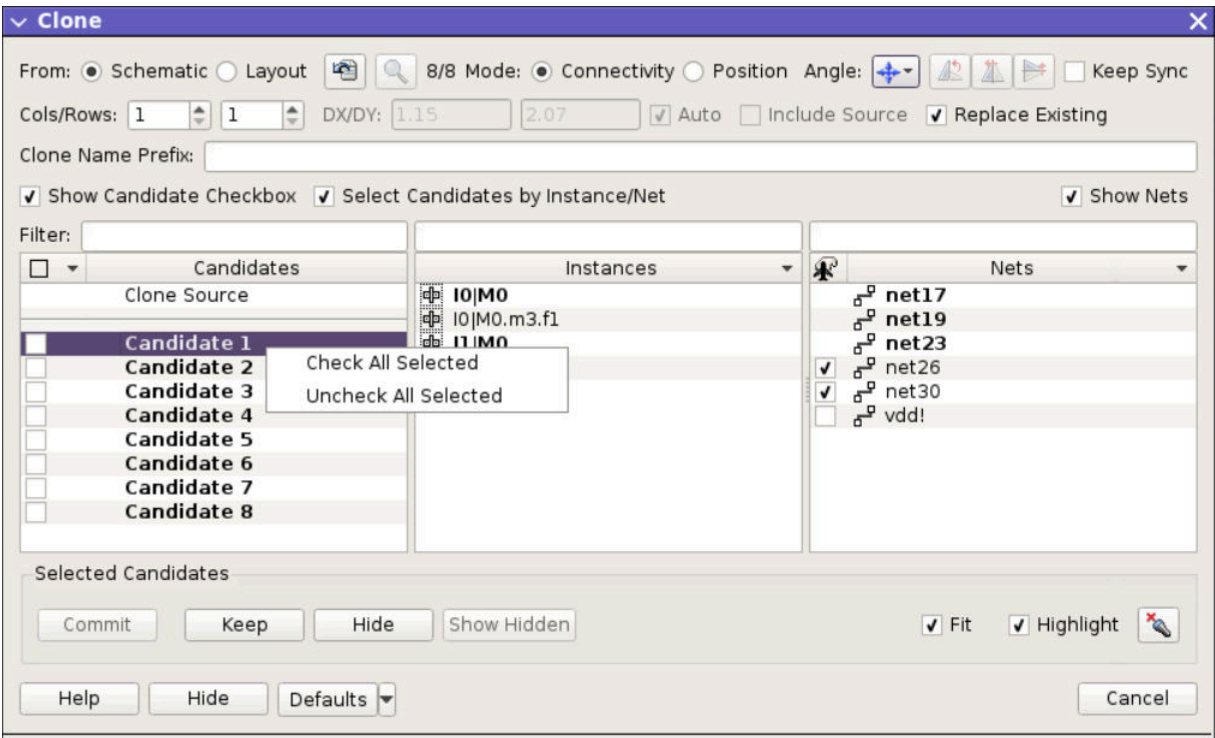
Clone Toolbar and Dialog Box Options

The **Clone** toolbar contains the following options:

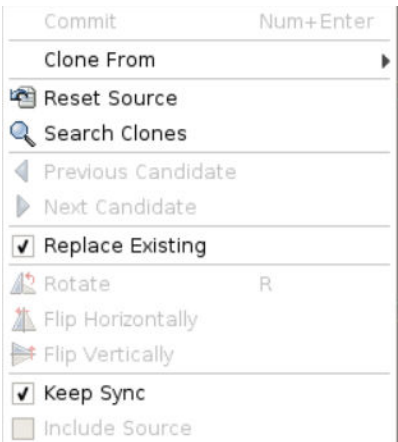


To see more options, click the **Eject** button  to open the dialog box.

The **Clone** dialog box contains the following options:

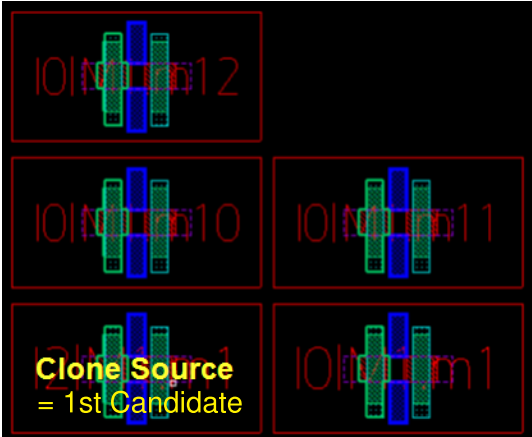


The **Clone** right-clicking menu contains the following options:



Option	Description
From	• <i>Schematic</i>: Specifies that target candidates should be searched from the schematic view. • <i>Layout</i>: Specifies that target candidates should be searched from the layout view.

Option	Description
Reset Source	Resets the current clone source and candidates. After resetting, the current selected device is the next clone source.
Search Clones	Identifies clone candidates in the schematic or layout.
Arrow buttons (Toolbar Only)	Let you cycle through the set of target candidates. Only available when the Cols or Rows are equal to 1.
Mode (Dialog Box Only)	• <i>Connectivity</i>: (default) Specifies that the connectivity algorithm is used to search for target candidates. • <i>Position</i>: Specifies that the position algorithm is used to search for target candidates. Device connectivity is not considered in this mode.
Replace Existing	When enabled (default), the clone target replaces the existing layout instances that name-collide with the instances in the clone target.
Angle	• Lets you control cursor snap locations for all points except the first, while cloning objects. • <i>Orthogonal</i> lets you place objects at multiples of 90 degrees. • <i>Diagonal</i> lets you place objects positioned at multiples of 45 degrees. • <i>Any Angle</i> lets you place vertices without any restrictions on the positions of the resulting objects.
Transform buttons	Let you rotate or flip the objects before placement.
Keep Sync	When enabled (default), the instances contained by the synthesized target clone are added to a syncFigGroup; otherwise they are added to a normal oaFigGroup.
Cols/Rows	Defines the number of columns and rows when placing multiple candidates in an array. The default for each is 1.
DX/DY (Dialog Box Only)	Sets the origin-to-origin spacing in the horizontal (DX) and vertical (DY) directions between items inside the candidates in an array. The default is the width (for DX) and height (for DY) of the bounding box of the source candidate enlarged to the manufacturing grid. For more information, see Array Placement Using the Clone Command.
Auto (Dialog Box Only)	When enabled (default), the Clone command automatically resets the DX and DY values based on the bounding box of the clone source when you execute the command or when the selection set is changed. For more information, see Array Placement Using the Clone Command.

Option	Description
Include Source	When enabled, the clone source is included in the current clone process and replaced with other candidates in array style. The clone source is rearranged as the first candidate when placing. This option is available when the Cols or Rows are larger than 1. 
Clone Name Prefix	Specifies the clone name prefix for clones newly created during the command cycle. This option is disabled if the clone source is a constraint figure group.
Show Candidate Checkbox	When enabled, displays the check boxed for the candidates. You can enable the check boxes to specify the target candidates to be realized. You can sort the checked candidates by clicking the arrow icon next to the Check/Uncheck All Candidates check box.
Select Candidates by Instance/Net	When enabled, selects instances/nets in their respective Instances or Nets boxes determines the selection of their corresponding candidates in the Candidates box. By default, this is disabled.
Show Nets	When enabled, displays the Nets filter and box. By default, this is disabled.
Filter	Filters candidates, instances, or nets by name.
Candidates (Dialog Box Only)	Displays all clone candidates found by search. The clone source is displayed at the top and separated from the remaining candidates.
Instances (Dialog Box Only)	When Select Candidates by Instance/Net is disabled, this box displays all instances of candidates currently selected. If no candidates are selected, it displays all instances of all candidates currently displayed. When Select Candidates by Instance/Net is enabled, this box always displays all instances of all unplaced candidates currently displayed.

Option	Description
Nets (Dialog Box Only)	This section is displayed when the Show Nets is enabled. When Select Candidates by Instance/Net is disabled, this box displays all nets of candidates currently selected. If no candidates are selected, it displays all nets of all candidates currently displayed. When Select Candidates by Instance/Net is enabled, this box always displays all nets of all unplaced candidates currently displayed. You can select the check boxes to display the flight lines for the selected nets. The check boxes are disabled when you deselect the Query > Flight Lines > Enable check box.
Selected Candidates (Dialog Box Only)	• Commit: Commits in-place selected candidates already existing in the layout view. • Keep: Keeps selected candidates and hides all others. • Hide: Hides selected candidates. • Show Hidden: Displays all previously hidden candidates. • Clear Highlights button : Clears all highlights.
Fit	When enabled (default), fits the current candidate in the logical (schematic or layout) view and candidates already existing in the layout view in their corresponding Schematic Editor and Layout Editor viewports.
Highlight	When enabled (default), highlights the current candidate in the logical (schematic or layout) view and candidates already existing in the layout view in their corresponding Schematic Editor and Layout Editor viewports.

Array Placement Using the Clone Command

The **Clone** command supports creating candidates in an array with a specified number of columns (**Cols**) and rows (**Rows**). It also supports specifying distances between columns (**DX**) and rows (**DY**) of the candidate array.

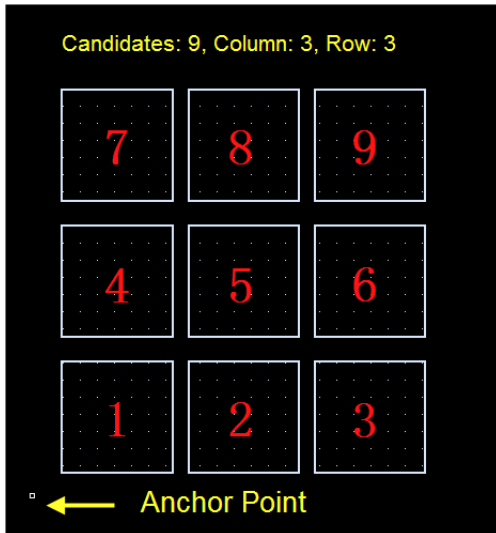
If columns or rows are set to more than 1, how clones are created depends on how you are invoking the **Clone** command:

- Clone toolbar — All candidates are considered as selected and the Clone command creates clones for *all candidates* in an array using the specified columns and rows,
- Clone dialog box — The **Clone** command creates clones for the *selected candidates* in an array using the specified columns and rows.

Runtime clones are composed of objects of the clone source. The objects of each selected candidate are treated as an integrated unit during placement. Selected candidates are

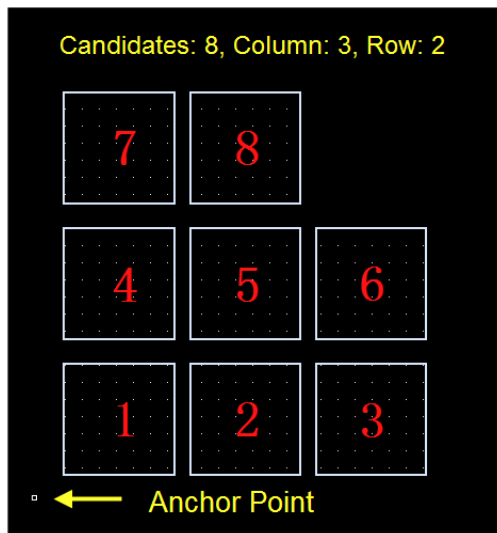
placed from left-to-right, bottom-to-top, in an array by their sequence in the candidates list (see [Figure 55](#)).

Figure 55 Position of Candidates Inside Array



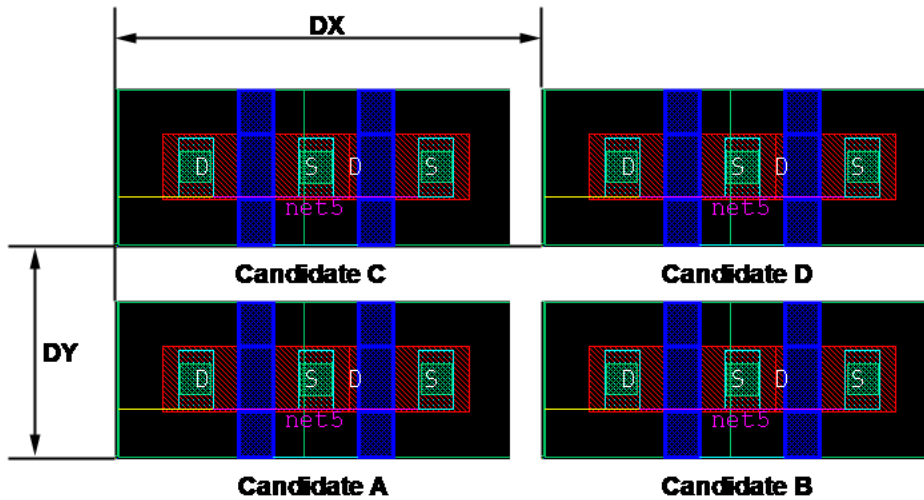
If the count of selected candidates is not equal to the number of columns multiplied by the number of rows, the **Clone** command uses the number of columns to place the selected candidates (see [Figure 56](#)).

Figure 56 Count of Candidates Not Equal to Columns Multiplied by Rows



The **Clone** command also supports specifying distances between columns (**DX**) and rows (**DY**) of the candidate array (see [Figure 57](#)).

Figure 57 *DX and DY Between Columns and Rows of Candidate Array*



If the **Auto** option is unchecked, the distance between columns and rows of the candidate array uses the specified values of **DX** and **DY**.

If the **Auto** option is checked, the distance between columns and rows of the candidate array uses the default values of **DX** and **DY**.

If the clone source exists and you turn off the **Auto** option, the values of **DX** and **DY** are reset with the default **DX** and **DY** of the current clone source.

If the runtime clones are already displayed and then you modify the value of **Cols** or **Rows**, the **Clone** command uses the new value of **Cols** or **Rows** to create candidates.

If the runtime clones are already displayed and then you modify the values of **DX** or **DY**, the Clone command uses the new value of **DX** or **DY** to create candidates.

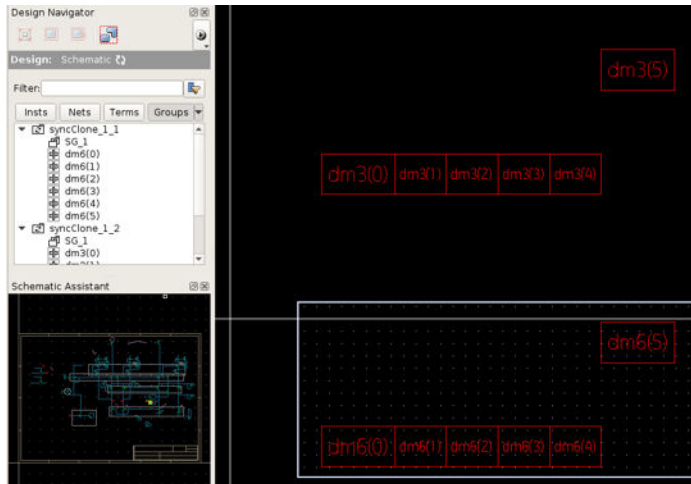
You can also use the Transform buttons to change the orientation of the runtime clones. This operation is effective for every selected candidate.

Re-editing the Clone Group Using Symbolic Editor

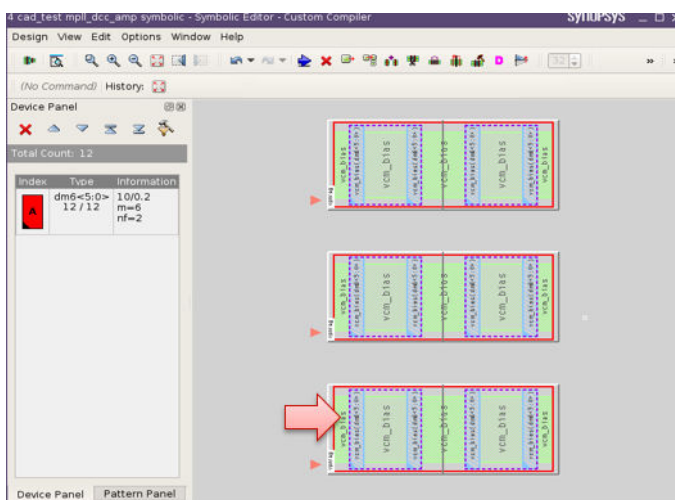
Symbolic Editor supports to keep synchronizing the clone groups after reediting the clone group if no device is added or deleted.

The typical use model for reediting the clone group is:

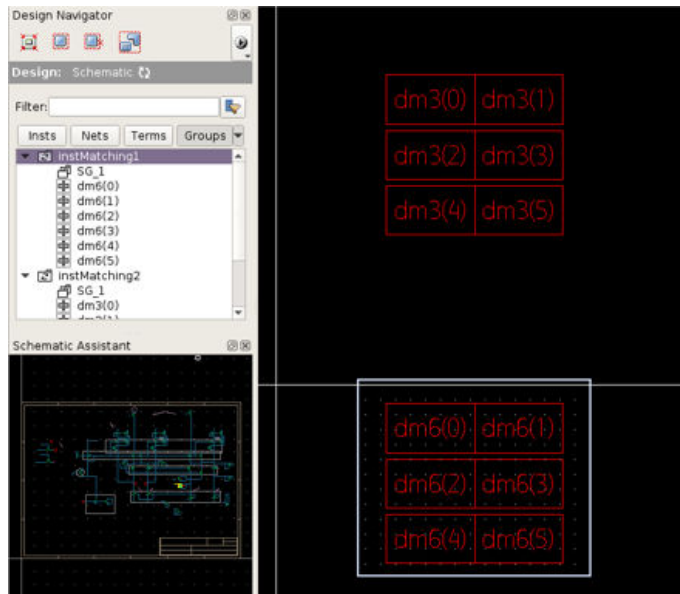
1. Create two syncClone groups which are synchronized.



2. Select a synchronized group and enable the Symbolic Editor.



3. Re-edit the group using the Symbolic Editor.
4. After realization, another syncClone group is also updated. Both clone groups are still synchronized.



5. If a device is incremental added when reediting using the Symbolic Editor, an extra instance is also created to another clone group after realization. The syncClone groups are out of synchronized.

If a device is deleted when reediting using the Symbolic Editor, the syncClone groups are out of synchronized after realization.

Splitting Devices

SDL supports two methods for splitting multi-fingered devices into multiple layout instances:

- The **Split Devices** option on the **SDL Options** dialog box lets you control whether devices are split as you place them in the design (see [Setting Schematic Driven Layout Options](#)).
- The **SDL > Split Device** command lets you split devices that have already been placed in the design (see [Splitting Placed Devices](#)).

Splitting Placed Devices

This section describes how to use the **SDL > Split Device** command.

The command action `lxSplitDevice` is located in the **Layout Editor** window of the SDL session, under the **SDL** menu.

For every instance received as input, the command verifies if the instance has a split parameter. For this, the command searches for the `lxComponentType` parameter and tries to find the value of `lxFingerParam`, which contains the name of the split parameter.

If the instance has a split parameter and the parameter value is greater than 1, then the command splits the instance into multiple instances based on the **Pattern** option. The default of the minimum finger (minimum split size) is 1. However, if `lxMinFinger` is defined in `lxComponentType`, the command considers the parameter value of `lxMinFinger` as the minimum split size.

The following rules apply:

- Only `oaScalarInst` instances are processed.
- Among a set of instances, only those having a split parameter with a value greater than 1 are processed.

The Split Device command supports the following two selection modes, as well as Infix:

- [Splitting Preselected Devices](#)
- [Splitting Post-selected Devices](#)

Note:

The Split Device command can also be accessed from the **Establish Correspondence** dialog box (see [Viewing Correspondence Relationships](#)).

Splitting Preselected Devices

Use the following procedure:

1. Select the instances you want to split.
2. Choose **SDL > Split Device**.

The tool splits the devices and adds the newly created fingers to the selection set, then the command ends.

Splitting Post-selected Devices

Use the following procedure:

1. Choose **SDL > Split Device**.

The **Split Device** toolbar appears.

2. Specify whether to allow external connections. For information about the option choices, see [Split Device Toolbar Options](#).
3. Select the instances you want to split.

The tool splits the devices and clears the selection set, and then prompts you for a new selection.

4. Press **Esc** to end the command.

Split Device Toolbar Options

The **Split Device** toolbar contains the following options:



Option	Description
External Connections	Determines if external connections are allowed when abutting the resulting fingers. • Auto (default): Contacts are generated on the abutted terminals of the resulting fingers, if their common net has more than two connections in the layout netlist. • Enable: Contacts are allowed when abutting the resulting fingers. • Disable: Contacts are not generated. • Pattern: Specifies finger numbers for the resulting fingers. Default is 1.

Adding and Editing SDL Mapping Rules

You can use the **SDL > SDL Mapping** command to add or edit the SDL mapping rules for the design in the **SDL Mapping** dialog box.

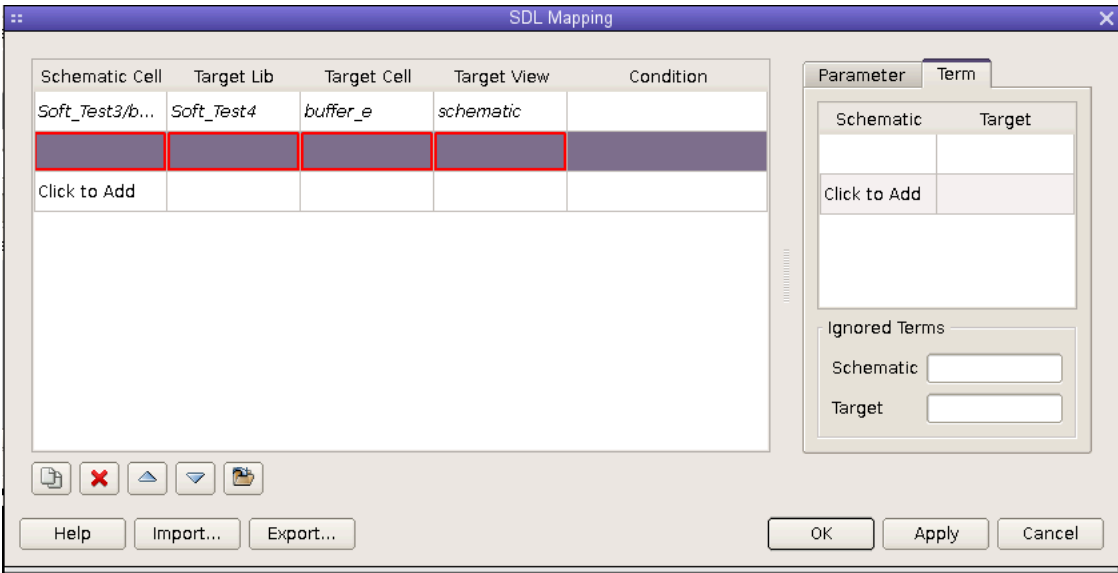
You also can use **Instance Mapping** to edit the mapping rules. For more information about **Instance Mapping**, see [Instance Mapping](#).

SDL Mapping is in cell scope and **Instance Mapping** is in instance scope. The priority for the mapping rules is Instance Mapping > SDL Mapping > Layout Config.

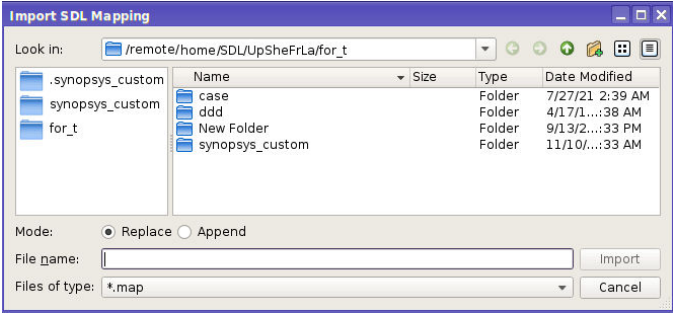
SDL Mapping Options

The following figure and table describe the **SDL Mapping** dialog box.

Figure 58 *SDL Mapping*



Option	Description
Schematic Cell	Selects the schematic cell to map. Wildcard character (*) is accepted for cell name matching.
Target Lib	Selects the library to which the schematic cell is mapped. The period (.) is supported to represent top library.
Target Cell	Selects the cell to which the schematic cell is mapped. Wildcard character (*) and wildcard matches (for example, A*) are accepted for cell name matching.
Target View	Selects the view to which the schematic cell is mapped.
Condition	You can specify the mapping condition by selecting parameters or ConfigBinding . The matched schematic cell is mapped.
Parameter	When specifying, the parameter in the Schematic column is used to replace the parameter in the Target column.

Option	Description
Term	When specifying, the terminal in the Schematic column is used to replace the terminal in the Target column. Ignored Terms: Specifies the schematic or target terminal in the Ignored Terms tab to ignore the mismatch checking. Wildcard character (*) is accepted for term name matching.
Import...	Click this button to import an SDL mapping file (YAML format) in the Import SDL Mapping dialog box.  • Mode: Specifies how to import the mappings from a file. ◦ In the Replace mode, all existing mappings are replaced by imported mapping file. ◦ In the Append mode, the tool appends the mappings from specified file to existing mapping.
Export...	Click this button to export SDL mapping information to a file (YAML format).

The example of SDL mapping file in YAML format:

```
---
- Version: 1.0.0
- Include: include_file_1.map
- Include: include_file_2.map
- Physical Mapping:
  - [placed_library1, placed_cell1]:
    - [switched_library1, switched_cell1, switched_view1]:
      Terminal Mapping:
        Target_Term1: Source_Term1
        Target_Term2: Source_Term2
      Ignored Schematic Terms: [Term_Name]
      ...
    - Parameter Mapping:
      Target_Parameter1: [Source_Parameter1]
      ...
    - ConfigBinding: [old_sw_lib1, old_sw_cell1]
  - [placed_cell2]:
    - [switched_library2, switched_cell2, switched_view2]:
      - Parameters:
```

```
param_name1: param_value1
param_name2: param_value2
- [switched_library3, switched_cell3, switched_view3]:
...
---
```

Note:

SDL mapping file supports include other mapping files:

- Both relative path and absolute path are acceptable for include files.
- The mapping rule describes first has higher priority than other mapping rules.

In the earlier example, the priority of the mapping rules is

include_file_1.map > include_file_2.map > the selected importing file.

- Include other mapping files only supports for **SDL Mapping > Import**.

Applying Preset Parameters

You can use the **SDL > Apply Preset Parameters** command to apply the preset parameters for the selecting layout instances.

[Figure 59](#) illustrates the **Apply Preset Params** toolbar.

Figure 59 Apply Preset Params Toolbar



You must set up the preset library before using this command. For more information, see **Parameter Preset** in [Main Tab](#).

Realizing Layouts Using Drag-And-Drop Operations

You can realize layout from schematic to layout using dragging. To perform a direct drag and drop operation, do the following:

1. Select the objects from Schematic Editor, Schematic Assistance, or SDL Browser.
2. While holding the middle mouse button, drag the objects to the layout canvas.
3. When placing the objects to the canvas, the **Pick and Place** command is invoked. See [Picking and Placing](#) for more information.

4. Or you can specify the value of the preference `lxDragAndDropMOS` to `SED`. The default is `PnP`. After all dragging objects are MOS devices, the Symbolic Editor with Chaining Mode is invoked when placing the objects to the canvas. See Using Chaining Mode for more details.

Triggering Callback Functions When Realizing Layout

The tool triggers the callback functions in order of the following precedence when the parameters are specified during the **Pick and Place** or **Generate Layout** operation.

1. `lxCDFCallbackParams`: This parameter is defined in CDF parameter. During the **Pick and Place** or **Generate Layout** operation, the callback functions are triggered for the parameters in the sequence specified in CDF parameter `lxCDFCallbackParams`.

The `lxTriggerCDFCallbackParams` preference controls `lxCDFCallbackParams`. The valid values of `lxTriggerCDFCallbackParams`:

- **Always**: When specified, the callback functions are always triggered.
 - **Auto**: When specified, the callback functions are triggered under the following conditions:
 - The switched master is a schematic PCell.
 - The instance uses `pPar`, `atPar`, or NLP Expressions for any parameters.
 - The default of the `lxCDFCallbackParams` is itself. For example, `lxCDFCallbackParams`.
 - **Never**: When specified, the callback functions are not triggered at all.
2. `lxFingerParam`: The finger parameter is defined in `lxComponentType`. This callback function is triggered, when the **Split Devices** option is turned on in the **SDL Options** dialog box and the finger parameter value is different between schematic and layout. For example, Schematic I1 has finger parameter and `nf = 2`. If placing with splitting, there are two layout instances I1.f1 and I1.f2, and `nf = 1` respectively after placing. The callback function of `nf` is triggered during the **Pick and Place** or **Generate Layout** operation.
 3. `lxDeviceWidth` or `lxDeviceFinNumber` for FinFET design: The width parameters are defined in `lxComponentType`. This callback function is triggered, when the **Fold** option is turned on in **Main** tab in the **Pick and Place** dialog box and the width or fin number parameter value is different between schematic and layout. For example, Schematic I1 has width parameter `w = 3u`. If placing with folding and threshold is `1u`, there are three layout instance I1.f1, I1.f2 and I1.f3, and `w = 1` respectively after placing. Thus,

the callback function of *w* is triggered during **Pick and Place** or **Generate Layout** operation.

4. When expanding m-factor or s-factor for an instance, its m-factor or s-factor parameter value is different between schematic and layout. **Pick and Place** or **Generate Layout** set the m-factor or s-factor parameter value to 1 and triggered its callback function.

Establish Correspondence Feature

You can view the correspondence between instances in a schematic netlist and a layout netlist using the Establish Correspondence feature. You can also use this feature to establish correspondence relationships between two elements and modify existing correspondence relationships.

For more information on the Establish Correspondence feature, see [Establishing Correspondence](#).

Cross-Object Referencing

This section describes the cross-object capabilities of SDL and explains how you can use the various cross-object capabilities.

It covers the following topics:

- [Cross-Selecting](#)
- [Cross-Probing](#)

Cross-Selecting

Cross-selection lets you easily select the equivalent objects of the current active object.

When cross-selection is enabled, all equivalent objects of the selected object are selected in corresponding design editor windows.

For example, when you select an instance in Layout Editor, the equivalent instance in the corresponding Schematic Editor window is selected.

Note:

Selecting an instance in a layout design that is below the current edit design in the schematic selects the parent schematic instance. Selecting a hierarchical symbol instance in the schematic cross-selects all objects representing the schematic hierarchy in the layout.

Enabling Cross-Selection

To enable cross-selection from Layout, click **Enable Cross Select**  on the toolbar.

Cross-Probing

Cross-probing lets you create hierarchical probes for equivalent objects. Probes help you visualize a net or other objects throughout the hierarchy.

When you create a probe for an instance, a net, or a terminal, probes are created for all equivalent instances, nets, or terminals in the corresponding design editor windows.

For example, when you create a probe for an instance in Schematic Editor, probes are created for all equivalent instances in the corresponding Layout Editor windows.

You can turn off the **Cross Probe via SDL (SDL > Probe > Add)** to disable the cross-probing.

For more information about probes and the **Probe** command options, see Probe Assistant in the *Custom Compiler™ Environment User Guide*.

Adding Probes

To add a probe:

1. Choose **SDL > Probe > Add**.

The 'Add Probe' dialog box is shown with the following fields and options:

- Type:** Any (dropdown)
- Name:** (text field)
- Packet:** (dropdown menu showing a yellow square icon)
- ☒ **Cycle**
- ☒ **Cross Probe via SDL**
- Trace Through Check:** ☒ **Property** ☐ **Cell**
- Advanced** (expanded section):
 - Description:** (text field)
 - Tool:** (text field)
 - Purpose:** other (dropdown)
 - Long Description:** (large text area)
- Remove All Probes** (button with a trash icon)
- Help**, **Hide**, **Defaults** (dropdown), **Apply**, **Cancel** (buttons)

2. Specify the options you want to apply.

For more information about using the **Add Probe** command, see Adding Probes in the *Custom Compiler Schematic Editor User Guide*.

3. (Options) Enable the **Cross Probe via SDL** option to cross-probe all equivalent objects. See [Cross-Probing](#).
4. Click **OK** or **Apply**.

Note:

Use the Probe Assistant to find and cycle through probes and their objects.

For more information about using the Probe Assistant, see Probe Assistant in the *Custom Compiler™ Environment User Guide*.

Removing All Probes

To remove all probes:

- Choose **SDL > Probe > Remove All**.

Finding Probes

Use the Probe Assistant to find and cycle through probes and their objects.

To access the Probe Assistant:

- Choose **SDL > Probe > Probe Assistant**.

For information about using the Probe Assistant, see Probe Assistant in the *Custom Compiler™ Environment User Guide*.

6

Using Templates and the Template Manager

The Template Manager helps you apply previous layout decisions to new designs. The Template Manager learns from the work done with the Placement Assistant and Custom Router. It intelligently recognizes circuits that are similar to ones that were already completed and enables you to apply the same placement and routing pattern as a template to the new circuits. The Custom Compiler environment provides a set of built-in templates for commonly used circuits, such as current mirrors, level shifters, and differential pairs.

The workflow for using templates and the Template Manager is described in the following sections:

- [Creating Templates](#)
- [Using the Template Manager](#)
- [Using the Template Browser](#)
- [Setting Template Attributes](#)

For information about Using Templates in the Schematic Editor, see the *Custom Compiler Schematic Editor and Text Editor User Guide*.

 [Show me](#) how to create templates and use them to manage design ECOs.

Creating Templates

The Create Template feature is described in the following sections:

- [Introduction to Create Template](#)
- [Using the Create Template Dialog Box](#)

Introduction to Create Template

As the very first part of the template solution, the Create Template feature provides the capability for you to construct your own template library. You can create a new template cell or add views to an existing template cell by selecting a partial layout and then invoking **Create Template**.

Note:

See [Batch Creating Templates](#) for information about the **Batch Create**

Template  command which scans all of the placement constraints in the editing design and then automatically adds or creates the proper template cell/view according to their netlist.

Template Modes

The Create Template feature supports the following modes:

- Regenerate mode (default)

In this mode, if there is only one output result from the children realizer, the Create Template feature just copies this input result to the output. If more than one children realizer generates the outputs, the template realizer takes all of the results and places them in one row.

- Static layout copy mode

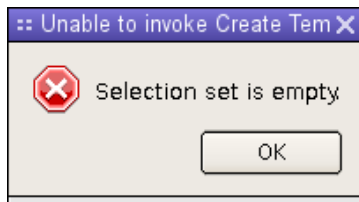
This mode copies all shapes and figures with the instance/net from the template layout and fixes the correspondence compared with the current SDL. In this mode, the template view is created for direct copying and the input designs from any children are ignored.

Choose this mode by selecting the **Static Template** field in the **Create Template** dialog box.

Prerequisites for Create Template

The Create Template feature cannot be invoked if the selection set is empty. If you select nothing and invoke the Create Template feature, the error prompt shown in [Figure 60](#) appears.

Figure 60 Create Template Error



Note:

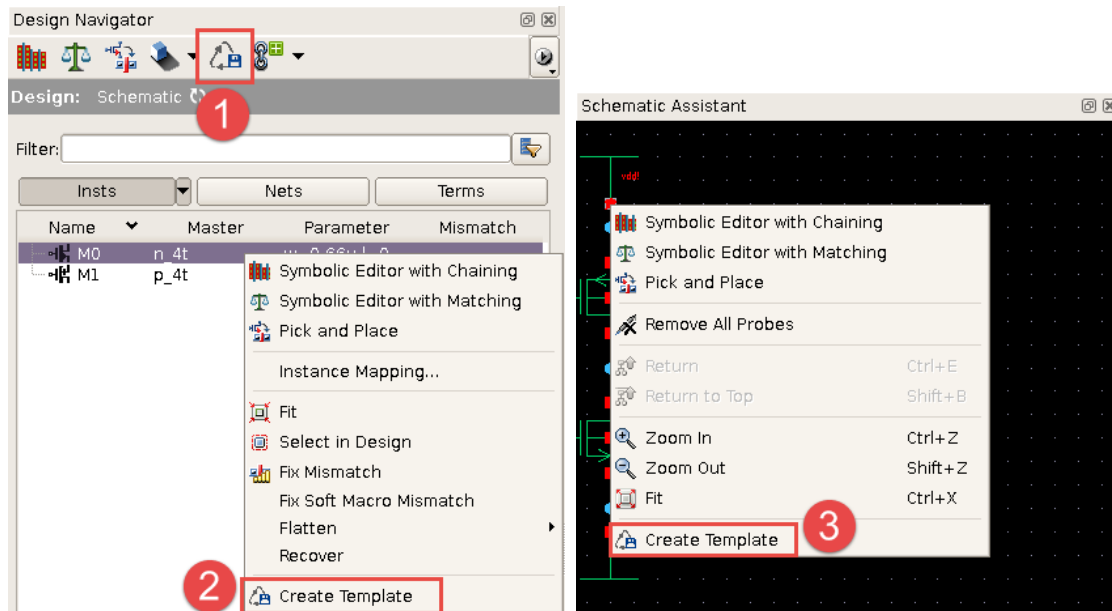
You must click **Enable Cross Select**  on the toolbar before invoking the Create Template feature to ensure that the stored netlist is synchronized with the stored layout.

Invoking Create Template

Figure 61 shows three different methods for invoking the Create Template feature:

1. Click **Create Template**  on the Design Navigator toolbar (see [Using Toolbar Icons for the Design Navigator in Schematic Mode](#)).
2. Right-click in the Design Navigator and choose **Create Template** from [Design Navigator Shortcut Menus](#).
3. Right-click in the Schematic Assistant and choose **Create Template** from the Schematic Assistant menu (see [Shortcut Menu on Canvas](#)).

Figure 61 Three Ways of Invoking the Create Template Feature



Using the Create Template Dialog Box

The **Create Template** dialog box (see [Figure 62](#)) supports the following use models:

- [Creating a New Template Cell](#)
- [Adding Views to an Existing Template Cell](#)

For both modes, a layout view is required, while a schematic view is optional. The major differences between the two modes are cell selection and description:

- When *creating a new template cell*, you enter the cell name (which should not conflict with existing cells) and the description.
- When *adding views to an existing template cell*, the cell should be from netlist-matched results selected from a list, while the description of the existing cell is loaded into the **Description** field. Optionally, you can modify the description.

Creating a New Template Cell

If the specified library and cell do not exist, a new template cell is created.

Figure 62 shows the **Create Template** dialog box.

Figure 62 Create Template Dialog Box

Table 8 describes the **Create Template** dialog box options when creating a new template cell.

Table 8 Create Template Dialog Box Options — Creating a New Template Cell

Option	Description
Library	Specifies the library name for the new template cell.

Table 8 *Create Template Dialog Box Options — Creating a New Template Cell*
(Continued)

Option	Description
Cell	When creating a new template, specifies a template cell name which does not exist in the library.
View	Specifies a view name for the template layout. The default view name is 'layout'.
Schematic	Specifies whether to save the schematic, allowing you to enter the schematic view name. The default schematic view name is 'schematic'.
Description	Specifies the description for the new template cell (such as the purpose of the cell, the type of design it is meant for, and components used). The description is on a per-cell basis.
Static Template	Controls a property on the new template cellview. • If selected, the template view is created for direct copying. • If not selected (default), the template view is created for regeneration purposes. The property is in the view level DM category and affects the realization process.
Bind Existing Layout	Specifies whether the currently selected layout figs are bound with the newly created template layout. If selected (default), a new template constraint is added to the corresponding constraints.

When creating templates, if more than one constraint tree exists, a Relative placement constraint is extracted automatically to combine these constraint trees into a single constraint tree under the Template placement constraint. You can turn off this function by setting the value of preference `lxTemplateManagerCreateTemplateExtractRp` to `False` (the default value is `True`).

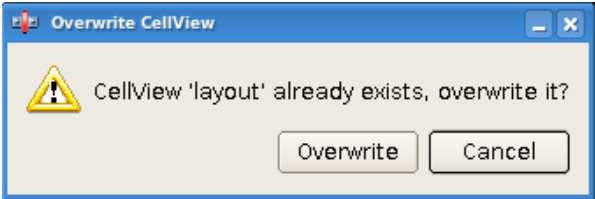
Also when creating templates, the [Extract Topology Constraints Command](#) is implicitly invoked to refine the current constraints. This feature can be turned off by deselecting the **Refine Constraints** option in the **Template Manager Options** dialog box (see [Setting Options for Template Manager](#)).

Adding Views to an Existing Template Cell

If the specified library and cell exist, a new view is added to an existing template cell. As with template creation, the [Extract Topology Constraints Command](#) may be implicitly invoked to refine the current constraints. You can type in an existing view name for the

target template cell. To handle this circumstance, an overwrite prompt (see [Figure 63](#)) is provided for you to choose whether you want to overwrite the existing view.

Figure 63 *Overwrite Prompt in Create Template Feature*



[Figure 62](#) shows the **Create Template** dialog box. [Table 9](#) describes the **Create Template** dialog box options when adding views to an existing template cell.

Table 9 *Create Template Dialog Box Options — Adding Views to an Existing Template Cell*

Option	Description
Library	Specifies the library name for the template cell.
Cell	Specifies an existing template cell name — either selected from a menu or entered manually — to which you are adding the new view.
View	Specifies a view name for the template layout. The default view name is 'layout'.
Schematic	Specifies whether to save the schematic, allowing you to enter the schematic view name. The default schematic view name is 'schematic'.
Description	Specifies the description for the template cell (such as the purpose of the cell, the type of design it is meant for, components used, and so on). The description is on a per-cell basis. If the selected template cell exists, the description for the cell is loaded and editable.
Static Template	Controls a property on the added template cellview. • If selected, the template view is created for direct copying. • If not selected (default), the template view is created for regeneration purposes. The property is in the view level DM category and affects the realization process.
Bind Existing Layout	Specifies whether the currently selected layout figs are bound with the added template layout. If selected (default), a new template constraint is added to the corresponding constraints.

Using the Template Manager

The Template Manager is described in the following sections:

- [Introduction to Template Manager](#)
- [Searching for Matches in the Active Design](#)
- [Browsing Matches in the Find and Bind Tab](#)
- [Working with Bound Templates in the Management Tab](#)
- [Previewing the Layout Realization](#)
- [Copying Template Results](#)
- [Saving Template Constraints](#)
- [Realizing a Template to the Editing Layout](#)
- [Batch Creating Templates](#)
- [Working With Static Flow](#)
- [Reconfiguring Templates](#)
- [Setting Options for Template Manager](#)
- [Showing and Hiding Sections of Template Manager](#)

Introduction to Template Manager

The Template Manager provides the capability to search based on a topological comparison for template matches, and then instantiate the template matches in the active design. The matches to the active design are listed in the [Results Viewer](#) and [Template Viewer](#). You can choose a preferred template match and use the toolbar to preview, save constraints, and realize the layout.

[Show me](#) how to reuse templates and template libraries.

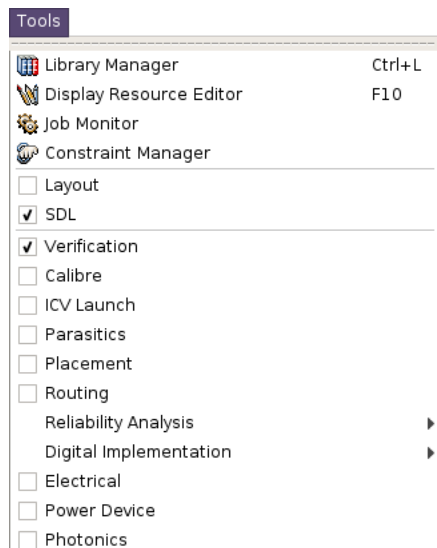
The Template Manager is outlined in the following sections:

- [Prerequisites for Template Manager](#)
- [Invoking Template Manager](#)
- [Using the Template Manager Interface](#)

Prerequisites for Template Manager

Before you can invoke the Template Manager, you must start an SDL session by choosing **Tools > SDL** (see [Figure 64](#)).

Figure 64 *Prerequisite of Template Manager*



When the SDL session is turned off with Template Manager online, the Template Manager is closed automatically.

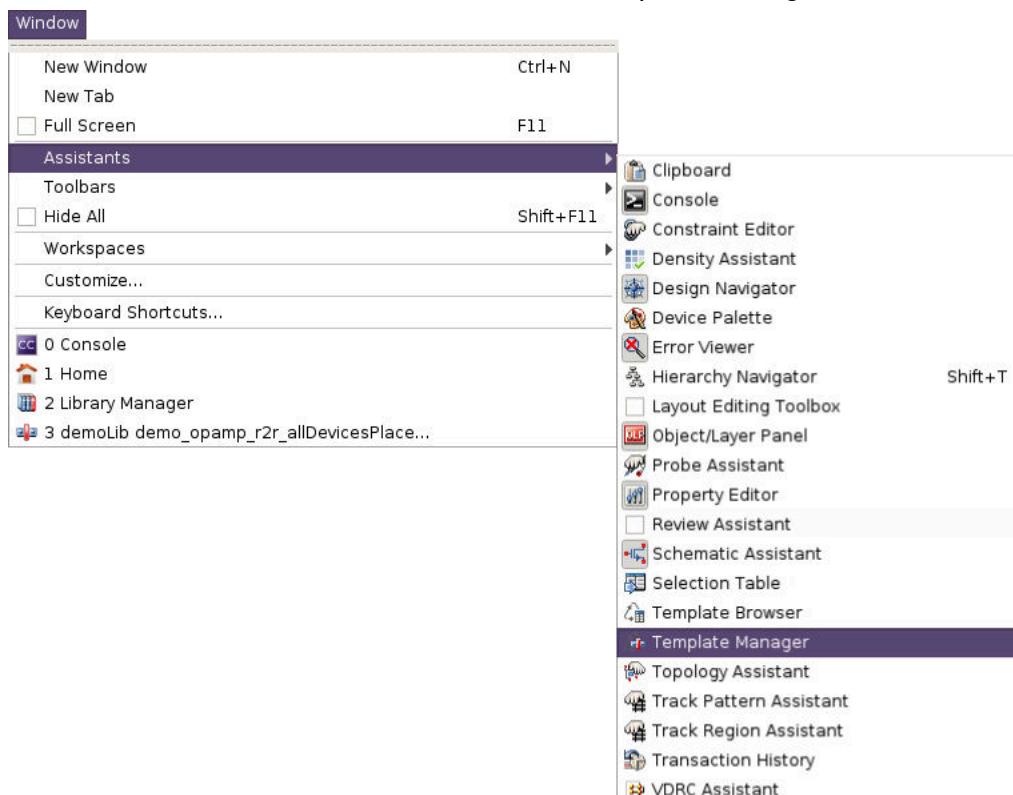
If you are [Using Templates and the Template Manager](#) realizer to realize some [Placement Constraints](#), the Placement feature must be enabled (see [Prerequisites for Placement Assistant](#)). Otherwise, an error message appears.

Invoking Template Manager

Start the Template Manager using either of the following methods:

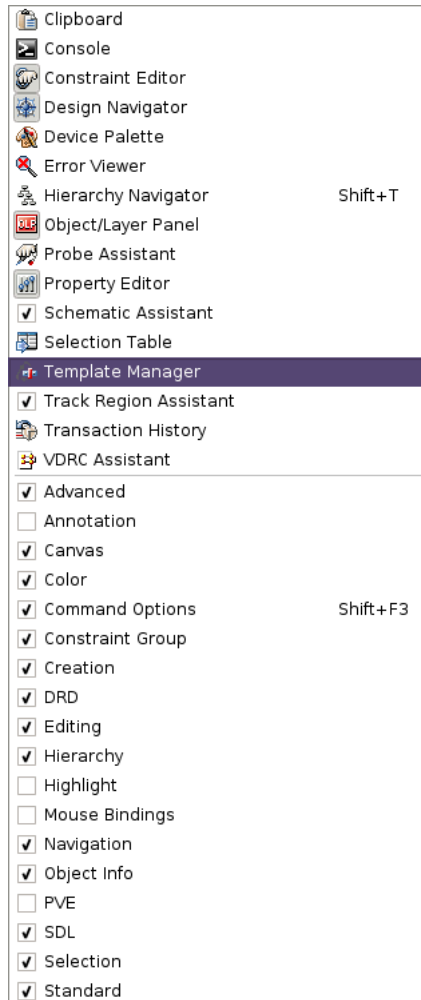
- Choose **Window > Assistants > Template Manager** (see [Figure 65](#)).

Figure 65 Invoke from Window > Assistants > Template Manager



- Right-click the design window toolbar and choose **Template Manager** (see [Figure 66](#)).

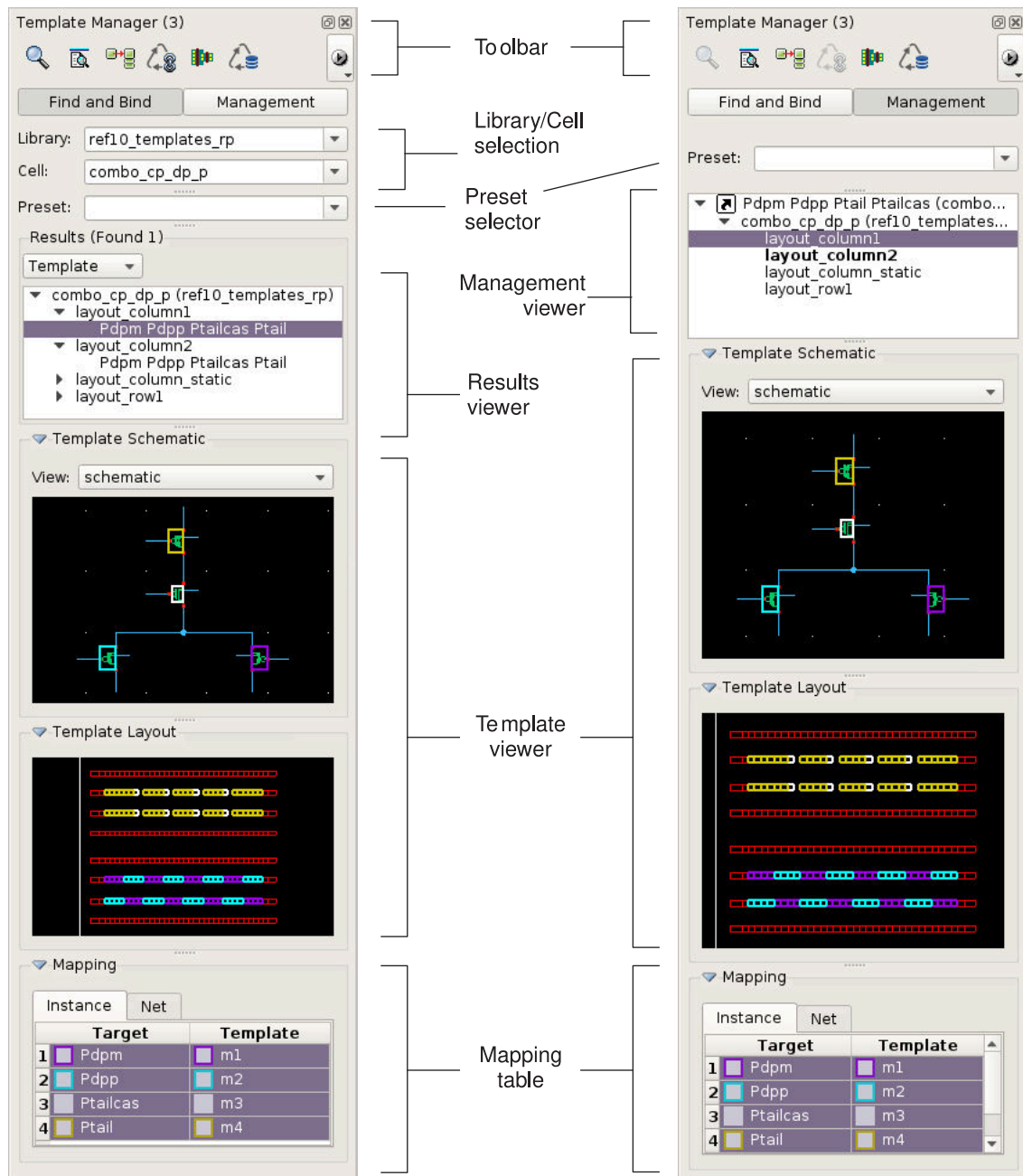
Figure 66 *Invoke from Design Window Toolbar Menu*



Using the Template Manager Interface

As shown in [Figure 67](#), the Template Manager is divided into two tabs: **Find and Bind** and **Management**. You can configure the sections to display; for more information, see [Showing and Hiding Sections of Template Manager](#) for information.

Figure 67 Template Manager Interface (Left: Find and Bind, Right: Management)



[Table 10](#) describes the toolbar of the Template Manager interface.

Table 10 *Template Manager Interface — Toolbar*

Toolbar Command	Icon	Description
Search Templates		Searches for templates that match the active design (requires a selection in the Design Navigator). See Searching for Matches in the Active Design . Note: This function is disabled in the Management tab.
Preview		Previews the realization result to check whether the realized result is consistent with what you expect (requires a selection in the Results viewer). See Previewing the Layout Realization .
Copy Result		After previewing template layouts, copies a selected result from the Template Gallery to the active design (requires a selection in the Template Gallery). See Copying Template Results .
Save Constraints		Saves constraints in the template cell to the active design (requires a selection in the Results viewer). See Saving Template Constraints . Note: This function is disabled in the Management tab.
Realize Template		Invokes the realization system to instantiate a template layout to the editing layout (requires a selection in the Results viewer). See Realizing a Template to the Editing Layout .
Batch Create Template		Scans all of the placement constraints in the editing design and then automatically adds or creates the proper template cell/view according to their netlist. See Batch Creating Templates .
Assistant Menu		- Displays the following: Template Manager Options dialog box See Setting Options for Template Manager. Show/Hide Sections dialog box. See Showing and Hiding Sections of Template Manager.

[Table 11](#) describes the **Find and Bind** tab of the Template Manager interface.

Table 11 *Template Manager Interface — Find and Bind Tab*

Tab Section	Description
Library and Cell fields	Provides the capability to limit the scope of target template cells which can be any template cells in any library or any template cells in a specific template cell. See Searching for Matches in the Active Design .

Table 11 Template Manager Interface — Find and Bind Tab (Continued)

Tab Section	Description
Preset selector	Specifies a preset cell to apply during realization. The field is prepopulated if you set a Parameter Preset library in the SDL Options dialog box, as shown in Figure 48 . You can optionally specify the preset cell by setting the preference <code>lxTemplateManagerPresetCell</code> .
Results viewer	Displays all of the matches from using Search Templates  in a three-level tree structure. Select one from the third level and then click Preview  , Save Constraints  , or Realize Template  to trigger the corresponding actions. See Results Viewer .
Template description	Shows the cell-level description which is saved in the template cell. The template cell description is added when using the Create Template feature. See Template Description .
Template viewer	Consists of a schematic selection and two preview panes: Template Schematic and Template Layout . When you select different matches in the Results viewer, the schematic and layout in the Template viewer are updated according to the selection. The Template viewer can also react with the selection in the Mapping table to highlight specific devices on the panes. See Template Viewer .
Mapping table	Consists of two tables in different tabs, Instance and Net , which show the detailed device/net mapping of the match that is selected in the Results viewer. The left column of the Mapping table shows the names in the active design and the right column shows the names in the matched template cell. When you select different results in the Results viewer, the Mapping table is updated accordingly. See Mapping Table .

[Table 12](#) describes **Management** tab of the Template Manager interface.

Table 12 Template Manager Interface — Management Tab

Tab Section	Description
Preset selector	Specifies a preset cell to apply during realization. The field is prepopulated if you set a Parameter Preset library in the SDL Options dialog box, as shown in Figure 48 . You can optionally specify the preset cell by setting the preference <code>lxTemplateManagerPresetCell</code> .

Table 12 Template Manager Interface — Management Tab (Continued)

Tab Section	Description
Management viewer	Displays the bound templates in a three-level tree structure. You can apply the actions on the toolbar to the selected templates. See Toolbar Actions in the Management Tab In addition, you can right-click the current assigned template layout view to display menu options. See Menu Actions in the Management Viewer .
Template description	The Template description on the Management tab functions the same as on the Find and Bind tab (see Template Description).
Template viewer	The Template viewer on the Management tab functions the same as on the Find and Bind tab (see Template Viewer).
Mapping table	The Mapping table on the Management tab functions the same as on the Find and Bind tab (see Mapping Table).

Searching for Matches in the Active Design

Search for templates that match the active design by clicking **Search Templates**  on the Template Manager toolbar.

Note:

The Template Manager requires that some instances are selected in the Design Navigator to identify the scope for the netlist matcher. If nothing is selected

in the Design Navigator when you click **Search Templates** , an error message appears and the Results viewer is empty.

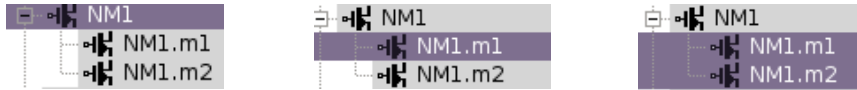
You can decrease the number of template candidates by specifying the target template library or template cell in the **Library** and **Cell** fields. After searching, all matched results are shown in the Results viewer (see [Template Viewer](#)).

Searching the Scope of the Active Design

The Template Manager requires that you select instances in the Design Navigator that possibly match with template candidates. If nothing is selected in the Design Navigator, an error message appears in the console. Because netlist matching is device-based, selecting leaf instances in the Design Navigator implies selecting the entire device.

For example, as shown in [Figure 68](#), device NM1 has m-factor 2. Selecting NM1 or NM1.m1 or NM1.m1 + NM1.m2 all result in the same searching scope: NM1.

Figure 68 Example — Selection of Design Navigator



Any selected devices that are fully or partially realized in the layout are omitted from the searching scope internally and the following warning message is shown:

Warning: Selected instance NM1 is filtered from Template Manager because it has been realized.

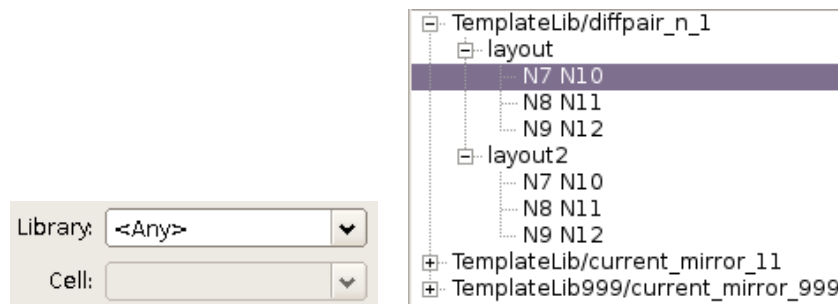
Searching Scope of the Template Candidates

The Template Manager provides the **Library** and **Cell** fields—both of which support [Wildcard Syntax](#)—to specify the target template cell candidates. It is helpful to reduce the search scope because it might be time-consuming to search matches of a large number of template cell candidates.

Either the **Library** or **Cell** field can be specified **<Any>**:

- When **<Any>** is selected in the **Library** field, all template cells under all libraries are template cell candidates (see [Figure 69](#)).

Figure 69 All template cells under all libraries are candidates

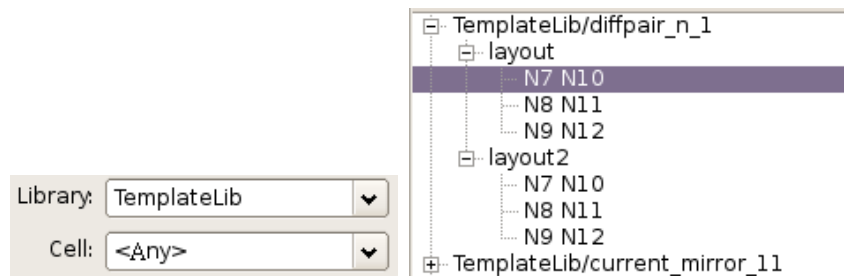


Note:

The **Library** field is set to **<Any>** with the **Cell** field unselectable by default.

- When **<Any>** is selected in the **Cell** field, all template cells under the specified library are template cell candidates (see [Figure 70](#)).

Figure 70 All template cells in TemplateLib are candidates

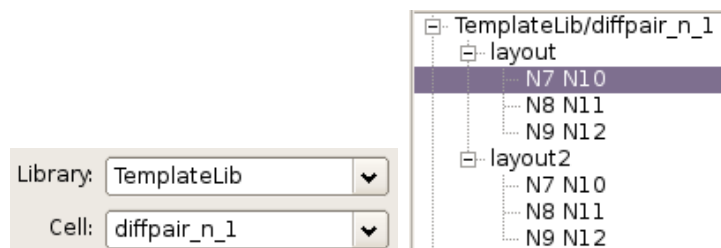


Note:

The **Cell** field is switched to <Any> automatically when the **Library** field is changed to avoid a meaningless selection.

- If both the **Library** and **Cell** fields are not <Any>, only the specific template cell is a candidate (see [Figure 71](#)).

Figure 71 Only TemplateLib/diffpair_n_1 is a candidate



Wildcard Syntax

To increase the efficiency of the search, the Template Manager supports the following wildcard characters:

- Asterisk (*) matches zero or more characters.

For example, `tmplRef10*` matches `tmplRef10HV` and `tmplRef10HV_rp` but not `tmplRef11`.

- Question mark (?) matches exactly one character.

For example, the pattern `dif?` matches `diff` but not `diffs`.

- Square brackets ([and]) match a single character within the set.

For example, `[A-Za-z]` matches any single uppercase or lowercase letter.

Browsing Matches in the Find and Bind Tab

When you select a match in the Results viewer of the **Find and Bind** tab, the corresponding instance and net mapping is shown in the Mapping table. You can click rows to show cross-highlighting between the template schematic, template layout, and the schematic of the active design with a system-specified color. The Mapping table is updated if the selection in the Results viewer is changed. By default, the Mapping table is in “select-all” mode to highlight all devices in different colors.

The interface elements of the **Find and Bind** tab are described further in the following sections:

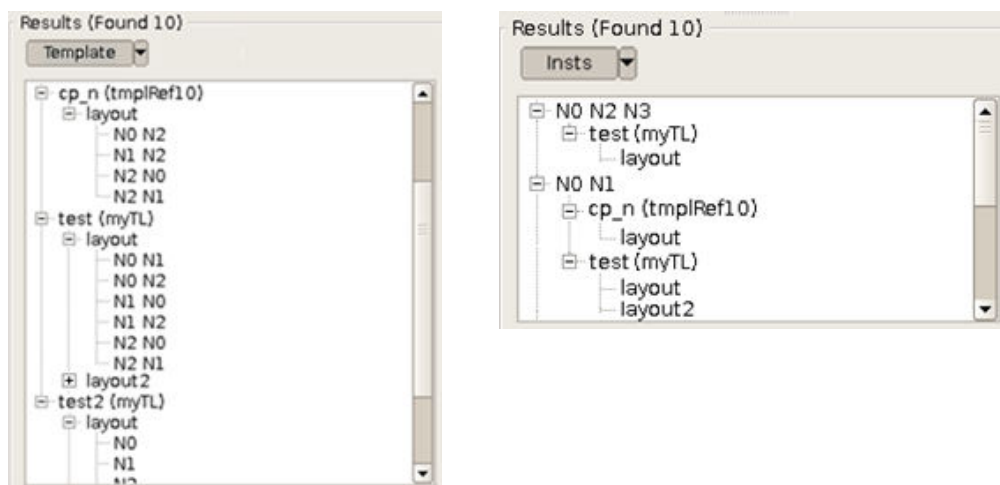
- [Results Viewer](#)
- [Template Description](#)
- [Template Viewer](#)
- [Mapping Table](#)

Results Viewer

The Results viewer provides the capability to browse the matching results in the active design. The matches are then presented by a three-level collapsible hierarchy tree.

As shown in [Figure 72](#), the Results viewer supports two display modes: **Template** and **Insts**.

Figure 72 Results Viewer: Template Mode (left) and Insts Mode (right)



The hierarchy of the Template mode display is as follows:

1. Root nodes: Template cell names with template library names in parentheses
2. Second-level nodes: Template layout view names
3. Leaf nodes: Matches of devices set in the active design

The hierarchy of the Insts mode display is as follows:

1. Root nodes: Matches of devices set in the active design
2. Second-level nodes: Template cell names with template library names in parentheses
3. Leaf nodes: Template layout view names

You can select a match from the leaf node level of the tree to **Preview** , **Save**

Constraints , or **Realize Template** . Selecting a root or second-level node implies selecting the first leaf under the node.

Multiple Selection in the Results Viewer

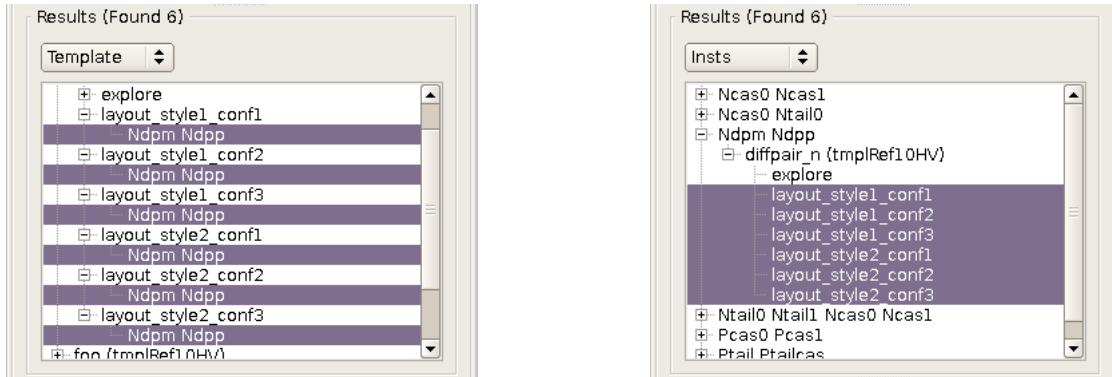
The Results viewer supports multiple selection as follows:

1. Select multiple candidates from the Results viewer (these can be from different libraries or cells), then click **Preview** .
2. Select the preferred result from the Template Gallery.
3. Click **Copy Result**  or use the shortcut key C to copy the selected result to the editing layout.

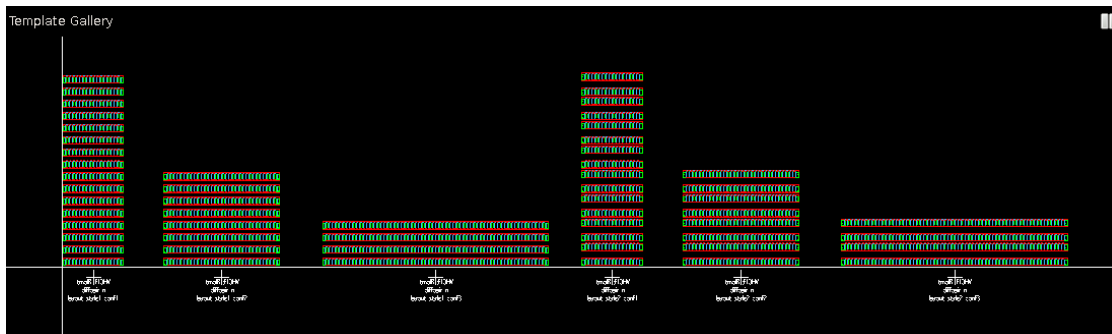
To identify the results, the library, cell, and view names are labeled under each instance in the Template Gallery in different lines.

Figure 73 shows an example for multiple selection in the Results viewer.

Figure 73 Example — Multiple Selection in the Results Viewer and Template Gallery



Multiple selection on Results viewer (left: Template mode, right: Insts mode)



Template Gallery showing preview result with multiple selection

Template Description

When you use the Create Template feature to create a new template cell or add a view to an existing template cell, you can add or edit a cell description in the **Description** field of the **Create Template** dialog box (see Figure 62).

The description is displayed in the following locations:

- **Template Description** field of both the **Find and Bind** tab and the **Management** tab
- Tooltips in the Results viewer when in Template mode
- **Description** field of the Template Attributes assistant

Template Viewer

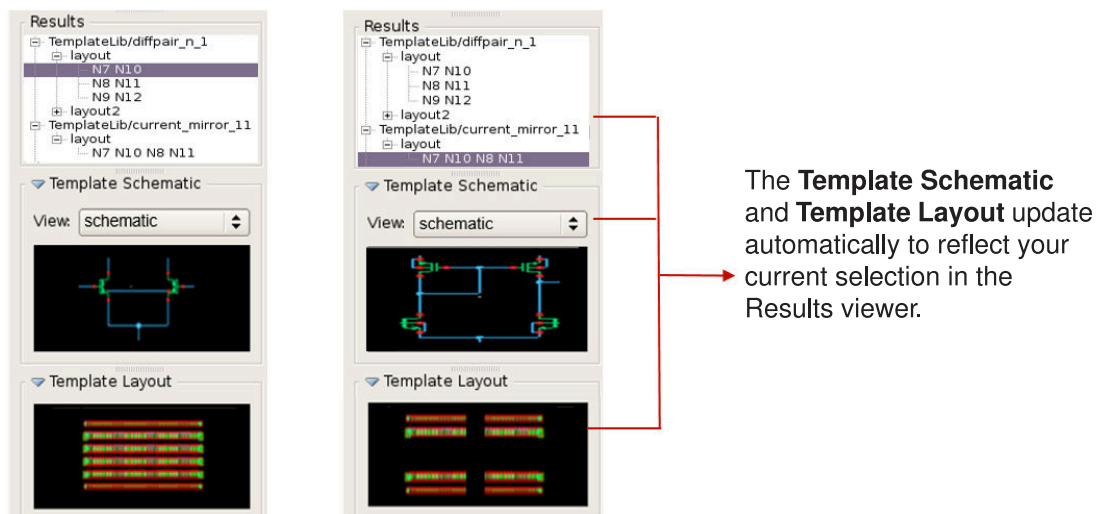
The Template viewer provides a way to select and preview the template schematic under the selected template cell in the Results viewer. A template layout is updated according to the second level of the Results viewer.

The layout selection in the Results viewer affects the constraints transfer (see [Saving Template Constraints](#)) and layout realization (see [Realizing a Template to the Editing Layout](#)) to the active design. The **View** menu of the Template Schematic only shows the schematic cellviews.

Figure 74 shows two examples of matches in the Results viewer (Template mode) and Template viewer:

- The example on the left shows that the selection, (N7, N10), matches with the template cell under TemplateLib/diffpair_n1/layout. Both (N8, N11) and (N9, N12) in the active design match with diffpair_n1, as well.
- The example on the right shows that when the selection changes to (N7, N10, N8, N11), which is matched to TemplateLib/current_mirror_11/layout; the template viewer is updated, as well.

Figure 74 Example — Matches in the Results Viewer

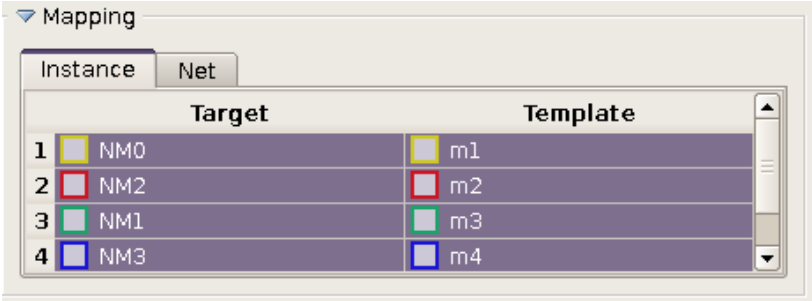


Mapping Table

The Mapping table presents the detailed device and net mapping of the selected match in the Results viewer.

As shown in [Figure 75](#), the Mapping table is divided into **Instance** and **Net** tabs. Every selection change in the Results viewer triggers the Mapping table to update the corresponding device/net mapping and changes the selection to all.

Figure 75 Mapping Table



The screenshot shows a software window titled "Mapping" with two tabs: "Instance" (selected) and "Net". Below the tabs is a table with two columns: "Target" and "Template". The table contains four rows of mappings, each with a colored square icon to its left. The rows are numbered 1 through 4 on the left side of the table.

	Target	Template
1	 NM0	 m1
2	 NM2	 m2
3	 NM1	 m3
4	 NM3	 m4

The **Instance** tab in the Mapping table shows the device mapping between the active design and the template source. A **Target** and **Template** in a row represent a device mapping pair between the active design and template source.

As shown in [Figure 76](#), the table shows the devices that map between the active design and template are {NM0, m1}, {NM2, m2}, {NM3, m3}, and {NM4, m4}. The order of the mapping table is sorted in a natural order (m1, m2, ..., m9, m10, m11) according to the right (Template) column. Each field has a color box  to identify the color of the devices when they are highlighted in the Template viewer.

Figure 76 Example — Instance Tab of Mapping Table

	Target	Template
1	 NM0	 m1
2	 NM2	 m2
3	 NM1	 m3
4	 NM3	 m4

Figure 77 shows an example of the **Net** tab. The nets that are mapped between the active design and the template source are put in a row. In contrast to the **Instance** tab, no color highlighting is supported in the **Net** tab.

Figure 77 Example — Net Tab of Mapping Table

	Target	Template
1	INN	net11
2	INP	net5
3	VOUT	net1
4	VREF	net12

Cross-Highlighting

You can pick some rows in the **Instance** tab of the Mapping table to cross-highlight the corresponding devices in the template schematic, template layout, and the schematic of the active design. The color box in the row identifies the color of the highlighting.

Figure 78 shows an example for cross-highlighting in a target schematic, template schematic, and template layout when you select row 4.

Figure 78 Example — Cross-highlighting in Instance Tab of Mapping Table

Mapping

Instance

Net

	Target	Template
1	Ncas0	m1
2	Pdpm	m10
3	Pdpp	m11
4	Ptail	m12
5	Ptail0	m13
6	Ptail1	m14
7	Ptailcas	m15
8	Ncas1	m2
9	Ndpm	m3
10	Ndpp	m4
11	Ntail	m5
12	Ntail0	m6
13	Ntail1	m7
14	Pcas0	m8
15	Pcas1	m9

Working with Bound Templates in the Management Tab

From the **Management** tab, you can view, manage, and realize the bound templates. Multiple selection is supported in this tab, similar to the **Find and Bind** tab (see [Multiple Selection in the Results Viewer](#)).

The **Management** tab is described further in the following sections:

- [Toolbar Actions in the Management Tab](#)
- [Display Styles in the Management Viewer](#)
- [Menu Actions in the Management Viewer](#)
- [Hierarchical Templates in the Management Viewer](#)

Toolbar Actions in the Management Tab

In the **Management** tab, **Search Templates**  and **Save Constraints**  are always disabled. The other commands are supported as follows:

- **Preview** : Previews templates in the Template Gallery, if at least one leaf node in the Management viewer is selected.

Note:

Preview  on the **Management** tab functions the same as on the **Find and Bind** tab. See [Previewing the Layout Realization](#) for more details.

- **Copy Result** : Copies a previewed layout into the editing design. After the layout is copied into the editing design, the Management viewer refreshes accordingly.

Note:

Copy Result  on the **Management** tab functions the same as on the **Find and Bind** tab. See [Copying Template Results](#) for more details.

- **Realize Template** : Reassigns the view of the template constraint and updates the layout at the same point or by following the cursor. If no corresponding placement group exists, the generated layout follows the cursor and waits for you to decide a location.

Note:

Realize Template  on the **Management** tab differs from how it functions the **Find and Bind** tab, which is detailed in [Realizing a Template to the Editing Layout](#).

This button is disabled when multiple leaf nodes are selected.

- **Batch Create Template** : Scans all of the placement constraints in the editing design and automatically adds or creates the proper template cell/view according to the netlist.

Note:

Batch Create Template  on the **Management** tab functions the same as on the **Find and Bind** tab. See [Batch Creating Templates](#) for more details.

Display Styles in the Management Viewer

The Management viewer provides a way to view, manage, and realize the template constraints. Unlike the Constraint Editor, the Management viewer only shows the template constraints in instances (Insts) mode. The template constraint and the contained device set are presented in a three-level collapsible hierarchy tree.

[Figure 79](#) shows an example of the Management viewer:

Figure 79 Management Viewer



The hierarchy of the Management viewer is as follows:

1. Root nodes: Instance name followed by the template constraint name
2. Second-level nodes: Template cell name followed by the template library name
3. Leaf nodes: All template layout views under the template cell in the second level nodes

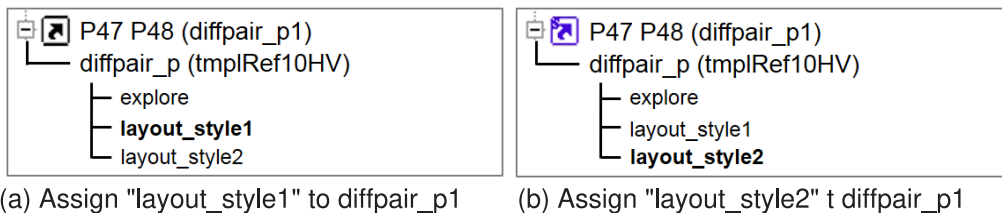
As with the constraint view of the Design Navigator, the Management viewer displays the template states in different styles:

- Gray background in root node: The template constraint is not yet realized. To determine whether a template constraint is realized, the Template Manager looks up the corresponding placement group on the layout.

For example, suppose there is a template constraint 'diffpair_p1'. If a placement group 'diffpair_p1' exists on the layout, then the template constraint is realized. Otherwise, the template constraint is unrealized and marked in a gray background.

- *Italics* in root node: The template constraint is transferred from SDL schematic elaboration.
- **Bold** in leaf node: The current assigned template layout view. In the example in [Figure 80](#), the template layout view 'layout_style1' is assigned to diffpair_p1 in (a) while 'layout_style2' is assigned to diffpair_p1 in (b).
- Blue icon  in front of the root node: The static template attribute is true on the template constraint. In the example in [Figure 80](#), the "Static Template" flag on diffpair_p1 is true for (b), so the icon is blue.

Figure 80 Example — Management Viewer Display Styles

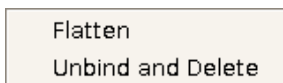


Because the Management viewer is not the only way to modify the template constraints, it reflects modifications from other features and always shows the latest state.

Menu Actions in the Management Viewer

Right-click the current assigned template view (displayed in **bold**) to display the Management viewer menu shown in [Figure 81](#).

Figure 81 Management Viewer Menu



Note:

Multiple selection is not supported for this feature.

[Table 13](#) describes the actions provided by the Management viewer menu.

Table 13 *Management Viewer Menu Actions*

Action	Description
Flatten	Unbinds (removes) the template constraint node from the current constraint hierarchy.
Unbind and Delete	Deletes the entire template constraint hierarchy and the corresponding layout.

Hierarchical Templates in the Management Viewer

For hierarchical template cells, the **Management** tab lists the entire template constraint hierarchy, but only the top template constraint can be previewed or realized. That is, for hierarchical templates, the **Preview**  and **Realize Template**  actions preview or realize the *entire* template hierarchy. Previewing or realizing a portion of the template constraint hierarchy is not supported, however you can switch the physical implementation from one layout to another which triggers the content update of the **Management** tab.

Menu Actions

As with the toolbar actions, the menu actions (**Flatten** and **Unbind and Delete**) are only supported for the top template constraint.

Static Templates

If one template constraint of the hierarchical template is a static template, the Template Manager treats this portion—that is, the static template and any children below it in the hierarchy—as a static template.

Previewing the Layout Realization

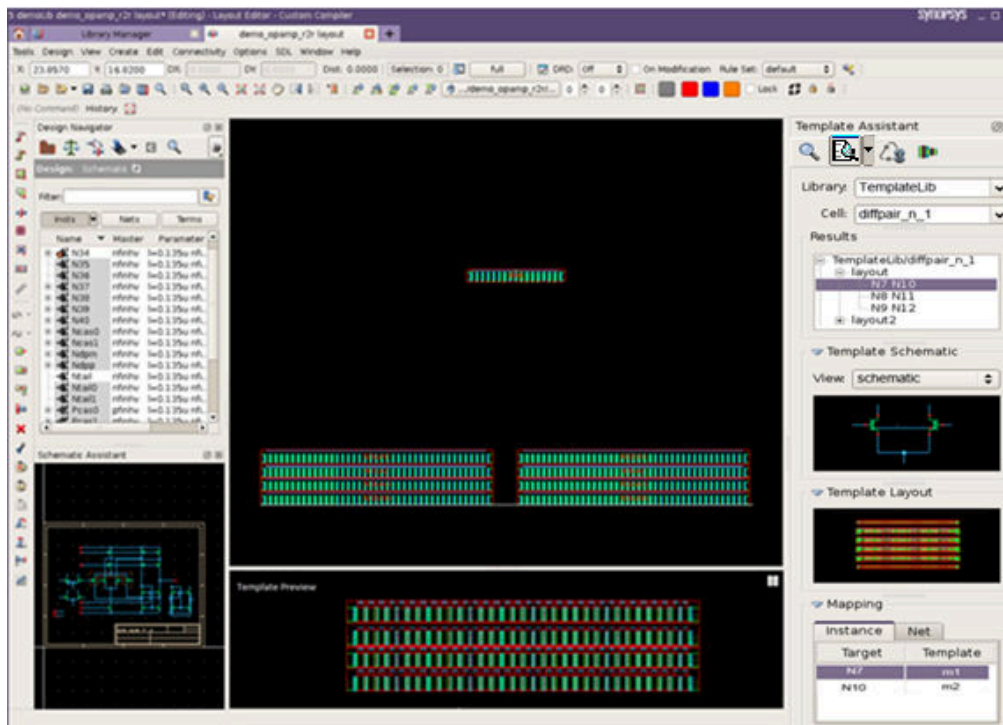
The Template Manager provides the capability to preview the realization result for you to easily check whether the realized result is consistent with what you expect.

To preview the layout, select one or more candidates from the Results viewer, then click

Preview  on the Template Manager toolbar. This causes the realization system to generate previews of the selected template candidates and then display the results on the Template Gallery as shown in [Figure 82](#). If the result is not what you expected, you can

choose another template candidate and click **Preview**  to regenerate the preview layout on the Template Gallery.

Figure 82 Layout Realization Preview



The preview layout on the Template Gallery is created as a cellview in a specific library and cell. The name of the library is the current library name with the postfix `_template` and the cell name is `current_cell_name@current_lib_name`. The following is an example:

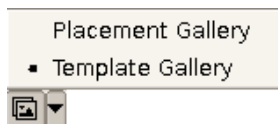
- Active design: demoLib/demo_opamp_r2r/layout
- Template preview library and cell: demoLib_template/demo_opamp_r2r@demoLib

The Template Gallery also supports cross-selection and cross-highlighting between the Template Gallery, main context, Design Navigator, and Schematic Assistant.

You can click **Show/Hide Gallery**  to toggle the display of the Template Gallery. All of the generated results are retained under the working library and cell, even if the Template Gallery is hidden.

As shown in [Figure 83](#), you can also use **Show/Hide Gallery**  to switch the gallery display between realization previews in the Template Gallery and placement results in the Placement Gallery (see [Viewing Placement Results](#)).

Figure 83 Switching Between Galleries



Copying Template Results

After previewing template layouts and selecting one, click **Copy Result**  or use the shortcut key C to copy it to the current design. The corresponding layout follows the position of the cursor until you click the canvas to position it on the active design.

Note:

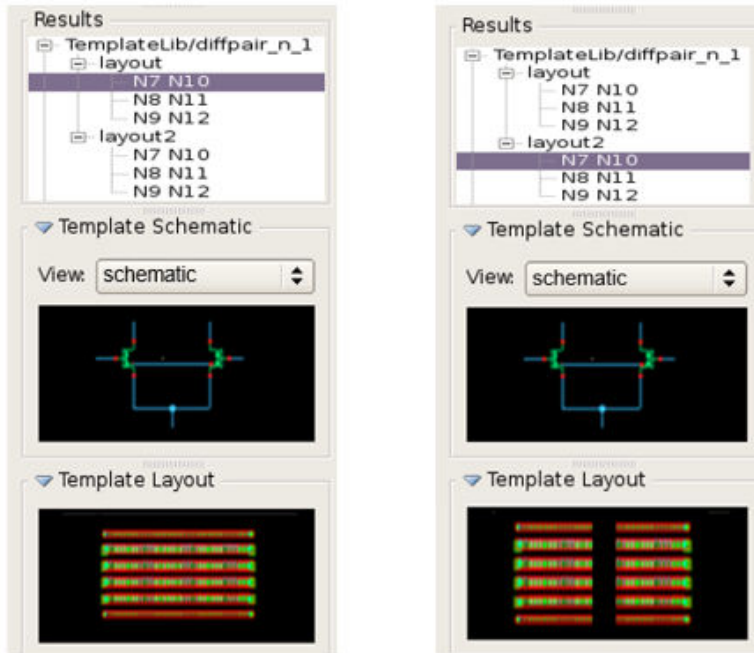
Selecting a portion of a previewed template to copy back to the editing design is not supported. The template preview is a hierarchical template result, so you must select the entire previewed template to copy back.

Saving Template Constraints

Click **Save Constraints**  to save constraints in the template cell to the active design. You can select the source template cellview from the Results viewer to pick the constraints to copy.

As shown in [Figure 84](#), different template layouts can contain different constraints to generate different layout styles. You can select different template layouts through the Results viewer. Schematics are provided only for reference to the layout being viewed.

Figure 84 Save Template Constraint from Different Layouts (layout and layout2)



Because the instance name in the template layout may be different from the active design, the Template Manager renames the corresponding devices when bringing over the constraints. As shown in [Figure 85](#) and [Figure 86](#), the devices (m1, m2) are renamed to (NM1, NM0) based on the information in the Mapping table.

Figure 85 Constraint in Template Cell

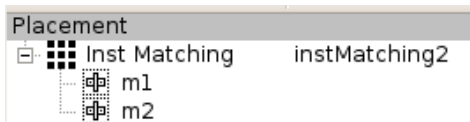
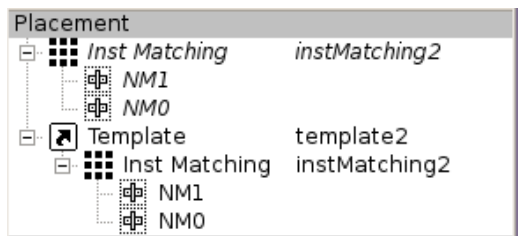
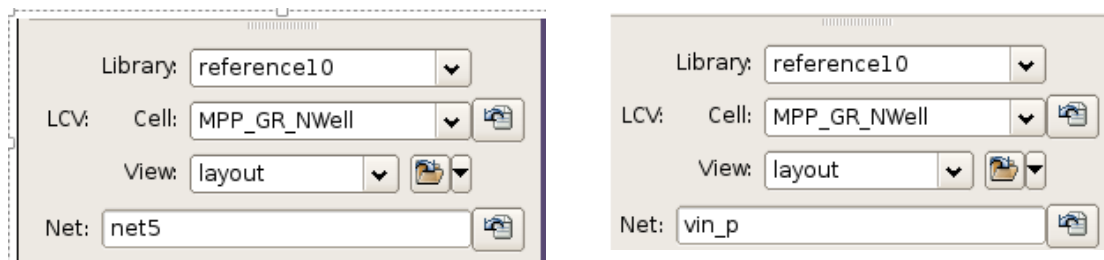


Figure 86 Save Template Constraint from Template Cell to Active Design



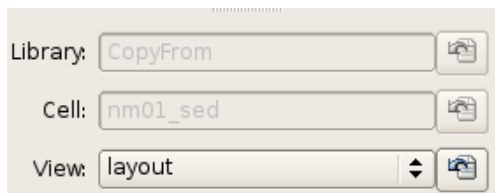
For [Guard Ring](#) and [Pattern Route](#) constraints, the net name stored is also renamed based on the mapping table. However, if the template netlist file does not contain this net, the net field remains the same. As shown in [Figure 87](#), because the Mapping table does not contain the information about user parameters (in this case, **Library**, **Cell**, and **View**), those fields remain the same.

Figure 87 Guard Ring Constraint Stored in the Template Layout and Editing Design



When instantiating a template, the Template Manager then creates a template constraint on the active design, which includes the mapping of instance and net names, and then the template constraint itself copies in the constraints that are contained in the template cell. You can check the constraint content through the Constraint Editor. As shown in [Figure 88](#), the source template library, cell, and view is stored.

Figure 88 Content of a Template Constraint



If there is any existing template constraint hierarchy which contains exactly the same contents as the current selection, the Template Manager tries to reuse it instead of creating a new constraint.

Post-Process Procedure

This procedure is called whenever the template constraint is created or realized in the Template Manager.

Syntax:

```
procedure_name { varListOfLists }
```

Arguments:

Table 14 *Post-process Arguments*

Name	Type	Description
templateLib	string	The template library name.
templateCell	string	The template cell name.
templateView	string	The template view name.
editingDesign	oaDesign	The SDL design in the layout window.
constraintDesign	oaDesign	The temporary design to put the constraint during realization process.
sourceConstraintName	string	The working constraint name mapped to the source template.
targetConstraintName	string	The working constraint name.

Example:

```
templateSavePostProc {data} {...}
```

Realizing a Template to the Editing Layout

After you choose a template layout to instantiate, click **Realize Template**  on the Template Manager toolbar to call the realization system. After the realization system is invoked, one or more realizers are called based on what kind of constraint is created. The realizer outputs a realized layout to the location of the cursor in the editing layout. After the layout is in the position you want, click the canvas and the layout is moved from the cursor to the editing layout. At the same time, the created template constraints from the realization system appear in the Constraint Editor.

Realize Toolbar

Clicking **Realize Template**  invokes the **Realize** toolbar, which allows you to control the origin and the orientation of the realized layout when committing it to the canvas.



Table 15 describes the options provided by the **Realize** toolbar.

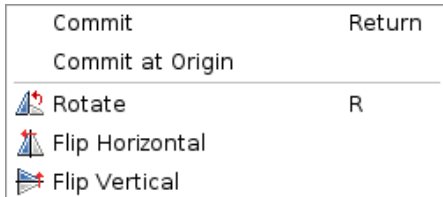
Table 15 Realize Template Toolbar Options

Option	Description
Origin	Specifies that the cursor points to the origin of the lower-left device in the realized layout. If this option is selected (default), the Justification option is disabled.
Justification	Specifies that the cursor points to one of the following instance boundaries of the realized layout: • Upper-Left, Upper-Center, Upper-Right • Center-Left, Center-Center (default), Center-Right • Lower-Left, Lower-Center, Lower-Right
	Rotates the realized layout 90 degrees counterclockwise.
Rotate	
	Mirrors the realized layout across the Y-axis.
Flip Horizontal	
	Mirrors the realized layout across the X-axis.
Flip Vertical	

Realize Menu

Right-click the canvas when using the **Realize Template** command to display the menu shown in [Figure 89](#).

Figure 89 Realize Template Menu



[Table 16](#) lists the actions provided by the **Realize Template** menu.

Table 16 Realize Template Menu Actions

Action	Description	Default Shortcut Key
Commit	Commits the realized layout to the canvas at the location specified by the cursor. This action is equivalent to clicking the canvas.	Enter
Commit at Origin	Commits the realized layout to the canvas at the origin (0,0).	
Rotate	Rotates the realized layout 90 degrees counterclockwise.	R or Middle-click
Flip Horizontal	Mirrors the realized layout across the Y-axis.	Shift-middle-click
Flip Vertical	Mirrors the realized layout across the X-axis.	Ctrl-middle-click

Post-Process Procedure

This procedure is called whenever the objects are previewed or realized in the Template Manager.

Syntax:

```
procedure_name { varListOfLists }
```

Arguments:

Table 17 Post-process Arguments

Name	Type	Description
templateLib	string	The template library name.

Table 17 Post-process Arguments (Continued)

Name	Type	Description
templateCell	string	The template cell name.
templateView	string	The template view name.
editingDesign	oaDesign	The SDL design in the layout window.
constraintDesign	oaDesign	The temporary design to put the constraint during realization process.
figDesign	oaDesign	The temporary design to put the result generated by realizer.
sourceConstraintName	string	The working constraint name mapped to source template.
targetConstraintName	string	The working constraint name.

Example:

```
templatePostProc {data} {....}
```

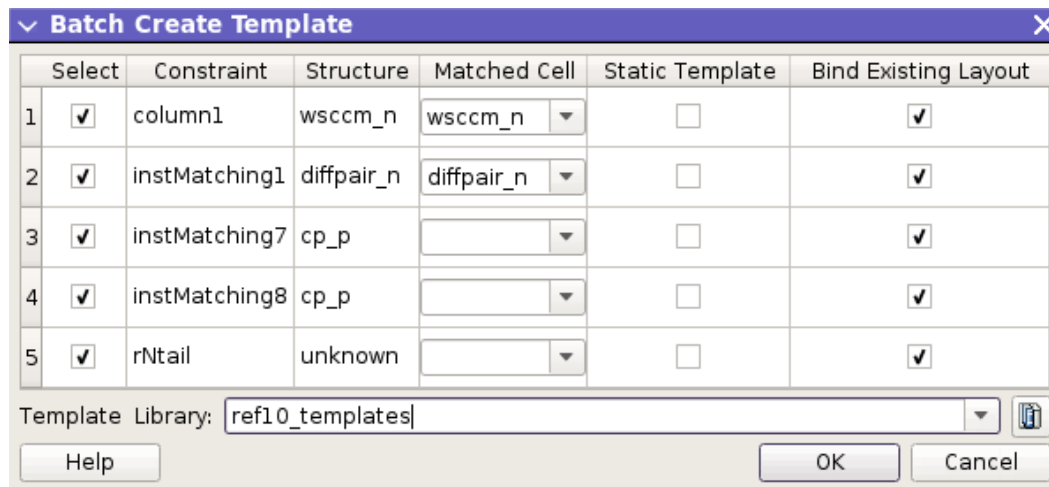
Batch Creating Templates

As described in [Creating Templates](#), you can use the **Create Template**  command and the **Create Template** dialog box to create a single template based on your selections in the Design Navigator, Schematic Assistant, or layout canvas.

However, you can also use the **Batch Create Template**  command to scan all of the placement constraints in the editing design and then automatically add or create the proper template cell/view according to their netlist.

Batch Create Template  scans all top-level, non-template, placement constraints that have corresponding placement groups and then lists them in the **Batch Create Template** dialog box.

Figure 90 Batch Create Template Dialog Box



The dialog box titled "Batch Create Template" contains a table with the following data:

	Select	Constraint	Structure	Matched Cell	Static Template	Bind Existing Layout
1	☒	column1	wscm_n	wscm_n	☐	☒
2	☒	instMatching1	diffpair_n	diffpair_n	☐	☒
3	☒	instMatching7	cp_p		☐	☒
4	☒	instMatching8	cp_p		☐	☒
5	☒	rNtail	unknown		☐	☒

Below the table, there is a "Template Library:" field with the value "ref10_templates". At the bottom are "Help", "OK", and "Cancel" buttons.

As shown in Figure 90, the tool lists all of the top-level, non-template constraints in the **Constraint** column.

The tool analyzes the structure for each constraint and lists them in the **Structure** column.

Note:

Not all structures are supported currently; if the constraint does not match any of the supported structures, `unknown` is displayed in the **Structure** column.

If you specify a **Template Library**, the tool finds any matching template cells in the specified template library and lists them in the **Matched Cell** column.

Use the checkboxes in the **Static Template** column to control the following property on the new template cellview:

- If selected, the template view is created for direct copying.
- If not selected (default), the template view is created for regeneration purposes.

Use the **Bind Existing Layout** column to specify whether the currently selected layout figs are bound with the newly created template layout. If selected (default), a new template constraint is added to the corresponding constraints.

After you click **OK**, the tool:

- Adds the template view into the selected template cell. If the matched cell is selected as an empty cell, then the tool creates a new template cell, named and numbered as follows:
 - `auto_structure_#` where *structure* is the recognized structure name
 - `auto_unknown_#` if the structure was not recognized
- Creates the template layout view into the selected template cell, named and numbered as follows:
 - `auto_layout_username_date_#` where *username* is your username and *date* is the current date.

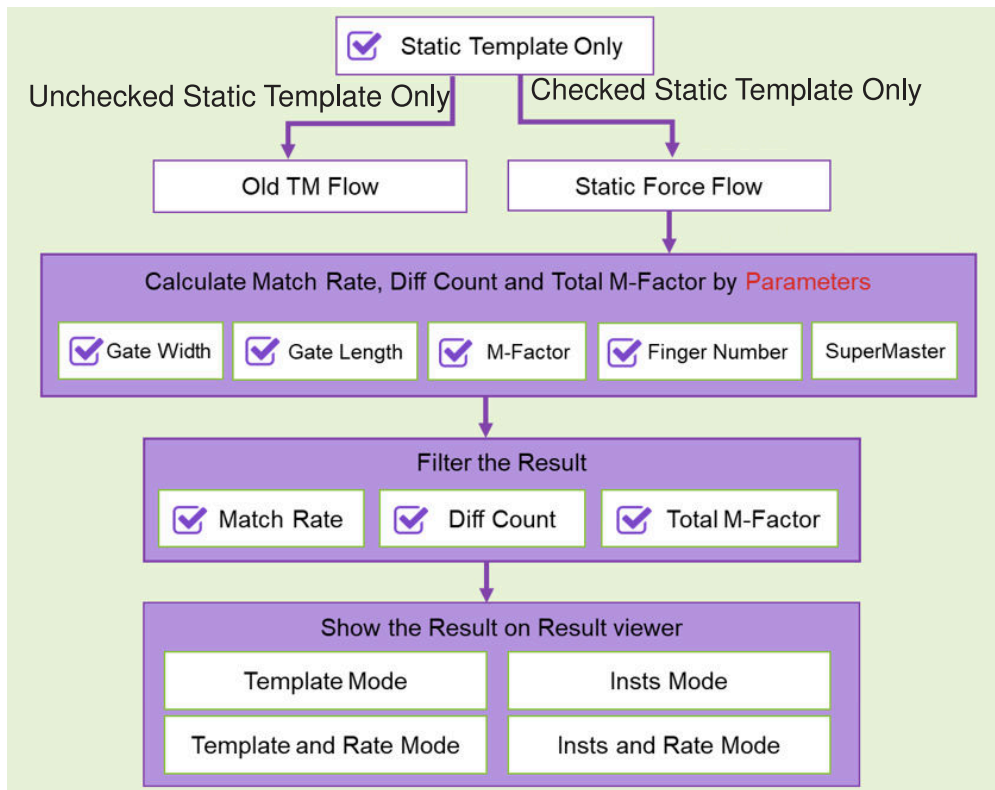
Working With Static Flow

This chapter includes the following sections:

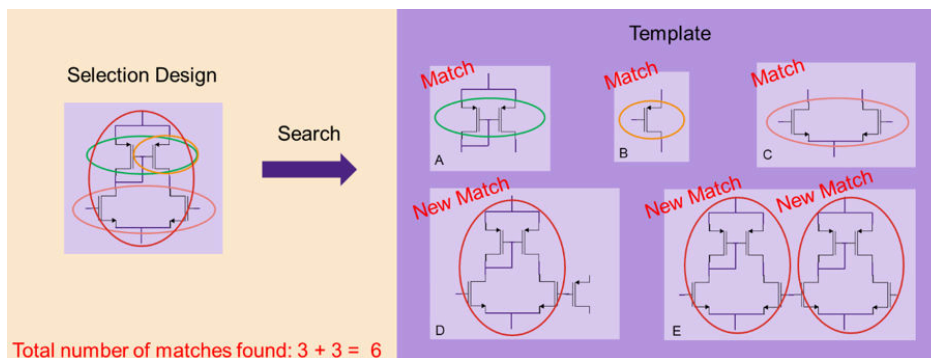
- [Enabling Static Template](#)
- [Using Matching Parameters and Criteria](#)
- [Browsing Matching Results](#)
- [Saving and Realizing Template Design](#)

Enabling Static Template

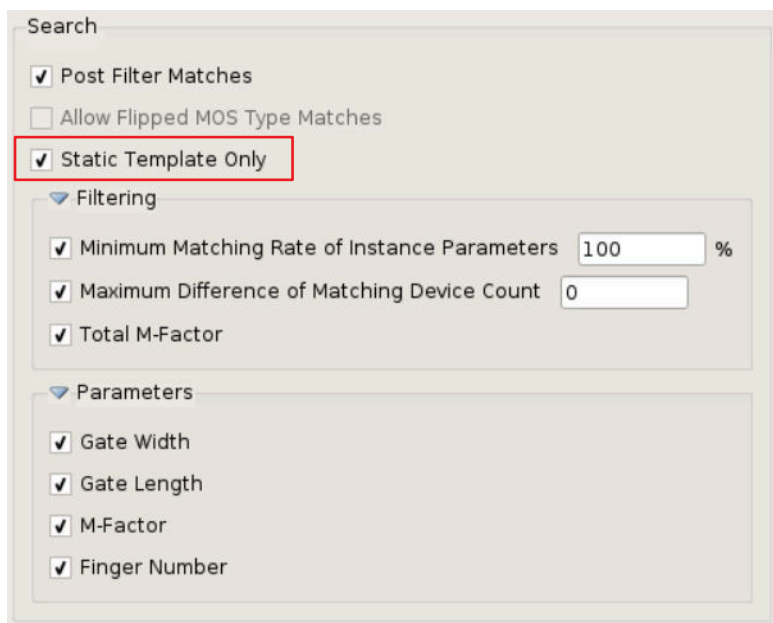
In this static flow, Template Manager can search template designs which are larger than selection design cases. The tool can find more potential candidates. You can also find the candidates very quickly by setting the filters and matching criteria.



When **Static Template Only** is selected in the **Template Manager Options** form, static force flow is enabled to search only the static template and to find more candidates with specified settings.



In the result viewer, there are two new display modes: Insts and Rate mode and Template and Rate. They are sorted by sorting priority, which means you can find the highly relevant candidates at the top and there are matching rate and difference count information next to the name of leaf node, which means you can clearly know the relevance between template design and selection design.



Search

☒ Post Filter Matches

☐ Allow Flipped MOS Type Matches

☒ Static Template Only

Filtering

☒ Minimum Matching Rate of Instance Parameters 100 %

☒ Maximum Difference of Matching Device Count 0

☒ Total M-Factor

Parameters

☒ Gate Width

☒ Gate Length

☒ M-Factor

☒ Finger Number

Using Matching Parameters and Criteria

To compare SDL parameters of devices between selection design and template design, using the following parameters: **Gate Width**, **Gate Length**, **M-Factor** and **Finger Number**. The matching rate is calculated when selecting the parameters options in the **Parameters** section in the **Template Manager Options** form. The following is the definition of parameters.

The Parameters section describes the following parameters:

- **Gate Width:** When selecting this option, the tool compares the gate width of template design to the gate width of selection design for matching rate calculation. If both values are the same, the tool returns matches. If the device doesn't define the gate width parameters, the tool uses total gate width or fin number instead. If both of them are not defined, the comparison result is false.
- **Gate Length:** When selecting this option, the tool compares the gate length of template design to the gate length of selection design. If both values are the same, the tool returns matches. If the device doesn't define this parameter, the tool uses the total gate length instead. If both of them are not defined, the comparison result is false.
- **M-Factor:** When selecting this option, the tool compares the M-factor of template design to the M-factor of selection design. If the target design is not a M-factor device, then the tool uses the instance counts instead. If the M-factor of template design is same as the M-factor of selection design, the tool returns matches. If not, the tool returns mismatches.

- **Finger Number:** When selecting this option, the tool compares the finger number of template design to the finger number of selection design. If the finger number of template design is same as the finger number of selection design, the tool returns matches. If not, return mismatches.

This section describes the equation of the matching criteria:

- **Matching Rate of Instance Parameters:** The equation below defines the matching rate.

$$\text{Match Rate} = \frac{\text{Matched device count}}{\text{Selection device count}}$$

Matched device means the parameters that users selected in **Template Manager Options** between selection device and template device are all the same. For example, when users select **Gate Width** and **Gate Length**, the parameters of the device D1 which is from selection design are all the same as the device D2 which is from template design. Then D1 is a matched device. The matched device count is the sum of matched devices and the selection device count is the sum of all selection devices. Matched device count is less than or equal to selection device count, so the match rate is not exceed 100%. Match rate 100% means the instance parameters of netlist matched instances are matched to edit design.

- **Difference of Matching Device Count:** The equation below defines the difference count.

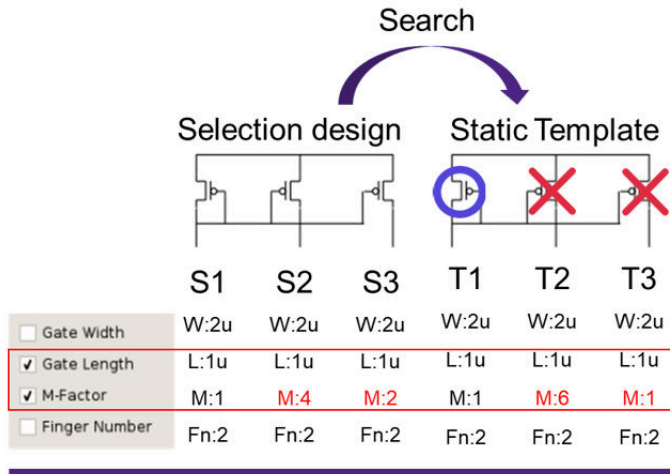
$$\text{Diff Count} = |\text{Selection device count} - \text{Template device count}|$$

Selection device count is the sum of all device of selection design and template device count is the sum of all device of template design. Difference count 0 means selected device counts are the same as the template.

- **Total M-Factor:** This option is the sum of M-factor of all devices. Total M-factor is related to layout area. Users would like to find the template that have the same or similar area.

For example: Users only select the **Gate Length** and **M-Factor** in **Parameters**. There are three devices match after template search. For device mapping, S1 maps to T1, S2 maps to T2 and S3 maps to T3. In matching rate, only the parameters of S1 is the same as the parameters of T1 and there are three devices in selection design so the matching rate is 33.33%. For difference count, both selection device count and template device count are 3 so the difference count is 0. For total M-factor, the total M-factor of selection design is the

$7 = 1 + 4 + 2$ and the total M-factor of template design is 8 so the total M-factor is not the same.



$$\text{Match Rate} = \frac{1}{3} = 33.33\%$$

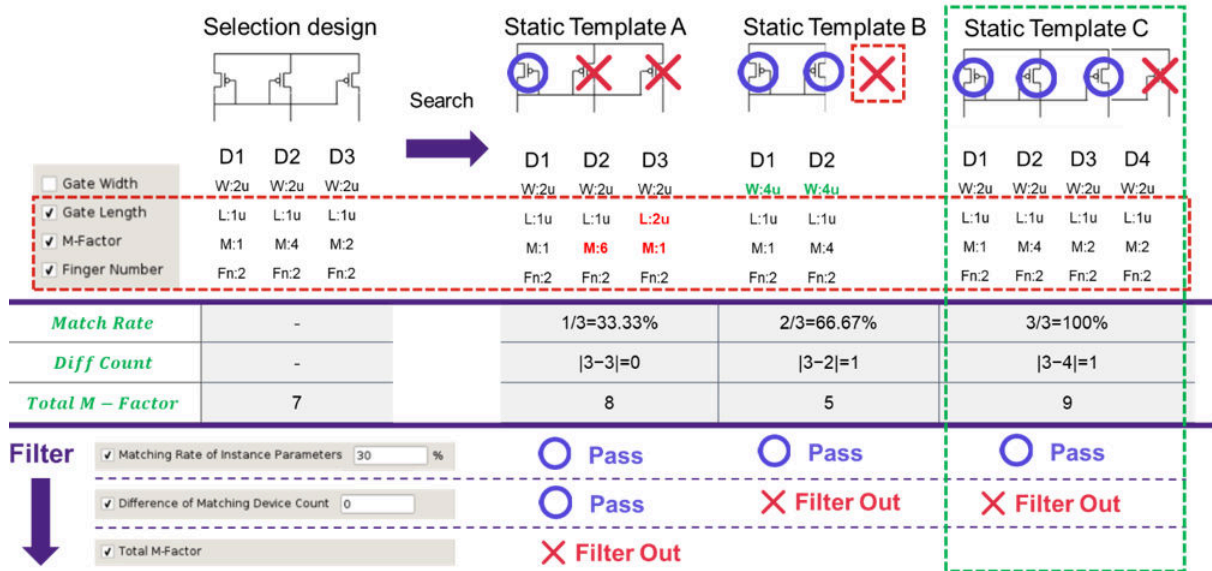
$$\text{Diff Count} = |3 - 3| = 0$$

$$\text{Total M - Factor} \Rightarrow 7 \neq 8 = \text{false}$$

The Filter section describes the following parameters:

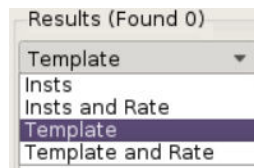
- **Minimum Matching Rate of Instance Parameters:** The valid value is from 0 to 100 and decimal is 6. When selecting this option and the matching rate of candidate is lower than user-specified value, the candidate is filtered out.
- **Maximum Difference of Matching Device Count:** The value is specified as an integer. The valid value is an integer than 0. When selecting this option and difference count of candidate is higher than user-specified value, the candidate is filtered out.
- **Total M-Factor:** When selecting this option and the total M-factor of candidate is not the same as the total M-factor of selection design, the candidate is filtered out.

In the following example, there are three candidates which are static template A, B and C and no candidate passes the filter. See the following figure:



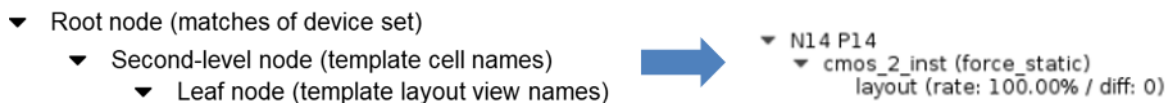
Browsing Matching Results

After getting the qualified static templates, Template Manager provides four modes to browse the matching results in the result viewer: **Insts**, **Insts and Rate**, **Template**, and **Template and Rate**.



The Insts and Rate and **Template and Rate** modes are sorted and grouped by grouping rules. Both modes are base on their structure to construct the result viewer tree.

- Tree structure of **Insts and Rate**



- Tree structure of **Template and Rate**



For the grouping rule, the tool sorts all the candidates by the following priority:

1. Matching rate
2. Difference count
3. Natural string order

For each candidate, if candidates have different matching rate, difference count or their first two-level name are different, candidates are put in different root node in the result viewer. Only if the matching rate, difference count and first two-level nodes are all the same, the candidates are put in the same root node.

If selecting the **Post Filter Matches** option in the **Template Manager options** form, the tool filters the symmetric matches out and keep the best candidates based on the sorting priority.

Saving and Realizing Template Design

When the template design is larger than the selection design, the extra template devices are named with the `template_static_extra_inst` prefix and integer suffix started from 0.



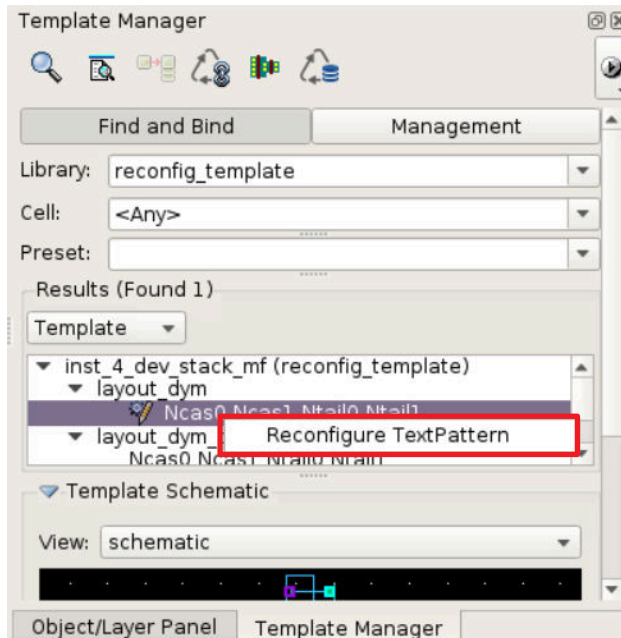
The tool also creates the extra net for those instances and the naming rule is the `template_static_extra_net` prefix with an integer suffix started from 0.

N14.m1	template_static_extra_inst0.m1
N14.m2	template_static_extra_inst0.m2
N14.m3	template_static_extra_inst0.m3
N14.m4	template_static_extra_inst0.m4

Reconfiguring Templates

Using the Reconfigure Template feature, you can reconfigure parameters to create temporary template samples to cover all kinds of combinations and easily adjust the placement.

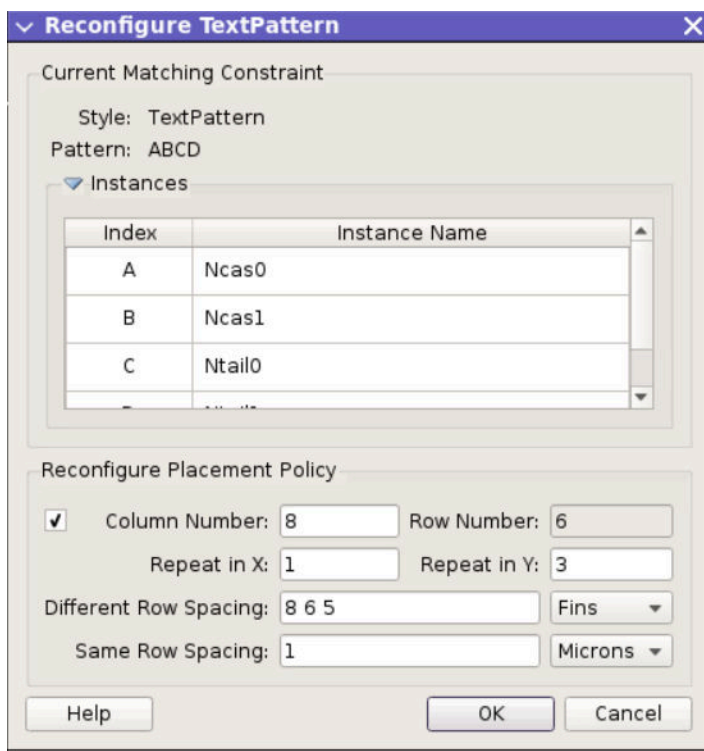
After you search the template, you can right-click the result in the result viewer and select the **Reconfigure TextPattern** option.



To enable **Reconfigure TextPattern**, the template needs to match the following two conditions:

- The template constraint of the selected result must contain only one matching constraint and the pattern style must be TextPattern.
- The template is dynamic template.

If you select more than one result or it is not a supported constraint or it is a static template, the **Reconfigure TextPattern** option is disabled.

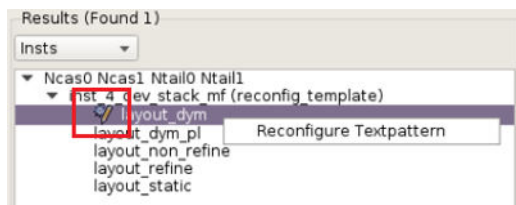


The **Reconfigure TextPattern** dialog box describes the **Current Matching Constraint** and **Reconfigure Placement Policy** sections.

The **Current Matching Constraint** section includes the following information of the current constraint: Pattern Style, Pattern, Instance Mapping Table.

The **Reconfigure Placement Policy** section includes the following modifiable parameters: **Column Number**, **Row Number**, **Repeat in X**, **Repeat in Y**, **Different Row Spacing**, the **Unit** of different row spacing, **Same Row Spacing**, and the **Unit** of same row spacing. In the beginning, the tool loads these parameters values from the current matching constraint.

After clicking **OK** for the modification, all the changes are updated and saved to a temporary template constraint. It is cleared when you search the template again. You can find an icon in front of its leaf name for a reconfigured result.



In the end, you can go to preview, save a constraint, or realize the template. All operations are based on this reconfigured template.

Setting Options for Template Manager

All Template Manager related options are centralized in a preference dialog box. Display the dialog box by choosing **Assistant Menu > Options** on the toolbar as shown in [Figure 91](#).

Figure 91 Assistant Menu > Options

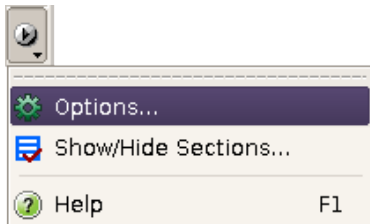


Figure 92 shows the **Template Manager Options** dialog box.

Figure 92 *Template Manager Options Dialog Box*

Template Manager Options

Creation

☒ Refine Constraints

Search

☒ Post Filter Matches

☐ Allow Flipped MOS Type Matches

☐ Static Template Only

Filtering

☒ Minimum Matching Rate of Instance Parameters %

☒ Maximum Difference of Matching Device Count

☒ Total M-Factor

Parameters

☒ Gate Width

☒ Gate Length

☒ M-Factor

☒ Finger Number

Realize

☒ Create Top-level Template Constraint

☐ Auto-delete Constraint

Reference Point:

X: Y:

☐ Realize to Specified Point

☐ Multiple Solutions

Max Results:

Table 18 describes the **Template Manager Options** dialog box options.

Table 18 Template Manager Options Dialog Box Options

Option	Description
Creation	Refine Constraints: Specifies whether to refine constraints from the current design when creating templates. By default, the Extract Topology Constraints Command is implicitly invoked to extract constraints for the selected layout and the extracted constraints are used to create templates. - Refinement is applied only to the following constraints: Inst Matching (performed based on the following pattern style priority: Device Group, Interdigitate, Common Centroid, Central Symmetric, Balanced Pattern, and Text Pattern) Resistor Bank (performed based on the Series Chain pattern style) CMOS If none of the preceding pattern styles can be identified from the current layout, then constraints from the current design are kept and used for template creation. Tcl preference: <code>lxTemplateManagerRefineConstraint</code>.
Search	Post Filter Matches: Affects the searching algorithm as follows: - Off: The searching algorithm shows all possible matches. - On: (default) The searching algorithm filters out some unreasonable matches based on internally decided priority. Tcl preference: <code>lxTemplateManagerPostFilterMatches</code>. Allow Flipped MOS Type Matches: Affects the searching algorithm as follows: - Off: (default) Only matching MOS type results are found and displayed. For example, an NMOS is not matched to a PMOS. - On: Matching of flipped MOS types is allowed. For example, if you select a <code>diffpair_n</code> structure in the editing design then both template cells, <code>diffpair_n</code> and <code>diffpair_p</code>, are matched. Any flipped matches are displayed with a <code>(flipped)</code> suffix in the Results viewer. Tcl preference: <code>lxTemplateManagerFlippedMOSType</code>. Static Template Only: Affects the searching algorithm as follows: the Filtering and Parameters sections. See Working With Static Flow.
Realize	Create Top-level Template Constraint: Specifies whether a top-level template constraint is created. - In the Find and Bind tab, the Save Constraints , Preview , and Realize Template  actions honor this option. - In the Management tab, the Realize Template  action does not honor this option and the original template constraint is retained. For the Preview  action, this option is honored.

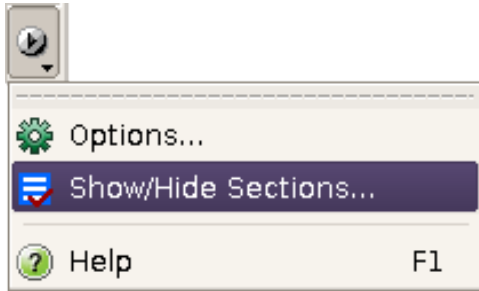
Table 18 *Template Manager Options Dialog Box Options (Continued)*

Option	Description
	Auto-delete Constraint: The Template Manager creates placement groups to cover all realized oaFig objects. An auto-delete flag on the placement group indicates that the corresponding constraints should be kept or deleted after the layout oaFig objects are deleted. If this option is turned <i>on</i>, the Template Manager sets the auto-delete flag to true when creating a placement group; otherwise it is false. By default, this option is turned <i>off</i>. Tcl preference: <code>lxTemplateAssistantAutoDeleteConstraint</code>. Reference Point: Specify the reference point for realizing the layout. In advance nodes, there are track patterns/regions on the layout and you may want to snap instances on track during realization. To avoid an off-track result, specify the reference point first; the realization system then passes this coordinate as the starting location for all placers. Placers put instances on track at the nearest legal location of this coordinate based on track definitions, placement constraints, and technology rules. Note that if you place the layout at a non-reference point, the result may be off track. Also, if the reference point itself is not a legal point (that is, not on track), then the Template Manager chooses the nearest point as the internal reference point. The default value (X, Y) is (0, 0). Set from Layout: Click this to hide the dialog box and display the text 'Reference Point' in follow-the-cursor mode, then click a point in the layout window to specify it as the reference point. After the point is specified, the dialog box reappears and the value of the (X, Y) is updated. Tcl preference: <code>lxTemplateManagerReferencePoint</code>. Realize to Specified Point: Sets the use model for determining the position of the realized layout as follows: • On: When you click Realize Template , the realized result generated is placed at the point specified in Reference Point. Note that guard ring instances and pattern route shapes are not placed to the reference point. • Off: (default) When you click Realize Template , the realized result is placed using follow-the-cursor mode, as if you had clicked Set from Layout. Tcl preference: <code>lxTemplateManagerRealizeToSpecifiedPoint</code>. Multiple Solutions: Allows to generate multiple solutions for one template candidate when users click Preview. The Multiple Solutions feature is supported in both Find and Bind and Management tabs. Tcl preference: <code>lxTemplateManagerMultipleSolutions</code>. Note: • This option only supports in <code>instMatching</code> and <code>resistorBank</code> constraints. • Max Results: Specifies the maximum results generated by Template Manager. The original placement result of the template is included in these multiple solutions. Tcl preference: <code>lxTemplateMaxNumResults</code>.

Showing and Hiding Sections of Template Manager

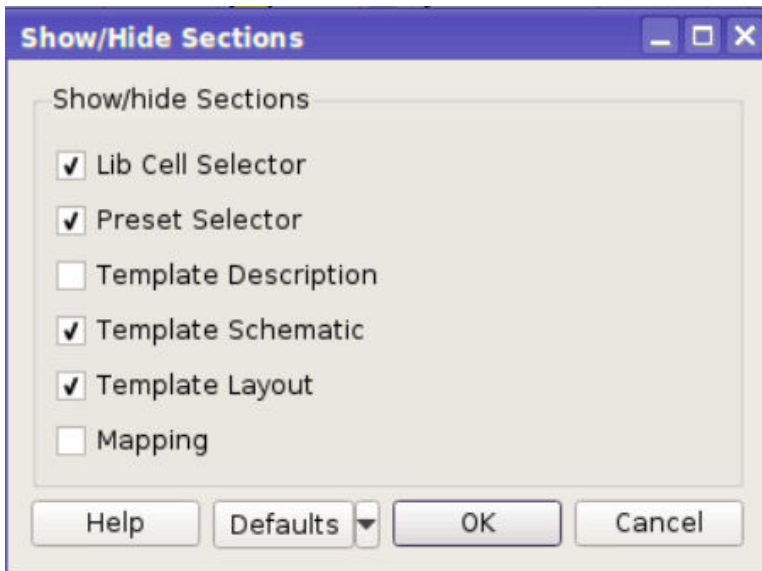
Use the **Show/Hide Sections** dialog box to control the visibility of components in the Template Manager. Display the dialog box by choosing **Assistant Menu > Show/Hide Sections** on the toolbar as shown in [Figure 93](#).

Figure 93 Assistant Menu > Show/Hide Sections



The **Show/Hide Sections** dialog box contains options, as shown in [Figure 94](#), which control the visibility of the corresponding Template Manager components.

Figure 94 Show/Hide Sections Dialog Box



The visibility control affects both the **Find and Bind** and **Management** tabs of all opened design contexts. However, the **Lib Cell Selector** option is redundant for the **Management** tab as it is only available on the **Find and Bind** tab.

By default, **Lib Cell Selector**, **Preset Selector**, **Template Schematic**, and **Template Layout** are shown, while **Template Description** and **Mapping** are hidden.

Using the Template Browser

The Template Browser is described in the following sections:

- [Introduction to Template Browser](#)
- [Working With Template Browser](#)

Introduction to Template Browser

The Template Browser is introduced to reuse the template which is not netlist-matched with target design. When the netlist of the two circuits are different, you can reuse the template to update the instance or net mapping easily using Template Browser. The Template Browser also supports cross-highlighting and automatically mapped instances. Both of which help you to fill in the mapping table effectively.

The Template Browser is outlined in the following sections:

- [Prerequisites for Template Browser](#)
- [Invoking Template Browser](#)
- [Using the Template Browser Interface](#)

Prerequisites for Template Browser

Before you can invoke the Template Browser, you must start an SDL session by choosing **Tools > SDL** (see [Figure 64](#)).

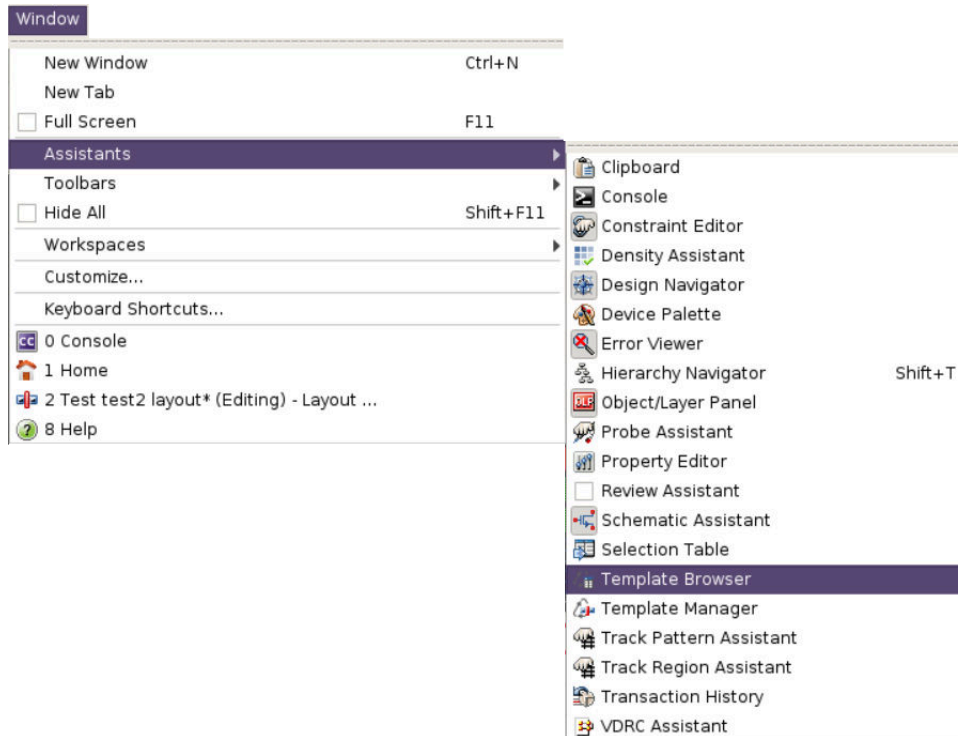
When the SDL session is turned off with Template Browser online, the Template Browser is closed automatically.

Invoking Template Browser

Start the Template Browser using either of the following methods:

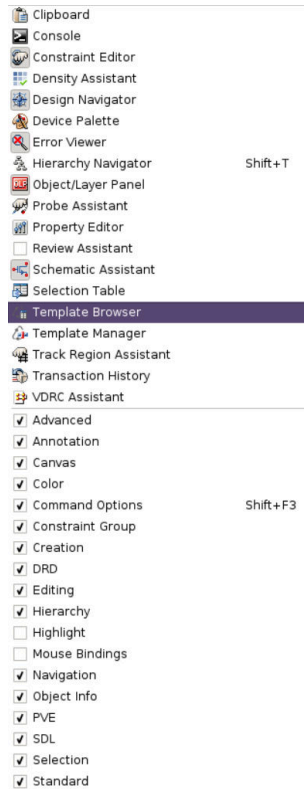
- Choose **Window > Assistants > Template Browser** (see [Figure 95](#)).

Figure 95 Invoke from Window > Assistants > Template Browser



- Right-click the design window toolbar and choose **Template Browser** (see [Figure 96](#)).

Figure 96 *Invoke from Design Window Toolbar Menu*



Using the Template Browser Interface

Figure 97 shows the Template Browser interface.

Figure 97 Template Browser Interface

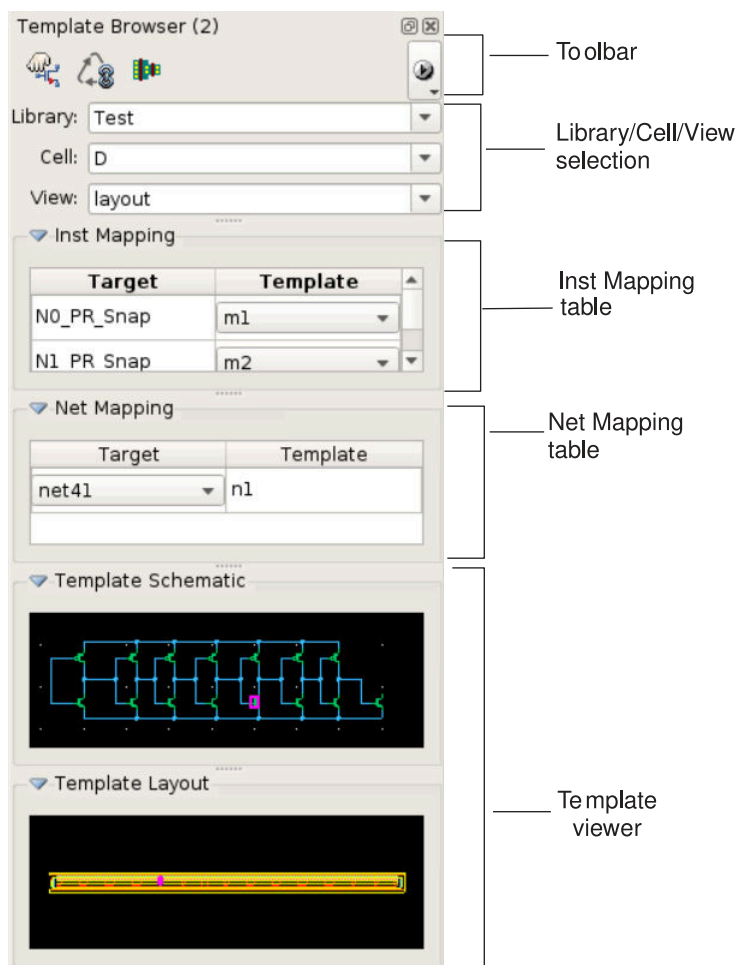


Table 19 describes the toolbar of the Template Browser interface.

Table 19 Template Browser Interface — Toolbar

Toolbar Command	Icon	Description
Grab Instances		Brings the selection from Design Navigator of the current design to the mapping tables. Before Grab Instances is triggered, Library , Cell , and View fields and mapping tables are disabled.

Table 19 Template Browser Interface — Toolbar (Continued)

Toolbar Command	Icon	Description
Save Constraints		Saves constraints in the template cell to the active design (requires the completed instance mapping tables). See Saving Template Constraints .
Realize Template		Invokes the realization system to instantiate a template layout to the editing layout (requires the completed instance mapping tables). See Realizing a Template to the Editing Layout .

[Table 20](#) describes the mapping information and Template Viewer of the Template Browser interface.

Table 20 Template Browser Interface

Tab Section	Description
Library, Cell, and View fields	Specifies the reused template layout which is a non-static template. Only libraries contain one or more template cells are listed in the Library field. Only cells that the instance count equals to the selected instance count are listed in the Cell field. The View field only displays the cellviews with the <code>maskLayout</code> type.
Instance and Net Mapping tables	Consists of two tables in different tabs, Instance and Net , which can be assigned the mappings. Once the Library , Cell , and View fields are specified, an initial mapping is brought to Inst Mapping and Net Mapping tables. You can click rows in the Inst Mapping table to show cross-highlighting between the template schematic, template layout, and the schematic of the active design with a system-specified color. See Mapping Table .
Template viewers	Consists of a schematic selection and two preview panes: Template Schematic and Template Layout . When you specify the template in the Library , Cell , and View fields, the schematic and layout in the template viewers will be updated. The Template Schematic shows the first schematic view under the specified template cell. When you click a device in the template schematic viewer, the device is highlighted on mapping table and Schematic Assistant. See Template Viewer .

Working With Template Browser

The Template Browser provides the capability to instantiate template with given selection from the current design, template layout, inst mapping, and net mapping.

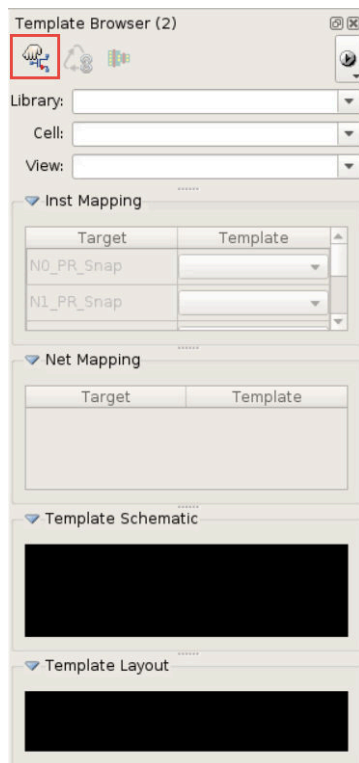
Note:

Before using template browser:

- A template need to be prepared for reuse.
- Template Browser only supports the non-static template when reusing.
- The grabbed instances cannot be realized.

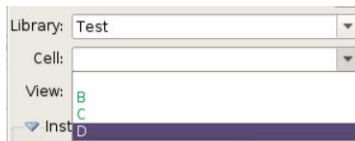
To reuse the template:

1. Selects instances to be realized.
2. Invokes Template Browser. See [Invoking Template Browser](#).
3. Clicks **Grab Instances** to grab the selected instances and then the **Library**, **Cell**, and **View** fields are enabled. See the **Template Browser Interface — Toolbar** table.

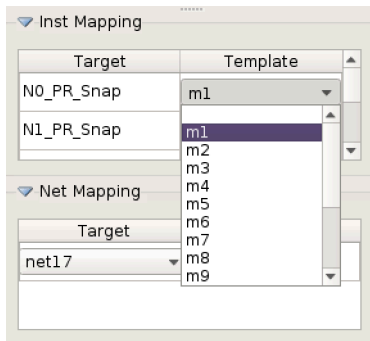


4. Specifies the reused template layout in the **Library**, **Cell**, and **View** fields. Only the cells that have the same instance count as the selected instance count are listed in the **Cell** field. See the **Template Browser Interface** table.

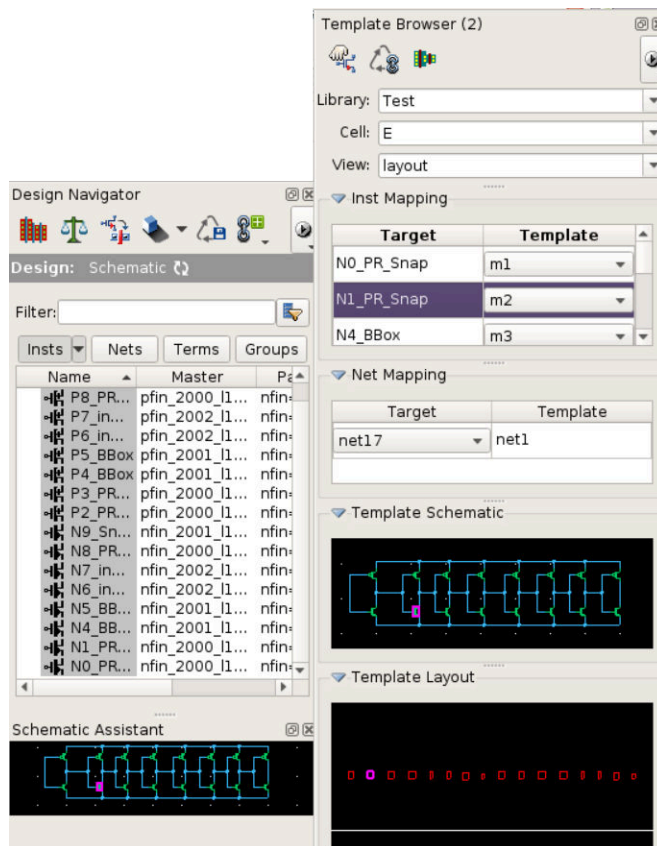
The instance count in cell B, cell C, and Cell D are the same.



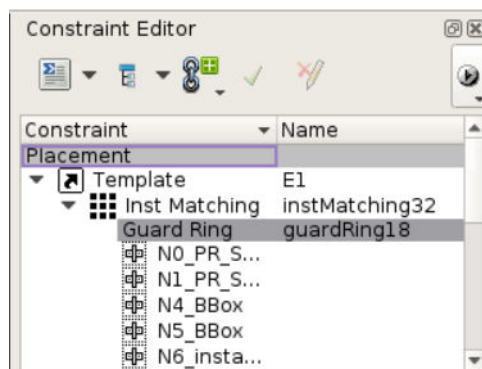
5. After specifying the reused template layout, the **Inst Mapping** and **Net Mapping** tables are enabled with the initial mapping and the reused template is shown in the **Template Schematic** and **Template Layout** views. You also can assign the mapping manually in the selection field.



When clicking rows to assign the mapping in the selection field, the relative object is highlighted between the **Schematic Assistant**, **Template Schematic** and **Template Layout**.



- After completing the mapping table, the **Save Constraints** and **Realize Template** are enabled. (Optional) When clicking Save Constraints, the constraint is saved under placement constraints. See the **Template Browser Interface — Toolbar** table.



- After completing the mapping, you can realize the design into layout window by clicking **Realize Template**. See the **Template Browser Interface — Toolbar** table.

Setting Template Attributes

Use the Template Attributes assistant to graphically edit template attributes. This assistant is only available for active designs whose cellviews are part of a template cell.

The Template Attributes assistant is outlined in the following sections:

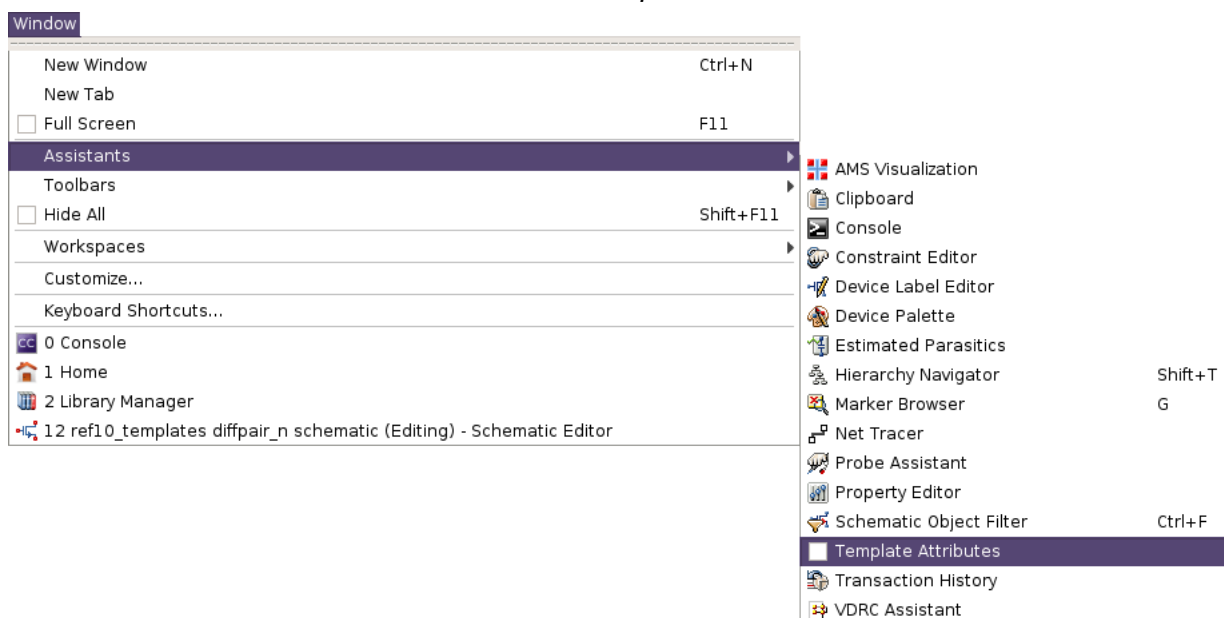
- [Invoking the Template Attributes Assistant](#)
- [Using the Template Attributes Assistant](#)

Invoking the Template Attributes Assistant

After opening a template layout view or template schematic, start the Template Attributes assistant using either of the following methods:

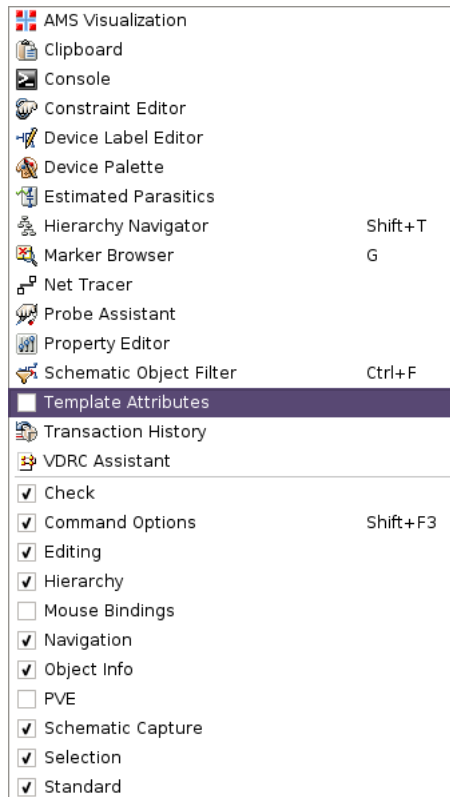
- Select **Window > Assistants > Template Attributes** (see [Figure 98](#)).

Figure 98 Invoke from Window > Assistants > Template Attributes



- Right-click the design window toolbar and choose **Template Attributes** (see [Figure 99](#)).

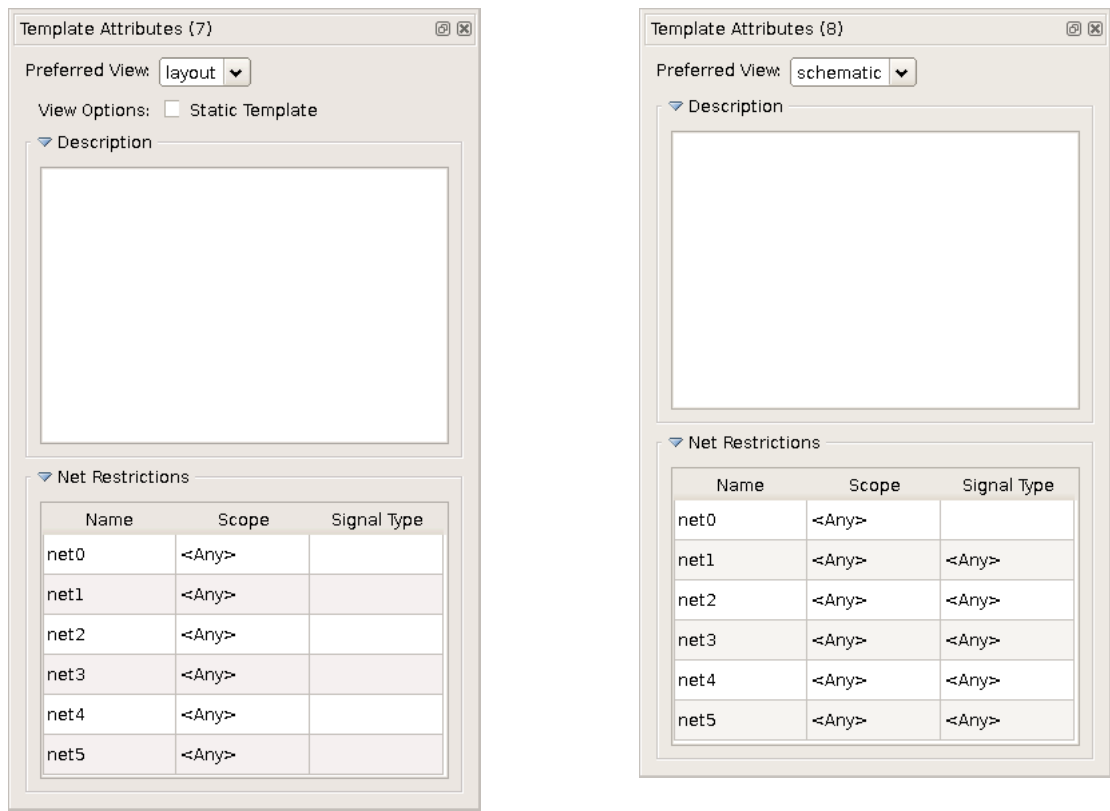
Figure 99 *Invoke from Design Window Toolbar Menu*



Using the Template Attributes Assistant

As shown in [Figure 100](#), the Template Attributes assistant presents different options depending on the view type (that is, whether it is displaying the attributes for a template layout view or template schematic).

Figure 100 *Template Attributes Assistant (Layout view on left, Schematic on right)*



[Table 21](#) describes the Template Attributes assistant interface.

Table 21 *Template Attributes Assistant Interface*

Option	Description
Preferred View	Contains all view names of the same view type as the active design in addition to an empty entry. This allows you to select the preferred schematic view or to have no preferred view set if the empty entry is selected.
View Options	When the active design is of view type maskLayout, this field is shown to allow you to indicate whether the active design should be used as a Static Template .

Table 21 *Template Attributes Assistant Interface (Continued)*

Option	Description
Description	Allows you to add documentation and comments about the views contained in the template.
Net Restrictions	Contains a table listing all the nets found in the template netlist. The table has the following columns: • Name: Holds the net names. All nets found in the template netlist are listed. • Scope: Displays the current state of the net with respect to whether that net is considered an internal, external, or unrestricted net. Any one net can be included in either the internalNets or externalNets attributes, but not both. Nets in the internalNets attribute are labeled as Internal Only, while nets in the externalNets attribute are labeled as Must Connect. Any net that is not in either attribute is designated with the label <Any>. • Signal Type: Allows specific signal type requirements for each net. The supported signal types are: power, ground, clock, and <Any>. The values from the signalTypes attribute are reflected in this column. The Net Restrictions table allows single row selection and when a row is selected, that net is highlighted in the active design.

7

Using the Placement Assistant

Layout Assistants speed layout creation and synthesis with visually guided automation of placement and routing. The Placement Assistant uses a new innovative approach to device placement. It allows you to make successive refinements, offering placement choices but leaving you in full control of the results—without requiring any up-front textual constraint entry.

The Placement Assistant is discussed in more detail in the following sections:

- [Introduction to Placement Assistant](#)
- [Create Placement Area Command](#)
- [Auto Place Command](#)
- [Place Pins Command](#)
- [Optimize Pins Command](#)
- [Placement Options Dialog Box](#)
- [Placement Results](#)
- [Placement Results Browser](#)
- [Copy Result Command](#)
- [Generating Constraints](#)
- [Extract Topology Constraints Command](#)

Introduction to Placement Assistant

The Placement Assistant is implemented as a tool in the Layout Editor and is completely integrated into the Custom Compiler environment. It provides a command (see [Auto Place Command](#)) for running the automatic placer on a given design or a selection set, either in batch or interactive mode.

The visualization of placement results is accomplished through a split-view of the current Layout Editor window, displaying the design that was placed (by default, at the top of the split-view), which allows you to do the following:

- Visualize placement results in a Placement Gallery (by default, at the bottom of the split-view).
- Copy placement results from the placement library back to the design library, which is referred to as the target library in this document.
- Cross-highlight between the Placement Gallery and layout, and also schematic if cross-highlighting is enabled between layout and schematic.

Note:

An SDL session is a prerequisite for the Placement Assistant.

After the results are produced and inspected, you can employ the interactive or the non-interactive command (see [Copy Result Command](#)) to copy the desired placement result from the placement library back to the target library.

Prerequisites for Placement Assistant

To use the Placement Assistant, you must take care of the following prerequisites:

- Define the layout hierarchy properly (see [Preparing the Layout Hierarchy](#)) — if you do not have the proper layout hierarchy defined, the Placement Assistant follows the SDL *config* to flatten the design for placement.
- Open a Schematic Driven Layout (SDL) session.

In addition, the PDK must be SDL-ready to be able to use the Placement Assistant (see [PDK Requirements for Placement Assistant](#)).

PDK Requirements for Placement Assistant

`lxCmpType` data is optional; any devices for which `lxCmpType` data is not defined, are treated as custom devices, as opposed to MOSFET devices, resistors, capacitors, and so on.

However, for MOSFET devices, resistors, capacitors, and other relevant devices, the Placement Assistant requires you to define the `lxComponentType` elements as follows:

- Cell name: (`"cells" ("cell_name1" "cell_name2" ...)`)
- Device type: (`"type" "type_name"`)
 - For MOSFET devices, `type_name` is anything matching the patterns: `[Pp][Mm][Oo][Ss].*` or `[Nn][Mm][Oo][Ss].*`

Examples:

```
("type" "PMOS")
("type" "nmos")
("type" "PMos_1")
("type" "nMosABC")
```

- For resistors, `type_name` is anything matching the pattern: `[Rr][Ee][Ss].*`

Examples:

```
("type" "Resistor")
("type" "RES")
("type" "res_1")
```

- For capacitors, `type_name` is anything matching the pattern: `[Cc][Aa][Pp].*`
- For diodes, `type_name` is anything matching the pattern: `[Dd][Ii][Oo].*`
- For inductors, `type_name` is anything matching the pattern: `[Ii][Nn][Dd].*`
- For bipolar devices, `type_name` is anything matching the patterns: `[Bb][Ii][Pp][Oo][Ll][Aa][Rr].*` or `[Bb][Jj][Tt].*`

The following `lxComponentType` elements are optional (except as noted):

- `lxDeviceLength`: Length parameter (mandatory for MOSFET devices, optional for all other types)
- `lxDeviceWidth` or `lxDeviceTotalWidth`: Width parameter per finger and total, respectively (mandatory for MOSFET devices, optional for all other types)
- `lxFingerParam`: Number of fingers

Note:

The m-factor is either specified through the Placement Assistant or uses the defaults `m` or `M`.

In addition, for MOSFET devices, resistors, capacitors, and so on, the Placement Assistant also requires you to set the device terminal order (terminal order is optional for custom devices). Set either of the following in search order:

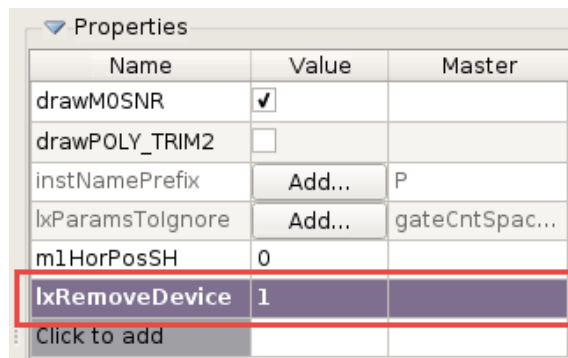
- `simInfo|sdl|termOrder`
- `simInfo|auLvs|termOrder`

The following is an example of the definition for a resistor:

```
((("cells" (rnpoly))) ("type" "resistor") ("lxDeviceLength" "l")
 ("lxDeviceWidth" "w") ("lxFingerParam" "nf")) ...)
```

For schematic instances that must be ignored by the Placement Assistant (that is, left open), you must set a property in the instance with a name found in the `lxIgnoreNames` preference: `nlAction`, `lvsIgnore`, or `lxIgnore` (see [Ignoring Objects](#)).

For schematic instances that must be ignored by the Placement Assistant (that is, left short), you must set a property in the instance with a name found in the `lxShortNames` preference: `lxRemoveDevice`.



See Property Editor Assistant in the *Custom Compiler Environment User Guide* for more information on adding and editing properties.

The Placement Assistant reads only the first type defined in the `lxComponentType` data; that is, each type name can only be used once in the `lxComponentType` data. This means the `lxComponentType` data for each library cannot have more than one group of cells with the same type name. For example:

```
( ( ("cells" ("p_4t")) ("type" "pmos") ... )
  ("cells" ("ph_4t")) ("type" "pmos") ... ) )
```

In the above example, there is an attempt to use the same type name `pmos` for two different groups of cells. The Placement Assistant issues a warning message, and only the first occurrence (that is, the group containing cell `p_4t`) is considered. Any attempt to lookup any cell from the second group (that is, the group containing cell `ph_4t`) results in an error message, and the Placement Assistant fails.

To avoid this situation, it is recommended that you use unique type names. For example:

```
( ( ("cells" ("p_4t")) ("type" "pmos1") ... )  
  ( ("cells" ("ph_4t")) ("type" "pmos2") ...) )
```

Preparing the Layout Hierarchy

In order for the Placement Assistant to be able to place hierarchically, the layout hierarchy must be prepared accordingly, following the SDL top-down flow and soft macro creation (see [Soft Macro Creation](#)).

The current/top design view, on which the Placement Assistant is applied, must be a maskLayout design view having a corresponding *config* view created using SDL functionality. Each level/cell in the top design hierarchy that you want to be automatically placed must also contain a layout design view having its *oaCellType* set to *softMacro* and its corresponding *config* view.

Note:

The *oaCellType* is ignored for the top design because you directly call the Placement Assistant on this design.

The placement of a soft macro layout design view is recreated every time the placement of any of its ancestor cells is recreated.

A layout design view that is not a soft macro in the top design hierarchy is not placed because it is considered a leaf in the hierarchy and used as is by the Placement Assistant. No corresponding *config* view is necessary for such a layout view.

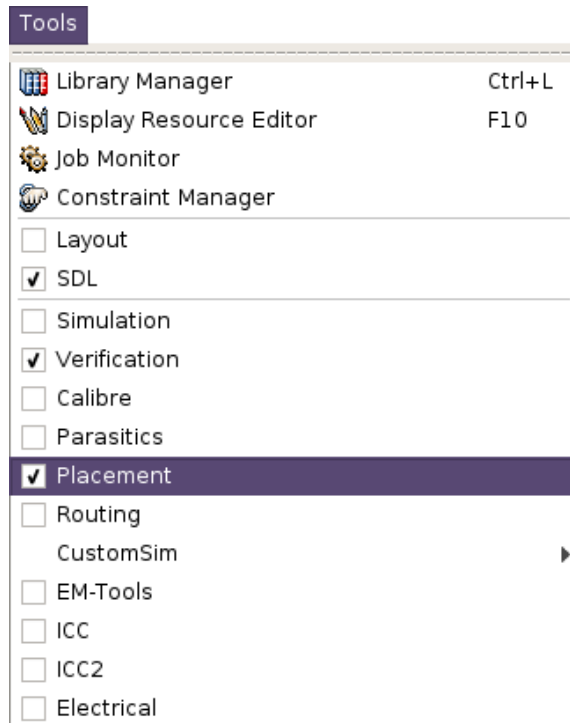
At each hierarchy level, the current *config* view semantics is the one used by the Custom Compiler environment. This means that the Placement Assistant generates the layout for the current cell with the same layout hierarchy as the one created when using SDL placement functionality such as **Pick and Place** (see [Picking and Placing](#)) or **Generate Layout** (see [Generating Layout](#)). Also, the Placement Assistant does not perform any hierarchy manipulation—the same behavior as in the SDL placement case.

Enabling the Placement Menu

Placement functionality is implemented as a tool in the Custom Compiler Layout Editor.

As shown in [Figure 101](#), choose **Tools > Placement** to enable the **Placement** menu in the Layout Editor.

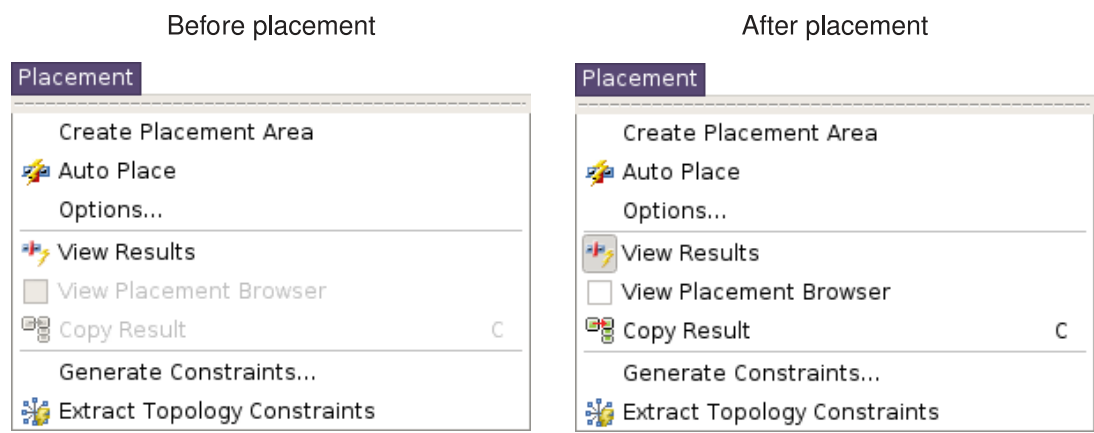
Figure 101 Tools Menu



Using the Placement Menu

As shown in [Figure 102](#), the **Placement** menu is displayed in the Layout Editor and gives you access to the Placement Assistant functionality.

Figure 102 Placement Menu



[Table 22](#) describes the **Placement** menu actions.

Table 22 Placement Menu Actions

Action	Description
Create Placement Area	Allows you to create a rectilinear shape in the main edit canvas that is used as a placement area when placing objects using the Auto Place command. See Create Placement Area Command .
Auto Place	Invokes the automatic placement of either an entire layout design view, or a portion of a layout design view, as determined by a selection of logical objects. See Auto Place Command .
Options	Specifies placement option preferences, including pin placement options. See Placement Options Dialog Box .
View Results	Displays placement results for the current cell (layout design view) in a Placement Gallery. See Placement Results .
View Placement Browser	Displays multiple placement solutions in tabular and graph format. See Placement Results Browser .

Table 22 Placement Menu Actions (Continued)

Action	Description
Copy Result	Copies a placement result from the placement library to the target library. See Copy Result Command .
Generate Constraints	Creates an initial set of constraints for the design by: applying templates from a template library, grouping standard cells according to which standard cell library they come from, and grouping other devices according to cell definition. See Generating Constraints .
Extract Topology Constraints	Extracts the layout topology from an existing layout, and creates Placement Constraints . This can help ensure that later runs of the Auto Place command re-create only similar results. See Extract Topology Constraints Command .

Create Placement Area Command

Choose **Placement > Create Placement Area** to activate the **Create Placement Area** command which allows you to create a rectilinear shape in the main edit canvas that is used as a placement area when placing objects using the [Auto Place Command](#).

Note:

The placement area shape is not a physical shape, such as a prBoundary, that is saved along with the design. Instead, it is a virtual shape that is made visible during its creation, and during its use by the interactive **Auto Place** and **Copy Result** commands. It is not saved with the design.

The **Create Placement Area** command is described in more detail in the following sections:

- [Placement Area Toolbar](#)
- [Using the Create Placement Area Command](#)

Placement Area Toolbar

The interactive [Create Placement Area Command](#) invokes the following toolbar:



[Table 23](#) describes the options provided by the **Placement Area** toolbar.

Table 23 Placement Area Toolbar Options

Option	Description
 Set Placement Area Shape Style to Rectangle	Sets the style of the placement area to be rectangular.
 Set Placement Area Shape Style to Polygon	Sets the style of the placement area to be polygonal.
Set from Viewport	Sets the placement area to be a rectangle with the same coordinates as the current viewport.
Clear	Removes any existing placement area.

Using the Create Placement Area Command

The [Create Placement Area Command](#) is analogous to the **Router > Create Routing Area** command. The command works in the following way:

1. Choose **Placement > Create Placement Area** to display the [Placement Area Toolbar](#).
2. If the desired shape style is rectangular, click , then click the main edit canvas to set the first corner point of the placement area, and click a second time to set the diagonally opposite corner. The shape automatically completes once the second corner point is set.

If the desired shape style is polygonal, click , then click multiple times to set the corner points of a rectilinear polygon. As more corner points are set, a rubber-banded shape displays the current outline. Once the desired shape has been achieved, press Enter to complete the shape.

Note:

If replacing an existing shape, the old shape is not removed until the new shape is complete. Pressing Esc at any time during the creation of the new shape cancels it and retains the old shape.

3. To use the current viewport (that is, the rectangular region currently visible within the canvas) as a placement area shape, click **Set from Viewport**.
4. To remove the existing placement area shape, without replacing it with a new shape, click **Clear**.

Note:

The coordinates of the corner points of the placement area shape are stored in a cellview-scoped Tcl preference, `lpPlacementArea`, for the duration of the session. The shape is not saved with the design, so it must be recreated each time the design is closed and reopened.

Auto Place Command

Choose **Placement > Auto Place** to activate the **Auto Place** command, which invokes the automatic placement of either an entire layout design view, or a portion of a layout design view, as determined by a selection of logical objects. This command prepares the input data for the selected objects, then runs to create placement results in the foreground (that is, blocking) mode.

The **Auto Place** command is described in more detail in the following sections:

- [Place Toolbar](#)
- [Using the Auto Place Command](#)

Place Toolbar

The interactive [Auto Place Command](#) invokes the following toolbar:



[Table 24](#) describes the options provided by the **Place** toolbar.

Table 24 Place Toolbar Options

Option	Description
Selected	Specifies whether to place only the selected objects, as opposed to the entire design. Tcl preference: <code>lpPlaceSelected</code>
Instances	Specifies whether to place instances. Tcl preference: <code>lxCreateInstances</code>
Pins	Specifies whether to place pins. Note: The Auto Place command only places pins when placing the entire layout design view. Therefore, if Selected is specified, then Pins is disabled. Tcl preference: <code>lxCreatePins</code>
Pin Labels	Specifies whether to create pin labels. Note: The Auto Place command can only create pin labels if pins are placed. Therefore, if Pins is not specified, then Pin Labels is disabled. Tcl preference: <code>lePinLabel</code>

Table 24 *Place Toolbar Options (Continued)*

Option	Description
Results	Specifies the maximum number of placement results to be generated and displayed in the Placement Gallery. Tcl preference: <code>lpMaxNumResults</code>
Packed	Specifies whether to treat those layout objects being placed as rectilinear blocks in order to create more packed placement results with extra effort at the top level. Tcl preference: <code>lpPlacePackedPartition</code>
Background	Specifies whether the placement jobs should be run in the background (that is, non-blocking). Tcl preference: <code>lpPlaceBackground</code>
Apply	Invokes placement on the current logical selection set.

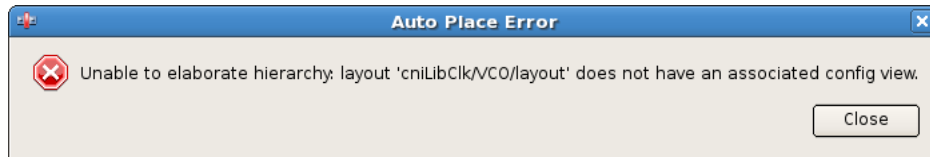
Using the Auto Place Command

The [Auto Place Command](#) requires that the target layout design is open, and that SDL has been initialized. The command works in the following way:

1. (Optional) Choose **Placement > Options** to open the [Placement Options Dialog Box](#) and set up the Placement Assistant's preferences, including the placement library, if needed.
2. Preselect (if desired) a set of objects (logical instances) in any of the schematic design views associated with the layout design view (via SDL session): Schematic Editor, Schematic Assistant, or Design Navigator (Design: Schematic).
3. Choose **Placement > Auto Place** to display the [Place Toolbar](#).

If necessary, you are asked to save the current design and/or its associated *schematic* or *config* designs.

- If no *config* view is associated with the current layout through SDL, the placement fails and the following error message box is displayed:



- For soft macro cell layouts referenced by the current cell layout hierarchy and which have no *config* view associated through SDL, the placement fails with the following error displayed in the console:

Error: Unable to elaborate hierarchy: layout '<LCV>' does not have an associated config view.

4. If **Selected** is unchecked, then **Apply**  is enabled and clicking it begins placement of the entire design.

If **Selected** is checked, then **Apply**  is not enabled until there is a valid selection, after which, clicking it begins placement of the selected objects only.

5. (Optional) Before clicking **Apply**  and beginning placement, you can set the following:
 - **Results**: Sets the maximum number of results to be generated.
 - **Packed**: Sets whether to treat those layout objects being placed as rectilinear blocks in order to create more packed placement results with extra effort at the top level.
6. Once satisfied, click **Apply**  to launch the placement job, which blocks further action until the placement results have been generated.
7. After the placement job for the current cell layout finishes, you can view the placement results by choosing **Placement > View Results** to trigger a split-view of the current layout window (see [Placement Results](#)).
8. During the split-view of the current layout window, you can choose **Placement > Copy Result** to start the interactive [Copy Result Command](#) to copy the desired result from the Placement Gallery.

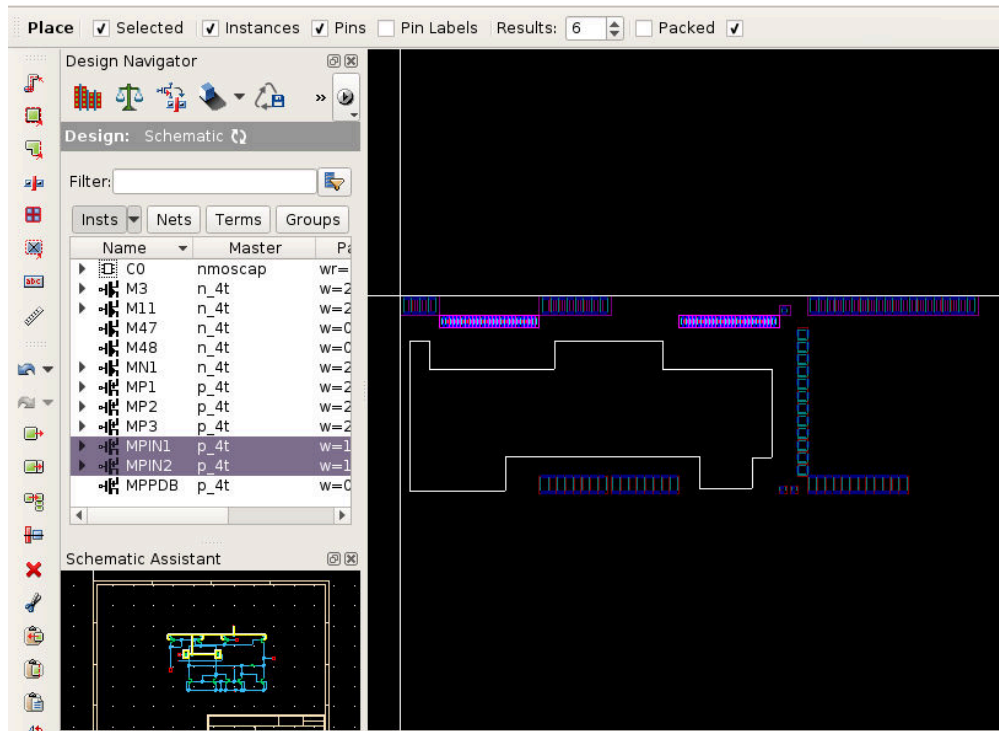
Auto Place Command Usage Notes

- The **Auto Place** command uses the [Placement Library](#) defined by the preference `lpPlacementLibrary`, and the same view names for the placement variants (for example, `layout000`, `layout001`, and so on) and the Placement Gallery (for example, `results`). If you wish to preserve older results, it is your responsibility to change

placement libraries or rename the `layout $nnn$` and `results` views generated in the previous run.

- Any constraint applied to an "as-is" `oaFigGroup` is ignored by the placer if the constraint creates an `oaFigure`. An "as-is" `oaFigGroup` is one where at least one of its members has its placement status set to `Fixed` or `Locked`. The **Auto Place** command places it as a layout piece without any change to its content.
- Preference `lpMaxNumUnconstrained` (default value 10) specifies the maximum number of unconstrained devices that can be selected for placement before a warning must be issued. When counting the number of unconstrained instances that have been selected for placement, the **Auto Place** command ignores instances that are members of constraints, as well as instances that are members of "as-is" `oaFigGroups`.
- Prior to launching the placement job, any open Placement Gallery is emptied of its contents. This action is required to allow the placement engine to overwrite the `layout000`, `layout001`, and `layout $nnn$` designs. Once placement is complete, the Placement Gallery is repopulated. If no Placement Gallery is open, a new Placement Gallery may be created depending on the value of Tcl preference `lpAutoOpenGallery` (set to `True` by default).
- Any schematic instances with parameter `lxIgnore` set to `true` are ignored during **Auto Place**.

- If a placement area has been defined using the [Create Placement Area Command](#), it is displayed during **Auto Place** as a visual reminder.



Placement Jobs Submission

The [Auto Place Command](#) ensures that the placement of a hierarchical top cell layout design is created correctly, in a bottom-up manner, starting with its leaf cells.

In the case of a hierarchical top cell layout design, the **Auto Place** command creates a list of dependencies between the cells that form the top cell hierarchy and, based on this list of dependencies, the placement jobs are submitted and executed one at a time.

If a cell is in View Results mode—that is, the layout window is in split-view mode displaying the Placement Gallery (see [Viewing Placement Results](#))—at the time **Apply**  is clicked, the Placement Gallery is cleared.

Placement Library

The placement library is managed by you and is the location where the Placement Assistant's input data files and results are created and stored. The name of the placement library is specified through the preference `lpPlacementLibrary`.

If the placement library does not exist, it is created automatically by the [Auto Place Command](#) during input data preparation phase (see [Create New Placement Library Dialog Box](#)).

The technology database `oaTechDB` of the target design library is attached to and used as the placement library's `oaTechDB`.

Note:

For backwards compatibility with older placement technology, if *santana.tech* is given, the placement library is created using this *santana.tech* file instead of `oaTechDB`.

Upon creation, the placement library is added to the currently open *lib.defs* file using a `DEFINE` statement.

A cell in the placement library is called a Placed Cell. In order to ensure uniqueness, the name of a Placed Cell is of the form *TargetCellName*`lpPlacedCellNameSeparator`*TargetLibName*.

Note:

For backward compatibility reasons, when the `lpPlacedCellNameSeparator` preference is set to an empty string, the Placed Cell name is set to the Target Cell name. The danger in using this setting is that two cells having the same name but located in different libraries also have the same Placed Cell name and, as a result, the Placement Assistant overwrites one with the other.

For a given top cell layout view being placed, the placement library contains at the end of placement:

- Input files used by the Placement Assistant
- Layout design views representing the placement results of the top cell
- Layout design views representing the placement results of all the cells having a soft macro layout view involved in the top cell layout hierarchy

Place Pins Command

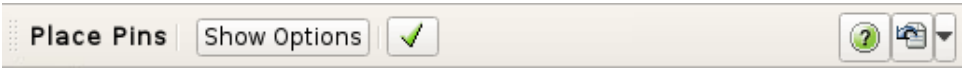
Choose **Placement > Place Pins** to activate the **Place Pins** command that places all pins on design.

The **Place Pins** command is described in more detail in the following sections:

- [Place Pins Toolbar](#)
- [Using the Place Pins Command](#)

Place Pins Toolbar

The interactive [Place Pins Command](#) invokes the following toolbar:



[Table 25](#) describes the options provided by the **Place Pins** toolbar.

Table 25 *Place Pins Toolbar Options*

Option	Description
Show Options	Invokes the Placement Options dialog box when clicking the option. Tcl preference: <code>lpPlacePinShowOptions</code>
Apply	Invokes placement on the current logical selection set. Tcl preference: <code>leApplyButton</code>

Using the Place Pins Command

The [Place Pins Command](#) requires that the target layout design is opened in the edit mode, and that PrBoundary has been created.

This feature creates pins based on connectivity and routability, with cell placement, pre-route, and blockages considered.

When tracks are defined on the target design, the created pin should snap to tracks. Assigned pins are snapped to their closest track according to pin constraint settings on the [Placement Options](#) dialog box. You can disable track snapping by setting the value of the preference `lpPlacePinSnapToTrack` to false.

For designs without track definition, the following formula is used by placer as default tracks:

track pitch = minimum width + minimum spacing

The command works in the following way:

1. Choose **Placement > Place Pins** to display the [Place Pins Toolbar](#).
2. You can edit pin constraints on [Placement Options](#) dialog box by clicking the **Show Options** option.
3. Once satisfied, click **Apply**  to launch the placement job.

Optimize Pins Command

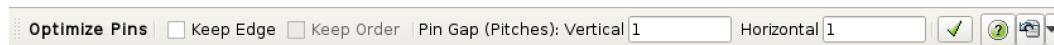
Choose **Placement > Optimize Pins** to activate the **Optimize Pins** command which optimizes the location of existing pins automatically. Pin optimization will be applied on selected pins, including top pins and instance pins.

The **Optimize Pins** command is described in more detail in the following sections:

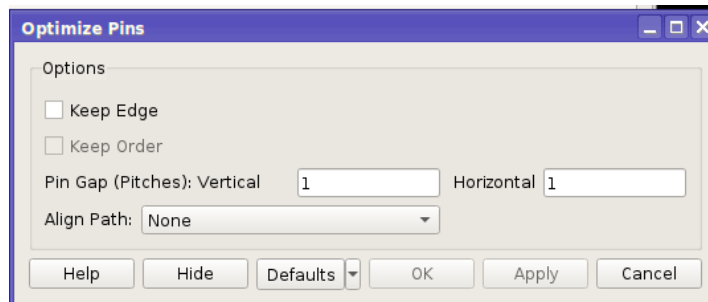
- [Optimize Pins](#)
- [Using the Optimize Pins Command](#)

Optimize Pins

The interactive [Optimize Pins Command](#) invokes the following toolbar:



Alternatively, choose **Placement > Optimize Pins** to open the **Optimize Pins** dialog box.



[Table 26](#) describes the options provided by the **Optimize Pins** toolbar.

Table 26 *Optimize Pins Options*

Option	Description
Keep Edge	Specifies whether to keep the edge assignment of the pins from original pin placement. When this option is turned off, the default pin layer, width, and length are set as defined in the Create Pins option of the Placement Options dialog box. When this option is turned on, the edge assignment and original layer are kept. Tcl preference: lpOptimizePinKeepEdge
Keep Order	Specifies whether to keep the order of original pin placement. When this option is turned off, the pins order can be permuted during pin optimization. When this option is turned on, the order of all pins on boundary are kept. Tcl preference: lpOptimizePinKeepOrder

Table 26 Optimize Pins Options (Continued)

Option	Description
Pin Gap	Specifies the pin gap on the vertical and horizontal of the PR boundary edge. Pin gap is the minimum number of pitches that need to be kept between every nearby pin on same layer. Tcl preference: lpOptimizePinVertPinGap, lpOptimizePinHorzPinGap
Align Path	Aligns the selected pin to the path in the top design that is in contact with the PR boundary. You can select one of the following options: • None: The pin is not aligned to any path. • Keep Original Layer: The pins are aligned to path in the original layer. • Use Path Layer: The pins are aligned to a path, and the pin layer changes to the path layer. Tcl preference: lpOptimizePinAlignPath. Note: If there are multiple paths available in the top design, then the conditions to select the path to align the pin are applied in the following order: • Edge: Left, Top, Right, Bottom • Layer number: Lower layer numbers are prioritized • Offset: Lower offset is prioritized
Apply	Invokes pin optimization on the current logical selection set. Tcl preference: leApplyButton

Using the Optimize Pins Command

The [Optimize Pins Command](#) requires that the pins are in the selection set. The following objects in the selection set are considered valid for this feature: top-level (0x) pins, second-level (1x) pins (instance pins), and any instance with valid master design.

Selecting an instance represents that all pins inside the instance are selected. All selected pins will be considered movable and their location may be changed to get a better pin placement.

The command works in the following way:

1. Choose **Placement > Optimize Pins** to display the [Optimize Pins](#).
2. Select (if desired) the instances which contain a set of pins in the layout design view to enable **Apply** .
3. Once satisfied, click **Apply**  to launch the optimization job.

Placement Options Dialog Box

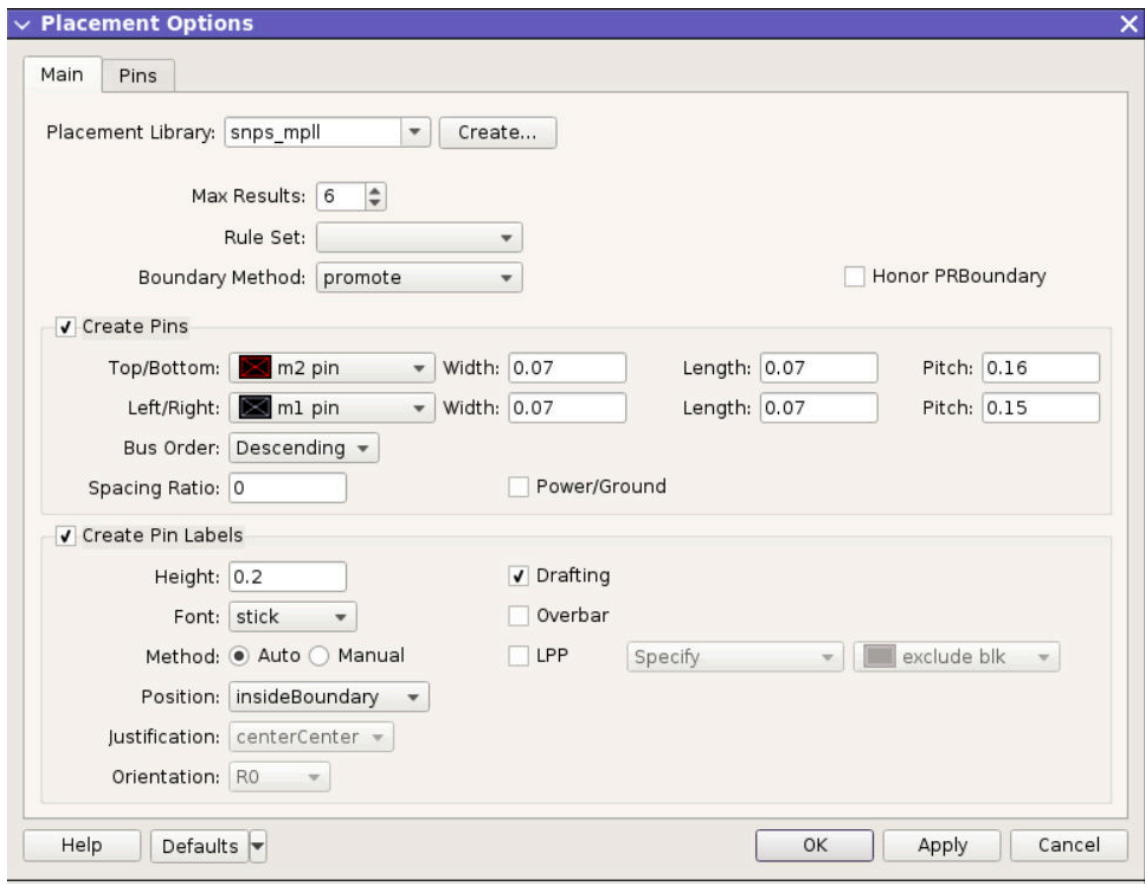
Choose **Placement > Options** to open the **Placement Options** dialog box, which is organized into the following tabs:

- [Main Tab](#)
- [Pins Tab](#)

Main Tab

Use the **Main** tab of the **Placement Options** dialog box (see [Figure 103](#)) to specify placement option preferences.

Figure 103 Placement Options Dialog Box — Main Tab



[Table 27](#) describes the options on the **Main** tab of the **Placement Options** dialog box.

Table 27 *Placement Options Dialog Box Options — Main Tab*

Option	Description
Placement Library	Specifies the placement library. Tcl preference: <code>lpPlacementLibrary</code>
Create	Displays the Create New Placement Library Dialog Box .
Max Results	Specifies the maximum number of placement results generated for each design and displayed in the Placement Gallery. The default is 6. Tcl preference: <code>lpMaxNumResults</code>
Rule Set	Specifies the name of the <code>oaConstraintGroup</code> to insert into constraint lookup. Tcl preference: <code>leUserDefinedConstraintGroup</code>
Boundary Method	Specifies how the <code>oaPRBoundary</code> is created: • promote (default): The size of the <code>oaPRBoundary</code> is created based on the <code>oaPRBoundary</code> inside instances. • bbox: The size of the <code>oaPRBoundary</code> is created based on the instance <code>bbox</code>, which should include all shapes in an instance. Tcl preference: <code>lpBoundaryMethod</code>
Honor PRBoundary	Specifies whether the <code>oaPRBoundary</code> in the current design should be honored by the Placement Assistant. This allows you to go back-and-forth between exploration and design refinement needs. Tcl preference: <code>lpHonorPRBoundary</code>
Create Pins	Specifies whether to create <code>oaTerm</code> , <code>oaPin</code> , and <code>oaPinFig</code> objects during placement. See Pins Tab . Tcl preference: <code>lxCreatePins</code>
Top/Bottom (LPP)	Specifies the LPP of pins shapes placed on the top or bottom side. The LPP selector contains valid/visible routing LPPs. The set of LPPs is restricted to LPPs where the layer is a visible routing layer, and the purpose is listed in <code>lePinShapePurposeList</code> . If <code>lxTopBottomPinLPP</code> is empty or invalid, then the value is the first vertical metal LPP in the LPP selector. Tcl preference: <code>lxTopBottomPinLPP</code>
Width (Top/Bottom)	Specifies the horizontal dimension in user units of the bounding box of a pin shape placed on the top or bottom side. This value is updated based on the value of <code>lxTopBottomPinLPP</code> whenever this changes and set to the minimum width rule value of the selected LPP. If no minimum width rule is defined for the selected LPP, the default is 0.35. Tcl preference: <code>lxTopBottomPinWidth</code>

Table 27 Placement Options Dialog Box Options — Main Tab (Continued)

Option	Description
Length (Top/Bottom)	Specifies the vertical dimension in user units of the bounding box of a pin shape placed on the top or bottom side. This value is updated based on the value of <code>lxTopBottomPinLPP</code> whenever this changes and set to the minimum width rule value of the selected LPP. If no minimum width rule is defined for the selected LPP, the default is 0.35. Tcl preference: <code>lxTopBottomPinHeight</code>
Pitch (Top/Bottom)	Specifies the spacing in user units between the centers of two consecutive pin shapes having the same LPP which are placed on the top or bottom side. This value is updated based on the value of <code>lxTopBottomPinLPP</code> and <code>lxTopBottomPinWidth</code> whenever they change and set to the Width value plus the minimum spacing rule value of the selected LPP. If no minimum spacing rule is defined for the selected LPP, the default is set to twice the value of <code>lxTopBottomPinWidth</code>. By default, neighboring pins on the same LPP are spaced apart at least by the minimum spacing rule for their common LPP. For optimized pins, Spacing Ratio is also used to determine the final pin positions. Tcl preference: <code>lxTopBottomPinPitch</code>
Left/Right (LPP)	Specifies the LPP of pin shapes placed on the left or right side. The LPP selector contains valid/visible routing LPPs. The set of LPPs is restricted to LPPs where the layer is a visible routing layer, and the purpose is listed in <code>lePinShapePurposeList</code>. If <code>lxLeftRightPinLPP</code> is empty or invalid, then the value is the first horizontal metal LPP in the LPP selector. Tcl preference: <code>lxLeftRightPinLPP</code>
Width (Left/Right)	Specifies the vertical dimension in user units of the bounding box of a pin shape placed on the left or right side. This value is updated based on the value of <code>lxLeftRightPinLPP</code> whenever this changes and set to the minimum width rule value of the selected LPP. If no minimum width rule is defined for the selected LPP, the default is 0.35. Tcl preference: <code>lxLeftRightPinWidth</code>
Length (Left/Right)	Specifies the horizontal dimension in user units of the bounding box of a pin shape placed on the left or right side. This value is updated based on the value of <code>lxLeftRightPinLPP</code> whenever this changes and set to the minimum width rule value of the selected LPP. If no minimum width rule is defined for the selected LPP, the default is 0.35. Tcl preference: <code>lxLeftRightPinHeight</code>

Table 27 Placement Options Dialog Box Options — Main Tab (Continued)

Option	Description
Pitch (Left/Right)	Specifies the spacing in user units between the centers of two consecutive pin shapes having the same LPP which are placed on the left or right side. This value is updated based on the value of <code>lxLeftRightPinLPP</code> and <code>lxLeftRightPinWidth</code> whenever they change and set to the Width value plus the minimum spacing rule value of the selected LPP. If no minimum spacing rule is defined for the selected LPP, the default is set to twice the value of <code>lxLeftRightPinWidth</code>. By default, neighboring pins on the same LPP are spaced apart at least by the minimum spacing rule for their common LPP. For optimized pins, Spacing Ratio is also used to determine the final pin positions. Tcl preference: <code>lxLeftRightPinPitch</code>
Bus Order	Specifies the placement order of bus pins: • Ascending • Descending (default) Tcl preference: <code>lxBusOrder</code>
Spacing Ratio	Controls spreading of the pins on the edges. The value must be between 0.0 and 1.0. A value of 1.0 means the pin shapes are placed as far apart from each other as possible, while a value of 0.0 means the placer does not make any effort to place them apart. Tcl preference: <code>lpSpacerPinRatio</code>
Power/Ground	Specifies whether to place power and ground pins. Tcl preference: <code>lpPlacePGPins</code>
Create Pin Labels	Specifies whether to create pin labels (oaAttrDisplay objects) for pin shapes (oaPinFig objects) to display the oaTerm name. Tcl preference: <code>lePinLabel</code>
Height (Pin Labels)	Specifies the height in user units of pin labels. The default is 0.2. Tcl preference: <code>leTextHeight</code>
Font	Specifies the pin label text font: • euroStyle • gothic • math • roman • script • stick (default) • fixed • swedish • milSpec Tcl preference: <code>leTextFont</code>

Table 27 Placement Options Dialog Box Options — Main Tab (Continued)

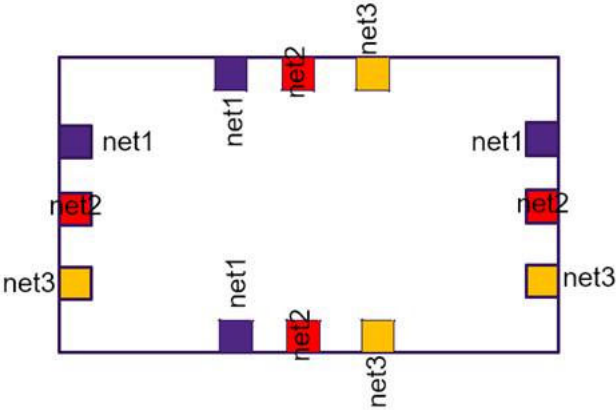
Option	Description
Mode	Specifies how the pin label text is justified and oriented: • Auto (default): Position is enabled; Justification and Orientation are disabled. • Manual: Justification and Orientation are enabled; Position is disabled. Tcl preference: <code>lePinLabelMode</code>
Position	If Mode is set to Auto, specifies that the pin label text is justified and oriented as follows: • Inside Boundary (default), as represented by 'net1' • On Boundary, as represented by 'net2' • Outside Boundary, as represented by 'net3'  The diagram shows a rectangular boundary. Inside the boundary, there are four purple squares, each labeled 'net1'. On the boundary, there are four red squares, each labeled 'net2'. Outside the boundary, there are four yellow squares, each labeled 'net3'. Tcl preference: <code>lePinLabelPosition</code>
Justification	If Mode is set to Manual, specifies the pin label text justification: • upperLeft • centerLeft • lowerLeft • upperCenter • centerCenter (default) • lowerCenter • upperRight • centerRight • lowerRight Tcl preference: <code>leTextJust</code>

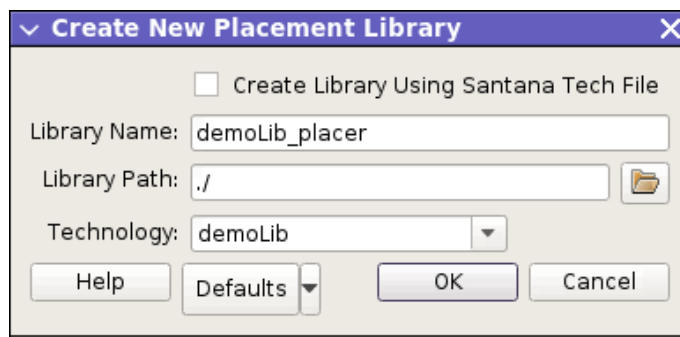
Table 27 Placement Options Dialog Box Options — Main Tab (Continued)

Option	Description
Orientation	If Mode is set to Manual, specifies the pin label text orientation: • R0 (default) • R90 • R180 • R270 • MX • MY • MXR90 • MYR90 Tcl preference: <code>leCreateOrientation</code>
Drafting	Specifies whether the drafting attribute should be used for the pin label text. Tcl preference: <code>leTextDrafting</code>
Overbar	Specifies whether the overbar attribute should be used for the pin label text. Tcl preference: <code>leTextOverbar</code>
LPP	When enabled, creates the pin label based on one of these options: • Specify: (default) The specified layer purpose pair. • Technology Library: The associated technology library. • Map: The file that defines the mapping of layer purpose pairs for shapes and text labels. See Defining the LPP Mapping File. When disabled, creates the label on the same layer as the pin shape, using the purpose defined by the <code>lePinPurposeList</code> option. The default purpose is <code>text</code>. If no valid purposes in <code>lePinPurposeList</code>, use the shape LPP for pin labels. Tcl preference: <code>leSpecifyPinLabelLPP</code>

Create New Placement Library Dialog Box

Select **Create** in the **Placement Options** dialog box to open the **Create New Placement Library** dialog box (see [Figure 104](#)), which allows you to create a new [Placement Library](#).

Figure 104 Create New Placement Library Dialog Box



[Table 28](#) describes the **Create New Placement Library** dialog box options.

Table 28 *Create New Placement Library Dialog Box Options*

Option	Description
Create Library Using Santana Tech File	Specifies whether to create the new placement library using the Santana Tech file or using an existing technology database.
Library Name	Specifies the placement library name. Tcl preference: <code>lpPlacementLibrary</code>
Library Path	Specifies the placement library path. Tcl preference: <code>dmAddLibDir</code>
Technology	Specifies the technology database of the placement library. - The field is hidden if you choose to create a new placement library using the Santana Tech file. - This field is displayed if you choose to create a new placement library using an existing technology, in which case it is set to the name of the current design library.

Pins Tab

If **Create Pins** is enabled, after all devices have been placed, pins are created on the edges of the layout and placed at optimal positions such that the total Manhattan distance from pins to instPins for all pins is minimized.

When track pattern information is available, pins are created on tracks. If track pattern information is not available, preferences for pin width, length, pitch, and LPP are used to create pins, as set on the [Main Tab](#).

Pins are created according to the following rules:

- If a terminal has an edge assignment, the Placement Assistant finds the optimal location on that edge.
- If a terminal has a position assignment, then this is honored in comparison to other position assignments such that the terminals are in the correct order as specified by the magnitude of the position value. This allows busses to be kept in the correct order.
- If the track or pin location has a blockage on the layer where the pin would be created, the Placement Assistant does not create a pin there.
- If a track or pin location intersects with an instPin on the LPP used for pin creation, the Placement Assistant creates the pin shape for that instPin on this pin location.

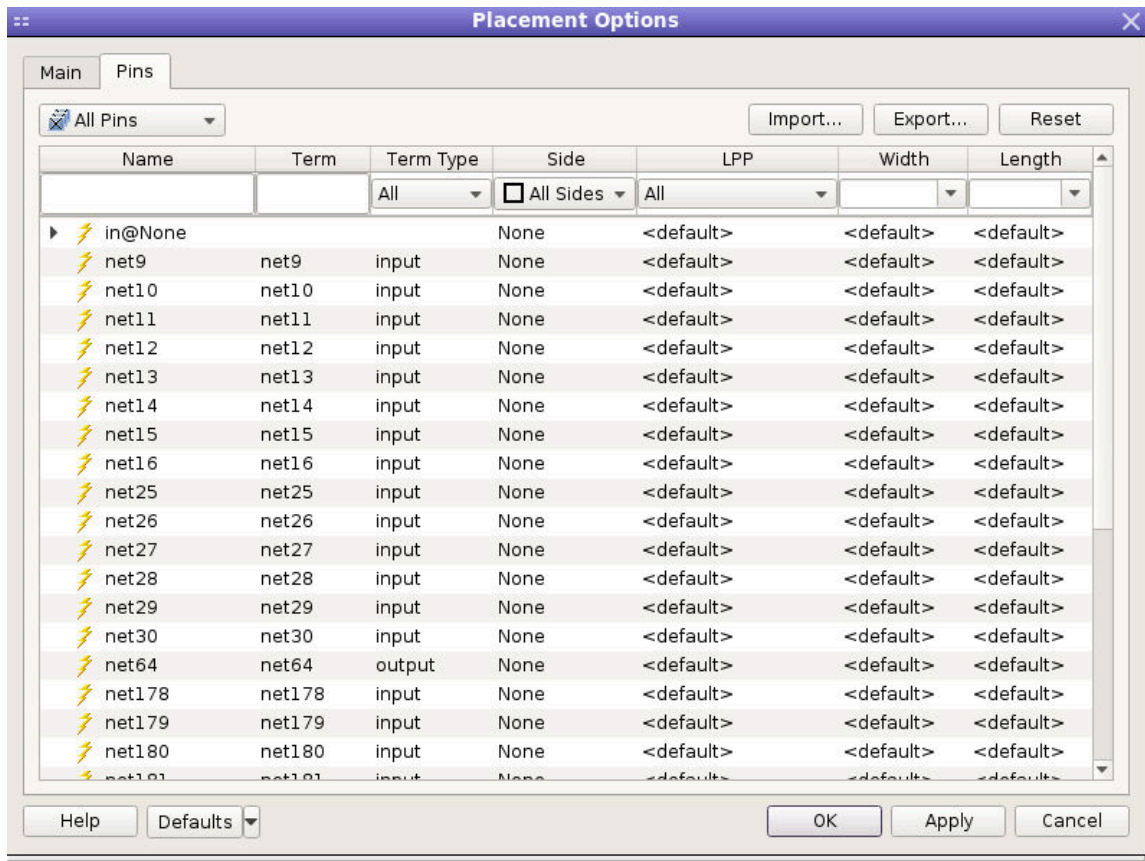
- For an orthogonal rectilinear prBoundary, if **Honor PRBoundary** is enabled on the [Main Tab](#), the Placement Assistant places pins along the edges based on the **Side** assignment.

Note:

This feature only works on top-level constraints.

Use the **Pins** tab of the **Placement Options** dialog box (see [Figure 105](#)) when [Modifying Pin Settings](#).

Figure 105 Placement Options Dialog Box — Pins Tab



[Table 29](#) describes the options on the **Pins** tab of the **Placement Options** dialog box.

Table 29 Placement Options Dialog Box Options — Pins Tab

Option	Description														
Pin Category  All Pins  All Pins  Auto Create  Assign  Existing Fixed	Filters by pin category the pins to be displayed in the Pins tab: • All Pins (default) displays all pins; uses only the columns containing the pin attributes common to all pin categories: Name, Term, Term Type, Side, LPP, Width, Length • Auto Create displays only those pins that are created and placed automatically; uses the following columns: Name, Term, Term Type, Side, LPP, Width, Length, Avoid Group • Assign displays only those pins that you explicitly assign to this category; uses the following columns: Name, Term, Term Type, Side, LPP, Width, Length, Offset, Pitch, Anchor • Existing Fixed displays only those pins existing in the current editing layout that have their Placement Status set to <i>fixed</i> or <i>locked</i>; uses the following columns: Name, Term, Term Type, Side, LPP, Width, Length, Origin The pin category determines which pins are displayed and which pin settings you can modify (see Modifying Pin Settings).														
Import	Displays the Import Pins dialog box which allows you to specify the CSV text file from which to import pin information. The first line of the file is expected to have the format created by the Export functionality. Pins in the Existing Fixed category are not imported. They are determined and collected from the current edit layout. The Term Type and Origin columns are not imported. They contain data collected either from the current edit layout or the schematic associated with it.														
Export	Displays the Export Pins dialog box which allows you to specify the CSV text file to which to export pin information. The first line in the file must have the following format: Name, Group, Side, LPP, Width, Length, AvoidGroup, Offset, Pitch, Anchor, Categ Pins in the Existing Fixed category are not exported. They are determined and collected from the current edit layout. The Term Type and Origin columns are not exported. They contain data collected either from the current edit layout or the schematic associated with it.														
Reset	Clears all pin overrides from the Pins tab and restores the initial, default pin attributes and order. The Assign pin category becomes empty. The pins and pin groups are sorted alphabetically in ascending order for each pin side, except bus bit pins which are ordered according to the <code>lxBusOrder</code> preference value inside their groups.														
Filter	Show or hide the pins which match or unmatch the filter values for all the columns. Multiple filters can be used at the same time. <table><tr><th>Name</th><th>Term</th><th>Term Type</th><th>Side</th><th>LPP</th><th>Width</th><th>Length</th></tr><tr><td></td><td></td><td>All</td><td>☐ All Sides</td><td>All</td><td></td><td></td></tr></table> See Setting General Options in the <i>Custom Compiler Environment User Guide</i> to specify Inputs > Filters for the function of text filters settings.	Name	Term	Term Type	Side	LPP	Width	Length			All	☐ All Sides	All		
Name	Term	Term Type	Side	LPP	Width	Length									
		All	☐ All Sides	All											

Modifying Pin Settings

As noted in [Table 29](#), the following pin categories determine not only which pins are displayed, but which pin settings you can modify:

- [All Pins](#)
- [Auto Create](#)
- [Assign](#)
- [Existing Fixed](#)

All Pins

For the **All Pins** category, you can employ drag-and-drop operations to change the position of pins or pin groups in the **Pins** tab. You can also override the pin group Name, pin Side, LPP, Width, and Length by editing the values in the corresponding columns.

The **Name** column is editable only for groups of pins. The group name must be unique among all pin and group names. You can edit only one group name at a time, even if multiple pins or pin groups are selected, and the new group name is set only to the group name currently being edited.

Groups of pins display data only on the line displaying the name of the group. The values in each column are common for all pins in the group and the child items are not editable. For more information on grouping and ungrouping pins, see the [Pins Tab Menu](#).

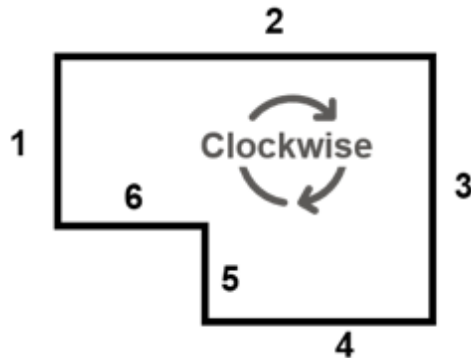
The **Term** column displays the terminal to which the pin is connected while the **Term Type** column displays the terminal type. Both columns are not editable.

The **Side** column displays the side of the boundary on which the pin is placed. The initial value is controlled by preference `lpInitPinSideFromSch`:

- `false` – (default) side value is initialized as "None" for all pins.
- `true` – side value is determined by pin location in the schematic.

The **Side** column shows different content according to number of boundary edges.

- Rectangular boundary (for example, the number of edges equals to 4) – the value can be either None, Top, Bottom, Left, or Right.
- Rectilinear boundary (for example, the number of edges greater than 4) – the value can be either “None” or an index of edge. The edges are indexed in clockwise order, starting from the left-bottom-most one. An example of how the edges are indexed:



You can edit the **Side**, **LPP**, **Width**, and **Length** values for a single row or multiple rows currently selected. When you select multiple rows, modifying a value in one of the rows modifies the values in the same column in all rows selected as soon as you finish editing the value currently being modified.

Auto Create

You can set pins to the **Auto Create** category using the [Pins Tab Menu](#).

As with the [All Pins](#) category, with the **Auto Create** category, you can employ drag-and-drop operations to change the position of pins or pin groups in the **Pins** tab. You can also override the pin group Name, pin Side, LPP, Width, and Length by editing the values in the corresponding columns. The pin Term and Term Type are not editable.

In addition, you can also edit the **Avoid Group** column which allows you to specify whether the Placement Assistant should avoid placing the pin (or the pins in a group) among the pins in any other group. When set for a group, it also means that no other pins are allowed to be placed inside the group.

Assign

You can set pins to the **Assign** category using the [Pins Tab Menu](#).

As with the [All Pins](#) category, with the **Assign** category, you can employ drag-and-drop operations to change the position of pins or pin groups in the **Pins** tab. You can also override the pin group Name, pin Side, LPP, Width, and Length by editing the values in the corresponding columns. The pin Term and Term Type are not editable.

In addition, you can also edit the following columns which allow you to assign pins to absolute or relative positions on the specified pin side:

- **Offset** specifies the spacing value in user units to be employed between the anchor and the edge of the pin.
- **Pitch** is editable only for groups of pins and specifies the spacing in user units to be employed between the centers of two consecutive pins in the group.
- **Anchor** specifies the pin anchor and is set to one of the following: <side low>, <side high>, <side center>, <previous pin>, and the names of all the pins with the same pin side that are placed before it. Given how valid anchor values are defined, the best way to edit them is to also filter the pins by their side to better see which pins are placed before a given pin in the table. If there are duplicate pins with same name, only one is listed and indicates the duplicate pin that is closest to current pin.

If the anchor value of a pin or group of pins is invalid, then the **Name** and **Anchor** fields are marked as invalid using the red foreground color. If the invalid anchor value of a pin or group of pins is not corrected, then the pins or the groups of pins are ignored (not created) and an error message is displayed.

The side low of all pin sides is the bottom-left corner of the bounding box of the area being placed. Therefore, the <side low> value for a pin or group of pins indicates the left-most point in case of Top and Bottom sides, such that pins are placed horizontally from left to right on the side/edge, and the bottom-most point in case of Left and Right sides, such that pins are placed vertically from bottom to top on the side/edge.

On contrary, the <side high> of all pin sides is the top-right corner of the bounding box of the area being placed. Therefore, when the value is <side high>, pins are placed from right to left on horizontal sides/edges and are placed from top to bottom on vertical sides/edges.

The <side center> value for a pin or group of pins indicates the pin or group of pins to be placed center-aligned on the side/edge they are assigned to. If the **Anchor** field of a pin/group of pins is <side center>, the pins are placed from side low to side high, and offset value of the first element is ignored.

The <previous pin> value for a pin or group of pins indicates the pin or group of pins placed immediately before it in the table and having the same pin side. If the <previous pin> is a group of pins, the anchor is the last pin in the group.

The default value is <previous pin> except for the first pin or group of pins of a given pin side in the table for which the only valid anchor value is <side low>.

An **Assign** pin is placed to the location according to settings in the [Placement Options](#) dialog box. Moreover, the pins are snapped to their closest track to ensure routability. You can disable track snapping by setting the preference `lpPlacePinSnapToTrack` to false.

Existing Fixed

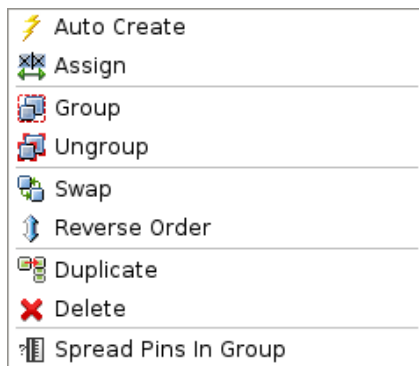
Pins in the **Existing Fixed** category are not editable because their attributes are collected from the oaPin objects found in the current edit layout design and their pin side is set to None.

The **Width** and **Length** columns display the horizontal and vertical dimensions in user units of the pin shape's bounding box, respectively, while the **Origin** column displays the pin shape bounding box's bottom-left point coordinates in user units.

Pins Tab Menu

Select one or more rows in the **Pins** tab then right-click to display the menu shown in [Figure 106](#).

Figure 106 Pins Tab Menu



Note:

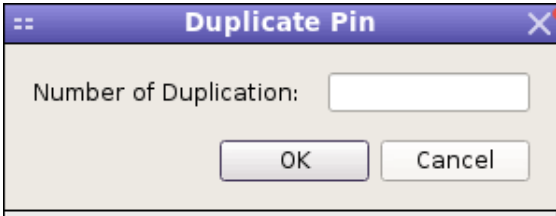
The **Pins** tab menu commands do not affect **Existing Fixed** pins.

[Table 30](#) describes the actions provided by the **Pins** tab menu.

Table 30 Pins Tab Menu Actions

Action	Description
Auto Create	Sets all pins and pin groups selected to the Auto Create pin category.
Assign	Sets all pins and pin groups selected to the Assign pin category.
Group	Groups all pins and groups of pins selected under a default pin group name of the form Group <i>n</i> . This action does not create multi-level groups. If a group is selected, all of the pins in the group are added to the new group. If multiple groups are included in the selection set, then all of the pins in the selected groups are added to the new group.
Ungroup	Ungroups all groups of pins and child pin items selected.

Table 30 Pins Tab Menu Actions (Continued)

Action	Description
Swap	Swaps the positions of two selected top-level pins or pin groups of the same pin category.
Reverse Order	Reverses the order of child pin items of the pin group selected.
Duplicate	Duplicates the selected pin by the number of duplication specified in the Duplicate Pin dialog box and places it under the same terminal. 
Delete	Deletes the selected duplicate pin.
Spread Pins in Group	Spreads pins evenly inside the selected Assign pin group and automatically calculates the pitch value. This action requires that the pin group already has a value set in the Offset column. If it does, specify a value larger than the group's offset in the Spread Pins in Group dialog box. 

Placement Results

In general, depending on how much is constrained, the Placement Assistant generates multiple placement solutions, in a Pareto set, for a given top layout design view. The maximum number of results for a given cell can be specified through the `lpMaxNumResults` preference.

Placement results are created by the executable in the [Placement Library](#) specified by the `lpPlacementLibrary` preference, so the layout design that was placed remains the same after the placement finishes. You must explicitly copy the placement result you prefer using the [Copy Result Command](#).

In every placement result (layout design view), each constraint has a corresponding `lePlacementGroup` object created.

For a given layout design to be placed, placement constraints must be valid. The Placement Assistant collects the constraints and performs specific validation, which is applicable to the placer.

For a detailed description of placement constraints supported, see [Placement Constraints](#).

Viewing Placement Results

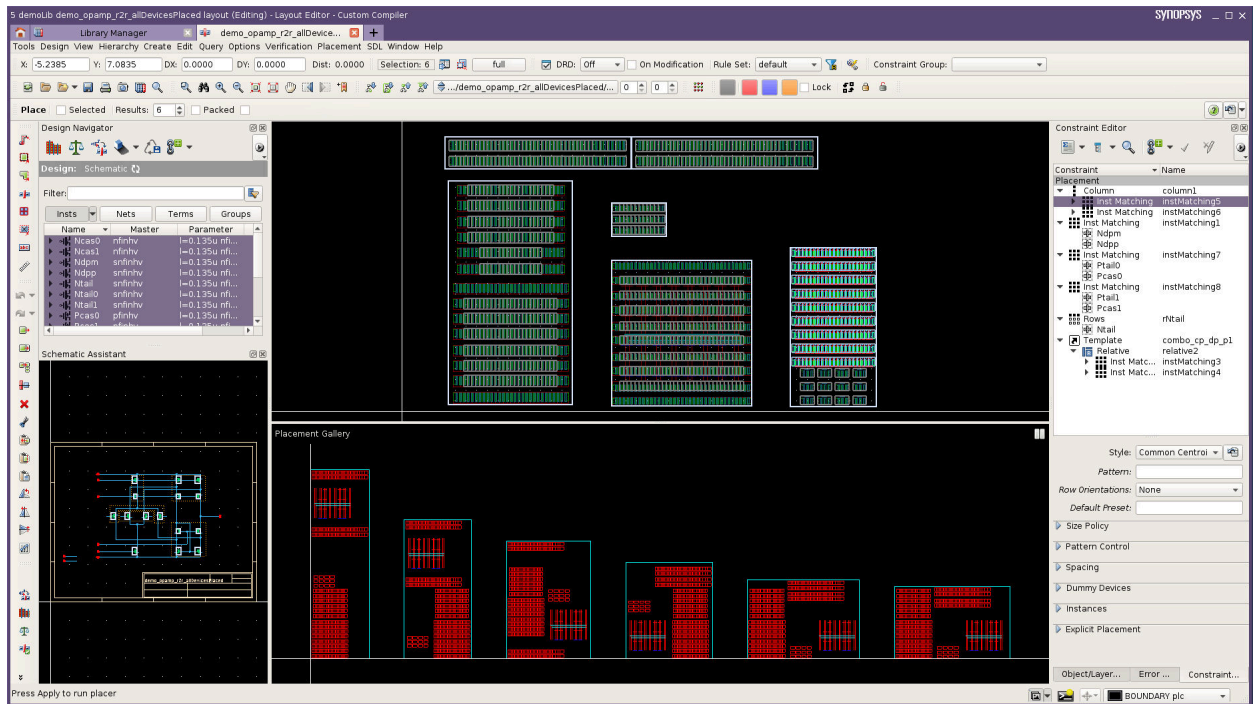
You can view placement results for the current cell (layout design view) by choosing **Placement > View Results** or by clicking **Show/Hide Gallery**  in the status bar. This enables the visualization of placement results, which is accomplished through a split-view of the current layout window (see [Figure 107](#)) displaying the design that was placed (by default, at the top of the split-view; it can be to the left if the vertical split-view is used), which allows you to do the following:

- Visualize placement results in a Placement Gallery (by default, at the bottom of the split-view; it can be to the right if the vertical split-view is used).
- Copy placement results from the placement library back to the target library.
- Cross-select and highlight between the Placement Gallery and the layout (and also the schematic during an SDL session); see [Selection and Highlighting of Placement Results](#).

Note:

If the preference `lpAutoOpenGallery` is set (by default), the Placement Gallery is displayed automatically when the placement job is complete.

Figure 107 Placement Gallery



Each placement result displayed in the Placement Gallery at the bottom of the split-view layout window is an instance of a layout design view for the current cell in the placement library. That is, each has an extra-level of hierarchy compared with the current cell that was placed. This top, extra-level of hierarchy is stripped off by the [Copy Result Command](#) when copying over the placement results from the placement library to the target library.

If a placement area has been defined in the layout design using the [Create Placement Area Command](#), then each placement result in the gallery is shown within the context of the placement area, for visualization purposes.

Occasionally, the Placement Assistant produces 'incomplete' results, for example, results that contain DRC errors in a sub-block. These incomplete results are displayed in the Placement Gallery alongside other results, but are tagged with an asterisk (*) for identification. Setting the preference `lpShowIncompleteResults` to `true` (default) can control whether to display incomplete results.

In some circumstances, you might want to label placement results, for example, to identify the template cell from which the constraints were derived for a particular result. When creating the Placement Gallery, the master design for each placement result is queried for the presence of a string property named `lpGalleryLabelString`. If this property is found, then a label is created from the property value, and placed next to the associated instance. By default, the `{text drawing}` layer/purpose pair is used for the label. The text height, orientation, and justification are determined based on the content of the Placement

Gallery. Any of these text attributes can be overridden by setting the following preferences: `lpGalleryLabelLPP`, `lpGalleryLabelHeight`, `lpGalleryLabelOrientation`, and `lpGalleryLabelJustification`.

Deselecting **Placement > View Results** or clicking **Show/Hide Gallery**  when the layout window is in split-view, closes the Placement Gallery and returns the canvas to its normal state which is to display the design that was placed.

You can also use the **Show/Hide Gallery**  menu (see [Figure 108](#)) to switch the gallery display between placement results in the Placement Gallery and realization previews in the Template Gallery (see [Previewing the Layout Realization](#)).

Figure 108 Switching Between Galleries



Selection and Highlighting of Placement Results

Active object cross-highlighting is supported between the Placement Gallery and the layout design.

- If the preference `lxCrossHighlighting` is `true`, then active object cross-highlighting is also supported between the Placement Gallery and the schematic design and Schematic Assistant.

Cross-selection is supported between the Placement Gallery and the layout design.

- If the preference `lxCrossSelectSchematic` is `true`, then cross-selection is also supported from the Placement Gallery to the schematic design, Schematic Assistant, and Design Navigator.
- If the preference `lxCrossSelectLayout` is `true`, then cross-selection is also supported from the schematic design, Schematic Assistant, and Design Navigator to the Placement Gallery.

Cross-selection has a special meaning in the Placement Gallery. If there are, for example, five placement results (that is, 0x instances) in the Placement Gallery, then one instance in the layout design corresponds to five instances in the Placement Gallery: one inside each placement result. Therefore, selection of one instance in the layout design cross-selects five instances in the Placement Gallery. However, there is no command that can operate on objects selected from multiple placement results. It does not make sense to add these objects to a selection set, so they are added to a highlight set. The packet style is chosen to have the same color as the default selection set, but to have a different line style: dashed versus solid. The intent is to convey the meaning that they are cross-

selected, but not truly selected. True selection in the Placement Gallery is only achieved by direct (that is, not cross) selection.

Object correspondence is determined by name. One object in the layout design typically corresponds to one object in the schematic design, and to one object in each placement result in the Placement Gallery. However, one object in the schematic design can correspond to multiple objects in the layout design, and multiple objects in each placement result in the Placement Gallery. For example, vectored instances in the schematic environment are scalarized in the layout environment. M-factored instances and multi-fingered instances are also fully expanded in the layout environment. In general, for non-vectored names, *name* in the schematic environment corresponds to *name* or *name.** in the layout environment.

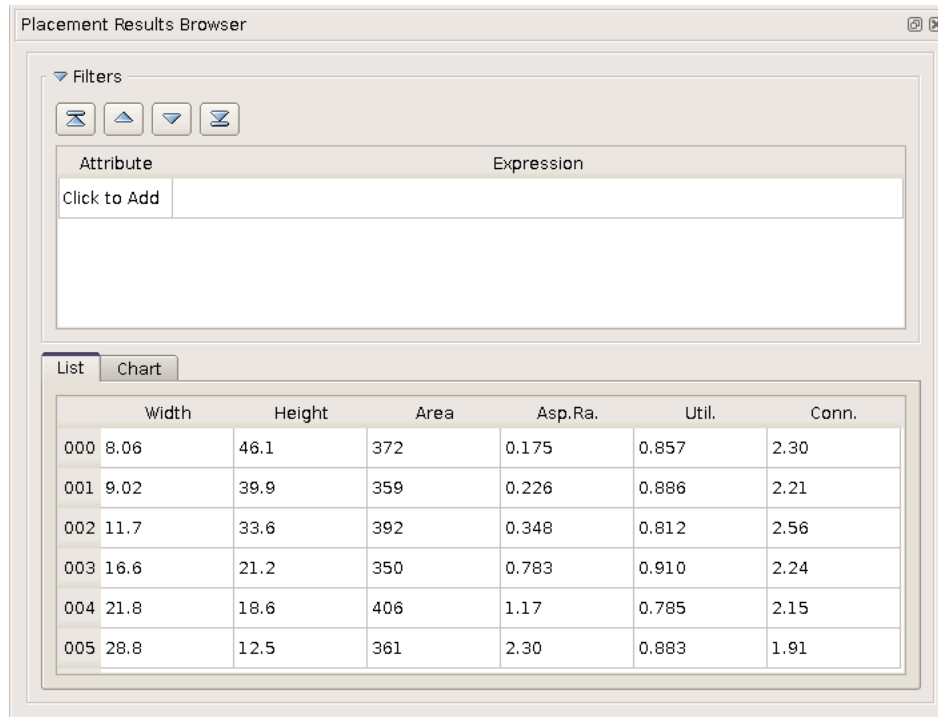
Examples:

- Schematic M1<2:0> corresponds to Layout M1(2), M1(1), M1(0)
- Schematic M2, m=2, nf=3 corresponds to Layout M2.m1.f1, M2.m1.f2, M2.m1.f3, M2.m2.f1, M2.m2.f2, M2.m2.f3
- Schematic M2, stack=3, nf=4 corresponds to Layout M2.STACK1, M2.STACK2, M2.STACK3, M2.STACK4

Placement Results Browser

Choose **Placement > View Placement Browser** to open the Placement Results Browser (see [Figure 109](#)), which helps you navigate through the multiple placement solutions generated by the Placement Assistant.

Figure 109 Placement Results Browser Assistant



The solutions are shown both in tabular ([List Tab](#)) and graph format ([Chart Tab](#)) and can be filtered ([Filters](#)) to remove solutions from the display that do not meet your criteria.

By default, the Placement Results Browser docks on the right side of windows. The orientation of the assistant is strictly vertical and it can be docked on the left or right side of a window.

By setting the preference `lpAutoOpenBrowser` to `true` (default is `false`), the Placement Results Browser can be set to open automatically along with the Placement Gallery through the **View Results** action described in [Viewing Placement Results](#).

The Placement Results Browser closes when the Placement Gallery is closed by deselecting **Placement > View Results**, but it remains open when the Placement Gallery is hidden by clicking **Show/Hide Gallery** .

The results in the Placement Results Browser are updated automatically when the results design is constructed, based on the context shown in the Placement Gallery.

The Placement Results Browser is made up of the following sections:

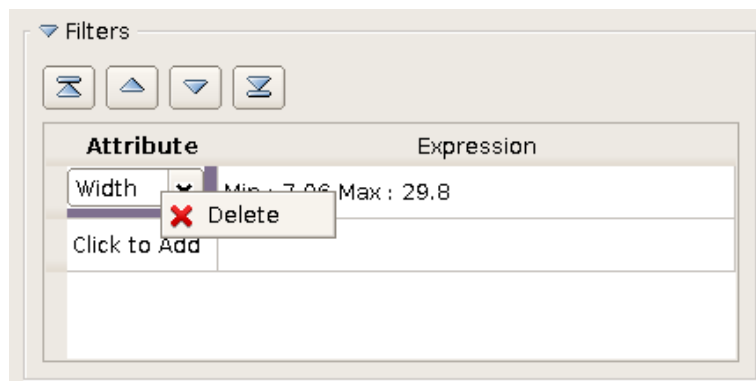
- [Filters](#)
- [List Tab](#)
- [Chart Tab](#)

Both the **List** and **Chart** tabs are populated by reading the data stored in the `synopsys_custom/gcpData/cell.cellinfo` file. This file is generated by the Placement Assistant.

Filters

Use the **Filters** section to refine the list of variants shown in the **List** and **Chart** tabs. As shown in [Figure 110](#), you can add multiple filters and each filter is applied sequentially from top to bottom in the list. In other words, the result from the first filter is input to the next filter.

Figure 110 *Filters*



You can add, modify, reorder, and delete filters as follows:

- To add a filter, click the **Click to Add** field in the **Attribute** column and select a filter from the list, as shown in [Figure 111](#).

Figure 111 Filters — Adding a Filter

Attribute	Expression
Width	Min : 7.06 Max : 29.8
Height	Min : 11.5 Max : 47.1
Height	
Area	
Asp.Ra.	
Util.	
Conn.	

- Specify or modify the expression to be used for the filter, which can be done in two ways:
 - *Range expression:* With **Target** unselected, specify minimum and maximum values for the attribute, as shown in [Figure 112](#).

Figure 112 Filters — Range Expression

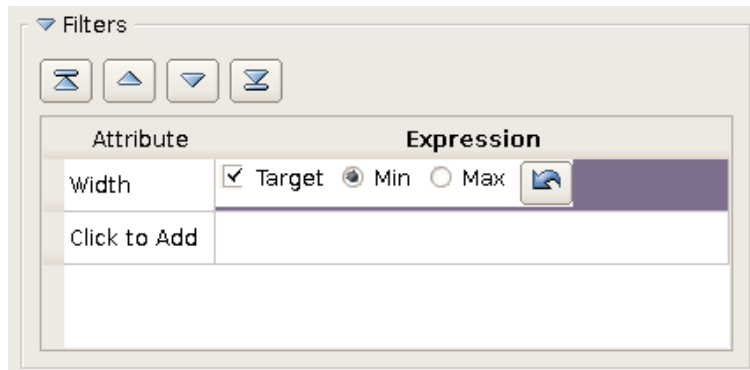
Filters

Attribute	Expression
Width	☐ Target Min: 7.06 Max: 29.8
Click to Add	

This filter returns the results satisfying the range of values for the attribute. Hence, there could be single or multiple or no results.

- *Min/Max expression:* With **Target** selected, select either the **Min** or **Max** option, as shown in [Figure 113](#).

Figure 113 Filters — Min/Max Expression

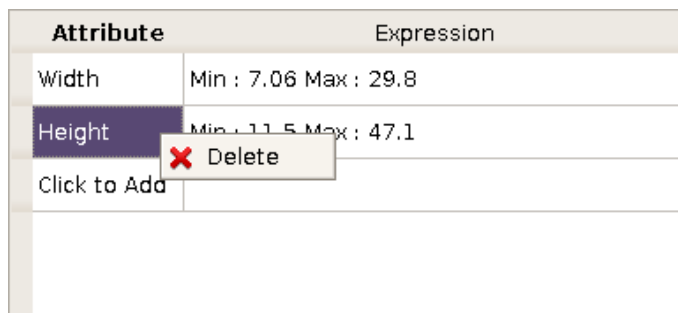


This filter always returns a single result. For the **Min** filter, the result is the one with the minimum value for the attribute. For the **Max** filter, the result is the one with the maximum value for the attribute. Typically, you would use this as the last filter to get a single result that meets all filter criteria.

Click **Undo** to undo the changes for the expression edited.

- You can reorder the filters by selecting the filter in the table and clicking **Up** , **Down** , **Top** , or **Bottom** .
- You can delete filters by selecting the filter in the table and right-clicking and choosing **Delete**, as shown in [Figure 114](#).

Figure 114 Filters — Delete Action



Note:

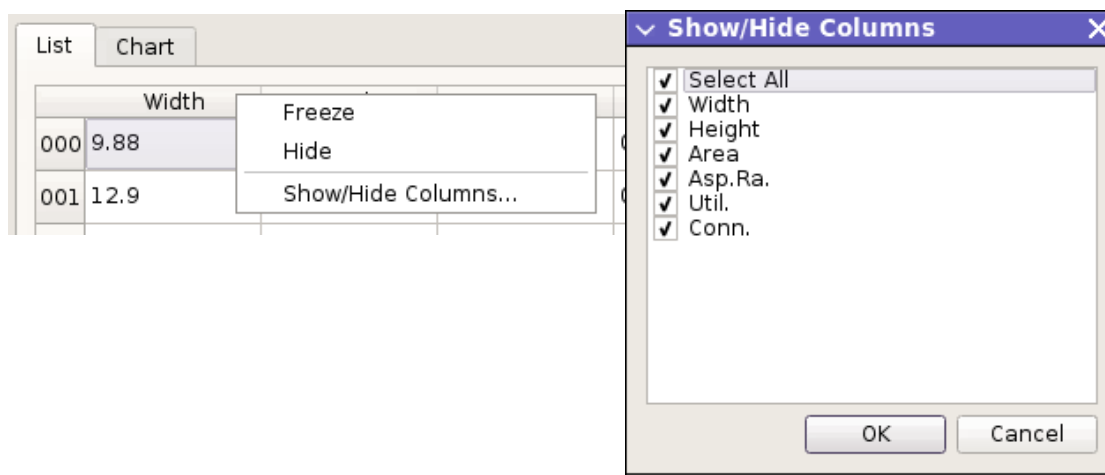
A list of default filters is added to the filter table when the assistant is created. This list is controlled using the `lpDefaultResultFilters` preference. This preference accepts a list of default filter attributes and expressions.

List Tab

The **List** tab shows the placement results in tabular format with column names: **Width**, **Height**, **Area**, Aspect Ratio (**Asp.Ra.**), Utilization Ratio (**Util.**), and Connectivity Metric (**Conn.**). You can rearrange the columns in the table by using a drag-and-drop operation on the column header.

As shown in [Figure 115](#), the column configuration can be modified by right-clicking the column header and choosing **Show/Hide Columns**.

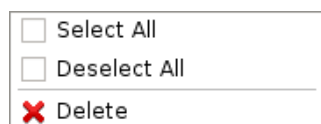
Figure 115 List Tab — Show/Hide Columns



The results table is read-only. You can select multiple rows in the table by using Shift key modifiers. You can deselect a row by using a Ctrl key modifier. All rows can be deselected by clicking the empty area in the table.

As shown in [Figure 116](#), you can right-click any row to display the **List** tab menu, which allows you to select all points by choosing **Select All** or to deselect all points by choosing **Deselect All**.

Figure 116 List Tab Menu



As shown in [Figure 117](#), the rows in the table can be sorted by clicking the column headers. This shows the arrowhead on the header to display the sorting column and ascending/descending status.

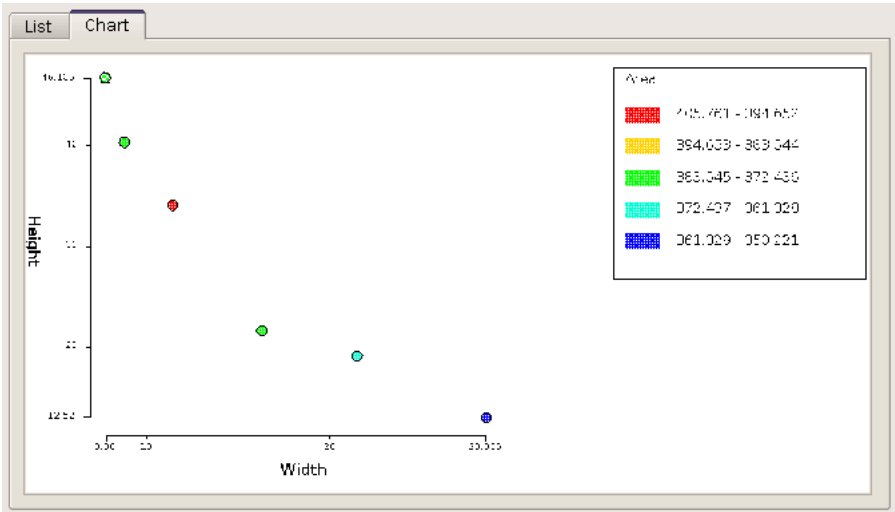
Figure 117 List Tab — Column Sorting

	Width ▾	Height	Area	Asp.Ra.	Util.	Conn.
000	8.06	46.1	372	0.175	0.857	2.30
001	9.02	39.9	359	0.226	0.886	2.21
002	11.7	33.6	392	0.348	0.812	2.56
003	16.6	21.2	350	0.783	0.910	2.24
004	21.8	18.6	406	1.17	0.785	2.15
005	28.8	12.5	361	2.30	0.883	1.91

Chart Tab

As shown in [Figure 118](#), the **Chart** tab displays a Pareto graph of the width versus height of the placement result generated by the Placement Assistant. The legend box shows the layout area of the results.

Figure 118 Chart Tab



You can click to select a point on the graph in **Chart** view. Multiple points can be selected by using Shift+Ctrl key modifiers or by clicking-and-dragging on the graph.

As shown in [Figure 119](#), you can right-click any point to display the **Chart** tab menu, which allows you to select all points by choosing **Select All** or to deselect all points by choosing **Deselect All**.

Figure 119 Chart Tab Menu



Copy Result Command

Choose **Placement > Copy Result** to activate the **Copy Result** command, which can copy a placement result from the placement library to the target library.

In the case of the interactive **Copy Result** command, the target layout design and all of its placement results are displayed in a split-view layout window. The command copies the entire or a part of a placement result from the Placement Gallery (by default, at the bottom of the window) to the current layout design view (by default, at the top of the window).

The **Copy Result** command has the following characteristics:

- Valid objects are the same as those for the **Copy** command.
- The **Copy Result** command supports only fully selected objects. Partially selected objects are ignored.
- The selection set is considered invalid and corresponding error messages are printed in the console when:
 - More than one 0x placement result instance is selected.
 - Both 0x and 1x objects are selected.
 - 1x objects selected come from more than one 0x placement result instance.

Copying multiple 1x objects from different 0x placement result instances cannot be done in a single command cycle; multiple command applications (transactions) are necessary.

- Clicking the layout design view commits the runtime objects to the canvas at the location specified by the mouse pointer.
- If a placement area has been defined using the [Create Placement Area Command](#), it is displayed during **Copy Result**, in both the edit design and the gallery, as a visual guide.

The **Copy Result** command is described in more detail in the following sections:

- [Copy Result Toolbar](#)
- [Copy Result Menu](#)
- [Using the Copy Result Command](#)

Copy Result Toolbar

The interactive [Copy Result Command](#) invokes the following toolbar:



[Table 31](#) describes the options provided by the **Copy Result** toolbar.

Table 31 *Copy Result Toolbar Options*

Option	Description
Replace Existing	Specifies whether the objects copied from the Placement Gallery may replace the objects already existing in the target layout. Tcl preference: <code>lpCopyResultReplaceExistingObjects</code>
Firm Constraints	Specifies whether constraints such as 'vpattern' should be 'firmed' when results are copied back from the Placement Gallery to the target layout. Tcl preference: <code>lpFirmConstraintsUponCopy</code>
 Rotate	Rotates the copied results 90 degrees counterclockwise.
 Flip Horizontal	Mirrors the copied results across the Y-axis.
 Flip Vertical	Mirrors the copied results across the X-axis.

Copy Result Menu

Right-click the canvas when using the **Copy Result** command to display the menu shown in [Figure 120](#).

Figure 120 Copy Result Menu



[Table 32](#) describes the actions provided by the **Copy Result** menu.

Table 32 Copy Result Menu Actions

Action	Description
Commit	Commits the runtime objects to the canvas at the location specified by the mouse pointer.
Commit at Origin	Commits the runtime objects to the canvas at the origin (0,0).
Commit at Offset	Commits runtime objects from the Placement Gallery to the canvas using the same offset, from either the lower left corner of the bounding box of a specified placement area or the origin of the existing prBoundary to the canvas origin (0,0). For example, if you use the Create Placement Area Command and then the Auto Place Command , the placed objects are offset to fit into the defined placement area and this command commits them to the canvas at the correct coordinates.
Rotate	Rotates the copied results 90 degrees counterclockwise.
Flip Horizontal	Mirrors the copied results across the Y-axis.
Flip Vertical	Mirrors the copied results across the X-axis.

Using the Copy Result Command

The interactive [Copy Result Command](#) works in both preselection and postselection modes and supports Infix mode.

In preselection and postselection modes, the command works in the following way:

1. Choose **Placement > Copy Result** or use the shortcut key C to display the **Copy Result Toolbar**.
2. Select objects to copy until the selection set is valid.
3. In postselection mode, you can decide whether to replace the target design's existing objects by selecting or deselecting the **Replace Existing** option on the toolbar without having any objects selected.
4. If the selection set is valid and contains only one 0x instance, or contains 1x objects from the same 0x placement result instance, runtime objects are created on the cursor and you must commit the runtime objects by specifying a destination point to complete the copy.
5. While the runtime objects are active, you can perform the following operations:
 - Select or deselect objects.
 - Decide whether to replace the target design's existing objects by selecting or deselecting the **Replace Existing** option on the toolbar.
 - Rotate or flip the runtime objects using the controls on the toolbar or the command's menu before committing them.
6. After the objects are copied or committed, the command functions as follows:
 - In preselection mode: Exits and the objects remain selected.
 - In postselection mode: Clears the selection set and enters a new cycle by prompting you to select objects to copy.

In Infix mode, when the command is activated using a shortcut key on the canvas, the use models change as follows:

- If the selection set is valid, and contains 1x objects from the same 0x placement result instance, then the command copies the objects to the target design at the current cursor location (assuming it is in the target design canvas). Otherwise, the use model remains the same.

[Table 33](#) describes **Copy Result** command prompts.

Table 33 Copy Result Command Prompts

State	Command Prompt
No valid objects in selection set	Select a single 0x object or multiple 1x objects to copy.
Valid objects selected	Specify destination point to complete copy.

Generating Constraints

You can create an initial set of constraints for the design when using **Generate Constraints** by:

- Applying templates from a template library
- Grouping standard cells according to the standard cell library from which they come
- Grouping other devices according to cell definition

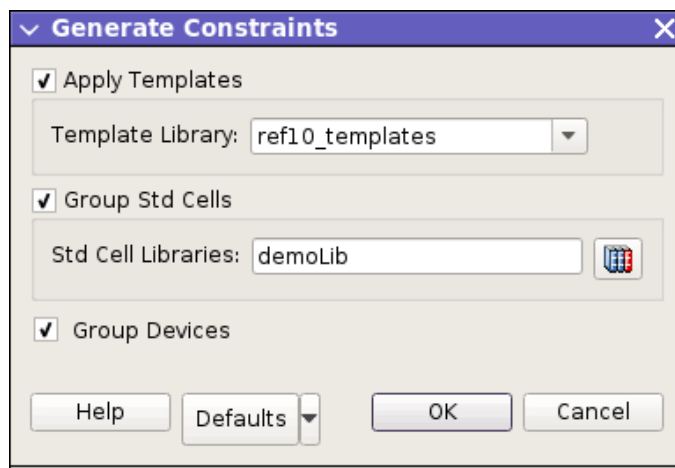
This chapter introduces the following sections:

- [Generate Constraints Dialog Box](#)
- [Create Constraint Area Command](#)

Generate Constraints Dialog Box

Choose **Placement > Generate Constraints** to open the **Generate Constraints** dialog box (see [Figure 121](#)) which allows you to create an initial set of constraints.

Figure 121 *Generate Constraints Dialog Box*



[Table 34](#) describes the **Generate Constraints** dialog box options.

Table 34 *Generate Constraints Dialog Box Options*

Option	Description
Apply Templates	Specifies whether to apply a template library for template matching. See Apply Templates Option . Tcl preference: <code>lpRunApplyTemplates</code>

Table 34 Generate Constraints Dialog Box Options (Continued)

Option	Description
Template Library	Specifies the template library to be used for template matching. Tcl preference: <code>lpTemplateLibrary</code>
Group Std Cells	Specifies whether to use a set of standard cell libraries to identify standard cells. Tcl preference: <code>lpRunGroupStdCells</code>
Std Cell Libraries	Specifies the set of standard cell libraries to be used to identify standard cells. Cells from the same library are grouped together. Tcl preference: <code>lpStandardCellLibList</code>
Group Devices	Specifies whether to group devices according to cell definition. See Group Devices Option Tcl preference: <code>lpRunGroupDevices</code>

Note:

You can generate constraints in a read-only design.

Apply Templates Option

The **Apply Templates** option implements automatic template matching and initial constraint generation for a layout design view. Templates are obtained from the specified **Template Library**, then applied individually against all unconstrained devices in the logical netlist. For this reason, the option requires that SDL has been initialized.

The order in which templates are applied is determined first by the number of devices in each template, and then alphabetically by template name. As each template match is processed, and constraints are generated in the layout design view, the number of unconstrained devices diminishes.

Group Devices Option

The **Group Devices** option generates constraint and uses a built-in template to match the current design.

Examples of the built-in structures:

- Cross-Coupled
- Current Mirror
 - Two devices: `cm_p`, `cm_n`, `cml_p`, `cml_n`
 - Three devices: `wcm_p`, `wcm_n`
 - Four devices: `cm_p`, `cm_n`, `cml_p`, `cml_n`

- Cascode Pair (two devices): cc_p, cc_n
- Differential Pair (two devices): diffpair_p, diffpair_n
- Level Shifter (two devices): ls_p, ls_n
- Voltage Regulator (two devices): vr_p, vr_n, vr_2p, vr_2n

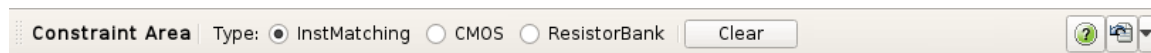
For the built-in structures with two devices, the tool creates an instMatching constraint with device group type. For the built-in structures with three devices, the tool creates an instMatching constraint with balanced pattern type. For the built-in structures with four devices, the tool creates two instMatching constraint and a column constraint at the top level.

For other instances that are not matched by the built-in template and the bboxes of multiple basenames are overlapped to each other, the tool only creates one placement constraint as below:

- Create instMatching constraints for those instances with MOS type.
- Create resistorBank constraints for those instances with resistor type.

Create Constraint Area Command

You can create constraint areas to give the constraint generator to create the specific constraint in the created areas using the **Placement > Create Constraint Area** command.

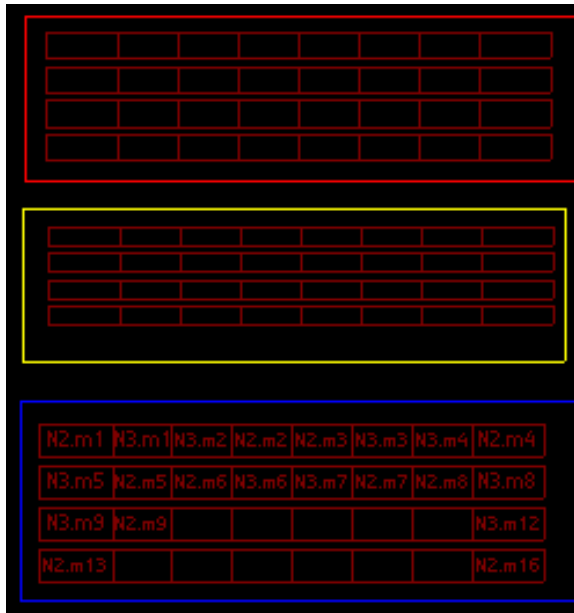


To create constraint areas:

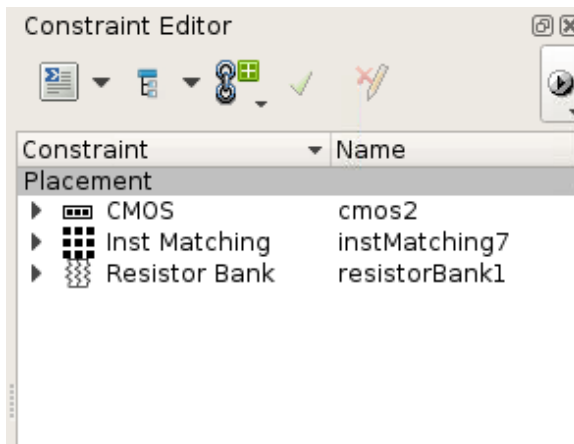
1. Choose one of the types from the **Create Constraint Area** command to group the instances to certain constraint type area. The area is created on deOverlay.

When creating the constraint area, the shadow mode is provided to highlight the areas. Different types of areas are highlighted with different color.

- The areas created by the **InstMatching** type are highlighted to red.
- The areas created by the **CMOS** type are highlighted to yellow.
- The areas created by the **ResistorBank** type are highlighted to blue.



2. (Optional) Click **Esc** to hide the constraint areas. The areas are restored and shown when this command is enabled.
3. Choose **Placement > Generate Constraint** to group the instances to the placement constraint based on the created areas.



4. (Optional) You can click **Clear** to remove all the constraint areas if the created areas are not expected.

[Table 35](#) describes the **Create Constraint Area** command options.

Table 35 Constraint Area Options

Option	Description
Type	Groups the instances with certain constraint type area by selecting one of the following types: InstMatching , CMOS , and ResistorBank . Tcl preference: lpConstraintAreaTypePerf
Clear	Specifies to remove all the constraint areas.

Extract Topology Constraints Command

Choose **Placement > Extract Topology Constraints** to activate the **Extract Topology Constraints** command which is used to extract the layout topology from an existing layout, and then create placement constraints. This can help ensure that later runs of the [Auto Place Command](#) recreate only similar results.

In the case of the interactive **Extract Topology Constraints** command, you can select any subset of instances in the layout window, from which layout topology is extracted. In the case of the non-interactive **Extract Topology Constraints** command, the entire design is used.

The command reads in the input layout based on the logical selection and creates the placement constraints listed in [Table 36](#) to represent the (partial) layout topology based on the **Mode** setting in the [Extract Topology Toolbar](#).

Table 36 Extract Topology Modes and Corresponding Placement Constraints

Placement Constraint	Extract Topology Mode						
	Relatives Only	Relatives	Clusters	Rows and Columns	Instance Matching	Resistor Bank	CMOS
Relative	X	X					
Cluster			X				
Rows		X	X	X			
Column		X	X	X			
Inst Matching					X		
Resistor Bank						X	
CMOS							X

Table 36 *Extract Topology Modes and Corresponding Placement Constraints (Continued)*

Placement Constraint	Extract Topology Mode						
	Relatives Only	Relatives	Clusters	Rows and Columns	Instance Matching	Resistor Bank	CMOS
Halo	X	X	X	X			
Guard Ring	X				X	X	X
Symmetric				X			

When the **Mode** setting in the [Extract Topology Toolbar](#) is set to:

- **Relatives Only:** The command does not generate any Cluster, Rows, or Column constraints.
- **Relatives:** The command does not generate any Cluster constraints.
- **Clusters:** The command does not generate any Relative constraints.
- **Rows and Columns:** The command does not generate any Cluster or Relative constraints.
- **InstanceMatching:** The command generates only an Inst Matching constraint based on the selection set from the layout. Any guard rings in the selection set are also extracted as Guard Ring constraints along with the Inst Matching constraint.
- **ResistorBank:** The command generates only a Resistor Bank constraint. Any guard rings in the selection set are also extracted as Guard Ring constraints along with the Resistor Bank constraint.
- **CMOS:** The command generates only a CMOS constraint. Any guard rings in the selection set are also extracted as Guard Ring constraints along with the CMOS constraint.

Therefore, the command can create different constraints depending on the behavior you specify. However, depending on the topology of the layout, it is still possible to get only Rows and Columns even if you select Relatives or Clusters mode. The converse is not true; the command does not generate Cluster or Relative constraints when you select Rows and Columns mode.

Clusters mode is more helpful in the early exploration phase when you may want to see minor modifications to the topology to help reduce the area.

Relatives and Relatives Only modes are used to preserve the topology as much as possible.

Rows and Columns mode makes minor adjustments to the topology in order to maximize alignments.

InstanceMatching mode extracts the specific layout pattern if it detects (in this search order) one of the following supported placement styles:

- Device Group
- Interdigitate
- Common Centroid
- Central Symmetric
- Balanced Pattern
- Text Pattern

This mode extracts the following [Inst Matching](#) constraint parameters:

- Row Orientations: None, Flipped, Odd Flipped, or Even Flipped
- Spacing Type: Microns or Tracks
- Different Row spacing
- Separation: Microns or Poly Pitch
- Abutment: None, Abut, or Optimal
- Same Row spacing
- Dummy Devices settings

ResistorBank mode extracts the following [Resistor Bank](#) constraint parameters:

- Unit Resistance
- Unit Height
- Unit Length
- Chain Style
- Spacing Type: Microns or Tracks
- Different Row spacing
- Same Row spacing

CMOS mode extracts the following **CMOS** constraint parameters:

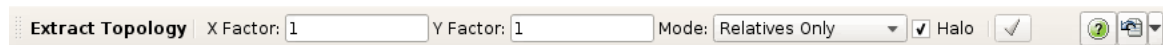
- Row Pattern: PN, NP, PNNP, or NPPN
- Dummy Devices settings
- Spacing Type: Microns or Tracks
- Different Row spacing
- Separation: Microns or Poly Pitch
- Abutment: Abut
- Same Row spacing

The **Extract Topology Constraints** command is described in more detail in the following sections:

- [Extract Topology Toolbar](#)
- [Using the Extract Topology Constraints Command](#)
- [Relative Constraint](#)

Extract Topology Toolbar

The interactive [Extract Topology Constraints Command](#) invokes the following toolbar:



[Table 37](#) describes the options provided by the **Extract Topology** toolbar.

Table 37 *Extract Topology Toolbar Options*

Option	Description
X Factor	Specifies the horizontal spacing scale factor with a positive real number. Tcl preference: lpExtractTopologyXFactor
Y Factor	Specifies the vertical spacing scale factor with a positive real number. Tcl preference: lpExtractTopologyYFactor

Table 37 Extract Topology Toolbar Options (Continued)

Option	Description
Mode	Specifies the mode used for extracting nonslicing portions of the layout, one of: • Relatives Only (default) • Relatives • Clusters • Rows and Columns • InstanceMatching • ResistorBank • CMOS Tcl preference: <code>lpExtractTopologyMode</code>
Halo	Specifies whether to use a halo to try to preserve alignment. Tcl preference: <code>lpExtractTopologyHalo</code>
Apply	Invokes the command on the current logical selection set.

Note:

The command supports only fully selected objects. Partially selected objects are ignored, if any. Partial selection is not and should not be enabled.

Using the Extract Topology Constraints Command

The [Extract Topology Constraints Command](#) works in both preselection and postselection modes. The command works in the following way:

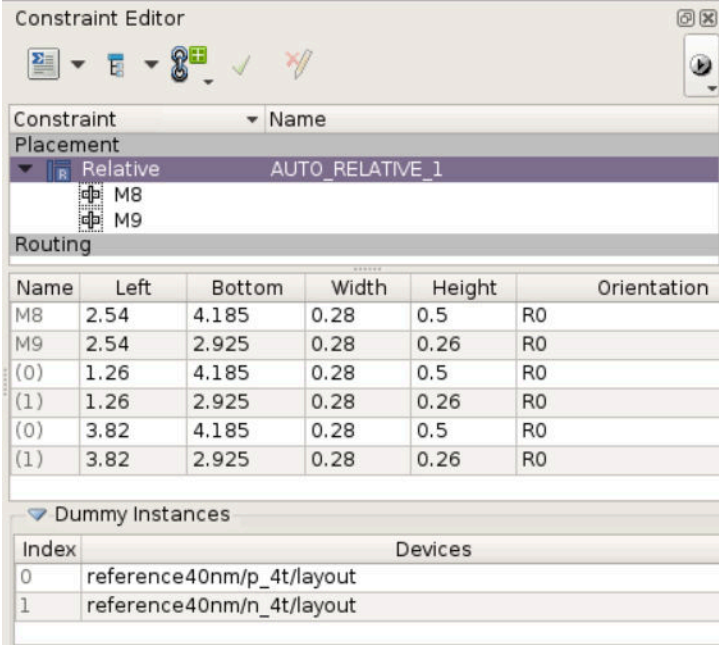
1. Choose **Placement > Extract Topology Constraints** to display the [Extract Topology Toolbar](#).
2. Select instances until the selection set is valid. **Apply**  is not enabled until there is a valid selection.
3. If the selection set is valid, either select more (or different) instances, or click **Apply**  to begin extraction.
4. After the extraction is complete, the command functions as follows:
 - In preselection mode: Exits and the objects remain selected.
 - In postselection mode: Clears the selection set and enters a new cycle by prompting you to select objects.

Relative Constraint

The Relative Constraint is created in the **Relative Only** mode from the Extract Topology Constraints command or using **Create Template** command. The relative intent allows a set of instances or instance groups to be placed in a relative fashion. All instances or instance groups associated with the intent have a box defining the position.

Figure 122 shows the Relative placement constraint as displayed in the Constraint Editor.

Figure 122 Relative Placement Constraint



Name	Left	Bottom	Width	Height	Orientation
M8	2.54	4.185	0.28	0.5	R0
M9	2.54	2.925	0.28	0.26	R0
(0)	1.26	4.185	0.28	0.5	R0
(1)	1.26	2.925	0.28	0.26	R0
(0)	3.82	4.185	0.28	0.5	R0
(1)	3.82	2.925	0.28	0.26	R0

Index	Devices
0	reference40nm/p_4t/layout
1	reference40nm/n_4t/layout

Table 38 describes the parameters for the Relative placement constraint.

Table 38 Parameters of Relative Placement Constraint

Parameters	Description
Name	Name of instance. Name of dummy instance is in the format of index.
Left	X coordinate of lower left corner of box.
Bottom	Y coordinate of lower left corner of box.
Width	Width of box.
Height	Height of box.
Orientation	Orientation guidance.

Table 38 *Parameters of Relative Placement Constraint (Continued)*

Parameters	Description
<i>Dummy Instances Settings</i>	
Index	Consecutive letter assigned to dummy instances.
Device	Library, Cell, and View (LCV) strings.

8

Updating Schematic From Layout

You can use the **SDL > Update Schematic From Layout** command to update the dummy devices, extra devices, and modified parameters of layout devices from layout to schematic. This command is provided to automatically synchronize between layout view and schematic view.

Auto updating schematic from layout is described in the following sections:

- [Updating Schematic From Dummy and Extra Devices](#)
- [Updating Schematic From Parameters Difference](#)
- [Update Schematic From Layout Options](#)

Updating Schematic From Dummy and Extra Devices

The typical synchronize between layout view and schematic view use model when using the **Update Schematic From Layout** command:

1. Click **SDL > Update Schematic From Layout** to open the **Update Schematic From Layout** dialog box. See [Figure 125](#).

You can choose the corresponding options to enable the **Dummy Devices** ([Figure 126](#)) and **Extra Devices** ([Figure 127](#)) tabs.

2. (Optional) You can specify target cellview by choosing the **Specify Schematic** option.
3. (Optional) You can specify connection of the schematic extra terminals in the **Map Terminals** text field for dummy and extra devices.

Note:

The detailed syntax and examples are shown in a yellow tip when the mouse cursor is placed over this text field.

4. In the **Dummy Devices** and **Extra Devices** tabs, you can choose **Cell** or **Instance** mapping to show the device mapping.

Note:

To display the candidates in the **Dummy Devices** tab, they must specify the following two attributes: **Usage** is specified to *Dummy* and **Physical Only** is enabled.

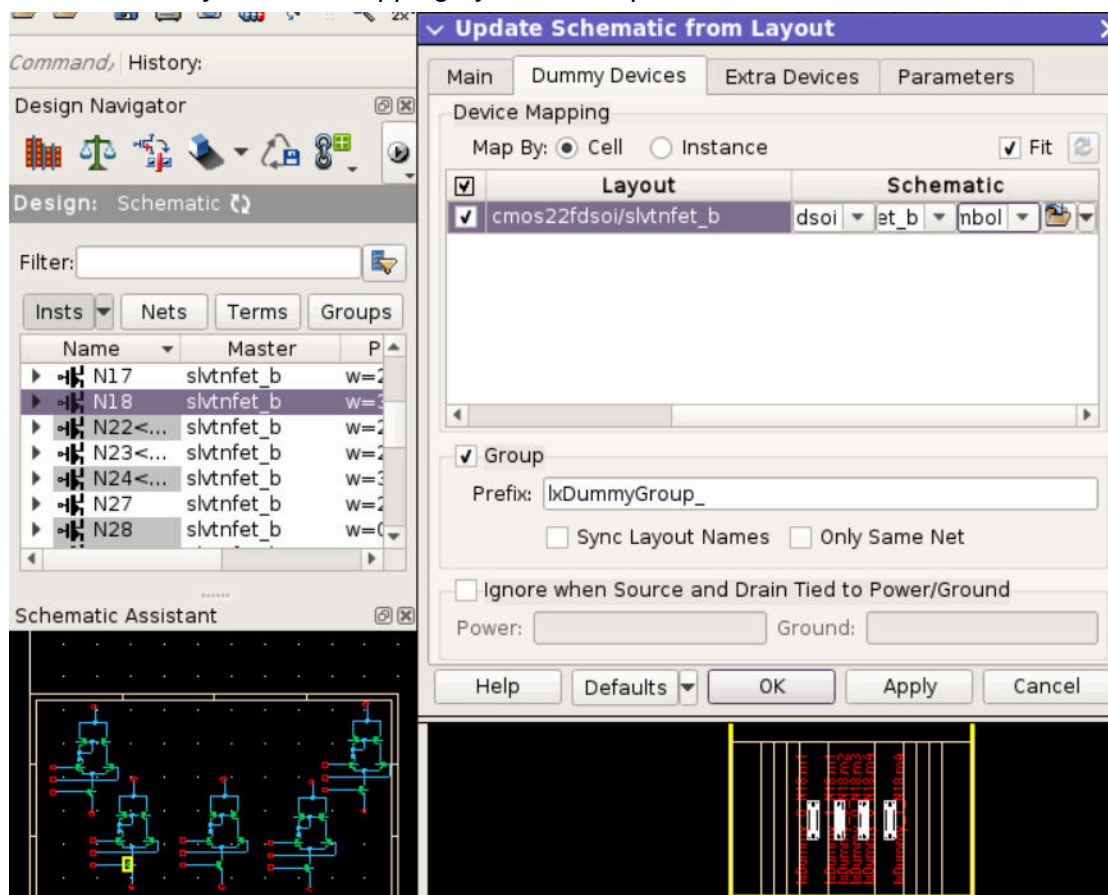
Attributes	
Prompt	Value
Library Cell View	reference40nm n_4t layout
Name	M3.m1
Origin	(0,0)
Orientation	R0
Array	☐
Placement Status	none
Depth Limit	Unlimited
Usage	Dummy
Physical Only	☒

Chapter 8: Updating Schematic From Layout

Updating Schematic From Dummy and Extra Devices

When mapping by **Cell**, every row of table represents a set of dummy devices which have same master cell and same mapping symbol. You can change the mapping symbol by clicking it in the **Schematic** column.

Figure 123 Dummy Devices Mapping by Cell Example

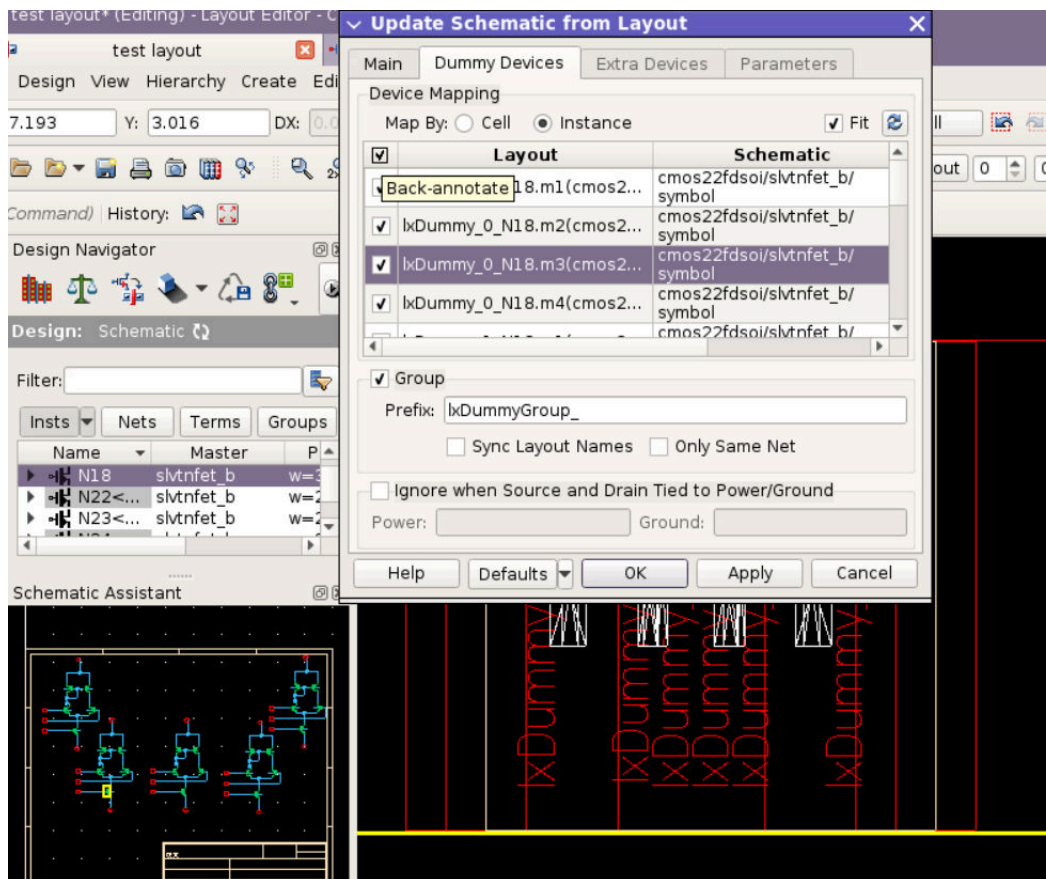


When mapping by **Instance**, every row represents a layout dummy and its mapping symbol. You can change the mapping symbol by clicking it in the **Schematic** column.

You can click the **Back-annotate** check box next to the **Layout** column heading to select or unselect all the check boxes. This check box lets you to decide what dummy devices need to be updated.

A tooltip appears and displays the mapping errors when moving the cursor on a cell or a dummy device.

Figure 124 Dummy Devices Mapping by Instance Example



- (Dummy Devices Only) Enable the **Group** option to group layout dummy devices which have same layout master and same parameter value. The grouped device in the schematic is named with the prefix `lxDummyGroup_`. You can enable the **Sync Layout Names** option to rename layout dummy devices in the layout so that they correspond to the schematic.
- (Extra Devices Only) The extra devices with the same base name and also M-factor, finger, or vector bit are grouped when updating to the schematic. If the parameters defined in `instParameter` of `simInfo` are not consisted for same base name, the devices cannot be grouped and updated to schematic. In this case, the devices are renamed with specified prefix `lxInst_` to the schematic when choosing **Update Layout Instance Name**.
- Click **OK**. The dummy devices are created in the target schematic with a red bounding box. The extra devices are created in the target schematic with a salmon bounding

box. The error messages are displayed when the devices are not updated to the schematic.

Note:

The updated dummy devices are displayed in the **Layout Dummy** group in the Design Navigator assistant.

Updating Schematic From Parameters Difference

Use the following procedure to update the layout parameters to schematic:

1. Click **SDL > Update Schematic From Layout** to open the **Update Schematic From Layout** dialog box. See [Figure 125](#).

In the **Main** tab, choose the **Parameters** option. The **Parameters** tab ([Figure 128](#)) is enabled.

2. (Optional) You can specify target cellview by choosing the **Specify Schematic** option.
3. To display the candidates in the **Parameters** table, they must have the following conditions:
 - The total width related parameters are different.
 - The value of the total width is identical between layout and schematic.
 - All the values of the width are identical in the layout.
4. The following total width related parameters are the candidates to be updated:
 - M-factor parameter defined in **M-Factor Names** in the **SDL Options** dialog box. See [Main Tab](#).

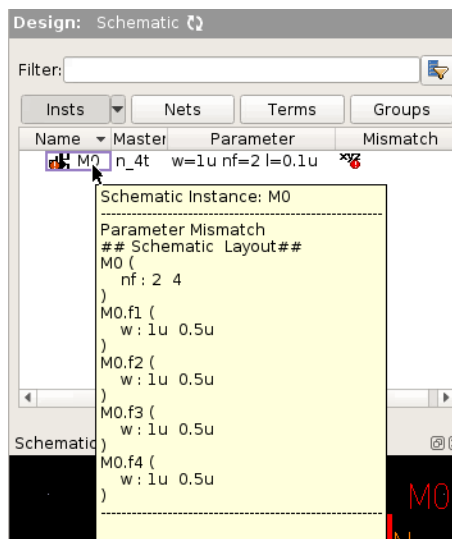
Note:

When the preference `lxExpandMFactor` is true, the devices with the M-factor is not updated.

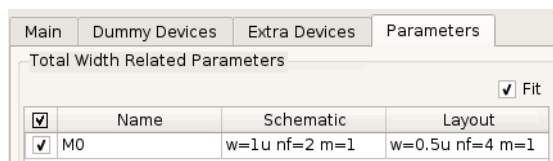
- `lxDeviceWidth`, `lxDeviceFinNumber`, and `lxFingerParam` defined in `lxComponentType`. See [Table 62](#).
5. Click **OK**. The layout parameters are updated to schematic.

The following example updates the parameters using **Update Schematic From Layout**.

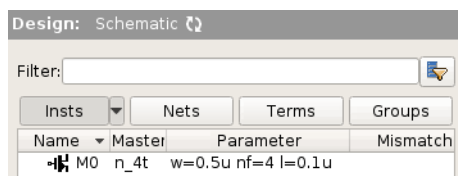
1. A parameter mismatch is found.



2. The parameter mismatch is displayed in the **Parameters** tab when clicking **SDL > Update Schematic From Layout**



3. Click **OK**. The parameters in the schematic are updated and the mismatch is fixed.



Update Schematic From Layout Options

This section describes the options of the **Update Schematic From Layout** dialog box:

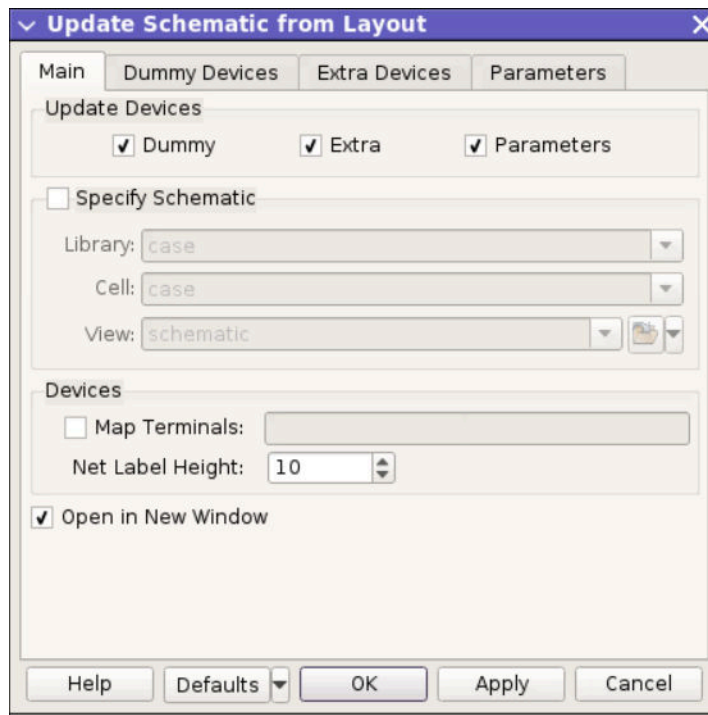
- [Update Schematic From Layout Options — Main Tab](#)
- [Update Schematic From Layout Options — Dummy Devices Tab](#)

- [Update Schematic From Layout Options — Extra Devices Tab](#)
- [Update Schematic From Layout Options — Parameters Tab](#)

Update Schematic From Layout Options — Main Tab

The following figure and table describe the **Main** tab of the **Update Schematic From Layout** dialog box.

Figure 125 Update Schematic From Layout Dialog Box - Main Tab



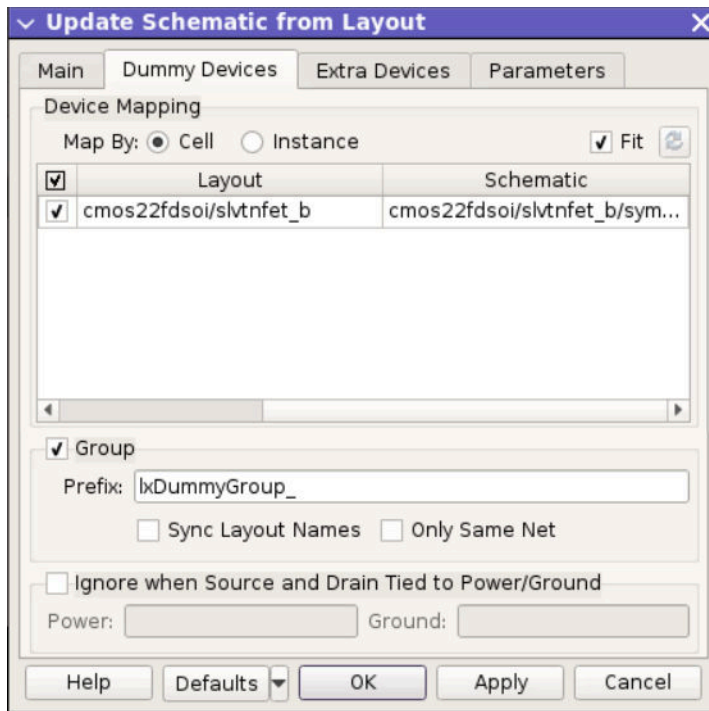
Option	Description
Update Devices	Lets you specify what devices that need to be updated to schematic. When the Dummy , Extra , and Parameters options are selected, the corresponding tabs are enabled.
Specify Schematic	Lets you specify the target schematic view in the Library/Cell/View fields. When enabled, the layout devices are updated to the system-created schematic or user-defined schematic. When disabled, the layout devices are updated to the original schematic.
Devices	This field is used and applied when updating layout dummy or extra devices.

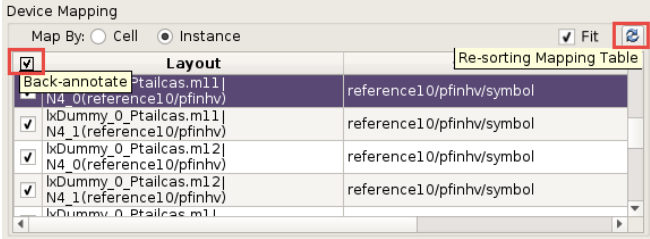
Option	Description
Map Terminals	Specifies the connection between the extra terminals and other terminals in the same instance with the following rules: • Extra terminals don't exist in the layer master. • Extra terminals and other specified terminals exist in the symbol master. The detailed syntax and examples are shown in a yellow tip when the mouse cursor is placed over this text field.
Net Label Height	Lets you specify the size of the net label when updating the devices in the schematic.
Open in New Window	When enabled, opens target cellview in a new window after updating.

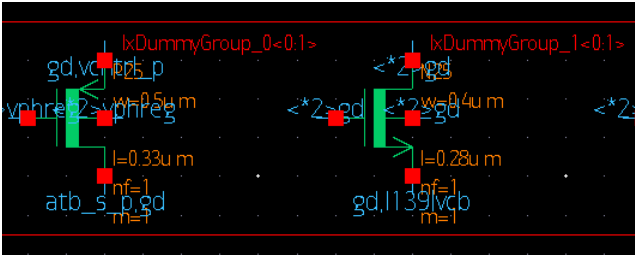
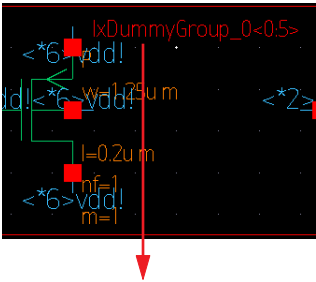
Update Schematic From Layout Options — Dummy Devices Tab

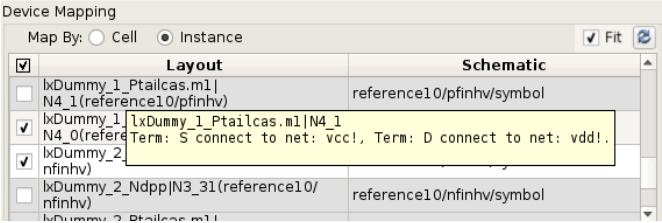
The following figure and table describe the **Dummy Devices** tab of the **Update Schematic From Layout** dialog box.

Figure 126 Update Schematic From Layout Dialog Box - Dummy Devices Tab



Option	Description
<i>Device Mapping</i>	Displays the dummy devices in the table. The dummy device is shown in red when the device information is missing. The dummy devices are shown in blue when the device information needs further investigation. The detailed missing information for the device is shown in a yellow tip when the mouse cursor is placed over a layout row in the Device Mapping table. 
Map By	Filter the dummy devices by selecting Cell or Instance. When mapping by cell, the Layout column represents layout master cell and the Schematic column represents the mapping symbol in the schematic view.
Fit	When enabled (default), fits the current candidate selected in the table in the layout view.
Re-sorting Mapping Table	Click the Re-sorting Mapping Table icon to re-sort the device mapping table.
Back-annotate	Click the Back-annotate icon to select all or deselect all dummy devices in the device mapping table. The selected dummy devices are back annotated from layout to schematic. The deselected dummy devices are ignored to back annotated from layout to schematic.

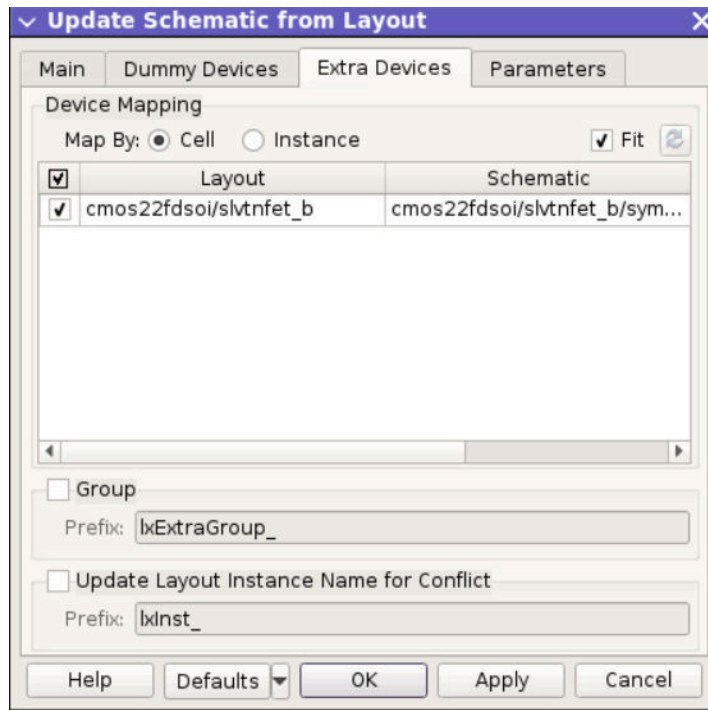
Option	Description
Group	Merges geometrically-identical dummy instances in the schematic, resulting in fewer visible instances in the schematic view. On by default. Two dummy groups in the schematic, each containing two dummy instances 
Prefix	If you deselect Group, each dummy instance in the layout will be visible in the schematic. Allows you to customize the prefix of grouped dummy device names. By default, the prefix is <code>lxDummyGroup_</code>. In the schematic, the prefix is followed by an index number and a number range that identify the dummy group and the number of dummy devices in that group. 

Option	Description
Sync Layout Names	When enabled, renames layout dummy devices in the layout so that they correspond to the schematic. For example, if layout dummy instances are grouped into <code>1xDummyGroup_0<0:5></code> in the schematic, the corresponding instances in the layout will be renamed to <code>1xDummyGroup_0(0)</code>, <code>1xDummyGroup_0(1)</code>, <code>1xDummyGroup_0(2)</code>, <code>1xDummyGroup_0(3)</code>, <code>1xDummyGroup_0(4)</code>, and <code>1xDummyGroup_0(5)</code>. When disabled, devices are updated to the schematic. However, you cannot find the updated devices in Design Navigator in Schematic mode because the <code>oaProp 1xIgnore</code> is added in the updated devices.
Only Same Net	When enabled, the dummy devices with same net are grouped. Tcl preference: <code>1xUpdateDummyGroupSameNet</code>
Ignore Dummies with Both Source and Drain Tied to Power/Ground Nets	When enabled, the dummy devices are ignored to back annotate from layout to schematic if the source and drain of the dummy devices are connected to the power or ground nets defined in the Power and Ground fields. The power or ground nets are applied from schematic connectivity automatically. A tooltip is shown when you move the cursor to the ignored candidate. 

Update Schematic From Layout Options — Extra Devices Tab

The following figure and table describe the **Extra Devices** tab of the **Update Schematic From Layout** dialog box.

Figure 127 Update Schematic From Layout Dialog Box- Extra Devices Tab



Option	Description
Device Mapping	This field is identical to the Device Mapping field in the Dummy Devices tab. See Figure 126 .
Group	Groups extra instances with specified prefix name. After extra devices are grouped to schematic design, the names of mapping layout extra instances will be renamed.
Updated Layout Instance Name	Renames extra instances with specified prefix name in the layout so that they correspond to the schematic.

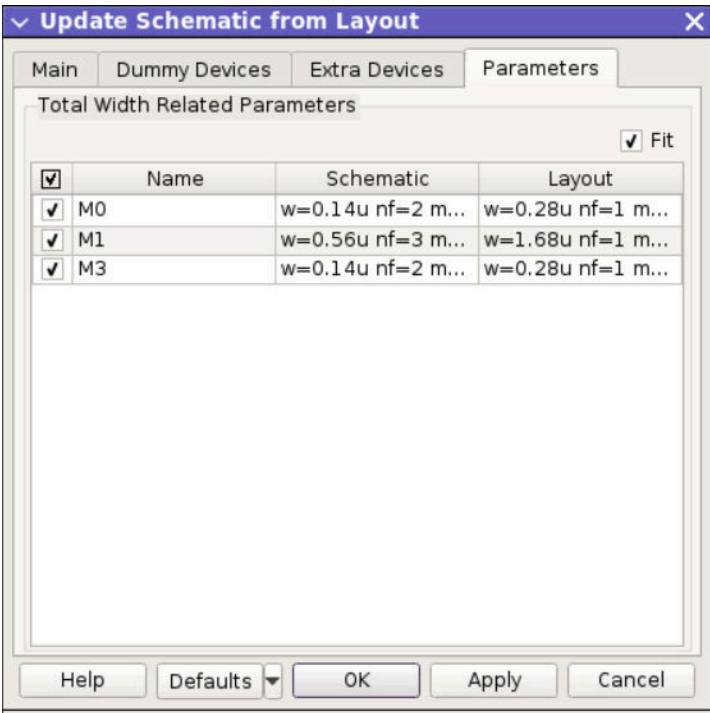
Update Schematic From Layout Options — Parameters Tab

The parameter update is designed for devices with the same total width values (gates been folded or merged). If the total width (width*finger) values are different between

schematic and layout, the device doesn't be shown in the **Parameters** tab. See the [Updating Schematic From Parameters Difference](#) section.

The following figure and table describe the **Parameters** tab of the **Update Schematic From Layout** dialog box.

Figure 128 Update Schematic From Layout Dialog Box- Parameters Tab



Option	Description
<i>Total Width Related Parameters</i>	Displays the candidates which can be updated from layout to schematic. • Fit: When enabled (default), fits the current candidate selected in the table in the layout view. • Click the Back-annotate icon to select all or deselect all candidates in the parameters table. The parameters of the selected candidates are back annotated from layout to schematic. The deselected candidates are ignored to back annotated from layout to schematic. • Name: Displays the schematic instances value. • Schematic: Displays the schematic instances where need to be updated. • Layout: Displays the updated value calculated from layout. These values are updated to schematic.

9

Using the Via Track Planner

This chapter explains how to use the Via Track Planner to create and delete vias, interconnects, and cut-poly segments in MOS transistor devices.

This chapter contains the following topics:

- [About Via Track Planner](#)
- [Enabling the Via Track Planner](#)
- [Starting Via Track Planner](#)
- [Creating and Deleting Vias in MOS Transistors](#)
- [Creating and Deleting Interconnects in MOS Transistors](#)
- [Creating and Deleting Cut-poly Segments in MOS Transistors](#)

About Via Track Planner

This feature supports a quick method to create vias in the predefined tracks. You can easily place vias by clicking a check box at a suggested via location, without zooming the design. After creating the vias, the routing can be finished using the router.

An Advanced license is required to invoke this feature. On the **Home** tab, choose **License > Advanced**.

Enabling the Via Track Planner

You must set the `lxViaTrackPlannerSetup` CDF parameter to use the Via Track Planner. This CDF parameter should be defined at the library, cell or cellview level. The parameter will be part of the data.dm for the cellview, cell or library (depending on where the CDF was defined.)

The parameter includes a list of track pattern definitions, and a prioritized list of layer definitions for annotations. Track pattern definitions are either horizontal or vertical, based on the foundry's requirements. The tracks typically define a horizontal grid to control the

horizontal connections, and the annotated layers control the vertical connections; together these create a matrix of connection points. It is expected that the tracks and the annotated layers are orthogonal to each other.

The syntax is as follows:

Name	Definition
lxViaTrackPlannerSetup	(trackPatternDef+ annotationDef+ [clickProcDef] [validateProcDef] connectionPointsDef+)
trackPatternDef	(trackPattern (name string) (attributes (attrLists)))
annotationDef	(annotation (name string) (priority int) (select (LPPList)) [(inside (LPPList))] [(outside (LPPList))] [(interact (LPPList))] [(not_interact (LPPList))] [(dir horizontal vertical)] [(geometryResize "float float")] [(displayResize "float float")] [(cpSize float)] (stretchable bool) (display bool) (type cpTypeEnum))
clickProcDef	(clickProc string)
validateProcDef	(validateProc string)
connectionPointsDef	(connectionPoints (trackPatterns stringList) (annotations stringList))
attrLists	attrList+
attrList	"attrNameValue+"
attrNameValue	attrName:attrValue
attrName	name layer color type pitch offset width direction orientation
attrValue	string real
stringList	"string+"
LPPList	LPP+
LPP	"string string" "string" "string *"
cpTypeEnum	empty filled erase local keepout

Default values:

Name	Default
clickProc	lx::clickProc (see Tcl Procedures, below)

Name	Default
validateProc	lx::validateProc (see Tcl Procedures, below)
resize	"0 0"
LPPList	" "
dir	vertical
stretchable	false
display	true

Tcl Procedures

You must define Tcl procedures with names defined by the parameters `clickProc` and `validateProc`, and they must be preloaded before the Via Track Planner is launched. The default names for these procs are `lx::clickProc` and `lx::validateProc`. An overview of the syntax follows.

<code>lx::clickProc { data }</code>	This user-defined process handles a click on a connection point and updates the PCell parameters and the PCell sub-master. The process is a list (data) with sub-lists that are name-value pairs. Valid argument names are <code>click_cp</code> and <code>neighbor_cp</code> which both have values as <code>lxConnectionPoint</code> objects. The procedure only needs to know the value of the <code>click_cp</code> connection point.
<code>lx::validateProc { data }</code>	This user-defined process checks the status of a neighboring connection point <code>nbr</code> after a click on connection point <code>p</code> has been handled. This process returns a Boolean indicating whether the neighboring point should be active or inactive. The process is a list (data) with sub-lists that are name-value pairs. Valid argument names are <code>click_cp</code> and <code>neighbor_cp</code> which both have values as <code>lxConnectionPoint</code> objects. The procedure needs to know the values of both the <code>click_cp</code> and <code>neighbor_cp</code> connection points to get their location in the design and return an appropriate Boolean status to be applied on <code>neighbor_cp</code> .

Example:

```
(
(trackPattern (name m1) (attributes ("name:mlh_e1 layer:m1" "name:mlh_e2
layer:m1")))
```

4. For more information about `lxConnectionPoint`, see *Schematic Driven Layout Tcl in the Custom Compiler™ Tcl Reference Manual*.

Chapter 9: Using the Via Track Planner

Enabling the Via Track Planner

```
(annotation (name GatekeepOut) (priority 0) (select ("POLY drawing")
  inside ("DIFF drawing")) (display false) (type keepout))

(annotation (name erase) (priority 1) (select ("POLY_CUT drawing"))
  (inside "") (outside "") (dir vertical) (display false) (type erase))

(annotation (name via) (priority 2) (select ("via0 drawing" "cont
  drawing"))
  (inside "") (outside "") (dir vertical) (display false) (type filled))

(annotation (name gate) (priority 3) (select ("POLY drawing"))
  (dir vertical) (display true) (type empty))

(annotation (name source_drain) (priority 4) (select ("DIFF drawing" "OD1
  *" "OD2"))
  (inside ("well drawing")) (outside poly) (dir vertical) (resize ("-0.005
  0.005")) (type empty))

(annotation (name local_interconnect) (priority 5) (select ("MODIFF2
  drawing"))
  (dir vertical) (stretchable true) (resize ("-0.010 0.000")) (type
  empty))

(connectionPoints (trackPatterns "m1") (annotations "GateKeepOut erase
  via gate source_drain local_interconnect"))
)
```

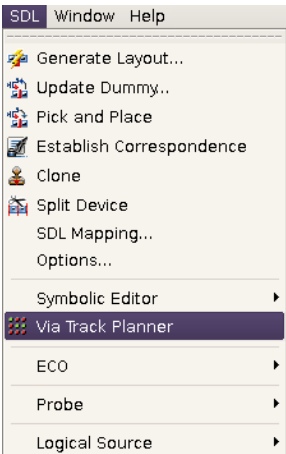
In this example, the annotation with the name `via` chooses all geometries on the `via0 drawing` or `cont drawing` LPPs from the devices. This annotation is defined with a lower priority number (2) than the annotation for `source_drain` (4), so the `via` annotation will take precedence over `source_drain`. That is, if there is also an annotation with `source_drain` at the same connection point, then the connection point will be deemed to be defined by "via" (and not "source_drain"). The geometry is not annotated because `display=false`. The annotation with name `source_drain` is defined by all geometries on the `DIFF drawing`, `OD1*`, and `OD2*` LPPs. The geometry of shapes from these LPPs that is outside the `poly` and inside the `well drawing` layers is computed. This computed geometry is resized by applying the `shrink` value (0.005) in the X direction and the `grow` value (0.005) in the Y direction, and the resulting geometry is annotated in the Layout Editor. The annotation for `local_interconnect` is applied at a `resize` value of "-0.010 0.000" on interconnect geometry. Since `local_interconnect` is defined as `stretchable`, the Via Track Planner will create a pair of connection points at the instance boundary of the two abutting instances. These connection points would be coincident and should be handled by the `clickProc` to stretch the underlying mask geometry (`MODIFF2` in this case) to the click point. The Via Track Planner calls the `clickProc` for both the connection points, to stretch both instances and create a local interconnection with `MODIFF2`. Annotations with a type of `keepout` identify the layers that define the keepout regions. In above example, the annotation `GateKeepOut` defines that wherever there is `POLY` inside the `DIFF` geometry, no connection points should be created inside that area.

Essentially this geometry is a keep-out area for the connection matrix. The track pattern names can be given in regular expression form.

Starting Via Track Planner

After you finish placement of transistors using SDL, there are no vias and local interconnections in the layout. The connectivity is depicted through flight lines. You can use **SDL > Via Track Planner** to create vias and connections.

Figure 129 SDL Menu - Via Track Planner



You also can start the feature by clicking the  button from the Layout Editor toolbar.

Figure 130 Layout Editor Toolbar



The Via Track Planner toolbar is displayed when the feature is invoked.

Figure 131 Via Track Planner Toolbar

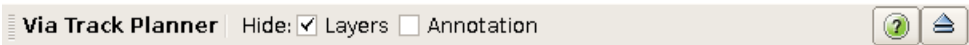


Figure 132 *Via Track Planner Example*

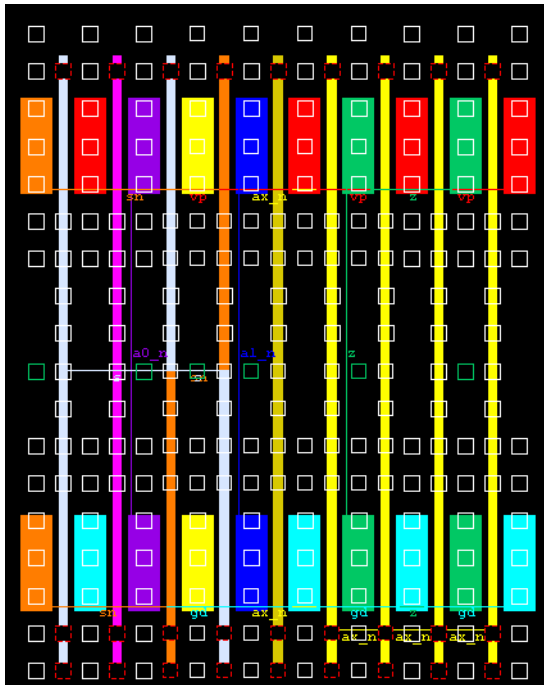
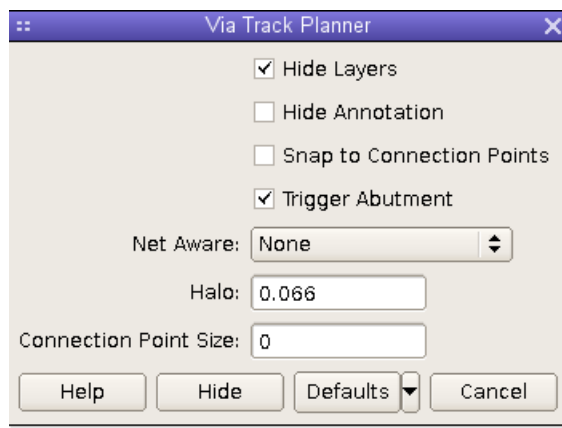


Figure 133 Via Track Planner



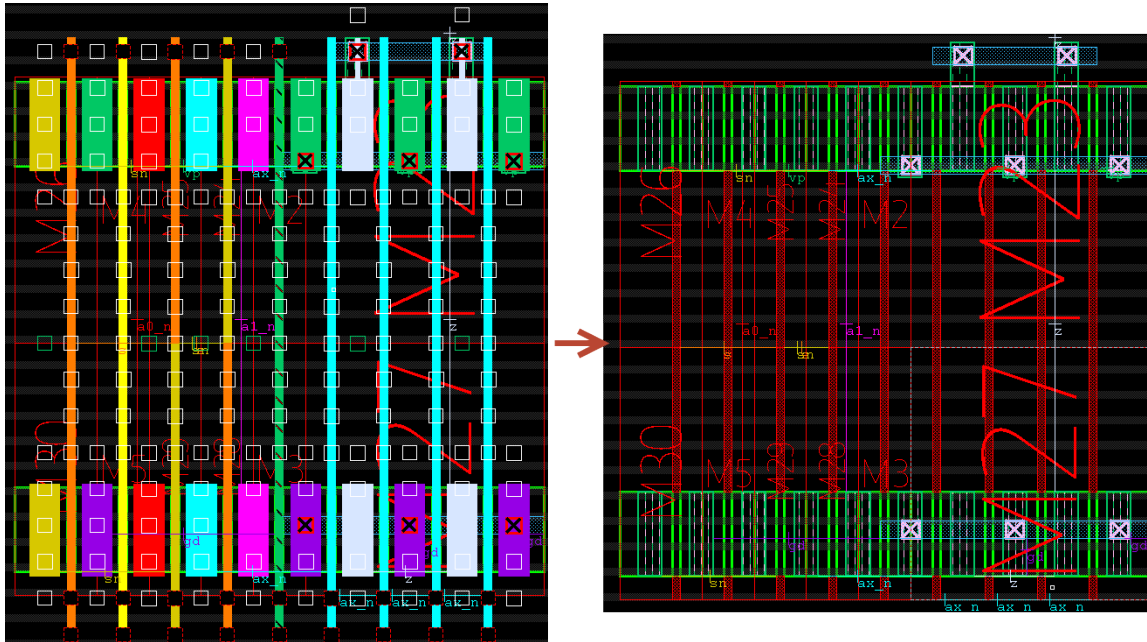
This dialog box includes the following options:

- **Hide Layers:** Clicking this option hides the drawing layers. The default is *on*.
- **Hide Annotation:** Clicking this option hides the annotation overlay created for net display.
- **Snap to Connection Points:** Clicking this option places cursor snap locations to the connection points.
- **Trigger Abutment:** Enables auto abutment for transistors. The default is *on*, so PCells will be modified to abut each other as needed, after the `clickProc` finishes updating the PCell parameters.
- **Net Aware:** Displays the nets information using the following selections:
 - **None:** This option displays all net information (annotations and connection points). The default is None.
 - **Annotations:** This option displays the annotations for the net that are selected in the **Nets** tab in the Design Navigator. All the connections are displayed.
 - **Annotations and CPs:** This option displays the annotations and the connection points for the net that are selected in the **Nets** tab in the Design Navigator.
- **Halo:** The **Halo** value defines an area (a “halo”) around any selected connection point, whenever you make a connection or place a cut-poly segment. The Via Track Planner checks within this area to determine whether any suggested connection points are now invalid because of the connection you enabled.
- **Connection Point Size:** Specify the size of the connection point. The default is 0.

Creating and Deleting Vias in MOS Transistors

After you launch the Via Track Planner, you can click the empty check boxes to drop vias at source, drain or gate fingers to make source connections, drain connections, or gate connections. You also can uncheck the boxes to delete the vias.

Figure 134 Create Vias

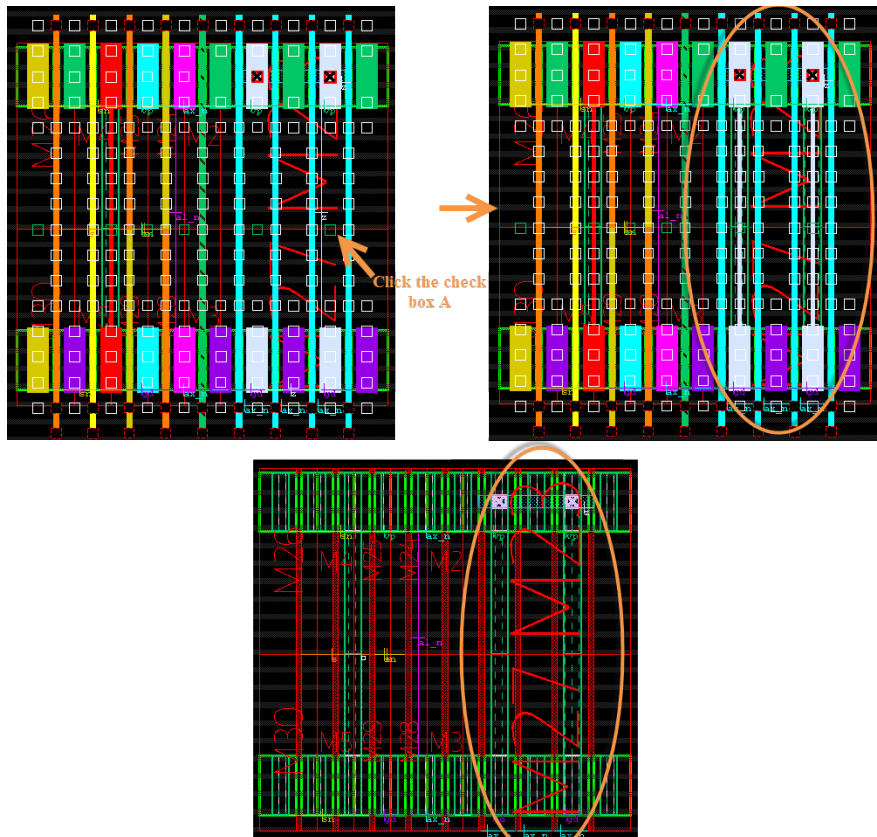


Creating and Deleting Interconnects in MOS Transistors

Figure 135 shows creating the vertical interconnects. You can create vertical interconnects on the diffusion layer in MOS transistors. Click a check box (A) next to a suggested

connection. The interconnect layer is created in the vertical direction to connect objects on the same net that are vertically above and below the connection point.

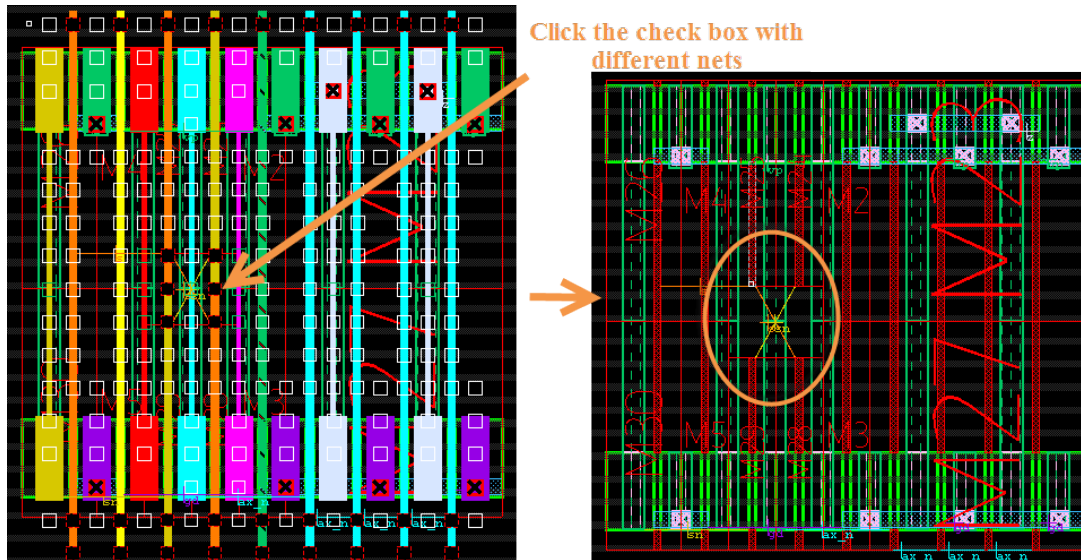
Figure 135 Create Vertical Interconnects



Creating and Deleting Cut-poly Segments in MOS Transistors

You can create cut-poly segments to erase portions of the poly layer that would otherwise short two objects on different nets. Click the box at the point where the objects on the two nets are touching, and a cut-poly segment is drawn, as shown in [Figure 136](#).

Figure 136 Create A Cut-poly Segment



10

Setting Up and Performing Verification

This chapter describes the features related to Engineering Change Orders (ECO) and best practices in working with errors.

 [Show me](#) how to use SDL ECO and **Auto Update**.

This chapter covers the following topics:

- [Mismatches Between Physical Layout and Logic Source](#)
- [Schematic Driven Layout Engineering Change Orders](#)

Mismatches Between Physical Layout and Logic Source

The SDL ECO feature finds mismatches between the target physical design and its logical source and helps you fix them.

While comparing the physical target with its logical source, SDL ECO checks for particular types of mismatches. For more information about the types of mismatches, see [Specifying Error Types and Auto Update Actions](#).

When the SDL ECO Verify operation is run, all mismatches are marked with an `oaMarker` object and displayed in the Error Viewer. For more information, see [Finding Mismatches between Source and Target Designs](#).

Schematic Driven Layout Engineering Change Orders

An Engineering Change Order (ECO) includes:

- Checking for mismatches between the physical and logical designs
- Resolving errors so that the logical and physical designs match

The following sections describe how to configure ECO, run the commands, and resolve mismatch errors.

- [Setting Up Schematic Driven Layout ECO](#)
- [Finding Mismatches between Source and Target Designs](#)
- [Resolving Mismatches between Source and Target Designs](#)

Setting Up Schematic Driven Layout ECO

Setting up the ECO feature in SDL serves three purposes:

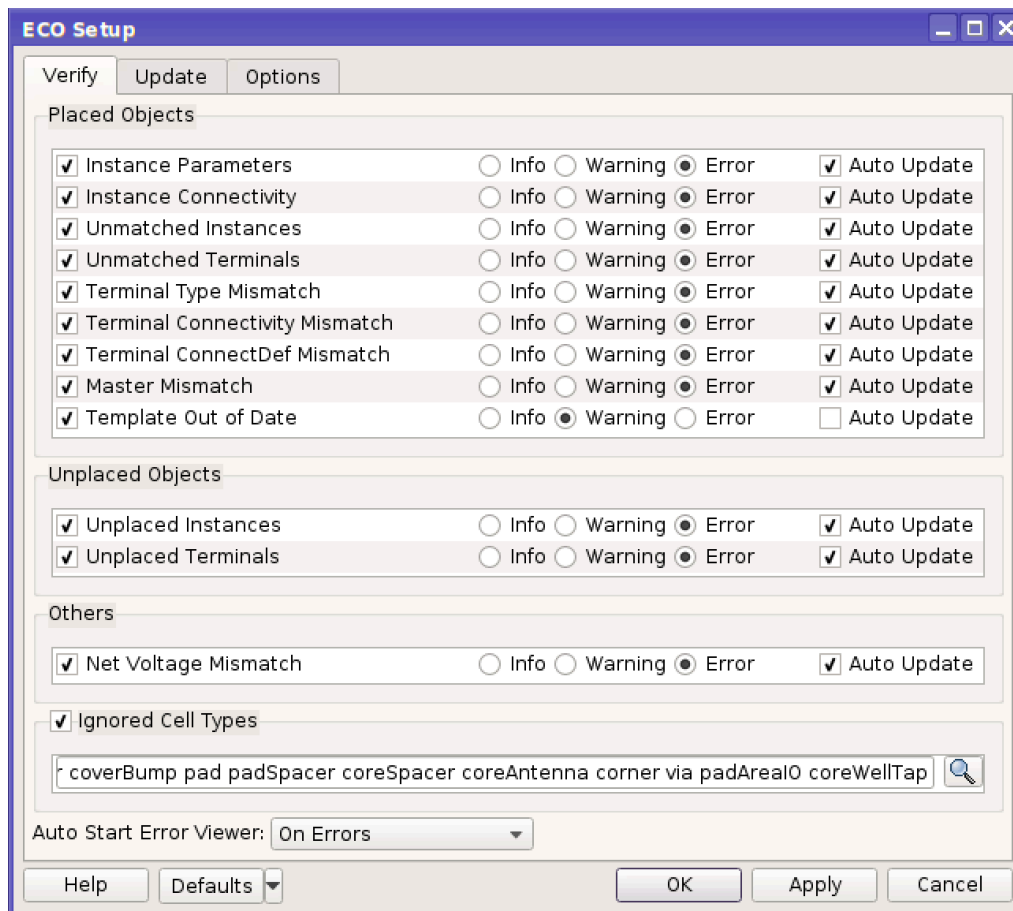
- To specify the types of errors between the physical layout and its logical source that SDL should look for in the design
- To specify the types of errors SDL should resolve automatically when the Auto Update command is executed
- To specify whether to open the Error Viewer for incremental updating

To set up ECO:

1. Select **SDL > ECO > Setup**.

The **ECO Setup** dialog box appears. See [Figure 137](#).

Figure 137 ECO Setup Dialog Box



2. Select the **Verify** tab and specify the options you want to apply (see [Specifying Error Types and Auto Update Actions](#)).
3. Select the **Update** tab and specify the update options you want to apply (see [Specifying the Update Options](#)).
4. Select the **Options** tab and specify the verify options you want to apply (see [Specifying the Verify Options](#)).
5. Click **OK**.

The SDL ECO setup is completed.

Specifying Error Types and Auto Update Actions

The **Verify** tab of the **ECO Setup** dialog box lets you specify which mismatches the **ECO > Verify** command should search for, the severity of the mismatch error, and whether the

ECO > Auto Update command should attempt to fix the mismatch. In addition, you can specify whether to ignore specific cell types during the verification operation.

The following tables list and describe the types of mismatches between the physical layout and its logical source that SDL ECO can identify.

- [Table 39](#) lists mismatches for placed objects and how the **Auto Update** command attempts to fix them.
- [Table 40](#) lists mismatches for unplaced objects.
- [Table 41](#) lists mismatches for net voltages.

To set up the **Verify** options:

1. Specify the rule violations that you want SDL to look for when you run the Verify command. For information about the rules and errors, see [Table 39](#).
2. (Optional) For each of the selected rules, specify the severity for errors that are found.
3. (Optional) Specify cell types to ignore during verification.
4. (Optional) For each of the selected rules, specify whether you want SDL to fix the error during the Auto Update process.
5. The **Auto Start Error Viewer** menu options let you control whether to open the Error Viewer to incrementally update mismatches:
 - **On Errors**: Automatically opens up the Error Viewer when SDL ECO Verify detects an error.
 - **On Warnings Or Errors**: Automatically opens up the Error Viewer when SDL ECO Verify detects an error or a warning.

Note:

With arrayed instances, the **ECO > Verify** command reports an error only in the console, not in Error Viewer.

- **Never**: Does not automatically open up the Error Viewer.

Note:

Alternatively, you can use the **SDL > ECO > Incremental Update** command to interactively resolve errors one at a time in the layout. For more information, see [Using the Incremental Update Command to Resolve Mismatches](#).

Table 39 Placed Objects

Mismatch Type	Description	Auto Update Action
Instance Parameters	Parameter values do not match between corresponding logical and physical instances.	Updates parameter values on instances. Note: If <code>lxEcoApplyPreset</code> is true, ECO Auto Update and Incremental Update apply the preset settings specified in Parameter Preset from SDL Options to the instances. Note: If <code>lxValidWidths</code> in <code>lxComponentType</code> is defined, the tool uses the defined valid widths when fixing parameter mismatch. Example: <pre>Schematic w = 2.4µ lxValidWidths = ("0.9µ" "0.6µ" "0.3µ") layout w = ("0.8u" "0.8u" "0.8u" "0.8u") After fixing, the width of each instance = 0.6µ</pre> Note: If <code>lxMinFinger</code> is defined in <code>lxComponentType</code> and <code>lxSplit</code> is turned on, the number of updated devices is greater than the specified value of <code>lxMinFinger</code>.
Instance Connectivity	The instance terminal of any instance has an incorrect net.	Updates the nets in the specific instance terminals.
Unmatched Instances	An instance is present in the layout, but it does not correspond to any instance in the schematic.	If <code>lxEcoDeleteExtraDevice</code> is true, deletes all unmatched instances. Otherwise, no operation.
Unmatched Terminals	A terminal is present in the layout, but does not match any terminal in the logical design.	If <code>lxEcoDeleteExtraDevice</code> is true, deletes all unmatched terminals. Otherwise, no operation.
Terminal Type Mismatch	The logical terminal type does not match with the physical terminal type. The logical pin type does not match with the physical pin type.	Sets the layout terminal type to match the logical terminal type. Sets the layout pin type to match the logical pin type.
Terminal Connectivity Mismatch	The corresponding nets of the logical and physical terminals do not match.	Sets the layout terminal net to match the logical terminal net.

Table 39 Placed Objects (Continued)

Mismatch Type	Description	Auto Update Action
Terminal ConnectDef Mismatch	The assignmentName or defaultName fields of the logical and layout terminals do not match.	Updates the layout terminal ConnectDef data to match the logical terminal.
Master Mismatch	The logical to physical master mapping does not match with elaboration.	If <code>1xEcoApplyPreset</code> is true, ECO Auto Update and Incremental Update apply the preset settings specified in Parameter Preset from SDL Options to the instances.
Template Out of Date	The template timestamp is out of date because the template was modified after creating template constraints. For hierarchy template, only verify the top-level template.	Regenerates the out of date template.

Table 40 Unplaced Objects

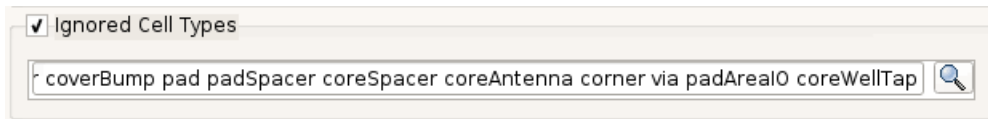
Mismatch Type	Description	Auto Update Action
Unplaced Instances	The logical instance master has no corresponding physical instance master. In such cases, SDL ECO Verify prints out the error to the log in the Custom Compiler console, but an error marker is not generated.	Places the unplaced instances. If <code>1xEcoApplyPreset</code> is true, ECO Auto Update and Incremental Update apply the preset settings specified in Parameter Preset from SDL Options to the instances when placing unplaced instances.
Unplaced Terminals	A terminal is present in the schematic, but not in the layout.	Places the unplaced Terminals.

Table 41 Others

Mismatch Type	Description
Net Voltage Mismatch	Voltage values do not match between the annotated value and the value in layout.

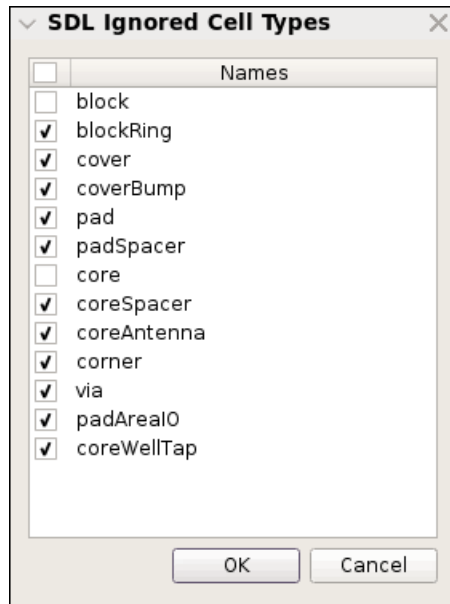
Ignoring Cells During Verification

By default, the tool ignores specific cell types during the verification operation, as shown in the **Verify** tab of the **ECO Setup** dialog box.



If you do not want to ignore any cells, deselect the **Ignored Cell Types** check box.

To specify which types of cells to ignore, click  to display the **SDL Ignored Cell Types** dialog box. Make your selections and click **OK**.



Alternatively, set the `lxEcoIgnoreCellTypes` preference to true and specify the cell types to ignore in the `lxEcoCellTypeIgnoredNames` preference.

Note:

If you specify cell types to ignore in SDL ECO, you cannot use the Unset Ignore right-click option in Design Navigator to stop ignoring instances of the specified cell types. You must modify the selection in SDL ECO.

Specifying the Update Options

The **Update** tab of **ECO Setup** window lets you specify how the **ECO > Auto Update** command attempts to fix the mismatch.

Figure 138 ECO Setup Dialog Box - Update Tab

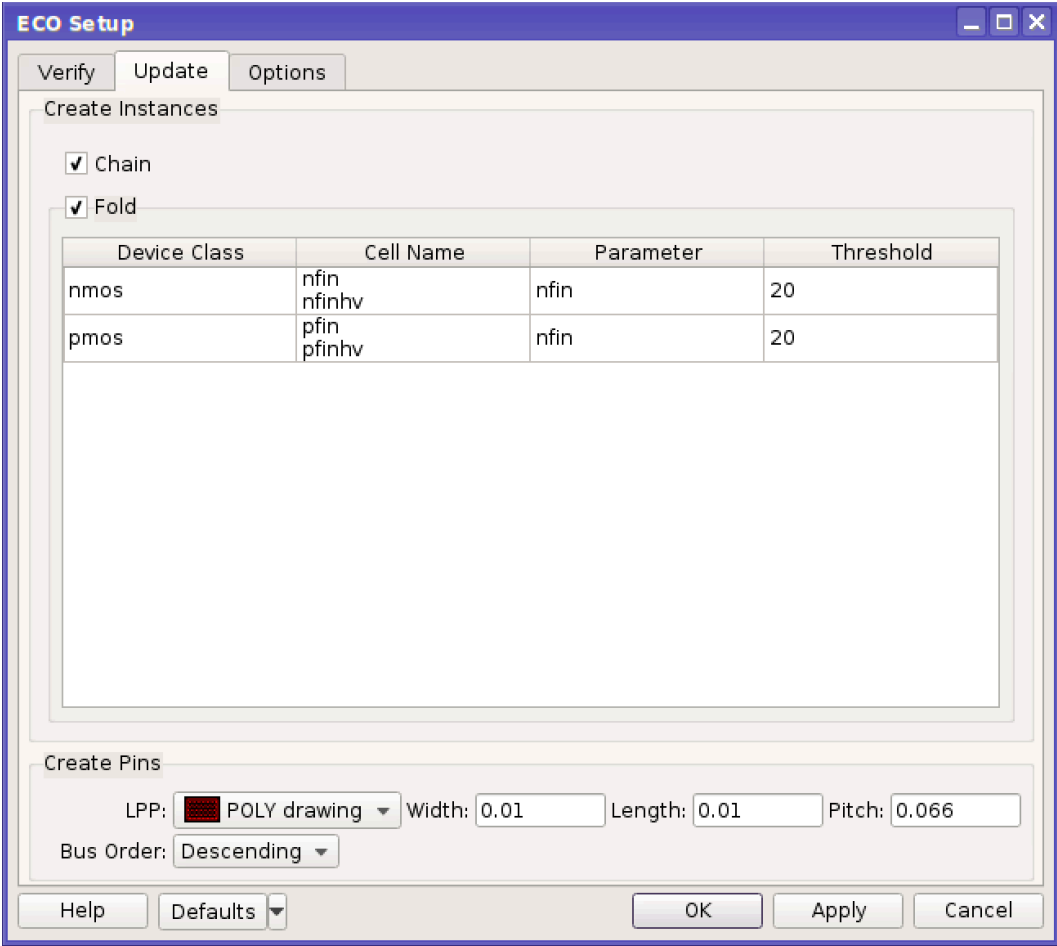


Table 42 lists the **Auto Update** options.

Table 42 Auto Update Options

Option	Description
Create Instances	Specifies whether all instances must be created automatically.
Chain	Controls whether the auto chaining feature must be enabled for selected objects; see Chaining Instances .

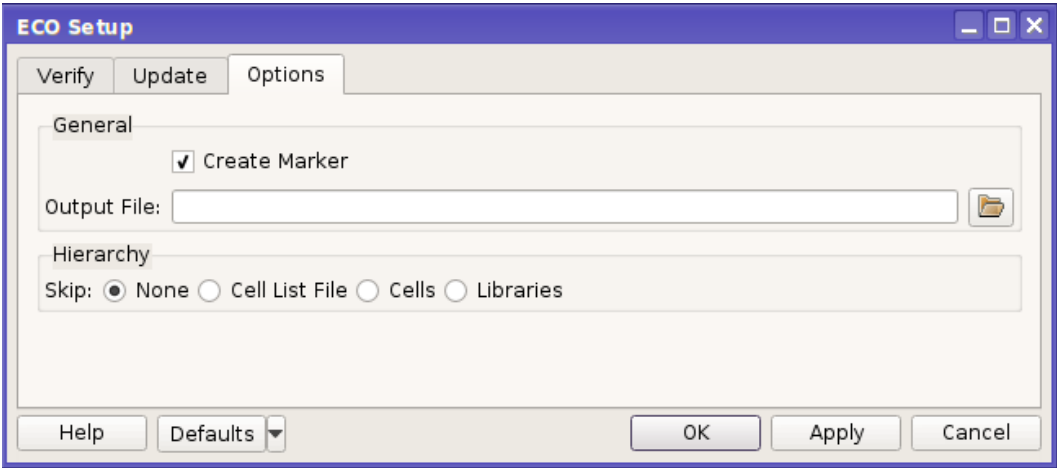
Table 42 Auto Update Options (Continued)

Option	Description
Fold	Controls whether the auto folding feature must be enabled for selected objects; see Folding Instances . • <i>Device Class</i>: Specifies the device class for which folding is made available. • <i>Parameter</i>: Specifies the parameter that is considered for folding. • <i>Threshold</i>: Lets you specify the folding threshold. If the instance parameter value is higher than the fold threshold, folding is initiated.
Create Pins	Controls whether pins are created automatically. • <i>LPP</i>: Specifies the layer purpose pair on which the pin shapes must be created. • <i>Width</i>: Specifies the width of the pin shapes, in user units. • <i>Length</i>: Specifies the height of the pin shapes, in user units. • <i>Pitch</i>: Determines the spacing between objects placed on the Top or Bottom side, in user units.
Bus Order	Controls the order of enumeration of buses.

Specifying the Verify Options

The **Options** tab of **ECO Setup** window lets you specify how the **ECO > Verify** and **ECO > Verify Hierarchy** commands attempt to fix the mismatch.

Figure 139 ECO Setup Dialog Box - Verify Options Tab



[Table 42](#) lists options in the **Verify Options**.

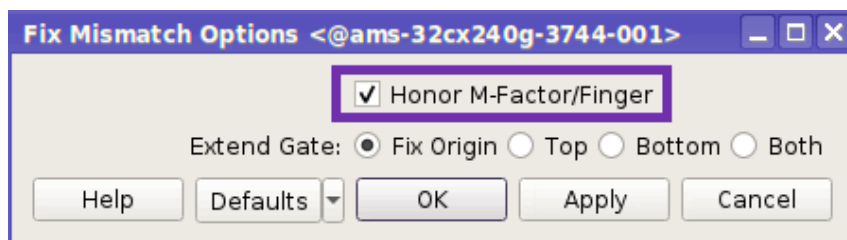
Table 43 Verify Options Tab

Option	Description
General	
Create Marker	Controls whether to create makers in the design. Markers are not saved in the design but saved in the log file for Hierarchy Verify .
Output File	Specifies a file to save the verify details. If this text field is empty, <code>./sdl_verify_hierarchy.log</code> is created to save the details.
Hierarchy	
Skip	Controls whether to skip the verify target by selecting one of the following options: • None: No object is skipped. • Cell List File: When this option is selected, you can choose a file in the selection field to skip the verification for the cells specified in the file. • Cells: When this option is selected, you can define a space-separated cell list in the text field. The Verify Hierarchy is skipped to verify the specified cells. • Libraries: When this option is selected, you can define a space-separated library list in the text field. The Verify Hierarchy is skipped to verify the specified libraries. This option only works for Hierarchy Verify .

Honor M-Factor/Finger

The **Honor M-Factor/Finger** option updates the count of instances in the layout to match the number of instances in the schematic to resolve the parameter mismatch.

1. Set the `lxECHonorMFactorFinger` as `true` to enable the **Honor M-Factor/Finger** option.
2. In the **Design Navigator**, right-click on an instance and select **Fix Mismatch Options** from the menu. The **Fix Mismatch Options** dialog box opens.



3. Select the **Honor M-Factor/Finger** check box.

Note:

- If the number of instances in the layout is greater than the number of instances in the layout, ECO renames the redundant instances in the layout as an additional instance.
- If the number of instances in the layout is less than the number of instances in the schematic, ECO creates new instances next to the instance with the largest f-number.

Extend Gate

The **Extend Gate** option allows you to change the boundary of an instance when fixing the parameters of the instance in the layout.

- In the **Design Navigator**, right-click on an instance and select **Fix Mismatch Options** from the menu. The **Fix Mismatch Options** dialog box opens.

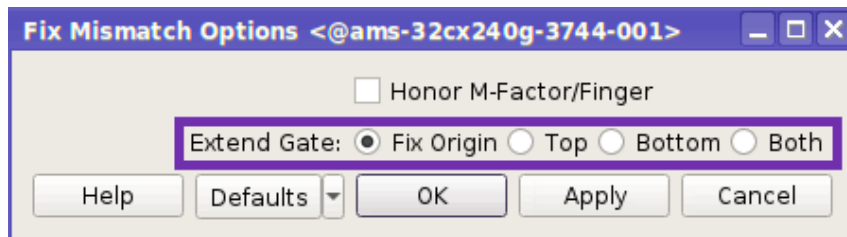


Table 44 describes the options available in **Extend Gate**:

Table 44 *Extend Gate Options*

Option	Description
Fix Origin	Change the boundary while keeping the origin fixed.
Top	Change the top and right sides of the boundary while keeping the bottom of the boundary fixed. Note: The origin of the boundary may change.
Bottom	Change the bottom and left sides of the boundary while keeping the top of the boundary fixed. Note: The origin of the boundary may change.
Both	Change the boundary while the keeping the middle of the instance fixed.

Finding Mismatches between Source and Target Designs

This chapter covers the following topics:

- [Verify Current Design](#)
- [Hierarchy Verify](#)

Verify Current Design

The **SDL > ECO > Verify** command applies a circuit matching algorithm to the netlists from the logical and physical designs and flags the mismatches.

To run ECO Verify from the layout design:

1. Choose **SDL > ECO > Setup**.

The **ECO Setup** window opens.

2. Specify the Verify and Update options that you want to apply. For more information, see [Specifying Error Types and Auto Update Actions](#), [Specifying the Update Options](#), and [Specifying the Verify Options](#).
3. Click **OK** to close the **ECO Setup** form.
4. Choose **SDL > ECO > Verify**.

SDL compares the logical and physical designs for errors that pertain to the rules selected during SDL ECO Setup and displays the errors.

The mismatches are written in the following forms:

- *oaMarkers*: SDL flags mismatches by creating markers in the design, which you can view using the Error Viewer. The markers are categorized based on the type of mismatch. For more information, see [Using the Error Viewer to Resolve Mismatches](#).

Note:

With arrayed instances, the **ECO > Verify** command reports an error only in the console, not in the Error Viewer.

- *Console*: **SDL Verify Summary** regarding the mismatches appears in the Custom Compiler Console window.
- *Log file*: Information regarding mismatches are also written to the Custom Compiler log file.

Hierarchy Verify

The **SDL > ECO > Verify Hierarchy** command applies a circuit matching algorithm to the netlists from the logical and physical designs and flags the mismatches by marks on overlay design.

To run ECO Verify Hierarchy from the layout hierarchy design:

1. Choose **SDL > ECO > Setup**.

The **ECO Setup** window opens.

2. Specify the Verify and Update options that you want to apply. For more information, see [Specifying Error Types and Auto Update Actions](#), [Specifying the Update Options](#), and [Specifying the Verify Options](#).
3. Click **OK** to close the **ECO Setup** form.
4. Choose **SDL > ECO > Verify Hierarchy**.

SDL compares the logical and physical designs for errors that pertain to the rules selected during SDL ECO Setup and displays the errors.

The mismatches are written in the following forms:

- *oaMarkers*: SDL flags mismatches by creating markers in the design, which you can view using the Error Viewer. The markers are categorized based on the type of mismatch. The markers are not saved in the design but saved in the log file. To view error markers from the top design hierarchy to down, see [Viewing Error Markers Hierarchically](#). For more information, see [Using the Error Viewer to Resolve Mismatches](#).

Note:

With arrayed instances, the **ECO > Verify Hierarchy** command reports an error only in the console, not in the Error Viewer.

- *Console*: **SDL Verify Hierarchy Summary** regarding the mismatches appears in the Custom Compiler Console window.
- *Log file*: Information regarding mismatches are also written to the Custom Compiler log file.

Resolving Mismatches between Source and Target Designs

There are three methods for resolving mismatches:

- [Using the Auto Update Command to Resolve Mismatches](#)
- [Using the Error Viewer to Resolve Mismatches](#)
- [Using the Incremental Update Command to Resolve Mismatches](#)

Using the Auto Update Command to Resolve Mismatches

You can use the **Auto Update** command to resolve mismatches.

To run the batch process:

1. Choose **SDL > ECO > Setup**.

The **ECO Setup** window opens.

2. Specify the Verify and Update options that you want to apply. For more information, see [Specifying Error Types and Auto Update Actions](#), [Specifying the Update Options](#), and [Specifying the Verify Options](#).

3. Click **OK** to close the **ECO Setup** form.

4. (Optional) Choose **SDL > ECO > Verify**.

Mismatch errors are highlighted in the design.

5. Choose **SDL > ECO > Auto Update**.

SDL resolves the mismatch errors according to the ECO Setup specifications.

Using the Error Viewer to Resolve Mismatches

You can use the Error Viewer to selectively fix mismatch errors.

To set up the SDL ECO to selectively fix mismatch errors using the Error Viewer:

1. Choose **SDL > ECO > Setup**.

The **ECO Setup** window opens.

2. Specify the Verify and Update options that you want to apply. For more information, see [Specifying Error Types and Auto Update Actions](#), [Specifying the Update Options](#), and [Specifying the Verify Options](#).

3. On the Verify tab, set the **Auto Start Error Viewer** option to **On Warnings or Errors**.

4. Click **OK** to close the **ECO Setup** form.

5. Choose **SDL > ECO > Verify**.

If there are any errors or warnings,

- Error markers are highlighted in the canvas.
- The Error Viewer opens and displays a hierarchical tree listing the error markers.

Note:

With arrayed instances, the **ECO > Verify** command reports an error only in the console, not in the Error Viewer.

6. In the Error Viewer, expand the hierarchical tree to see the details for each error.

7. Select the markers that represent the mismatch errors that you want SDL to resolve.

- The viewport zooms to display the selected markers.
- The **Fix the selected markers** button on the Error Viewer is enabled.

8. Click the **Fix the selected markers** button on the Error Viewer.

SDL automatically resolves the mismatches denoted by the selected set and removes the markers from the hierarchical tree.

Note:

Alternatively, you can right-click on a marker in the canvas and select the **Update** command from the shortcut menu.

Using the Incremental Update Command to Resolve Mismatches

You can use the **Incremental Update** command to interactively fix mismatch errors. The command lets you select individual objects or create selection sets for updating. You can select objects either before or after starting the command, as described in the following sections.

- [Pre-selecting Objects for Incremental Update](#)
- [Post-selecting Objects for Incremental Update](#)

Pre-selecting Objects for Incremental Update

To pre-select objects for incremental updating:

1. Choose **SDL > ECO > Setup**.

The ECO Setup window opens.

2. Specify the Verify and Update options that you want to apply. For more information, see [Specifying Error Types and Auto Update Actions](#), [Specifying the Update Options](#), and [Specifying the Verify Options](#).

The **Incremental Update** command fixes the following mismatch types differently:

- **Unmatched Instances:** If `ECODeleteExtraDevice` is true, deletes instances. Otherwise, the instances are marked as ignored.
- **Unmatched Terminals:** If `ECODeleteExtraDevice` is true, deletes terminals. Otherwise, the terminals are marked as ignored.

3. Click **OK** to close the ECO Setup form.

4. Choose **SDL > ECO > Verify**.

If there are any errors or warnings,

- Error markers are highlighted in the canvas.
- The Error Viewer opens and displays a hierarchical tree listing the error markers.

Note:

With arrayed instances, the **ECO > Verify** command reports an error only in the console, not in the Error Viewer.

5. In the canvas, select the object(s) that you want to update.

6. Choose **SDL > ECO > Incremental Update**.

All errors in the selected set are fixed and the command quits.

Post-selecting Objects for Incremental Update

To post-select objects for incremental updating:

1. Choose **SDL > ECO > Setup**.

The ECO Setup window opens.

2. Specify the Verify and Update options that you want to apply. For more information, see [Specifying Error Types and Auto Update Actions](#), [Specifying the Update Options](#), and [Specifying the Verify Options](#).

The **Incremental Update** command fixes the following mismatch types differently:

- **Unmatched Instances:** If `ECODeleteExtraDevice` is true, delete instances. Otherwise, the instances are marked as ignored.
- **Unmatched Terminals:** If `ECODeleteExtraDevice` is true, delete terminals. Otherwise, the terminals are marked as ignored.

3. Click **OK** to close the ECO Setup window.

4. Choose **SDL > ECO > Verify**.

If there are any errors or warnings,

- Error markers are highlighted in the canvas.
- The Error Viewer opens and displays a hierarchical tree listing the error markers.

Note:

With arrayed instances, the **ECO > Verify** command reports an error only in the console, not in the Error Viewer.

5. Choose **SDL > ECO > Incremental Update**.

6. In the canvas, select the object(s) you want to update.

The error associated with each selected object is fixed and the command remains active.

7. (Optional) Continue to select objects to update.

8. Press Esc to quit the command.

A

Placement and Resistance Constraints

This chapter covers the following topics:

- [Placement Constraints](#)
- [Resistance Constraints](#)

Placement Constraints

Placement results quality is mainly driven by the placement constraints. This appendix describes each of the placement constraints, their parameters, and how they are accessed through the Constraint Editor.

The following sections list all placement constraints supported when [Using the Placement Assistant](#) and the result produced by each constraint:

- [Affinity](#)
- [Alignment](#)
- [Block Size](#)
- [Cluster](#)
- [CMOS](#)
- [Column](#)
- [Edge Assign](#)
- [Enclosing Shape](#)
- [Guard Ring](#)
- [Halo](#)
- [Inst Matching](#)
- [Orientations](#)

- [Pattern Route](#)
- [Placement Grid](#)
- [Resistor Bank](#)
- [Routing Stripes](#)
- [Rows](#)
- [Standard Cells](#)
- [Symmetric](#)

Affinity

The Affinity placement constraint specifies groups of devices that generally attract or repel each other for placement purposes. This is basically gravitational attraction between groups of devices in the physical layout.

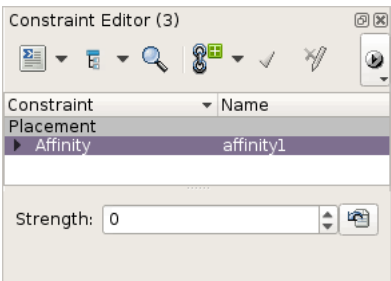
This constraint can be applied to instances or instance groups. The Placement Assistant gives an error if members of this constraint are members of any other placement constraint; that is, all member instances or instance groups must be unconstrained for this constraint to be accepted by the Placement Assistant.

Note:

The affinity of groups of devices to one another can also be influenced by the **Soft Cluster** optimization objective in the [Placement Options Dialog Box](#).

[Figure 140](#) shows the Affinity placement constraint as displayed in the Constraint Editor.

Figure 140 Affinity Placement Constraint



[Table 45](#) describes the parameters for the Affinity placement constraint.

Table 45 Parameters of Affinity Placement Constraint

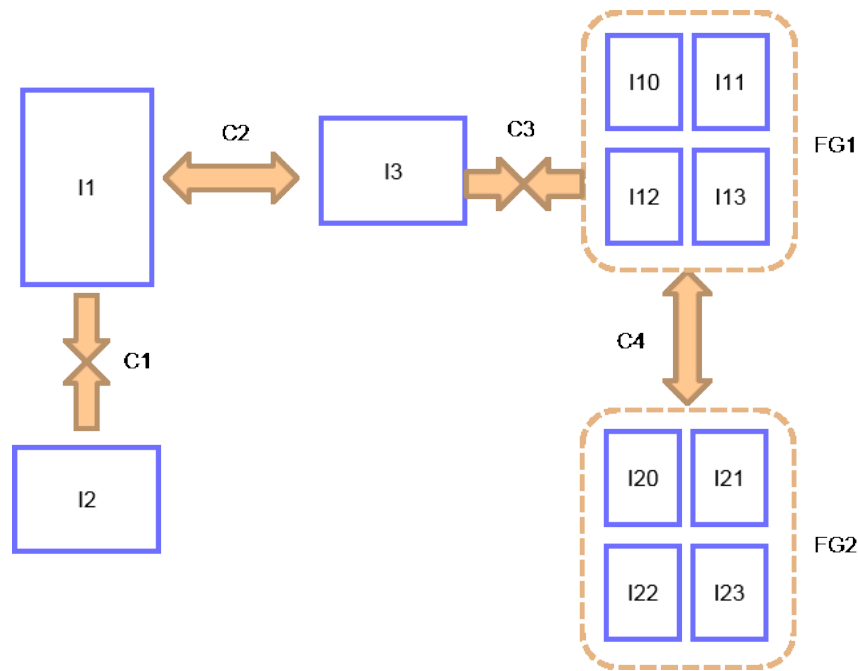
Parameters	Description
Strength	Specifies the gravitational attraction between groups of devices in the physical layout. A positive value defines an attractive force, while a negative value defines a repulsive force.

Example — Affinity Constraint

Figure 141 shows an example of using the Affinity placement constraint to achieve the following:

- C1: Instance I1 is attracted to Instance I2
- C2: Instance I1 is pushed away from Instance I3
- C3: figGroup FG1 is pulled towards Instance I3
- C4: figGroup FG1 is pushed away from figGroup FG2

Figure 141 Example — Affinity Placement Constraint



Alignment

The Alignment placement constraint constrains points coincident on a specific edge or the center point of each bounding box to have the same X or Y value as all other edges or center points from objects in the group for a given set of two or more objects.

This constraint supports oaPinFig and oaFigGroup objects for aligning shapes or FigGroup extents and supports cmlnst (intents) objects allow alignment of instance group extents.

Figure 142 shows the Alignment placement constraint as displayed in the Constraint Editor.

Figure 142 Alignment Placement Constraint

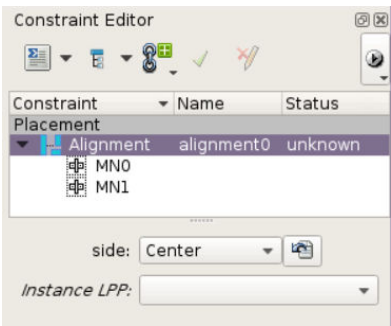


Table 46 describes the parameters for the Alignment placement constraint.

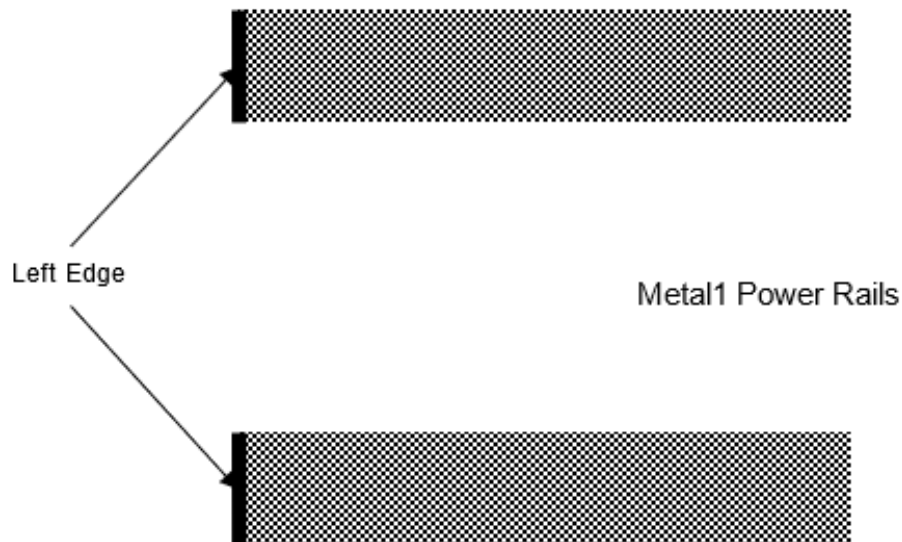
Table 46 Parameters of Alignment Placement Constraint

Parameters	Description
Side	Specifies the alignment of all grouped objects: • Center (default) • Left Edge • Right Edge • Top Edge • Bottom Edge
Instance LPP	Defines the layer purpose pair (LPP) of the enclosing shape.

Example — Alignment Constraint

Figure 143 shows an example of a set of objects to align **Left Edge**.

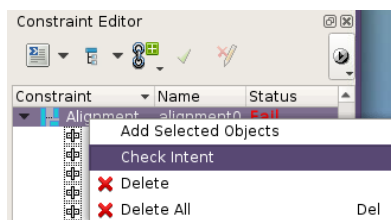
Figure 143 Example — Alignment Placement Constraint



Checking Intent

Figure 144 shows the **Check Intent** option in the menu:

Figure 144 The **Check Intent** Option



To check the alignment:

1. Right click the Alignment placement constraint after specifying the constraint.
2. Choose **Check Intent** to determine whether the BBox of instances or figure groups on the layout meets the settings of alignment constraint: **Side** and **Instance LPP**.

- 3. If the layout matches the Alignment placement constraint, **Pass** is displayed in the **Status** column.
- 4. If the layout does not match the Alignment placement constraint, **Fail** is displayed in the **Status** column. The differences are displayed with error markers added by the checker in the design, and you can view the violations using the Error Viewer.

Block Size

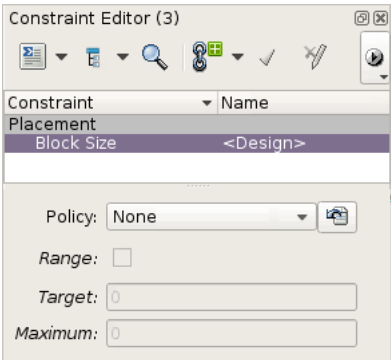
The Block Size placement constraint allows you to give sizing guidance for an entire block. This constraint is applied to the overall design. It cannot be applied to an instance or instance group.

In the [Auto Place Command](#), the precedence is:

- 1. Placement area created by the [Create Placement Area Command](#)
- 2. Block Size placement constraint
- 3. PRBoundary

[Figure 145](#) shows the Block Size placement constraint as displayed in the Constraint Editor.

Figure 145 Block Size Placement Constraint



[Table 47](#) describes the parameters for the Block Size placement constraint.

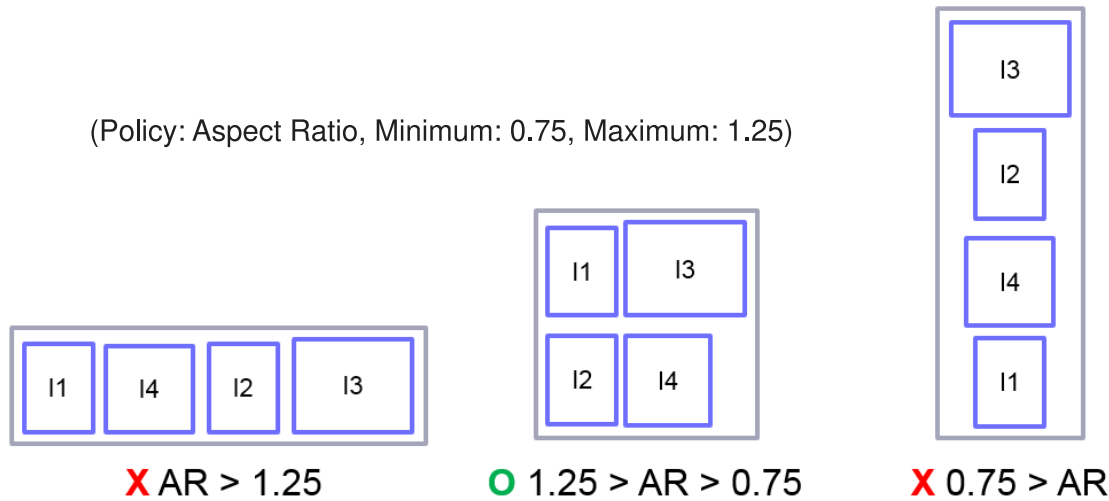
Table 47 Parameters of Block Size Placement Constraint

Parameters	Description
Policy	Defines the sizing of the resultant component: • None (default) • Aspect Ratio • Rows • Columns • Width • Height Note: The Placement Assistant supports only the Aspect Ratio, Width, and Height values. If Policy is not None, and Range is not enabled, you can specify a value for Target. If Policy is not None, and Range is enabled, you can specify values for Minimum and Maximum.

Example — Block Size Constraint

Figure 146 shows an example of using the Block Size placement constraint.

Figure 146 Example — Block Size Placement Constraint



Cluster

The Cluster placement constraint provides support for a collection of instances to be placed as a unit.

Note:

A Cluster placement constraint with only one member has no effect; the Placement Assistant ignores such constraints.

Figure 147 shows the Cluster placement constraint as displayed in the Constraint Editor.

Figure 147 Cluster Placement Constraint

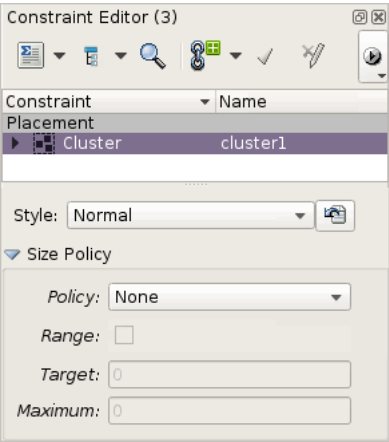


Table 48 describes the parameters for the Cluster placement constraint.

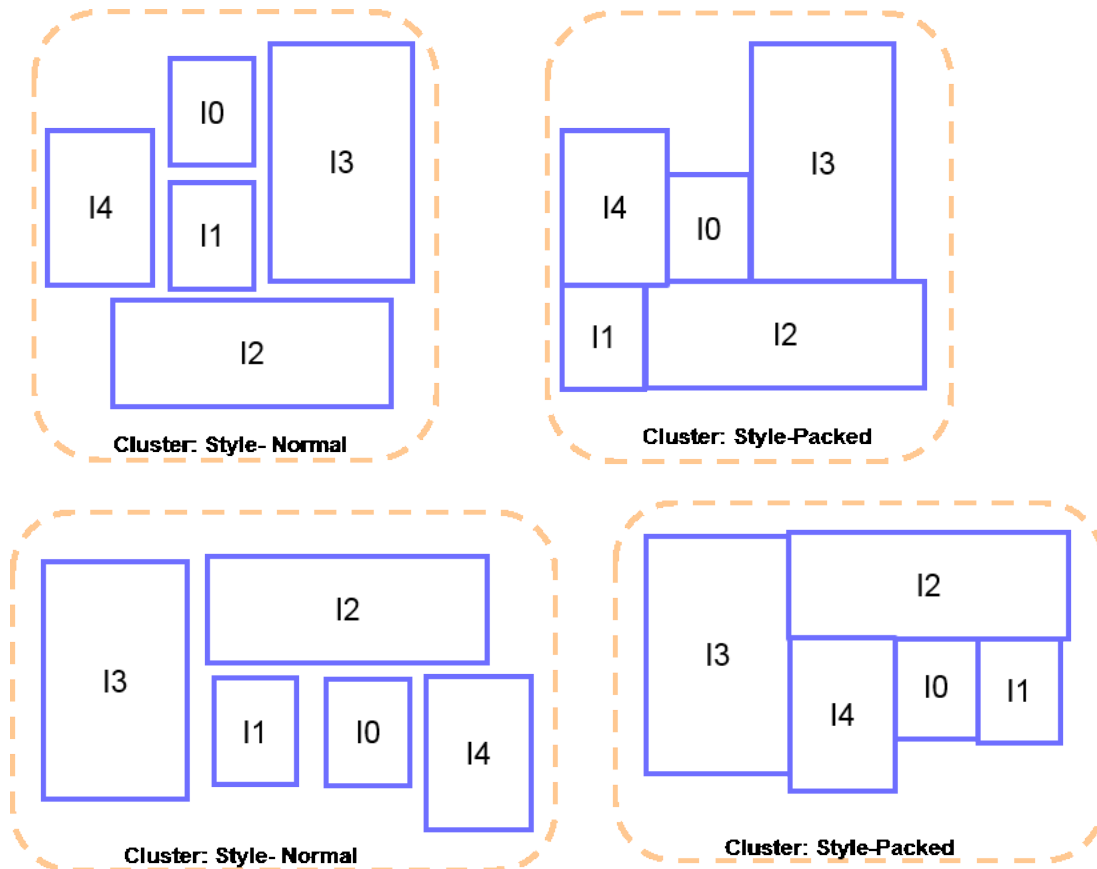
Table 48 Parameters of Cluster Placement Constraint

Parameters	Description
Style	Specifies the type of placement to be done with the associated instances: • Normal (default) • Packed • Symmetric X • Symmetric Y • Symmetric X or Y If set to one of Symmetric X, Symmetric Y, or Symmetric X or Y, then Symmetric placement constraints in the design are processed and their elements are then placed accordingly in the Cluster placement constraint.
Size Policy Settings	
Policy	Defines the sizing of the resultant component: • None (default) • Aspect Ratio • Rows • Columns • Width • Height If Policy is not None, and Range is not enabled, you can specify a value for Target. If Policy is not None, and Range is enabled, you can specify values for Minimum and Maximum. Note: Unused by the Placement Assistant.

Example — Cluster Constraint

Figure 148 shows examples of using the Cluster placement constraint.

Figure 148 Example — Cluster Placement Constraint



CMOS

The CMOS placement constraint is typically used to place custom digital circuits that include complementary PMOS and NMOS devices (CMOS).

The constraint creates PMOS and NMOS rows and aligns them vertically as much as possible. PMOS and NMOS devices can also be abutted automatically in each row.

Note:

A CMOS placement constraint with only one member has no effect; the Placement Assistant ignores such constraints.

Figure 149 shows the CMOS placement constraint as displayed in the Constraint Editor.

Figure 149 CMOS Placement Constraint

Constraint Editor (3)

Constraint Placement

CMOS cmos2

Gate Alignment: Center

Row Pattern: PN

User Pattern:

Default Preset:

Dummy Devices

Right Dummies: 0

Left Dummies: 0

OD Sharing: ☐

Index	Devices
-------	---------

Spacing

Spacing Type: Microns

Different Row: 1

Separation: Microns

Abutment: Abut

Same Row: 1

Size Policy

Policy: Rows

Range: ☐

Target: 2

Maximum: 0

Instances

Index	Instance Name	Fingers	Row Locations
A	M6	0	

Rows

Alignment	Height	Preset	Placement	Dummy Size
Top	0		*^# *^# A+ A + [	m/M m/M
Bottom	0			

Table 49 describes the parameters for the CMOS placement constraint.

Table 49 *Parameters of CMOS Placement Constraint*

Parameters	Description
Gate Alignment	Specifies whether the Placement Assistant must align the gates from complementary MOSFETs between rows: - Center (default) - Left - Right - None
Row Pattern	Defines the pattern of row types: - PN (default) - NP - PNNP - NPPN - custom
User Pattern	Defines a row pattern that you can enter when Row Pattern is custom.
Default Preset	Defines an optional preset cell name.
<i>Dummy Devices Settings</i>	
Right Dummies	Specifies the number of dummies to include on the right. The default is 0.
Left Dummies	Specifies the number of dummies to include on the left. The default is the value of Right Dummies .
OD Sharing	Indicates whether sharing OD is applied to the placed dummy instances.
Index	Index of each Library, Cell, and View (LCV).
Devices	Library, Cell, and View (LCV) strings.
<i>Spacing Settings</i>	
Spacing Type	Specifies the type of value given in the Different Row parameter: - Microns (default) - Tracks The Placement Assistant only honors the placement grid if Spacing Type is Tracks.

Table 49 Parameters of CMOS Placement Constraint (Continued)

Parameters	Description
Different Row	Defines a list of one or more spacing values that apply cyclically to rows. The default is 0, meaning that the Placement Assistant separates NMOS and PMOS devices by the minimum design rule spacing. The Placement Assistant picks only the first value as the distance between P-diffusion and N-diffusion shapes. - You can also use this parameter to change the row pattern, as follows: - By default, P and N rows are arranged from bottom to top as NP-PN-NP and so on. The first device row is N; to switch the first device row to P, set this parameter to 0 -1 1. Note that -1 is the special sign to the Placement Assistant to look for flags. - In advanced nodes, you might want to split NWELLS by alternating P and N devices in each row, for example, NP-NP-NP. To produce such a pattern, set this parameter to 0 -1 0 1. - The following example illustrates the two preceding options used together. Setting this parameter to 0 -1 1 1 produces the pattern PN-PN-PN. Note: The Symbolic Editor with Chaining Mode interprets -1 differently, so you must update this parameter if you intend to use the Symbolic Editor.
Separation	Specifies the separation setting between instances in a row: - Microns - DRC (default) - Poly Pitch
Abutment	Specifies the type of abutment performed on instances in the same row: - None - Abut (default) - With Contacts - Optimal
Same Row	Defines the gap between instances in the same row or the number of pitches if Separation is Poly Pitch. The default is 0, meaning that the gap is defined according to the minimum spacing rules and the applicable grids.
<i>Size Policy Settings</i>	
Policy	Defines sizing of the resultant component: - None (default) - Aspect Ratio - Rows - Width - Height If Policy is not None, and Range is not enabled, you can specify a value for Target. If Policy is not None, and Range is enabled, you can specify values for Minimum and Maximum.
<i>Instances Settings</i>	
Index	Assigned letter and number, if needed.
Instance Name	Name of instance.

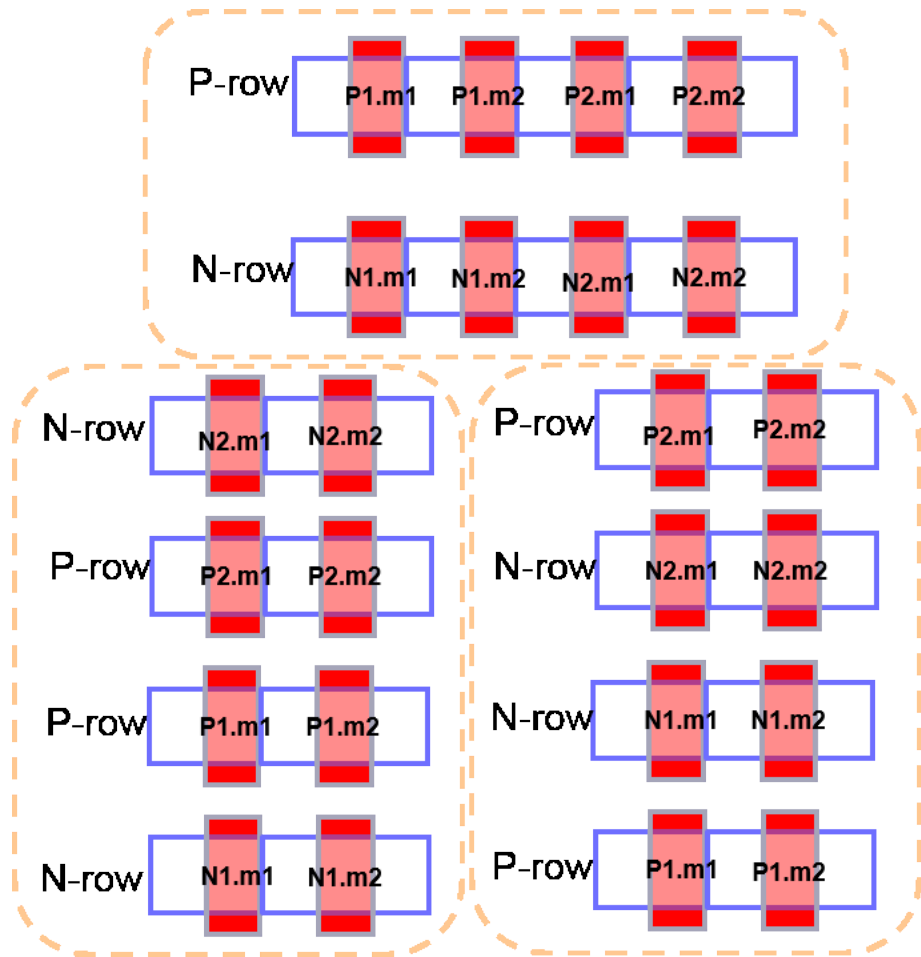
Table 49 *Parameters of CMOS Placement Constraint (Continued)*

Parameters	Description
Fingers	Number of fingers to use for this instance. The default is 0. Note: Unused by the Placement Assistant.
Row Locations	Allows justification in a given row to be specified for the instance.
Rows Settings The Placement Assistant regards the first row as an NMOS row and alternates further between PMOS and NMOS rows. Note: Do not manually update any of the data in <i>Rows Settings</i> .	
Alignment	Specifies the alignment of the devices: • Center (default) • Top • Bottom Alignment of all PMOS rows comes from the first PMOS row. Alignment of all NMOS rows comes from the first NMOS row.
Height	Defines the height of the row. The default is 0, meaning that the Placement Assistant sizes the given row according to the sizes of the devices placed on it and applicable design rules.
Preset	Optional preset name to use by default for this row.
Placement	Explicit placement information. Note: Unused by the Placement Assistant. The Symbolic Editor realizer is used to generate the layout.
Dummy Size	Character or floating number defining the width of dummies in a row.

Example — CMOS Constraint

Figure 150 shows examples of using the CMOS placement constraint.

Figure 150 Example — CMOS Placement Constraint



Column

The Column placement constraint specifies the placement of elements in an ordered column.

Figure 151 shows the Column placement constraint as displayed in the Constraint Editor.

Figure 151 Column Placement Constraint

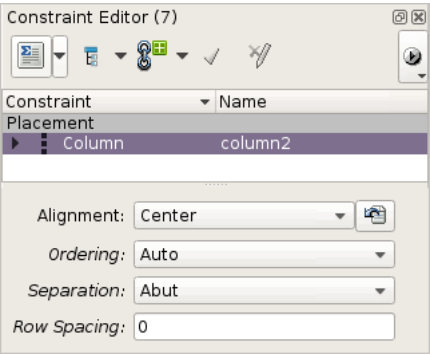


Table 50 describes the parameters for the Column placement constraint.

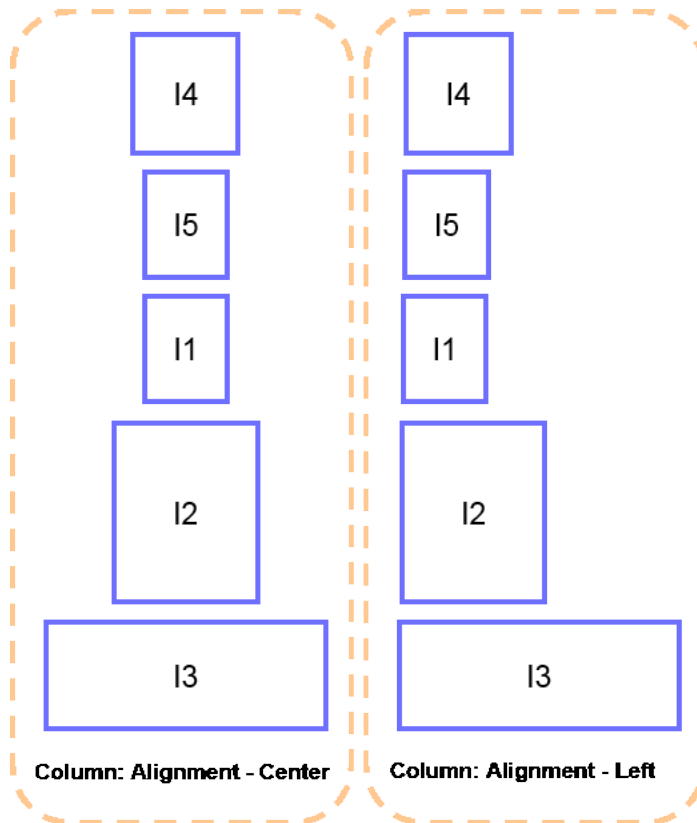
Table 50 Parameters of Column Placement Constraint

Parameters	Description
Alignment	Specifies the horizontal alignment of instances: • Center (default) • Left • Right
Ordering	Specifies whether the order you give should be used or the Placement Assistant can reorder: • As Is • Auto (default)
Separation	Specifies the separation setting between instances of the column: • Abut (default) • Abut Keep Contacts • DRC • Fixed If set to Abut, then instances are abutted.
Row Spacing	If Separation is Fixed, this value defines the spacing between instance boundaries. The default is 0.

Example — Column Constraint

Figure 152 shows examples of using the Column placement constraint.

Figure 152 Example — Column Placement Constraint



Edge Assign

The Edge Assign placement constraint allows you to define a specific edge to be considered a virtual pin location for a specific net.

Note:

This constraint is valid only if associated with a single instance group; that is, it can have only one member and that member must use one of the following placement constraints: [Cluster](#), [CMOS](#), [Column](#), [Rows](#), or [Standard Cells](#). The Edge Assign placement constraint is then applied while placing the members of that instance group.

Figure 153 shows the Edge Assign placement constraint as displayed in the Constraint Editor.

Figure 153 Edge Assign Placement Constraint

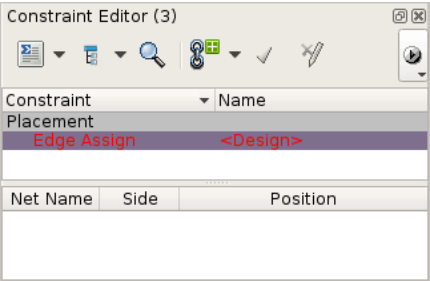


Table 51 describes the parameters for the Edge Assign placement constraint.

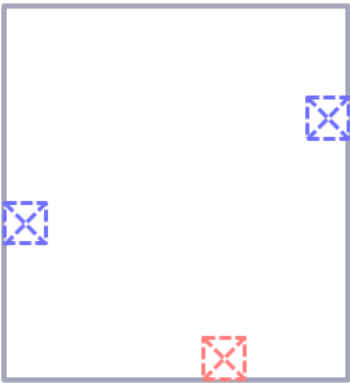
Table 51 Parameters of Edge Assign Placement Constraint

Parameters	Description
Net Name	Name of net for assignment.
Side	Assigned side for a virtual pin.
Position	Relative distance from the left (bottom) corner for the position of a virtual pin on the top/bottom (left/right) side.

Example — Edge Assign Constraint

Figure 154 shows an example of using the Edge Assign placement constraint.

Figure 154 Example — Edge Assign Placement Constraint



Enclosing Shape

The Enclosing Shape placement constraint allows you to define a shape on a specific layer to be added based on the objects associated with the constraint. The shape is defined on a per-side basis.

Note:

This constraint is valid only if associated with a single instance or instance group.

Figure 155 shows the Enclosing Shape placement constraint as displayed in the Constraint Editor.

Figure 155 Enclosing Shape Placement Constraint

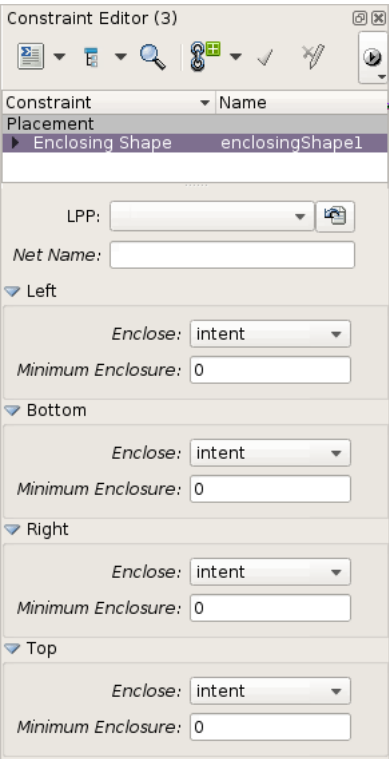


Table 52 describes the parameters for the Enclosing Shape placement constraint.

Table 52 Parameters of Enclosing Shape Placement Constraint

Parameters	Description
LPP	Defines the layer purpose pair (LPP) of the enclosing shape.

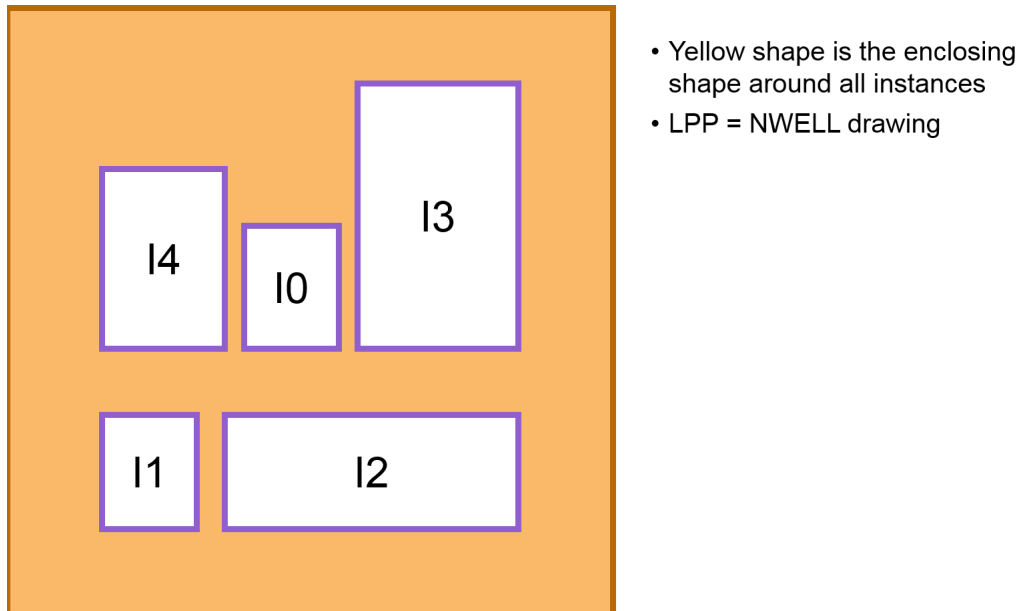
Table 52 *Parameters of Enclosing Shape Placement Constraint (Continued)*

Parameters	Description
Net Name	(Optional) Net name to assign to the enclosing shape. Note: If it is a WELL layer, you must provide a name.
<i>Left Settings</i>	
Enclose	- intent (default): Minimum Enclosure defines the minimum edge overhang on the left side. - layer: Enclosing shape is snapped to shapes contained on the Fill layer.
Minimum Enclosure	Minimum edge overhang on the left side. Applicable if Enclose = intent. The default is 0.
<i>Bottom Settings</i>	
Enclose	- intent (default): Minimum Enclosure defines the minimum edge overhang on the bottom. - layer: Enclosing shape is snapped to shapes contained on the Fill layer.
Minimum Enclosure	Minimum edge overhang on the bottom. Applicable if Enclose = intent. The default is 0.
<i>Right Settings</i>	
Enclose	- intent (default): Minimum Enclosure defines the minimum edge overhang on the right side. - layer: Enclosing shape is snapped to shapes contained on the Fill layer.
Minimum Enclosure	Minimum edge overhang on the right side. Applicable if Enclose = intent. The default is 0.
<i>Top Settings</i>	
Enclose	- intent (default): Minimum Enclosure defines the minimum edge overhang on the top. - layer: Enclosing shape is snapped to shapes contained on the Fill layer.
Minimum Enclosure	Minimum edge overhang on the top. Applicable if Enclose = intent. The default is 0.

Example — Enclosing Shape Constraint

Figure 156 shows an example of using the Enclosing Shape placement constraint.

Figure 156 Example — Enclosing Shape Placement Constraint



Guard Ring

The Guard Ring placement constraint allows you to define a guard ring around an instance or instance group.

Figure 157 shows the Guard Ring placement constraint as displayed in the Constraint Editor.

Figure 157 Guard Ring Placement Constraint

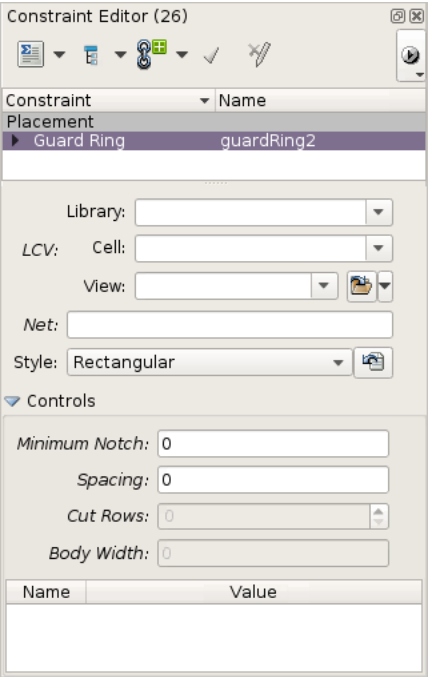


Table 53 describes the parameters for the Guard Ring placement constraint.

Table 53 Parameters of Guard Ring Placement Constraint

Parameters	Description
LCV	Defines the Library, Cell, and View of the object to be used for creating the guard ring.
Net	Defines the net name to assign to the guard ring.
Style	Specifies the style of the guard ring: - Rectangular (default) - Conformal Note: The Placement Assistant supports only the Rectangular value.
Controls Settings	
Note: The Placement Assistant does not support these parameters.	
Minimum Notch	Smallest notch allowed for the body centerline. The default is 0.
Spacing	Additional spacing added between the guard ring boundary and the extents being guarded. The default is 0.
Cut Rows	Number of cut rows to include for MCell guard rings. The default is 0.

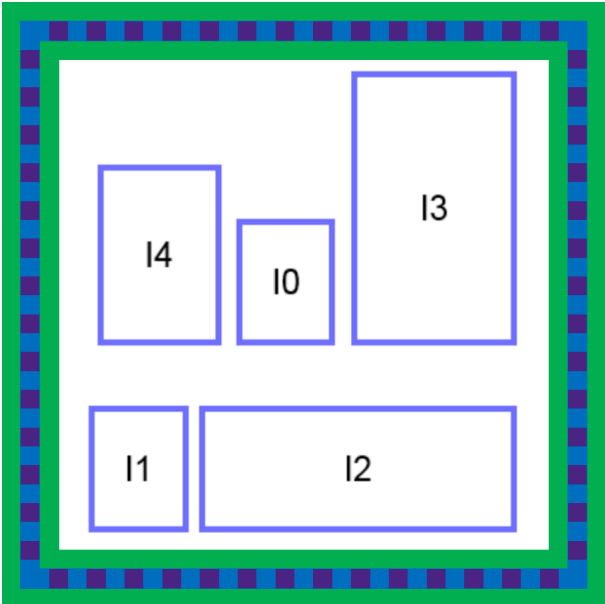
Table 53 Parameters of Guard Ring Placement Constraint (Continued)

Parameters	Description
Body Width	Minimum width of body for MCell guard rings. The default is 0.
Name	Parameter name (paired with Value) to be used by editable PCell guard rings.
Value	Parameter value (paired with Name) to be used by editable PCell guard rings.

Example — Guard Ring Constraint

Figure 158 shows an example of using the Guard Ring placement constraint.

Figure 158 Example — Guard Ring Placement Constraint



Halo

The Halo placement constraint allows you to define placement blockage regions around an instance or instance group. When applied to more than one object, the constraint parameters are simply shared among all objects as opposed to the data being applied to all objects as a single unit.

Figure 159 shows the Halo placement constraint as displayed in the Constraint Editor.

Figure 159 Halo Placement Constraint

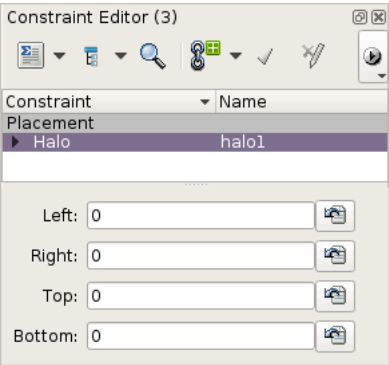


Table 54 describes the parameters for the Halo placement constraint.

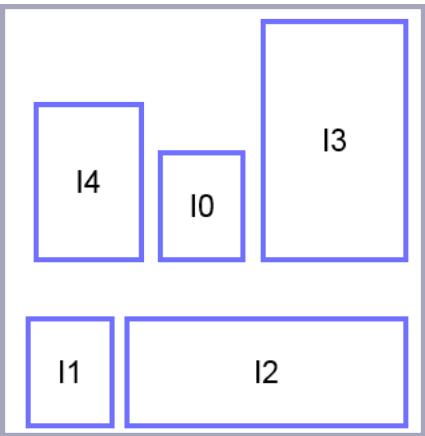
Table 54 Parameters of Halo Placement Constraint

Parameters	Description
Left	Specifies the left offset. The default is 0.
Right	Specifies the right offset. The default is 0.
Top	Specifies the top offset. The default is 0.
Bottom	Specifies the bottom offset. The default is 0.

Example — Halo Constraint

Figure 160 shows an example of using the Halo placement constraint.

Figure 160 Example — Halo Placement Constraint

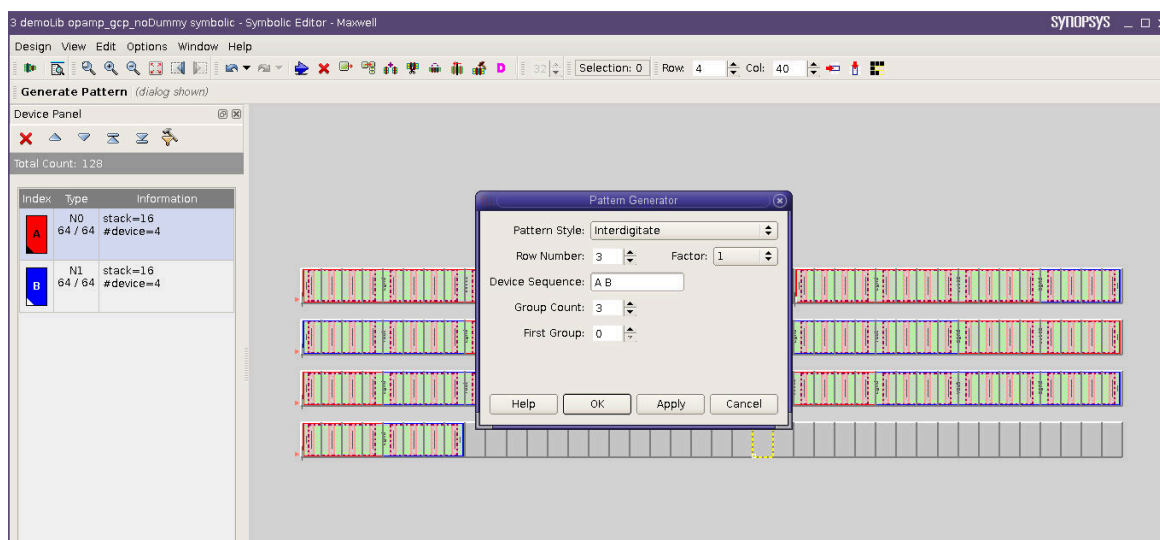


Inst Matching

The Inst Matching placement constraint specifies a set of devices to be placed using a specific pattern. The constraint allows you to specify how the devices are placed. You are also provided with a set of predefined patterns which can be autogenerated in the tool.

The default creator of this constraint is the Symbolic Editor (see For Matching Mode for more details) which uses the following procedure:

1. Select the devices in the Design Navigator (DN-Schematic).
2. Click  **Symbolic Editor with Matching Mode** to open the Symbolic Editor.
3. In the Symbolic Editor, choose **Edit > Generate Pattern** to open the **Pattern Generator** dialog box.



4. After you enter a pattern style and its associated parameters, click **Apply** to view the generated pattern. See Generating Patterns (Matching Mode Only) for detailed information about pattern styles and their respective parameters.
5. After you are satisfied with the generated pattern, choose **Design > Save Constraint** to save the constraint. The Placement Assistant then reads the parameters from the constraint and generates the placement accordingly.

Note:

If the Symbolic Editor is not used for creating the constraint — that is, if you have directly created the constraint in the Constraint Editor — then the parameters associated with pattern generators are read by the Placement Assistant and the pattern is automatically generated during automatic

Appendix A: Placement and Resistance Constraints

Placement Constraints

placement. It is, however, *strongly* recommended that the Symbolic Editor be used for generating the Inst Matching constraint.

Figure 161 shows the Inst Matching placement constraint as displayed in the Constraint Editor.

Figure 161 Inst Matching Placement Constraint

Constraint Editor (4)

Constraint: Placement

Inst Matching instMatching1

Style: Device Group

Pattern:

Row Orientations: None

Default Preset:

Size Policy

Policy: None

Range:

Target: 0

Maximum: 0

Pattern Control

Group Count: 1

First Group: 0

Chain Style:

Start Instance: A

Spacing

Spacing Type: Microns

Different Row: 0

Separation: DRC

Abutment: Abut

Same Row: 0

Dummy Devices

Left Dummies: 0

Right Dummies: 0

Top Dummies: 0

Bottom Dummies: 0

Dummy Net:

Dummy Sizes:

OD Sharing:

Index	Devices
A	M6
B	M7
C	M8
D	M9
E	M10

Instances

Index	Instance Name	Fingers	Split
A	M6	0	
B	M7	0	
C	M8	0	
D	M9	0	
E	M10	0	

Explicit Placement

Alignment	Placement	Dummy Size
Bottom	A# B# C# D# E+ E # B# A+ A # B#	
Bottom	- - *# E+ E # B# A+ A # E + E# A+	m/M m/M
Bottom	- - *# - - *# E# A# - - *# - - *# *	m/M m/M m/M m/M

Table 55 describes the parameters for the Inst Matching placement constraint.

Table 55 Parameters of Inst Matching Placement Constraint

Parameters	Description
Style	Specifies the type of Pattern to use: • Common Centroid • Central Symmetric • Interdigitate • Device Group (default) • Defined Pattern • Custom • Text Pattern • Balanced Pattern • Series Chain If the constraint is not created through the Symbolic Editor, you must set this field in the Constraint Editor. If Style is Custom, then instances are placed according to the Device Group pattern style.
Pattern	Defines a logical pattern to apply to instances if Style is one of Central Symmetric, Interdigitate, Defined Pattern, or Text Pattern. Note: Defined Pattern and Text Pattern are unused by the Placement Assistant.
Row Orientations	• Defines the default row orientations: • None (default) • Flipped • Even Flipped • Odd Flipped
Default Preset	Defines an optional preset cell name.
<i>Size Policy Settings</i>	
Policy	Used by some of the pattern styles for generating patterns: • None (default) • Rows • Columns If Policy is not None, and Range is not enabled, you can specify a value for Target. If Policy is not None, and Range is enabled, you can specify values for Minimum and Maximum. The Placement Assistant uses only the Rows value to create patterns. By default, Target mode is set, which creates a single pattern with the number of rows specified. The Range parameter can be set to create multiple patterns with a number of rows ranging between the minimum and maximum values specified. The number of rows for each pattern is a factor of the total number of contained devices. The number of patterns generated has a value less than or equal to the number of factors of the total number of contained instances. The Central Symmetric style depends on the number of contained instances and is independent of the Policy parameter. Only one pattern is created for this pattern style. If Policy is set to None, then multiple patterns are created with the number of rows ranging from 1 to the number of instances contained.

Pattern Control Settings

Table 55 *Parameters of Inst Matching Placement Constraint (Continued)*

Parameters	Description
Group Count	Number of fingers to repeat in a single group. The default is 1.
First Group	Number of fingers to use in first group. The default is 0.
Chain Style	Specifies the chaining style: ☒ None  S1 Upper Left Horizontal Snake  S3 Upper Right Horizontal Snake  S5 Upper Left Vertical Snake  S7 Upper Right Vertical Snake  S2 Lower Left Horizontal Snake  S4 Lower Right Horizontal Snake  S6 Lower Left Vertical Snake  S8 Lower Right Vertical Snake  Z1 Upper Left Horizontal Zigzag  Z3 Upper Right Horizontal Zigzag  Z5 Upper Left Vertical Zigzag  Z7 Upper Right Vertical Zigzag  Z2 Lower Left Horizontal Zigzag  Z4 Lower Right Horizontal Zigzag  Z6 Lower Left Vertical Zigzag  Z8 Lower Right Vertical Zigzag
Start Instance	Defines the starting instance letter assignment for the chaining style. The default is A.
Spacing Settings	
Spacing Type	Specifies the type of value given in the Different Row parameter: • Microns (default) • Tracks
Different Row	Defines a list of one or more spacing values that apply cyclically to rows (bottom-up, including dummy rows). The default is 0. • If Spacing Type is Microns, then this parameter specifies spacing in microns between rows. • If Spacing Type is Tracks, then spacing between rows is according to the number of tracks specified. The size of tracks is according to fin grids.
Separation	Specifies the separation between instances in a row: • Microns • DRC (default) • Poly Pitch

Table 55 *Parameters of Inst Matching Placement Constraint (Continued)*

Parameters	Description
Abutment	Specifies the type of abutment performed on instances in the same row: • None • Abut (default) • With Contacts • Optimal Note: The abutment information in the <i>Explicit Placement</i> settings supersedes this value. That is, if Abutment is Abut, but a pair of instances is not set to Abut in <i>Explicit Placement</i>, then those are not abutted by the Placement Assistant.
Same Row	Defines the spacing between instances in the same row or the number of pitches if Separation is Poly Pitch. The default is 0.
<i>Dummy Devices Settings</i>	
Left Dummies	Specifies the number of dummies on the left. The default is 0.
Right Dummies	Specifies the number of dummies on the right. The default is 0.
Top Dummies	Specifies the number of dummies on the top. The default is 0. Note: The Placement Assistant creates only one dummy row if the value is greater than or equal to 1.
Bottom Dummies	Specifies the number of dummies on the bottom. The default is 0. Note: The Placement Assistant creates only one dummy row if the value is greater than or equal to 1.
Dummy Net	Net to connect dummy device terminals.
Dummy Sizes	Specifies 0, 1, 2, or 4 values of the format "<W>/<L>", where <W> is width and <L> is length units. W/L value is a float greater than 0.0f or m M character.
OD Sharing	Indicates whether sharing OD is applied to the placed dummy instances.
Index	Index of each Library, Cell, and View (LCV).
Devices	Library, Cell, and View (LCV) strings.
<i>Instances Settings</i>	
Index	Consecutive letter assigned to instance.
Instance Name	Name of instance.
Fingers	Number of fingers to use for this instance. The default is 0. Note: Unused by the Placement Assistant.
Split	Indicates whether the instance should be split into multiple instances, one per finger. Note: Unused by the Placement Assistant.

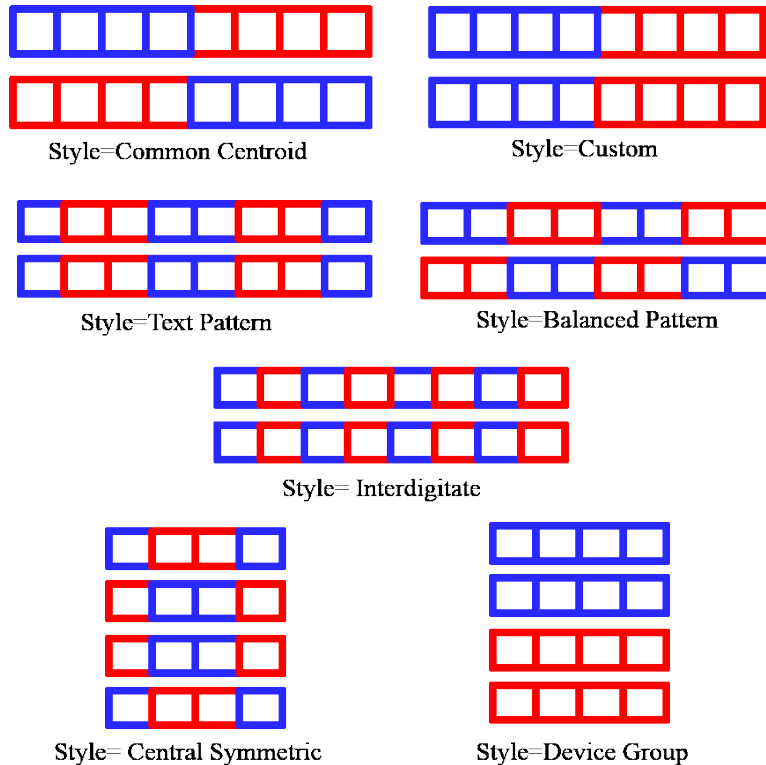
Table 55 Parameters of Inst Matching Placement Constraint (Continued)

Parameters	Description
<i>Explicit Placement</i> Settings	
If <i>Explicit Placement</i> is empty and Style is Custom or Defined Pattern, then the Placement Assistant generates multiple patterns using the pattern style Device Group and the number of rows is according to the Policy parameter.	
Note:	
Do not manually update any of the data in <i>Explicit Placement</i> settings.	
Alignment	Specifies the alignment of the devices: • Center (default) • Top • Bottom The Placement Assistant uses single alignment, so the value is picked from the first row.
Placement	Explicit placement information.
Dummy Size	Character or floating number defining the width of dummies in a row.

Example — Inst Matching Constraint

Figure 162 shows examples of patterns used by the Inst Matching placement constraint.

Figure 162 Example — Inst Matching Placement Constraint



Orientations

The Orientations placement constraint lets you limit the orientations allowed to be used on the instances or instance groups.

When applied to more than one object, the constraint parameters are simply shared among all objects as opposed to the data being applied to all objects as a single unit.

Figure 163 shows the Orientations placement constraint as displayed in the Constraint Editor.

Figure 163 Orientations Placement Constraint

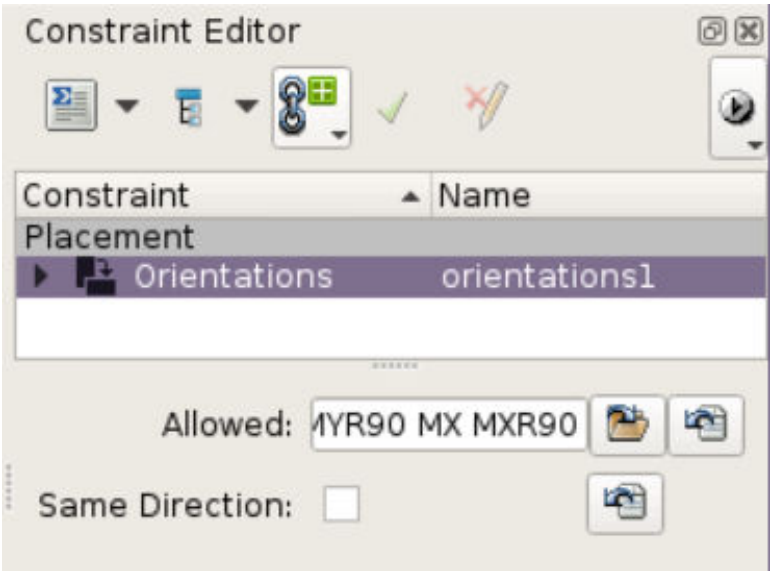
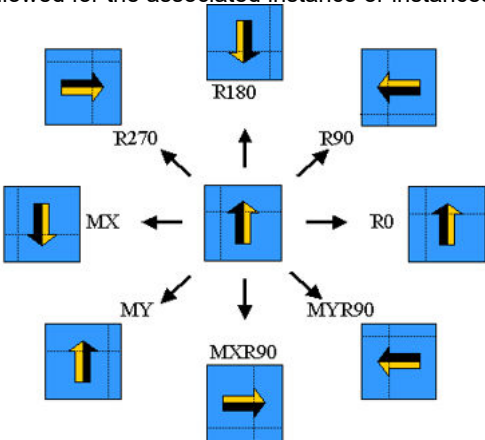


Table 56 describes the parameters for the Orientations placement constraint.

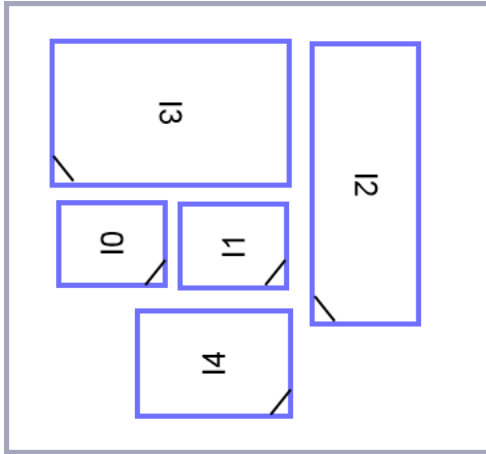
Table 56 Parameters of Orientations Placement Constraint

Parameters	Description
Allowed	List of orientation abbreviations (R0, R90, R180, R270, MX, MY, MXR90, and MYR90) allowed for the associated instance or instances. 
Same Direction	The parameter indicates that all instances are placed same direction. The default is false.

Example — Orientations Constraint

Figure 164 shows an example of using the Orientations placement constraint.

Figure 164 Example — Orientations Placement Constraint



Pattern Route

The Pattern Route Constraint is documented in the *Custom Compiler Custom Router User Guide*.

Placement Grid

The Placement Grid placement constraint can be applied to an instance or instance group or on the overall block. It defines the grid(s) on which the member elements can be placed in the design.

When applied to more than one object, the constraint parameters are simply shared among all objects as opposed to the data being applied to all objects as a single unit.

When applied on the overall block, the Constraint Editor shows <Design> in the Name column. In this case, the grid applies only when the block is placed in a parent cell.

Figure 165 shows the Placement Grid placement constraint as displayed in the Constraint Editor.

Figure 165 Placement Grid Placement Constraint

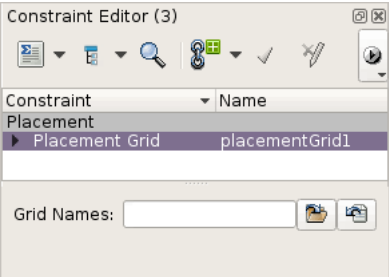
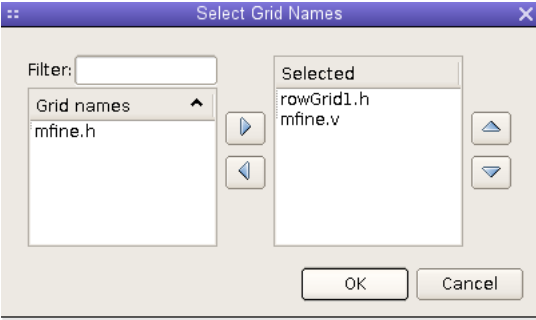


Table 57 describes the parameter for the Placement Grid placement constraint.

Table 57 Parameters of Placement Grid Placement Constraint

Parameters	Description
Grid Names	Defines a space-separated list of track group definition (placement grid) names. Enter the list of grid names directly in the field or click Select  to open the Select Grid Names dialog box and select them from the list.  Note: This dialog box displays only placement grids that are in global track regions; it filters out local track regions and routing grids.

Placement grids are track groups on the instance layer with one or more tracks. Placement grids can be defined in the technology or in the design data. For more information on instance tracks and track groups, see the Track Region Assistant in the *Custom Compiler Environment User Guide*.

In a gridded flow, instances are expected to be placed on a grid as defined by Placement Grid constraints. Instances with a smaller layout, in general, require a finer grid, while instances (or instance groups) with a larger layout require a coarser grid.

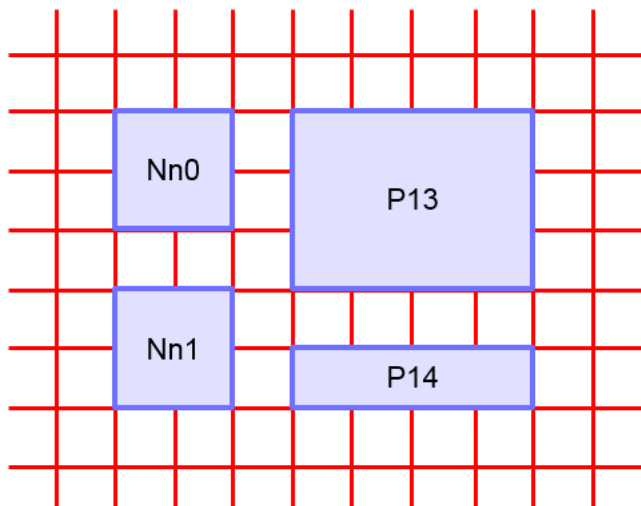
For larger instances (that is, blocks or cells as opposed to devices), the Placement Assistant picks the first instance track as the block (or cell) placement grid and ignores the other instance tracks, if any.

In the latest FinFET nodes, even primitive devices require a Placement Grid constraint. The Placement Assistant supports placement grids with multiple instance tracks in the following constraints: [Column](#), [Inst Matching](#), and [Rows](#).

Example — Placement Grid Constraint

[Figure 166](#) shows an example of using the Placement Grid placement constraint.

Figure 166 Example — Placement Grid Placement Constraint



Resistor Bank

The Resistor Bank placement constraint helps you to place resistors by breaking up each resistor device into discrete units.

Figure 167 shows the Resistor Bank placement constraint as displayed in the Constraint Editor.

Figure 167 Resistor Bank Placement Constraint

Constraint Editor (4)

Constraint Placement

Resistor Bank resistorBank1

Unit Resistance: 0

Unit Height: 0

Unit Length: 0

Unit Tolerance: 0

Default Preset:

Pattern Control

Chain Style:

Start Instance: A

Size Policy

Policy: None

Range:

Target: 0

Maximum: 0

Spacing

Spacing Type: Microns

Row Spacings: 0

Default Separation: Abut

Gap: 0

Dummy Devices

Index	Devices
-------	---------

Instances

Index	Instance Name	Topology
A	Pcas1	
B	Pcas0	
C	Ptail0	
D	Ptail1	

Explicit Placement

Alignment	Placement	Dummy Size
-----------	-----------	------------

Table 58 describes the parameters for the Resistor Bank placement constraint.

Table 58 Parameters of Resistor Bank Placement Constraint

Parameters	Description
Unit Resistance	Defines the resistance value for individual series resistors. The default is 0.
Unit Height	Defines the height of individual series resistors. The default is 0.
Unit Length	Defines the length of individual series resistors. The default is 0.
Unit Tolerance	Specifies the percentage value defining when to truncate the remainder. The default is 0.
Default Preset	Defines an optional preset cell name.

Pattern Control Settings

Chain Style Specifies the chaining style:

☒	None	
	S1	Upper Left Horizontal Snake
	S3	Upper Right Horizontal Snake
	S5	Upper Left Vertical Snake
	S7	Upper Right Vertical Snake
	S2	Lower Left Horizontal Snake
	S4	Lower Right Horizontal Snake
	S6	Lower Left Vertical Snake
	S8	Lower Right Vertical Snake
	Z1	Upper Left Horizontal Zigzag
	Z3	Upper Right Horizontal Zigzag
	Z5	Upper Left Vertical Zigzag
	Z7	Upper Right Vertical Zigzag
	Z2	Lower Left Horizontal Zigzag
	Z4	Lower Right Horizontal Zigzag
	Z6	Lower Left Vertical Zigzag
	Z8	Lower Right Vertical Zigzag

Start Instance Defines the starting instance letter assignment for the chaining style. The default is A.

Size Policy Settings

Policy Defines the number of rows in the resulting component:

- None (default)
- Aspect Ratio
- Rows
- Columns
- Width
- Height

If **Policy** is not None, and **Range** is not enabled, you can specify a value for **Target**.
If **Policy** is not None, and **Range** is enabled, you can specify values for **Minimum** and **Maximum**.

Spacing Settings

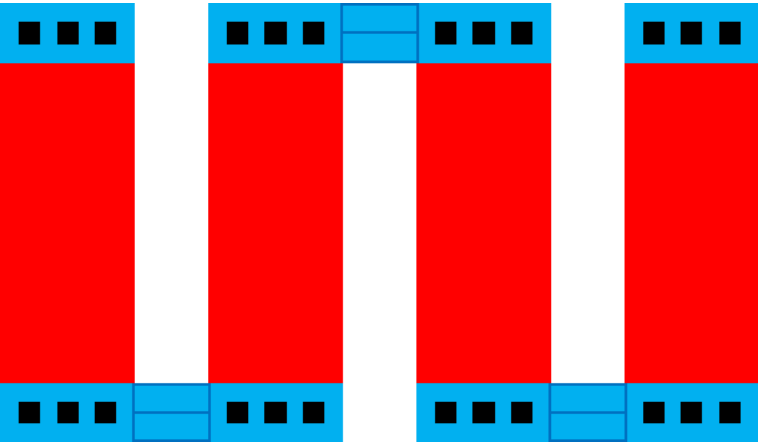
Table 58 *Parameters of Resistor Bank Placement Constraint (Continued)*

Parameters	Description
Spacing Type	Specifies the type of Row Spacings parameter: • Microns (default) • Tracks
Row Spacings	Defines a list of one or more spacing values that apply cyclically to rows. The default is 0.
Default Separation	Defines the separation setting between instances in a row: • Abut (default) • Abut Keep Contacts • DRC • Fixed
Gap	Specifies the gap between instances in the same row. The default is 0.
<i>Dummy Devices Settings</i>	
Index	Index of each Library, Cell, and View (LCV).
Devices	Library, Cell, and View (LCV) strings.
<i>Instances Settings</i>	
Index	Consecutive letter assigned to the instance.
Instance Name	Name of the instance.
Topology	Optional setting defining the series and parallel unit resistors to use to represent the instance's resistance.
<i>Explicit Placement Settings</i>	
If <i>Explicit Placement</i> is empty, then the Placement Assistant generates multiple patterns using the pattern style Device Group and the number of rows is according to the Policy parameter.	
Note: Do not manually update any of the data in <i>Explicit Placement</i> settings.	
Alignment	Specifies the alignment of the devices: • Center (default) • Top • Bottom Note: The Placement Assistant uses single alignment, so the value is picked from first row.
Placement	Explicit placement information.
Dummy Size	String or floating number defining the characteristics of dummies in a row. Use a space-separated list of values to control each dummy in order.

Example — Resistor Bank Constraint

Figure 168 shows an example of using the Resistor Bank placement constraint.

Figure 168 Example — Resistor Bank Placement Constraint



Routing Stripes

The Routing Stripes placement constraint allows you to define a set of one or more stripes to be generated along the periphery of the boundary defined by the associated instance group.

For each stripe, the net, layer, width, and offset can be defined. The stripes are ordered to control their generation from the boundary in either an outward or an inward direction.

Figure 169 shows the Routing Stripes placement constraint as displayed in the Constraint Editor.

Figure 169 Routing Stripes Placement Constraint

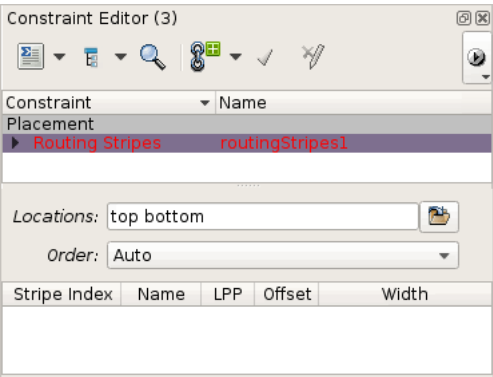


Table 59 describes the parameters for the Routing Stripes placement constraint.

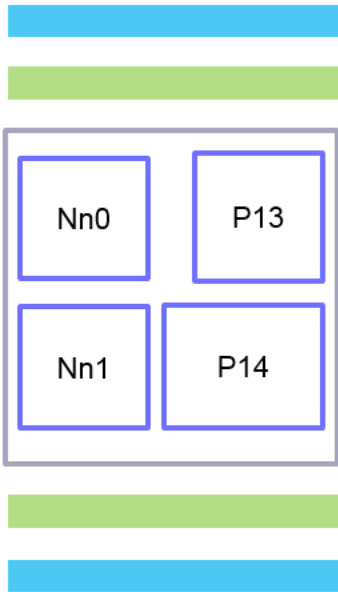
Table 59 Parameters of Routing Stripes Placement Constraint

Parameters	Description
Locations	Defines a space-separated list of locations (top, bottom, left, and right) where routing stripes are required.
Order	• Auto (default) • Inward • Outward
Per Net Settings	
Stripe Index	Consecutive letter assigned to net. Note: Unused by the Placement Assistant.
Name	Net name for a stripe.
LPP	Layer purpose pair (LPP) for a stripe.
Offset	Additional spacing added to the minimum spacing required.
Width	Width of a stripe.

Example — Routing Stripes Constraint

Figure 170 shows an example of using the Routing Stripes placement constraint.

Figure 170 Example — Routing Stripes Placement Constraint



Rows

The Rows placement constraint specifies placement of elements in an ordered row or rows.

Figure 171 shows the Rows placement constraint as displayed in the Constraint Editor.

Figure 171 Rows Placement Constraint

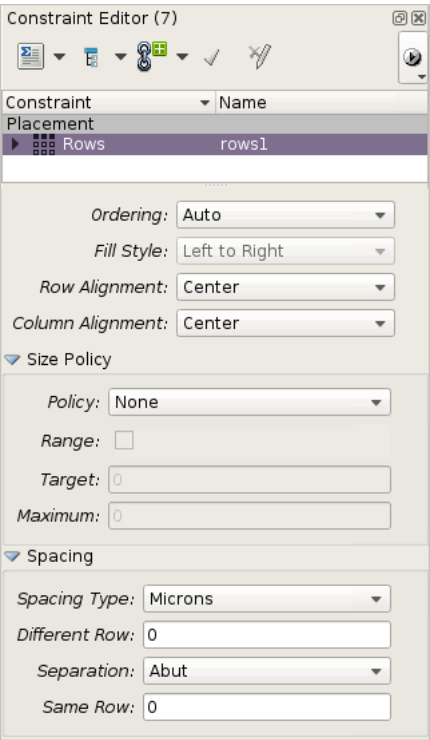


Table 60 describes the parameters for the Row placement constraint.

Table 60 Parameters of Rows Placement Constraint

Parameters	Description
Ordering	Specifies whether the order you give should be used or the Placement Assistant can reorder. • As Is • Auto (default)

Table 60 *Parameters of Rows Placement Constraint (Continued)*

Parameters	Description
Fill Style	Specifies how the ordered set of instances are placed: • Optimal • Left to Right (default) • Right to Left • Left then Right • Right then Left • Top to Bottom • Bottom to Top • Top then Bottom • Bottom then Top Note: Used only if Rows > 1 or = 0.
Row Alignment	Specifies the vertical alignment of instances: • Top • Bottom • Center (default)
Column Alignment	Specifies how columns are aligned across rows: • Center (default) • Left • Right Note: Used only if Rows > 1 or = 0.
<i>Size Policy Settings</i>	
Policy	Defines the number of rows in the resulting component: • None (default) • Aspect Ratio • Rows • Columns • Width • Height If Policy is not None, and Range is not enabled, you can specify a value for Target. If Policy is not None, and Range is enabled, you can specify values for Minimum and Maximum. Note: Only the Rows value is used for deciding the number of rows in the final placement.
<i>Spacing Settings</i>	
Spacing Type	Specifies the type of Different Row parameter: • Microns (default) • Tracks Note: Used only if Rows > 1 or = 0.
Different Row	List of one or more spacing values that apply cyclically to rows (bottom-up, including dummy rows). The default is 0, meaning that the Placement Assistant spaces rows according to applicable design rules.

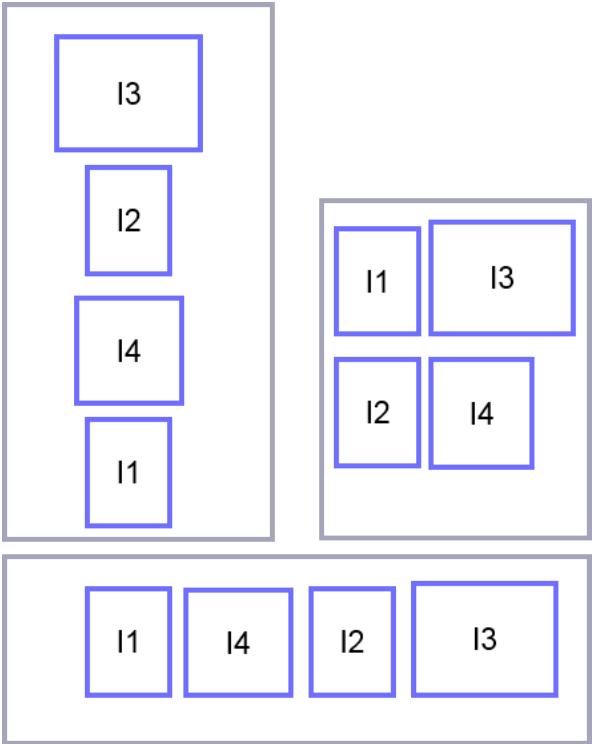
Table 60 Parameters of Rows Placement Constraint (Continued)

Parameters	Description
Separation	Specifies the separation between instances in a row: • Abut (default) • Abut Keep Contacts • DRC • Fixed
Same Row	Defines the spacing between instance boundaries. The default is 0.

Example — Rows Constraint

Figure 172 shows examples of using the Rows placement constraint.

Figure 172 Example — Rows Placement Constraint



Standard Cells

The Standard Cells placement constraint specifies that all of the elements be placed in one or more rows, using a standard-cell placement algorithm.

Standard cells are typically used to implement control logic or other types of digital functions that do not have the same type of critical performance or timing requirement as usually found in analog circuits.

Note:

This constraint supports cells with a height that is two times the standard cell height. In fact, cells of any height (5x, 3x, 2x, 1.5x, even 1.23x) can be mixed to get a quick placement where the PRBoundary of each cell is abutted to others.

In contrast to other placement constraints, the contents of a cell are not checked for DRC when placing two cells next to each other.

The cells or their instances can also be constrained with [Placement Grid](#) constraints to achieve gridded placement for advanced nodes.

Figure 173 shows the Standard Cells placement constraint as displayed in the Constraint Editor.

Figure 173 Standard Cells Placement Constraint

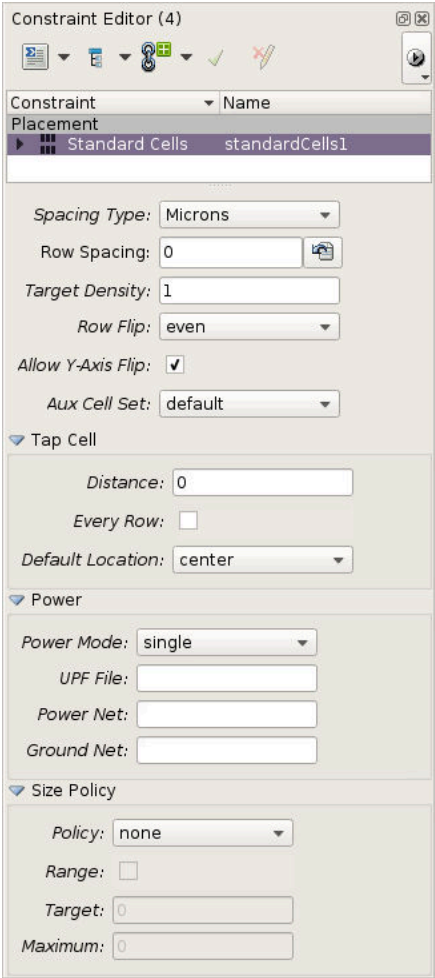


Table 61 describes the parameter for the Standard Cells placement constraint.

Table 61 Parameters of Standard Cells Placement Constraint

Parameters	Description
Spacing Type	Specifies the type of Row Spacing parameter: • Microns (default) • Tracks
Row Spacing	Defines the separation of rows. The default is 0, which means to abut PRBoundaries.
Target Density	Defines the placement density target. The default is 1.

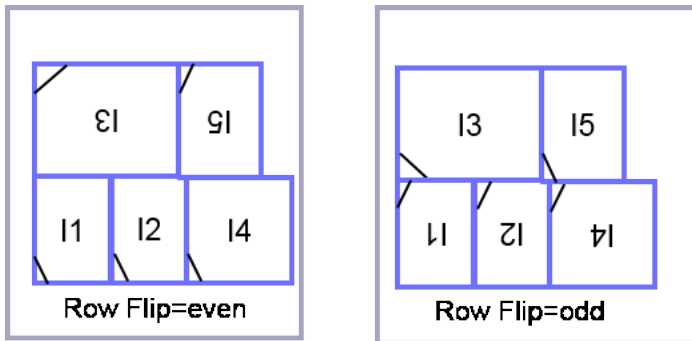
Table 61 *Parameters of Standard Cells Placement Constraint (Continued)*

Parameters	Description
Row Flip	Defines how a pair of rows are flipped: • none • even (default) • odd • all
Allow Y-Axis Flip	Specifies whether instances can be flipped about the y-axis in a row.
Aux Cell Set	If given, defines the name of the auxiliary cell set to use. CDF property <code>auxCellConnectivityMap</code> defines available cell sets. Tap cells, fill cells, and end-cap cells information is stored in such a cell set.
<i>Tap Cell Settings</i>	
Distance	Defines the tap cell distance. The default is 0.
Every Row	Specifies whether a tap cell is needed in every row.
Default Location	Specifies the default location for tap cells in a row: • left • right • center (default)
<i>Power Settings</i>	
Power Mode	Specifies the type of power supply to use: • single (default) • UPF Note: Unused by the Placement Assistant.
UPF File	Specifies the name of a UPF file. Note: Unused by the Placement Assistant.
Power Net	Specifies the name of a single supply power net.
Ground Net	Specifies the name of a single supply ground net.
<i>Size Policy Settings</i>	
Policy	Defines the number of rows in the resulting component: • none (default) • aspectRatio • rows • columns • width • height If Policy is not None, and Range is not enabled, you can specify a value for Target . If Policy is not None, and Range is enabled, you can specify values for Minimum and Maximum . Note: Only the rows value is used for deciding the number of rows in the final placement.

Example — Standard Cells Constraint

Figure 174 shows examples of using the Standard Cells placement constraint.

Figure 174 Example — Standard Cells Placement Constraint



Symmetric

The Symmetric placement constraint allows you to define a single instance or a pair of instances as being symmetric with respect to an arbitrary symmetric axis.

In the case of a single instance, the member is placed to center align to the symmetry axis. This is called a *self-symmetric* instance.

In the case of a pair of instances, the two members are placed in a mirror symmetric configuration. This is called a *symmetric pair*.

All symmetric instances contained in one [Cluster](#) are symmetric across a single symmetry axis.

Multiple [Cluster](#) constraints can be created to get a layout with multiple symmetry axes.

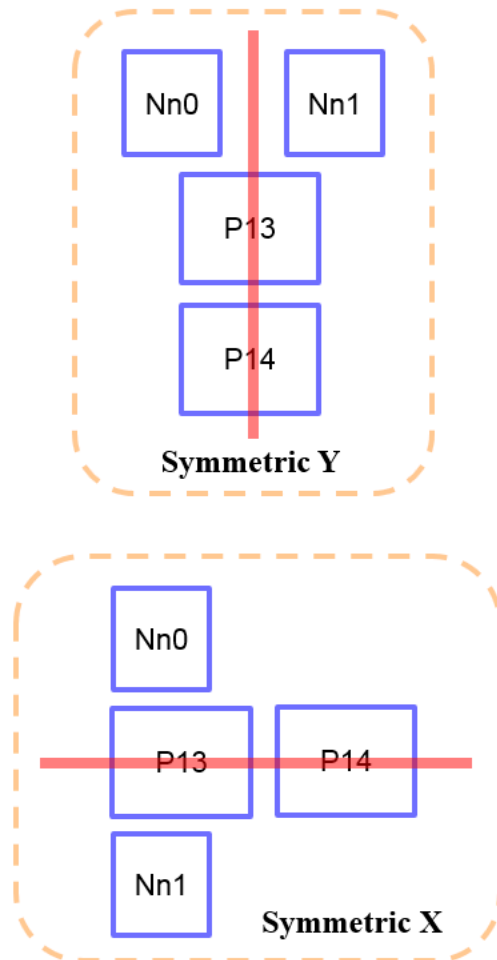
Note:

The existence of this constraint is only of value if the associated instances are also direct members of a [Cluster](#) constraint whose **Style** is set to Symmetric X, Symmetric Y, or Symmetric X or Y.

Example — Symmetric Constraint

Figure 175 shows examples of using the Symmetric placement constraint.

Figure 175 Example — Symmetric Placement Constraint



Resistance Constraints

For information about the Resistance constraint, see Setting Resistance Constraints.

B

Setting up a Foundry Tool Kit for SDL

This chapter describes details of how logical to physical mapping can be established and how mapping is carried out in SDL.

This chapter covers the following topic:

- [Overview of Logical to Physical Mapping](#)
- [About IxComponentType](#)

Overview of Logical to Physical Mapping

This section explores the various methods by which logical to physical mapping is established in SDL.

Basic View Switch / Stop Lists

When there is an existing design containing identical library, cell, and view information, logical to physical mapping is established by a defined process. For more information, see Hierarchy Editor Tool in the *Custom Compiler Environment User Guide*.

The exceptions are the following:

- IxUseCell
- IxIgnoreNames and IxShortNames
- IxUseCell = "cellname"

Treat it exactly as if there had been an instance scoped cell name selection

- IxUseCell = "libname cellname"

Treat it exactly as if there had been instance scoped selections for lib and cell name

- IxUseCell = "libname cellname viewname"

Treat it exactly as if there had been instance scoped selections for lib, cell, and view name

If viewname is not on switch view then stop. If viewname is on switch view and not on stop, then switch.

Library Search List

Often the logical symbols or schematics for a design are placed in a different library than the physical layouts. To enable easy switching between logical and physical libraries, you can specify a library search list.

Note:

The library search list is relevant only if the layout library is different from the schematic library.

The library search list is specified by the preference `lxLibrarySearchList`. The entry "%PLACED" denotes the library name of the placed master. For more information, see Hierarchy Editor Tool in the *Custom Compiler Environment User Guide*.

Ignoring Objects

You may want to restrict some instances from being used in SDL.

You can instruct SDL to ignore these instances in one of the following ways:

- Specify an `oaBooleanProp` on the instance
- Specify an `oaBooleanProp` on the master
- Create a parameter with a name found in the `lxIgnoreNames` preference: `nlAction`, `lvIgnore`, `lxIgnore`

Note:

The `lxIgnoreNames` preference is an ordered, space delimited string.

- Set the Physical Only attribute of the instance. Physical-only instances are ignored by default.
- Use the **ECO Setup** dialog box to select the Ignored Cell Types option and specify the ignored cell types in the **SDL Ignored Cell Types** dialog box.

Alternatively, set the `lxECOIgnoreCellTypes` preference to true and specify the cell types to ignore in the `lxECOCellTypeIgnoredNames` preference.

Ignoring Terminals

Some terminals are added in the schematic design for post simulation needs. You can specify any terminals that SDL must ignore by creating an oaProp on the oaTerm with any property name found in the lxIgnoreNames preference.

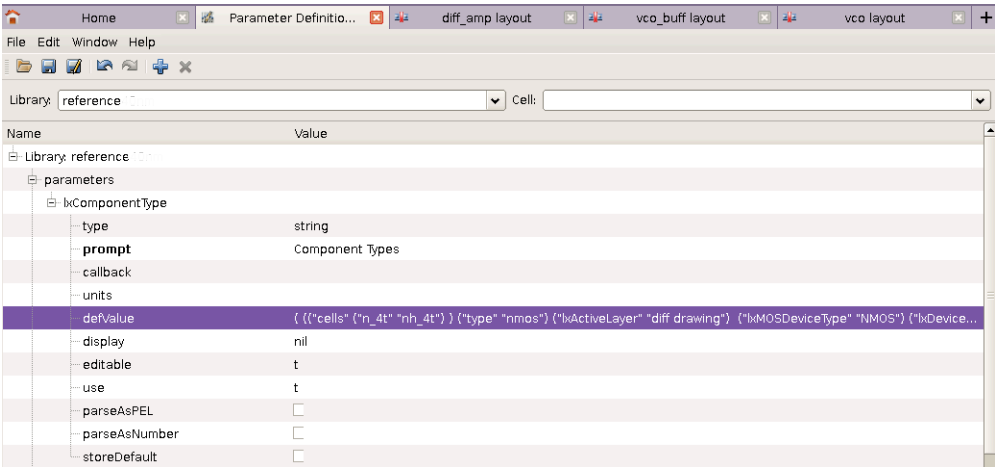
Logical to Physical Mapping Using Configurations

You can override all schematic annotations for logical to physical mapping by defining configurations using the Hierarchy Editor in Custom Compiler. For more information regarding the Hierarchy Editor and editing configurations, see Hierarchy Editor Tool in the *Custom Compiler Environment User Guide*.

About lxComponentType

This CDF attribute is used to define parameters for SDL. SDL looks in the library CDF for the lxComponentType parameters, which contains a list of lists (see [Example 1](#)). You can view lxComponentType parameters in defValue from **Parameter Definition Editor > Library > Parameters > lxComponentType**. See Parameter Definition Editor Tool in the *Custom Compiler Environment User Guide* for more details.

Figure 176 Parameter Definition Editor



- The outer list contains sets of cells in a device class.
- The inner list contains the class attributes, which are defined in [Table 62](#).

Table 62 Class Attributes for Transistor

Attribute ⁵	Description
cells	List of cell names that belong to a specific class. Supports "libName cellName" pairs. libName is optional. Example: ("cells" ("libName1 cellName1" "libName2 cellName2" "cellName3"))
type	Type of class.
lxActiveLayer	Specify a LPP as a boundary layer. If it is defined, the spacing will be calculated based on this boundary layer; otherwise, instance bounding box is used for spacing.
lxDeviceFinNumber ⁶	Fin number parameter. Used for FinFET when folding. The MOS is treated as FinFET if the wideAlignment is defined.
lxDeviceLength	Length parameter. Used when editing dummy in Symbolic Editor.
lxDeviceTotalFin	Total fin number parameter. Used for FinFET when folding. The MOS is treated as FinFET if the wideAlignment is defined.
lxDeviceTotalWidth	Total width parameter.
lxDeviceWidth	Finger width parameter. Used when folding.
lxDummyMaxLength	Maximum length value of dummy MOS.
lxDummyMinLength	Minimum length value of dummy MOS.
lxFingerParam	Finger number parameter. Used for splitting or folding.
lxMinFinger	Minimum finger size (used for splitting). If lxMinFinger is defined in lxComponentType, instances are split according to the specified value. The finger number of the split devices should be larger than the parameter value of lxMinFinger. For example, consider an instance containing 8 fingers that are to be split. If lxMinFinger is not defined, the instance is split into 8 devices, each with 1 finger. If lxMinFinger is defined with a value 2, the instance is split into 4 devices, each with 2 fingers.
lxLeftODExt	Parameter for left OD extension to use for gate alignment in Symbolic Editor.
lxMaxFin	Maximum value of fin number. Used when folding FinFET.

5. SDL does not consider the other parameters (such as "width per finger" or "folding parameter") as accumulative. Consequently, SDL compares those other parameter values on each layout instance against the schematic value.
6. If lxDeviceFinNumber and lxMaxFin are not specified, lxDeviceWidth and lxMaxWidth are used for the Auto Folding feature. For example, lxDeviceFinNumber is nfin in the parameter definition and nfin is 8 in the instance. In the Pick and Place dialog box, the Fold table shows that the Threshold is 3. The original device is folded to 3 devices. The values of nfin for each segment are 3, 3, 2.

Table 62 Class Attributes for Transistor (Continued)

Attribute ⁵	Description
lxMaxWidth	Maximum value of finger width. Used as default folding threshold or used when merging.
lxMFactorParam	If not found then preference <code>lxMFactorNames</code> is used to repeat the lookup precedence.
lxMinFin	Minimum value of fin number. Used when folding FinFET.
lxMinWidth	Minimum value of finger width. Used when folding.
lxMOSDeviceType	Unknown use at this time.
lxRealMasterParameter	Parameter name for the real master. Example: <pre>(("cells" ("analog_cell")) ("lxRealMasterParameter" "srcCell"))</pre> SDL checks the parameter <code>srcCell</code> to get the real master name.
lxRealMasterLibrarySearchList	List of the order of the library to search the cell. Example: <pre>(("cells" ("cell1")) ("lxRealMasterParameter" "cellNameParam") ("lxRealMasterLibrarySearchList" "Lib1 Lib2"))</pre> SDL searches the cell name defined in the <code>cellNameParam</code> parameter in the library <code>Lib1</code> . If no cell is found, SDL searches the cell in the library <code>Lib2</code> . It displays original cell name in the schematic mode of Design Navigator if the cell cannot be found.
lxPlacementOrderParam	Specify the custom placement order in the PyCell schematic sub-master. The CDF/OA property defines a list of instance names as the placement order. Example: <pre>pre_abutment_order = M0 M1 M3 M4 DM</pre> To consider the precedence, if the cell-level CDF has no keyword, the SED chaining mode checks the OA property in schematic view. If there is no any specified custom placement order or the CDF/OA property is empty, SED chaining mode works as before. To consider the precedence, if cell-level CDF has no keyword, SDL checks the OA property in schematic view during elaboration. If the property value changes, you need to re-initialize the SDL environment to get the latest value. After you define <code>lxPlacementOrderParam</code> in the library, the row-based placement follows the order. See the example in About lxComponentType.
lxRightODExt	Parameter for right OD extension to use for gate alignment in Symbolic Editor.

5. SDL does not consider the other parameters (such as "width per finger" or "folding parameter") as accumulative. Consequently, SDL compares those other parameter values on each layout instance against the schematic value.

Table 62 Class Attributes for Transistor (Continued)

Attribute ⁵	Description
lxSDSwap	Swap source and drain terminals. The format is ("lxSDSwap" "ParamName" ("SwapValue" "NonSwapValue") ("termNamesS" "termNamesD")) SwapValue and NonSwapValue are optional. Example: ("lxSDSwap" "swapTerminal" ("true" "false")) termNamesS and termNamesD are optional. Default is S and D.
lxStacking	A Boolean to state whether a device has a stacking structure.
lxTermMap	List of name/value pairs to define the mapping of terminal names to common names. Example: ("lxTermMap" (("drain" "D_term") ("source" "S_term") ("gate" "G_term") ("bulk" "B_term") ("leftpode" "E1") ("rightpode" "E2"))) If not found then termOrder is used lookup precedence using lxToolSearchList. If no terminal map specification is found or determined, then a default terminal mapping will be used. The terminal names will not be single names but instead a regular expression. Any terminal name matching the regular expression will fulfill the mapping and identify the terminal type. Table 63 defines the default terminal maps for each component type.
lxTap	Specify the cell to be tap cell. Example: ("lxTap" "t")
lxIgnoredSyncParams	List of ignored parameters.
lxLayoutDummyParams	Specify the layout dummy parameters. The format is ("lxLayoutDummyParams" ("Param1" "Value1") ("Param2" "Value2") ...) Example: ("lxLayoutDummyParams" ("dmy_switch" "ON")) The parameter dmy_switch is set as ON for dummy devices.
lxValidWidths	Specify one or several folding widths. The format is ("lxValidWidths" ("width1" "width2" "width3" ...)) Example: ("lxValidWidths" ("0.9u" "0.6u" "0.3u"))

Table 63 Default Terminal Maps for Each Component Type

Component Type	Terminal Name	Terminal Type
nmos pmos	^([Dd])DRN)\$	drain
	^([Ss])SRC)\$	source

5. SDL does not consider the other parameters (such as "width per finger" or "folding parameter") as accumulative. Consequently, SDL compares those other parameter values on each layout instance against the schematic value.

Table 63 Default Terminal Maps for Each Component Type (Continued)

Component Type	Terminal Name	Terminal Type
res cap	^([Gg])GATE GT)\$	gate
	^([Bb])BULK SUB)\$	bulk
	^([Pp])PLUS plus)\$	plus
	^([Mm])MINUS minus)\$	minus

Example 1

```
(("cells" ("nch1" "nch2")) ("type" nmos) ("lxActiveLayer" "OD drawing")
("lxMOSDeviceType" "NMOS") ("lxDeviceWidth" "w") ("lxMaxWidth" 5e-06)
("lxDeviceTotalWidth" "wtot") ("lxFingerParam" "nf"))

((("cells" ("pch1" "pch2")) ("type" pmos) ("lxActiveLayer" "OD drawing")
("lxMOSDeviceType" "PMOS") ("lxDeviceWidth" "w") ("lxMaxWidth" 1e-05)
("lxDeviceTotalWidth" "wtot") ("lxFingerParam" "fingers"))
```

Example 2

```
(("cells" ("nfin" "nfinhv")) ("type" nmos) ("lxActiveLayer" "DIFF
drawing") ("lxMOSDeviceType" "nmosClass") ("lxTermMap" (("drain"
"D") ("gate" "G") ("source" "S") ("bulk" "B"))) ("lxDeviceFinNumber"
"nfin") ("lxMaxFin" "20") ("lxDeviceTotalFin" "total_nfin")
("lxFingerParam" "nf"))
```

Example 3

```
(("cells" ("snfin" "snfinhv")) ("type" "nmosStack") ("lxStacking" "t"))
```

Example 4

```
( ( ("cells" ("n_4t" "nh_4t")) ("type" "nmos") ("lxActiveLayer"
"diff drawing") ("lxMOSDeviceType" "NMOS") ("lxDeviceWidth" "w")
("lxDeviceTotalWidth" "wtot") ("lxDeviceLength" "l") ("lxFingerParam"
"nf") ("lxTermMap" (("drain" "D") ("source" "S") ("gate" "G") ("bulk"
"B"))) ("lxMaxWidth" "1.5u") ("lxMinWidth" "0.5u") ("lxDummyMaxLength"
"0.5u") ("lxDummyMinLength" "0.2u") ("lxPlacementOrderParam"
"pre_abutment_order"))) )
```

Table 64 Class Attributes for Resistor

Attribute	Description
cells	List of cell names that belong to a specific class. Example: ("cells" ("rnpoly"))

Table 64 *Class Attributes for Resistor (Continued)*

Attribute	Description
type	The type of cells in this class. Example: <code>("type" "res")</code>
lxActiveLayer	Specify a LPP as a boundary layer. Example: <code>("lxActiveLayer" "ODdrawing")</code>
lxDeviceWidth	Device width parameter. Used when folding.
lxMaxWidth	Maximum width of the finger. Used as default folding threshold or used when merging.
lxDeviceTotalWidth	Total width parameter.
lxDeviceTotalResistance	Total resistance parameter.
lxMFactorParam	If not found then preference <code>lxMFactorNames</code> is used to repeat the lookup precedence.
lxTermMap	List of name/value pairs to define the mapping of terminal names to common names. Example: <code>("lxTermMap" (("plus" "PLUS") ("minus" "MINUS") ("bulk" "BULK")))</code> If not found then the <code>termOrder</code> is used lookup precedence using <code>lxToolSearchList</code> .
lxDeviceLength	Length of the device.
lxDeviceResistance	The resistance for the device.
lxDeviceTotalLength	Total length of the device.
lxSeriesParam	Series parameter.
lxParallelParam	Parallel parameter.
lxConnectionType	Connection type. The format is <code>("lxConnectionType" ("serial" "paramName" "paramValue") ("parallel" "paramName" "paramValue"))</code> Example: <code>("lxConnectionType" ("serial" "connection" "Series") ("parallel" "connection" "Parallel"))</code>
lxSegmentParam	Segment parameter.
lxEvalModes	Calculation mode. Example: <code>("lxEvaModes" (("WR ("ResCalc" "Res_&_w") ("ns" "1") ("np" "1")) ("RL" ("ResCalc" "Res_&_w")) ("LR")))</code>

Table 64 *Class Attributes for Resistor (Continued)*

Attribute	Description
lxResDirection	Resistor direction. The format is ("lxResDirection" ("horizontal" "paramName" "paramValue") ("vertical" "paramName" "paramValue")) Example: ("lxResDirection" ("horizontal" "orientation" "P"))

Example 5

```
( ("cells" ("rnpoly")) ("type" "res") ("lxActiveLayer" "poly
drawing") ("lxTermMap" (("plus" "PLUS") ("minus" "MINUS") ("bulk"
"BULK")) ("lxResDirection" ("horizontal" "orientation" "P"))
("lxDeviceResistance" "r") ("lxDeviceWidth" "w") ("lxDeviceLength"
"l") ("lxDeviceTotalLength" "Leff") ("lxSeriesParam" "series")
("lxParallelParam" "np") ("lxSegmentParam" "segments")
("lxConnectionType" ("serial" "connection" "Series") ("parallel"
"connection" "Parallel")) ("lxEvalModes" (("WL") ("LW") ("RL")))) )
```

Table 65 *Class Attributes for Guard Ring*

Attribute	Description
cells	List of cell names that belong to a guard ring. Example: ("cells" ("MPP_GR_NWell"))
type	The type of cells in this class. Example: ("type" "guardring")
lxExposedParams	Specify the parameters used in the Generate Guard Ring command. Example: ("lxExposedParams" ("grWidthFinger" "grWidthFin"))

Example 6

```
( ("cells" ("MPP_GR_NWell")) ("type" "guardring")
("lxExposedParams" ("grWidthFinger" "grWidthFin"))) )
```